6TH FORM TIME MANAGEMENT AND APPLICATION HELPER SOFTWARE

Nair, Aaryan (13 Iz)

ST BERNARDS CATHOLIC GRAMMAR SCHOOL

Contents

Contents 1
Analysis 4
The problem 4
Stakeholders 4
Why is this problem solvable using computational methods? 5
Interview 7
Interview Questions 7
Interview Results 8
Research 11
Existing Solutions 11
Features of proposed solution 15
Stakeholder Feedback 19
Requirements 19
Hardware and Software requirements 19
Success Criteria 22
Design 24
Algorithms 24
Top-Down Diagram 24
Subroutines 29
Pseudocode and Flowcharts 29
User Interface 49
Usability Features 56
Stakeholder Feedback 57
Variables and Data Structures 58
Entity Relationship Diagram 64
Validation 68
Testing Method 70
Test data to be used during development 70
Further data to be used in post development 78
Development and Testing 79

Stage 1: Database 79
Initial iteration implementation creating the database 79
Second iteration implementation creating the USERS table 81
Third iteration implementation creating the other tables 83
Stage 1 Review 86
Stage 2: Signup Page 87
Creating the GUI of the Page 88
Connecting to the database and creating session variables 93
Initial iteration implementation using Javascipt 94
Second iteration implementation using PHP 96
Stage 2 Review 105
Stage 3: Login Page 106
Creating the GUI of the Page 106
Initial iteration implementation – updating the form attributes and fields 108
Second iteration implementation – implementing the formhandler 109
Third iteration implementation – implementing the functions 112
Stage 3 Review 118
Stage 4: Calendar Page 119
Initial iteration implementation of replace_(month name) 122
Second iteration implementation – adding the header row 125
Third iteration implementation – fixing the date value formatting 126
Fourth iteration implementation – making the code more modular 127
Fifth iteration implementation – adding colour coding 130
Sixth iteration implementation – adding the legend 132
Stage 4 Review 133
Stage 5: Input Form Page 136
Creating the GUI of the Page 136
Initial iteration implementation – implementing radiobuttonoutput 141
Second iteration implementation – implementing inputformhandler 142
Third iteration implementation – implementing inputform_input 147
Stage 5 Review 156
Stage 6: Skills Page 158

Creating a prototype GUI of the page 159
Second iteration of the GUI 161
Initial iteration implementation using JavaScript 162
Second iteration implementation using PHP for textbox output with values 163
Third iteration implementation – formhandler 166
Fourth iteration implementation – navigation bar 169
Fifth iteration implementation - logout 172
Stage 6 Review 173
Stage 7: Timeline Page 175
Creating a prototype GUI of the page 175
Initial iteration implementation using JavaScript 177
Second iteration implementation using PHP – timeline.inc.php 178
Third iteration implementation using PHP – timeline_input.inc.php 181
Fourth iteration implementation using JS – save as pdf feature 183
Stage 7 Review 188
Stage 8: Final Testing 190
Stage 9: Stakeholder Testing 201
Stage 9 Review 205
Evaluation 205
Success Criteria Review 205
Success Criteria 205
Evidence 206
How could unmet criteria be addressed in future iterations? 210
Usability Features 215
Evidence 215
How could unmet features be implemented in future iterations? 217
Limitations 218
How could these limitations be addressed? 218
Potential improvements/changes that could be made? 219
Maintenance 220
Final Code 222
Bibliography 257

Analysis

The problem

In 6th Form, there is a great emphasis placed on preparing for the future. Students are presented with many opportunities such as courses and leadership roles; however, they are not provided with a way to manage them. Many students struggle with balancing their academic responsibilities whilst also participating in extracurricular, super-curricular and co-curricular activities for their personal statements and CVs. Calendars may be forgotten to be updated each day and producing a definitive list of every extracurricular activity or placement when needed for applications is challenging. This problem affects many students, including myself.

Furthermore, writing a personal statement is one of the most important aspects of university applications, even for students who have many things to talk about. Few people can easily keep track of relevant events whilst also being able to elaborate on the skills they gained from them. A subpar personal statement can lead to the student's application being rejected, even before they can appear for an interview.

When talking in an interview, students may forget some course or work experience that they have done which can lead to them underselling themselves. One piece of evidence can make a difference in deciding between two candidates of similar calibre, and this is especially pressing for those applying to competitive institutions with 1000s of applicants from all around the world.

My program aims to solve this issue. It will be a calendar system where students can plot upcoming events. These events will then be accessible in the skills table where they can be grouped together based on the skills the students have developed during those events which would allow students to easily link them whilst writing their personal statements or CVs. Finally, all the events from the calendar will be displayed chronologically on a timeline page which will serve as a reminder to the students of everything they have done and when they have done it. I believe that this project could be an extremely helpful and crucial tool for all 6th form students, if completed well.

Stakeholders

My clients would be 6th form students who manage multiple responsibilities and curricular or extracurricular events. Due to the variety of A level and extracurricular choices, my clients would be a random sample of students willing to test this software.

The primary stakeholders for this project would be the students; however, some important secondary stakeholders include their carers/guardians and their teachers who may want to view the student's logs and therefore, having a simple UI would be key.

Many of these students express frustration with the pre-existing tools provided by the school, such as Unifrog, for several reasons such as confusing UI, presenting too much information and the process of managing activities being confusing. Therefore, I would like my project to avoid these issues whilst also improving on them, for example having simple, well-defined input fields for logging activities.

My stakeholders will be able to use this program as an all-in-one solution to the problem of stressful 6th form applications. They will be able to use the calendar system to organise events and as a structured plan which will help them manage their time effectively. The skills table will help them in writing their personal statements and other applications as the events they have logged onto the calendar system will be linked to skills which would simplify their writing process. The timeline will serve as a reminder of old events and experiences and will allow them to plan which events they should do next based on what they have done. This would help solve the issue of time-management and forgetting events that they have done whilst also helping them in writing their personal statements.

Why is this problem solvable using computational methods?

This problem is solvable by computational methods due to several reasons. Firstly, there are well-defined inputs (date-time, event description, skills, text describing the event) and requirements that can only be efficiently met using subroutines and algorithms (sorting algorithms to order events chronologically and searching algorithms to locate events tagged with a certain skill).

Another reason is because I believe that the UI of the program should be a website. Using a website for the user interface would allow for effective visualisation of data, for example, the date-time data can be displayed using an interactive calendar system. Furthermore, it will be easy to use and access for my clients who are of an age demographic familiar with the internet and navigating websites.

Furthermore, this problem involves the storage and retrieval of data which can be effectively implemented using a database. The program will need to be able to take the data relating to events the user has completed and input them as a row in the database associated with a certain username (as multiple users will use this program). It will then need to be able to retrieve that input event data and output that data on the calendar and timeline pages to serve as a reminder for the user. These interactions with a database will need to be implemented using computational techniques and methods, making the problem solvable using computational methods.

Problem Recognition

The main problem is that there is not an all-in-one resource that can help students balancing many extracurriculars. Calendars are only useful for remembering upcoming events, there is no perfect way to remember previous events, and some people may struggle to link events to skills when necessary.

Problem Decomposition

Problem decomposition is the process of breaking a complex problem into smaller, more manageable sub-problems that can be solved individually.

In this case, the problem can be decomposed into 3 key subproblems:

1. A way to be reminded of upcoming/ongoing activities
2. A way to remember past events
3. A way to link these events to the skills demonstrated/gained

The general solutions being:

1. A calendar to log events
2. A table to log skills and events (which links to the calendar and timeline)
3. A timeline system to view all events (organised chronologically)

Divide and Conquer

Divide and Conquer is when a problem is split into two or more smaller, similar subproblems, which can then be solved recursively.

After having decomposed the problem, each individual part can be tackled using Agile software development methodology (which emphasises delivering working software in cycles which can be improved on incrementally) so that I can get feedback at each stage of development from the stakeholders to iteratively improve the program.

I will work on one of the 3 sub-problems until it is completed to a satisfactory standard for my clients before starting work on the next sub-problem. This is the best way to tackle the project as it provides a solution to an incredibly pressing problem, and it should be completed to a standard that satisfies the clients' needs.

Abstraction

Abstraction is when excessive details are removed to simplify a problem which allows programmers to focus on the core aspects of a solution rather than worrying about unnecessary details.

There is a lot of unnecessary information that does not need to be immediately visible to the user. For example, the calendar system does not need to show data for the entire year all at once. It can show an overview based on the month and year selected and then clicking on a day would bring only the events of that day into focus.

Also, the calendar can be further simplified by excluding the reflections on the event and instead including it on the timeline as it will reduce clutter whilst still including valuable information in an easy to view location. I believe that this decision would be important as the timeline is designed to serve as a reminder of previous events and therefore including the reflections on the timeline would help serve as an additional reminder.

Interview

Interview Questions

I will outline the questionnaire sent to each client but during the interview I may have asked follow-up questions or asked them to explain their answers. The purpose of the questionnaire is to see if they already use tools like this project; to get their opinion on what they are currently using; and to get their opinion on the proposed features of the project and any other features they may want to see.

1. Do you believe that you can manage extracurriculars (webinars, work experiences, taster days, clubs, etc) easily?
2. If so, which tools do you use to help with managing them?
3. What are the benefits of those tools?
4. What can be improved in those tools?
5. Do you find that you can easily recall events (and the skills demonstrated/gained in them) when needed?
6. What are your thoughts on using my program (A calendar system to log the events which would link to a skills table where you can organise the events present in the calendar)?
7. What features would you like in a program such as this?
8. Would you like this program to be a website or something else (if so, elaborate)?
9. Would you like a machine learning model which can try and extract skills it thinks you may have gained/demonstrated from a brief description of an event to be implemented (if time permits)?
10. Do you have anything else to add?

Questions 1 and 5 are to check whether they will benefit from using this program and to identify if they are my primary stakeholders.

Questions 2 to 4 are to determine which tools they are currently using and to identify the benefits and shortcomings of those tools that can be implemented and worked on in this project.

Questions 6 to 10 are to determine whether they are satisfied with the proposed project, and to get their opinions on any further features that can be added to it.

Interview Results

1. Do you believe that you can manage extracurriculars (webinars, work experiences, taster days, clubs, etc) easily?

"**No**" (3), "**Yes**" (3), "**Sometimes**" (2)

Apart from those who do not do any extracurricular activities, the sample follows a trend where those who do fewer extracurriculars have an easier time managing them as opposed to those who do multiple.

2. If so, which tools do you use to help with managing them?

"**Memory**" (2), "**Unifrog**" (2), "**Calendars**" (2), "**Written reminders**" (3), "**Other cues (images, etc.)**" (1), "**None**" (1)

There was repeated mention of the platform Unifrog and online Calendars such as Google calendar where those who used those platforms spoke in detail about them being beneficial.

3. What are the benefits of those tools?

"Infinite and doesn't disappear" (in regards to memory and visual cues**), "Good organisation of extracurriculars, can add extra notes, and helps start extracurricular reading"** (in regards to Unifrog)**, "Make it easy to plan, prevent overlap and gives structure and is easy to read"** (in regards to calendars)**, "Reliable and no work needed"** (in regards to memory)**, "If it is something online, such as a webinar it gives a link that can be accessed directly from the calendar"** (in regards to Google Calendar)**, "Prevents me from forgetting"** (in regards to written cues)**, "Simplistic"** (in regards to written cues)

From this we can determine the characteristics my project should have:

- Non-volatile

- Simple structure
- Acts as a reminder
- Helps with planning
- Has some Quality-of-Life features

4. What can be improved in those tools?

I will only include relevant information that I can use as a guide for my own project, however, I will include answers such as "Nothing" as those answers will help me determine which alternative programs are already helpful so I can research them in more detail.

"Nothing" (regarding Unifrog)**, "A timeline would help remember the order in which tasks were completed. Also, it is sometimes difficult to assign skills to an extracurricular that is quite obscure" (**regarding Unifrog)**, "An easier to use UI"** (regarding digital calendars)**, "Better control of colour coding"** (regarding Google Calendar)**, "I don't really write the skills I use"** (regarding written cues)**, "Paper can only be used to plan so far, could also be better for the environment"** (regarding written cues)

From this we can determine some more characteristics my project should have:

- A timeline
- A wide variety of skills in the skills table to cover obscure extracurriculars
- Easy to use UI
- Colour Coding
- Linking skills to events easily

5. Do you find that you can easily recall events (and the skills demonstrated/gained in them) when needed?

"**Yes, both**" (5), "**Yes, skills**" (1), "**Yes, events**" (1), "**No**" (1), "**Sometimes**" (1)

This helped me determine that a 'reminders' feature for past events would be mostly unnecessary if I am planning to implement a timeline that would log all events chronologically. Furthermore, for those who do not remember everything, I think that including a brief description of the event on the timeline and linking it to the skills shown will be another useful feature to implement.

6. What are your thoughts on using my program (A calendar system to log the events which would link to a skills table where you can organise the events present in the calendar)?

All the responses were happy with the idea, but I would like to include some of the extra details they were particularly interested in and would like to see.

“Calendar to skills table connectivity”, “Chronological ordering and linking to skills table”, “Helping organise revision”, “Emphasis on reflecting on the event afterwards"

7. What features would you like in a program such as this?

“**Colour coding”, “Example extracurriculars for each skill”, “Adding future events”, “Drop-down menu in the calendar for each event”, “adding reminder notifications”, “searching for events that occur on a certain day and displaying them on the calendar”, “Reflections on how exactly you gained the skills from the event”, “Skills suggester based on the activities you have logged”, “Optional light/dark mode toggle”**

All are very interesting features; however, I may not be able to implement all of them due to time constraints.

8. Would you like this program to be a website or something else (if so, elaborate)?

“**Yes**” (8), “**App**” (2)

Most people chose websites due to them being easier to access. I will make this program a website as it will be easier to code a website than an app in the limited time given.

9. Would you like a machine learning model which can try and extract skills it thinks you may have gained/demonstrated from a brief description of an event to be implemented (if time permits)?

“**Yes**” (5), “**No**” (2), “**No opinion**” (2)

Most of the clients seemed to be happy with the idea of it. The only concerns were with the viability of completing it in the given time and that it detracts from individual, personal reflection.

10. Do you have anything else to add?

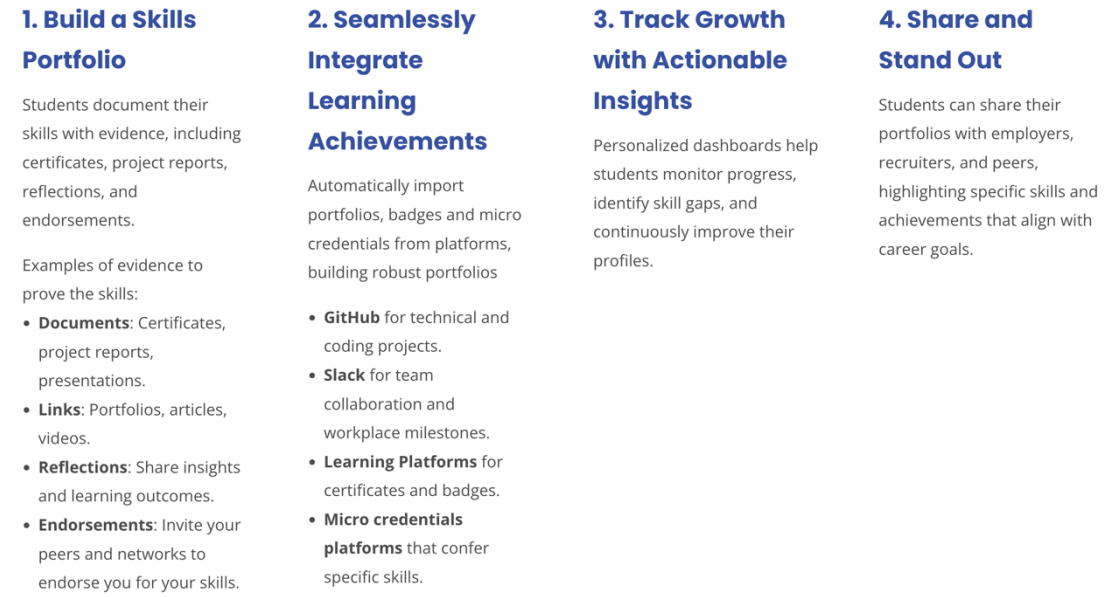
“Skills ‘tags’ should be included for each event in the calendar”, “Adding future extracurriculars”

This section tended to include certain additions that were forgotten during earlier questions and general wishes of luck for the project.

Research

Existing Solutions

Skills-Log



https://skillslog.com.au/student-features/

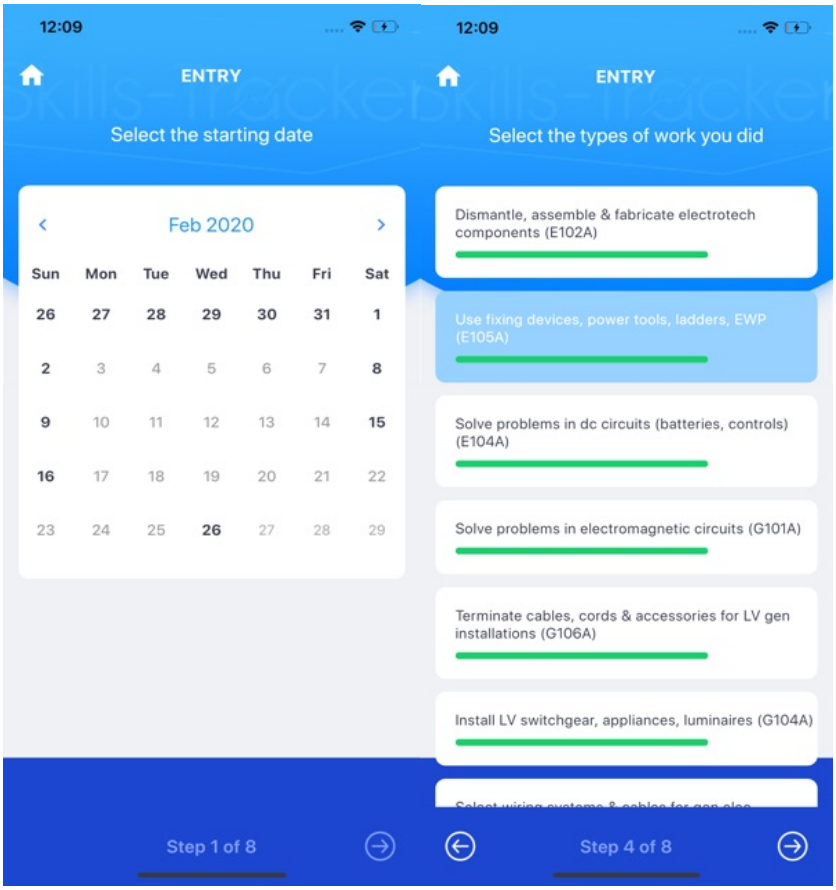
Overview:

A platform that allows students to upload evidence of their skills by adding links, documents, and reflections. It also integrates with various platforms and imports credentials from them to provide further evidence of their skills. The dashboards of the students are personalised and can be shared with peers and employers.

Parts that I can apply to my solution:

I would like to try and implement a 'reflections' feature to the calendar. When an event is added to the calendar a brief description/reflection can be added where the client can talk about how exactly they showed the skills the event will be linked to and how the benefitted from the event. This would allow them to think about what went well and what could have been improved. Furthermore, I would like to add a light/dark mode feature to allow for (although basic) customisation as most websites have this feature and it is a useful quality of life feature to have (for example, for light-sensitive people or when my clients want to use this website at night.

Skills Tracker



https://www.skills-tracker.com/students/

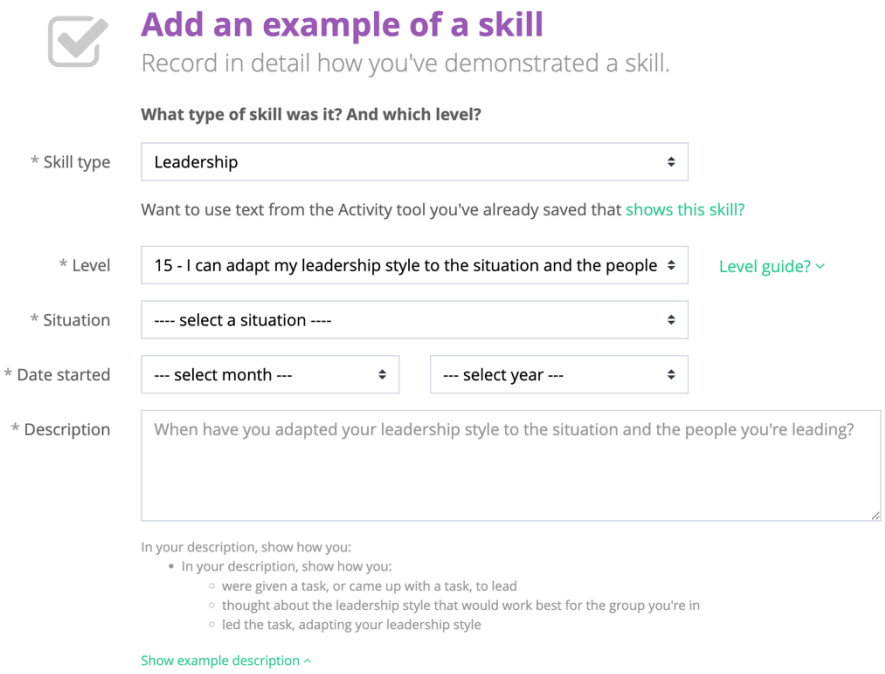
Overview:

Skills tracker is an app that allows employees to log the date-time details of tasks that they have completed in their role allowing them to form a list of evidence when asked to provide proof of their skills. This app is targeted at employees with well-defined job requirements and common tasks. This app also requires a form of verification from the employee's team leader before the task can be marked as completed which is an effective way of providing evidence for the employee's skills.

Parts that I can apply to my solution:

Although a verification system would be a safe way to evidence my clients' additions to the calendar, it is not an essential feature as students can receive references from teachers which is a more effective piece of evidence at this stage of their academic journey. I will use a similar calendar feature on the home page which will allow the client to add past, present and future dates for events and will also allow them to colour code the events to correspond with the type of events/skills they have gained as it is a key aspect of solving the problem of time-management.

Unifrog



https://www.unifrog.org/student/skills

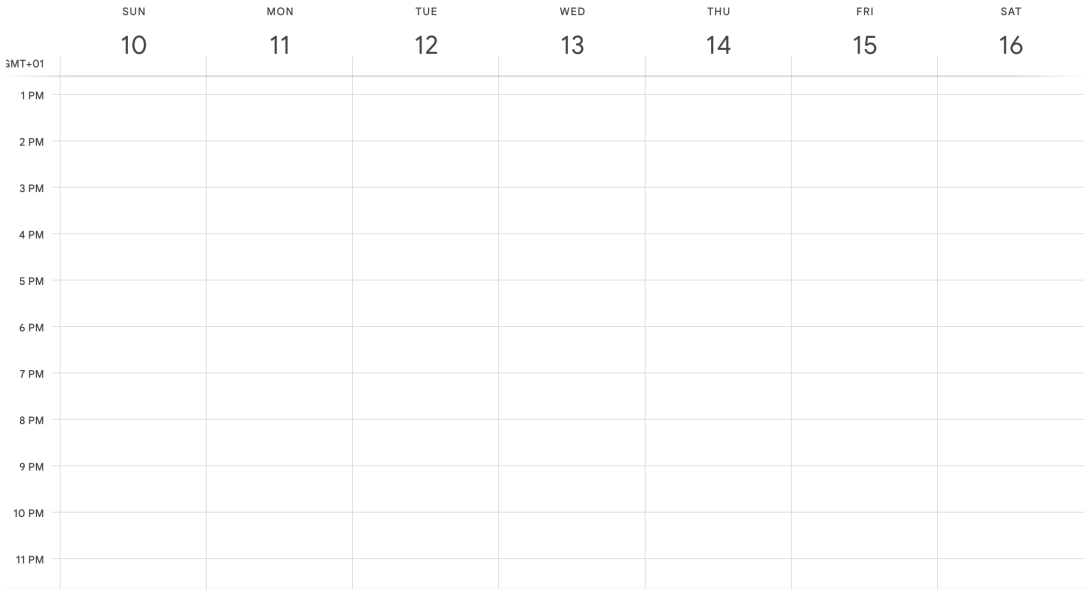
Overview:

Unifrog is a platform that helps students with managing their responsibilities and making career decisions before entering university. Unifrog is a multifaceted software that has various features that can be useful for students such as University shortlists, a Careers Library and Reference Notes. One of the features on their platform that can serve as a reference for this project is the skills log. Students can select the type and level of the skill and then add details regarding an experience where they have shown it. The details include the situation, the date-time information, and a brief description of how they demonstrated the skill. It also includes a guide on some of the details that the student may want to include in the description and it also includes an example description that can act as a guide.

Parts that I can apply to my solution:

Using the Skills log feature on Unifrog as a baseline for the Skills Table page on my website would be incredibly useful, as various clients have stated that they use this software and one client even said that this feature has no real faults. Therefore, I would like the addition of events to the Skills Table to be like this. Logging an event to the Skills Table would require the date-time information (taken from the calendar) and a brief description of how they showed the skill. I would also include an example description or guideline on how to write the description for those who are struggling.

Google Calendar



https://calendar.google.com/calendar/

Overview:

Google Calendar is a helpful digital calendar system. It has modular time blocks that you can drag and resize for your commitments which makes structuring and planning time extremely easy. Another useful feature is the ability to include links to meetings/webinars that use Google Meet and documents such as important forms or certificates to the event logged in the calendar. It also includes many common calendar features such as reminders and colour coding of events.

Parts that I can apply to my solution:

Although the resizable time chunks are an incredibly intuitive and effortless way to plan events, it would be incredibly challenging to recreate with the given time conditions. There are many other features that I would like to refine to the clients' requirements, and I am unsure whether I will be able to recreate this whilst also completing all the other features to a good standard. However, I would like to include the reminders feature and the ability to colour code events based on the skills shown or the type of

event it was. These are features that the clients have mentioned that they would like to see and are essential features in most calendar programs.

Features of proposed solution

Initial Concept of solution based on research

My solution will be a website where the user can sign up/login for their account and be taken to the homepage which will be a calendar system where events can be logged for the past, present and future. These events will be able to be colour coded (by type of event specified by event tags). These events will link to the skills table where a brief description of how the skill was developed/shown will be displayed alongside an example description for each skill which would help clients write their own descriptions and help with grouping. These pages will link to the final page which will be a timeline which will display every event from the calendar chronologically (based on start-date) and will include the date-time information and a brief reflection on the event. This page will be able to be saved as a pdf which would be useful for user records. The whole website will also have toggle-able light/dark modes for customisation purposes and reminder notifications will pop-up a day before an upcoming event (if time permits).

Inputs:

- Username
- Password (input twice to implement a 'confirm password' feature)
- Name of the event
- Date-time information of the event
- A brief description of the event
- Type of event (for use in colour-coding)
- Skills linked to the event (for use in colour-coding)
- A description of how they demonstrated the skill they have logged (displayed in the skills table)
- A reflection on the event (displayed on the timeline)

Backend:

- SQL Database to store all the input data
- Login/Signup credential checking algorithms (check if the credentials are correct, if the username is already taken etc.)
- Searching algorithms to search for events tagged with a certain skill for the event to appear in the skills table drop-down menu
- Sorting algorithms to sort all the logged events in the calendar and to display them chronologically in the timeline

- Calendar creation function (function to create the clickable button cells of the calendar, each of which would redirect to an input form allowing the user to input event data)
- A logout function (which would allow only the user to access their account and prevent accidental access of user accounts)
- Functions to retrieve data stored in the database (to output colour coding and event skill tags etc.)
- Two switchable CSS files storing the light and dark mode stylesheets which can be toggled using a radio button system on the website

Outputs:

- Error Messages appearing on the Login/Signup pages (if the user enters incorrect credentials or a taken username)
- Success Messages appearing on the Signup/Calendar pages (if the user created an account successfully or logged in successfully)
- Displaying the input event data on the calendar and timeline in the correct place based on their date time information
- Events being displayed with the right colour coding based on the tagging done by the user
- The events tagged with a certain skill are displayed in the correct part of the skills table with the description and example description next to it
- Logged events automatically appearing on the timeline once they have been logged on the calendar
- Stored event reflections appearing underneath the event names on the timeline page

Justification and Explanation:

- An accounts system and login/signup pages would be helpful when storing sensitive user events data in the database as it can be associated with a user's login credentials.
- Error and success messages will make the website feel more responsive and will be helpful in informing the user of what changes they need to make to make the input data valid.
- The calendar, skills table and timeline are all the key features of this program and are necessary requirements to solve the problem.

- Colour coding is a feature that has been highly requested by the clients as it allows them to visually organise and group their events which helps add structure to the calendar.
- The descriptions feature on the skills table is a feature that I thought was impressive from Unifrog's Skills Log system and would serve as justification on how exactly the client has displayed the skill that they have grouped the event under.
- The example description is another feature I found impressive from Unifrog's Skills Log System as it would help the clients who are unsure about grouping events and would also serve as a guideline on how they can write their own descriptions.
- I believe that implementing these features from Unifrog is worthwhile and important as during the Interview, many clients said that they are currently using that platform and one client even said that the platform has no real faults.
- The reflections feature would be a useful tool that would allow the clients to describe their thoughts on the event after it has happened. It would allow them to write down what they believe was beneficial about the event and what could have been done better.
- The save as PDF feature on the timeline page would be a useful tool as it would allow the user to have a copy of all the events they have done. They can then use it for their own reference by printing it out or they could attach it as a part of their CV.
- The reminder notifications are there to remind the clients to prepare any necessary resources for the event and are a useful tool present in most calendar systems that I researched.
- Finally, the light/dark mode feature is a feature that would give the clients an aspect of customisation. Most websites have light/dark mode features, and it would be a nice quality of life feature for the clients.

Limitations

This program features some limitations which are mainly caused by time constraints. They include:

1. A large limitation of the program is that multiple events won't be able to be input for one day.
2. Additional stylistic features such as an expandable/collapsable calendar and more customisation options will not be prioritised features of the program.
3. Adding images/certificates for each event (similar to some of the existing solutions) may not be possible to implement.

4. I have also decided not to include the additional machine learning program that would suggest the skills gained from a description of the event as an essential feature.
5. I have chosen to implement a login page before the user can access the website instead of the skills generator feature.

Justification of Limitations

The main cause of the limitations of this program is time. There are many features that I would like to add, however they would require learning new programming techniques and languages which will not be possible to achieve within the given time.

The biggest limitation of the program it that multiple events cannot be input per day. This is to prevent clashes when retrieving data from the SQL database based on the date, and it would require implementing two or more outputs per day in the calendar which would be complicated to implement. Currently, implementing one output per day will be easier. Furthermore, as this program focuses on extracurricular and super curricular events, I am currently satisfied with my initial project only allowing for one event to be input per day as activities such as courses and competitions tend to take at least one full day to complete.

For the stylistic features, I am not experienced enough with CSS and web development in general to be confident in creating multiple website themes and styles and I do not know how to provide the user with a button/system to switch between these theme files. This would require me to need more time to learn CSS and languages such as PHP to allow for dynamic switching of stylesheets and allowing the user to define their own themes and styles.

Allowing the user to upload files which may serve as proof of their events is a good feature but leads to various time-consuming issues. Firstly, I would need to research how to implement an upload feature using web development languages. Furthermore, the documents and files will need to be stored in a database after they are uploaded which would require me to do further research on if SQL allows for files to be uploaded and how they can be retrieved. Finally, I would also need to implement various validation checks to see if the file the user submitted is not malicious, which would require checking extensions and scanning file content which would be too difficult to do within the time limit.

Training a machine learning program requires a lot of data (which very time consuming to gather). A lot of testing will need to be done, and I will need to improve the model's accuracy which would be very hard to do in the given time. If I were to implement this, I would need to put less time into the other features of the program which would lead to a decrease in the quality of the other features of the program which would negatively

affect the clients. Therefore, I have decided to include it as a desirable feature and have not included it in the initial concept.

This will allow each user to store their data in a server-side database to decrease the risk of data interception and SQL injections. I will use a hashing algorithm to encrypt the user's passwords and username before they are stored in a linked list or database to prevent login details from being stolen. Although the user data in this case relates to extracurriculars, the interception of the data would be a violation of the Data Protection Act (1998) which states that it is the right of an individual to protect their data from unauthorised distribution.

Stakeholder Feedback

I sent a copy of the initial concept of solution to my clients to get their feedback on it.

The overall feedback was incredibly positive; one client particularly liked the light/dark mode feature saying that "It seems very useful and is very nice to have". Another client said they liked the addition of the colour coding feature saying, "colour coded is good".

One critique on the listed features included one client saying, "what's the point of a timeline when you have a calendar". The calendar and timeline are both equally as useful as the calendar serves as a time-management and planning system whilst the timeline acts as a reminder and reflection system.

Another critique of the listed features included changing the notifications system to where the user can choose when they are reminded, which is a challenging addition, but I believe is a valid criticism as it is a key part of most calendar systems.

Also, one client suggested including a navigation bar at the top of the pages to help traverse between the pages which I believe is a valid quality of life feature that I would like to include if time permits.

Therefore, I have decided to proceed with this concept and the listed features alongside the feedback from the stakeholders.

Requirements

Hardware and Software requirements

Hardware Requirements

A computer capable of running the software with an active internet connection – A computer with a fast enough processor to allow for real-time editing and uploading of data will be needed. This is not an issue as most computers are capable of this.

Essential Hardware

- **A PC or laptop–** A PC or laptop will be needed to run the code files.
- **A Network Interface Card (NIC)** - Allows the device to connect to a network and process data. Built into most devices.
- **Standard peripherals (keyboard, mouse and monitor) -** Allows the user to interact with the website such as click on buttons and type data into input fields.

General Hardware Specifications

- **Processor (CPU)** - Most modern processors are powerful enough for this purpose. An Intel i5 equivalent or better are good recommendations for this.
- **RAM (Random Access Memory)** - 8 GB of RAM is good for smooth operation and running multiple tabs at once.
- **Storage (Hard Drive)** - An SSD (Solid State Drive) is recommended for faster loading times; however, a moderately sized drive is sufficient.
- **Graphics Card (GPU)** - For a standard user, the integrated graphics on a typical processor are sufficient for displaying web content.

Software Requirements

- **Web Browser** – An up-to-date version of Chrome, Safari or Microsoft Edge is necessary to display websites correctly.
- **An Internet Connection** – A reliable internet connection is necessary. Having good upload and download speeds will be good for this website as it involves inputting data and an output visualisation.
- **A Local Host Server (XAMPP)** - This will be used to host the PHP files and will allow for my program to have functionality and database integration.
- **Windows, Mac or Linux OS** – These are the OS types supported by XAMPP.
- **NOT ESSENTIAL - A PHP/JS/HTML/CSS interpreter (Visual Studio Code)** – The frontend of the website will be made using HTML and CSS whilst the backend will be made using PHP and JS

Justification of requirements

Requirement	Justification
PC or Laptop with standard peripherals	My program involves inputting data, clicking on buttons (days in the calendar) and seeing the visual output of data being retrieved from the database (timeline page, skills descriptions, colour coding).

	Therefore, standard peripherals such as a keyboard, mouse and monitor are necessary for using my software.
Computer with the following minimum specs: Intel i5 equivalent or better CPU, 8 GB of RAM, 512 GB Storage, Integrated GPU is sufficient, NIC	These specs are sufficient for running a program such as mine which isn't incredibly hardware intensive. The important aspects are enough storage to download XAMPP, and enough processing power to allow for dynamic websites to run.
Web Browser	A web browser will be required as my program will be a website and will involve interactive HTML and JS elements which are client-side languages which are run in the browser.
An Internet Connection	An internet connection is required to run the web browser which will be used to locally host the PHP files. Without an internet connection, importing anything such as fonts from google fonts will also not work.
A Local Host Server (XAMPP)	Downloading XAMPP is the recommended live server. This is for a few specific reasons: XAMPP comes with a MySQL database built into it, XAMPP is incredibly easy to install (only one .dmg/.exe file needs to be downloaded) and it is very simple to use (with start options for Apache Web Server and MySQL database). Overall, XAMPP is a single, simple download which comes with built in DB and PHP running capabilities.
Windows, Mac or Linux OS	These are the OS supported by XAMPP which is the live server. The exact OS specifications are Windows Server 2008 and later; Windows Vista and later; Mac OS X 10.6 and later; and Linux.
A PHP/JS/HTML/CSS interpreter	This is only for clients who may want to change aspects of the program such as stylistic options etc. Using an interpreter such as **Visual Studio Code** will be ideal as it can run all four types of files. PHP will be the language used for the backend due to easy linkage of the SQL database and frontend integration and linking using

	modules and libraries. Furthermore, I will be unable to learn a new language within the time constraints.

Success Criteria

Essential Features	*Justification and Explanation*
Login Page with hashing algorithm	The Login Page is necessary to prevent the risk of data breaches due to client-side reasons. All the data in this program will be stored in a server-side database. Therefore, to access the information, data must be associated with an account and therefore a login page must be present. Also, when a user is signing up, the password they entered will be hashed before it is compared with the password stored in the database as that will be hashed.
Signup Page with hashing algorithm	The Signup Page is necessary to allow the user to create an account. This page will check two input passwords and compare them (a confirm password feature) as a usability feature. This would allow the user to remember their password and prevent the risk of accidentally setting a password. Also, to prevent the use of stolen passwords, the password will be hashed before it is stored in the database.
Calendar System for past, present and future events	The calendar system is necessary as it will be the main feature that would allow the user to structure and plan their events. The calendar system will give the user an interface which will allow them to view and change their stored events during the year. The user will be able to select days from the calendar which would redirect the user to an input field where they can input the details of the event and can colour code the event.
Colour coding of events	This feature is necessary as it would help the user identify events by colour coding events of similar types and skills. It was also one of the most requested features by the stakeholders. It would allow the user to select a tag for an event in the input form and based on the tag, a colour will be applied to the event.
Skills table that links to calendar	This feature is necessary as it would be a tool that would allow the user to have a visual overview of how many skills they have developed and which skills they need to further develop. It allows the clients to see events where they have demonstrated certain skills which would be useful for writing personal statements and applications.
Skills description feature	The skills description feature is necessary as it would allow the user to provide evidence for the skills they tagged the event with. The user would be able to memorise the descriptions when asked to give examples of occasions where they showed certain skills. When adding an event from the calendar to the skills table, the

	input field will include a text input box. Here, the user will be able to input a brief description of how the skill the event is being tagged with was shown.
Example description of skills and extracurriculars	The example description of skills and extracurriculars is necessary as it would help guide the user on what to write. Including a blank skills page might confuse the user, therefore adding some guidance would be a helpful usability feature. This would be an example paragraph on how a certain skill was shown in an example of a certain extracurricular to help clients link extracurriculars to skills effectively.
Timeline page	The timeline page would allow the user to have a visual reminder of all the events they have logged in the calendar. It would be a chronologically ordered list of events from the calendar.
Reflection for each event on the timeline	The event reflection feature would be a useful feature to include on the timeline page as it would allow the user to have a post-event reflection which they can look back on to inform future plans. It was also one of the suggested features by the stakeholders in the initial questionnaire. After an event is added to the timeline, the client can add a brief description/reflection where they can discuss how they developed skills from the event and how they benefited from it.
Navigation bar	A navigation bar is necessary as it would be the only way for the user to traverse the pages of the website. This feature will not be difficult to implement as I have previously implemented this before and therefore, I will not need to do any further research. The navigation bar would be at the top of the website and that allows the user to go from one page to another by clicking on the name of the page they want to go to.

Desirable Features	*Justification and Explanation*
'Save as PDF' feature on the timeline page	This feature would be useful to implement as it would allow the user to save a copy of their timeline page which they can use as an overview of the events they have done. The user could then attach this overview to applications, CVs or use it for their own reference. It would be a feature that would save the timeline section of the timeline page in a pdf format with the styling intact.
Notifications system	This feature would be useful to implement as it is a common feature in another calendar system that I have researched (Google calendar) and is a helpful feature to have for any planner system. This would help the client prepare any materials required the day before the event and would also serve as a reminder to the user.
Light/Dark mode toggling	Light/Dark mode toggling would be useful to implement as it would give the user a sense of customisation when using the program. It will allow the user to switch between two CSS files allowing a sense of customisation.

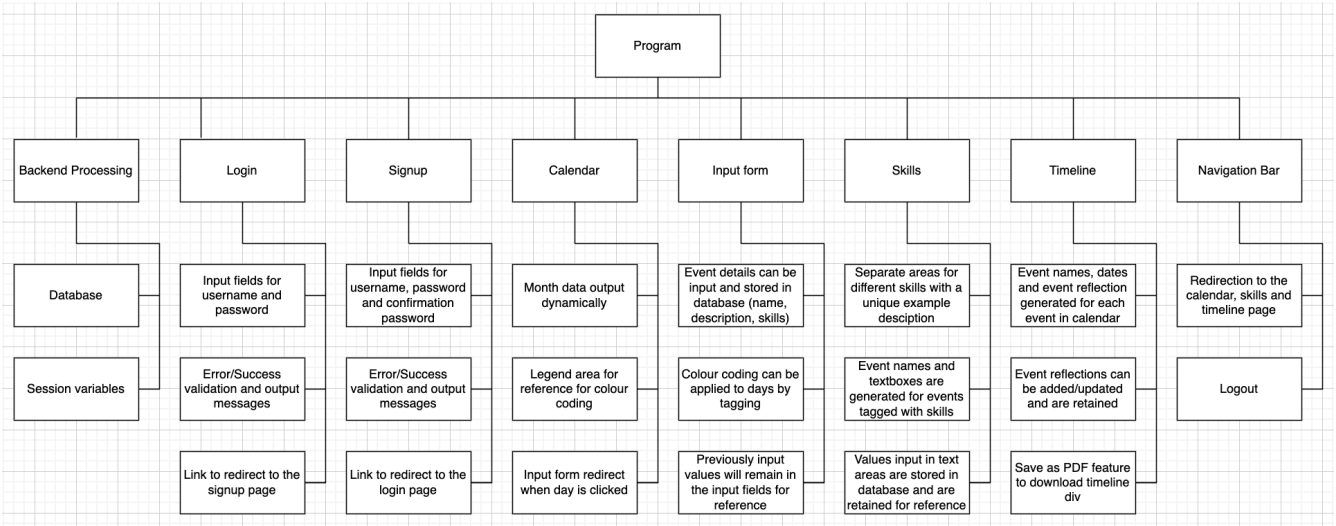
Skills Generator	This would be useful to implement as it would allow users who may be confused about which skills they showed during an event get an idea of what to tag the event with. This would be a machine learning program that would extract the skills it thinks that you have gained based off a brief description of the event.

Design

Algorithms

Top-Down Diagram

I will plan the backend processing of the website before I work on creating example UIs. The main problem being linking the database to the PHP/JS backend and then linking the PHP/JS backend to the HTML/CSS frontend. The main program, however, focuses more on the fetching, sorting and displaying of data which can be broken down as follows:



The program is secured by an accounts system so the processing can be broken down into pre- and post-login processing. Breaking this down further, page-by-page, looks like this:

Backend Processing/Navigation Bar:

- Create the database
- Create tables to store the data
- Create a session
- Create session variables

For the general backend processing I will be using a database to store all of the user information. Using a database allows the data to be accessed securely following the ACID rules of transaction processing. Furthermore, adding a file for session variable is

useful as session variables in PHP can be used as a secure way of declaring global variables through the use of a session. This would be useful in storing data such as the user id and errors in login.

- Buttons can be pressed to be redirected to the selected page
- The logout button will safely close the user session

The navigation bar is a useful usability feature which would allow the users to traverse between the pages easily. The bar would also contain the option to logout which would allow the user to securely leave the website without needing to directly close the tab or worrying for other users on the same device to access their account.

Login Page:

- Take a username and password input
- Compare it with stored username and password
- If both match, let user into the program
- If not, prompt the user to try again

For the login page, the validation of the password and username is key. The input username and password will be taken and compared with the privately stored username and hashed password. To do this, the password must be hashed using the same hashing algorithm and then compared with the private password. Only if both the input username and password match the private username and password, will the program allow the user to access the main program. Outputting success and error messages are good for usability and make the program feel more robust and therefore I will implement these features.

This validation algorithm is to ensure that only the user can access their data. This program will contain event data and other potentially sensitive data which can be linked back to the user and therefore, user validation will be essential in keeping this data protected. The real username and password will be stored in the database. However, the password will be hashed before it is stored to prevent hackers who compromise the database to login using the stolen credentials (the password will be unreadable).

Sign-up Page:

- Take username, password, and confirmed password input
- Compare the password and confirmed password
- If they are the same, hash the password
- Store the username and password as private variables

- If not, tell the user that the passwords don't match and prompt them to enter it again

For the sign-up page, ensuring that what the user entered is correct is the key aspect. The password and confirm password inputs are compared. If they are the same, the password input will then be hashed and stored privately in a class. If they are not, the user will be told that the passwords do not match and that they should try again. Implementing this feature is helpful to the user as it allows the user to confirm the credentials that they used to create their account. If the user accidentally entered the wrong password, without a confirm password option, they wouldn't be able to access the account. This is important for usability as an incorrect signup would ruin the user experience (as they would not be able to use their preferred username as it is already in the system) and implementing correct signup validation would help the program be more robust in nature.

Hashing and storing the password privately prevents the risk of the password being used for illegitimate login attempts. Even if it was compromised, they would find the hashed password which couldn't be reversed to find the original (as I will be using industry standard hashing algorithms which are built into PHP) which ensures the maximum level of security. Hashing is also a feature present on all of the account creation system for the other similar projects I researched in the analysis stage. Therefore, to create a suitable program and to maximise user security, I will need to implement this feature.

Calendar/Input Form:

- The month is navigated to
- The day button is pressed, redirecting the user to the input form
- The details of an event are inputted into a form
- The form results are stored in the SQL database

The calendar page will contain the calendar data of all of the months in 2026. The month data will be dynamically generated by pressing next/previous month buttons which will generate a calendar system with buttons arranged in the format of the month data for 2026. To store an event in the calendar I am thinking of making each day node in the calendar a clickable button, which would redirect the user to an input form for the specified day. This would allow for modularity, as I would only need to code the input form once. I could pass the date information of the node to the form on each redirect, so that the form could display the date as the title (a helpful usability feature) and so that it could input the data in the database with the correct date. Furthermore, implementing an input form would allow for abstraction and simplification of the calendar GUI which would also be great for usability.

- The day button is pressed
- The database is queried for event data corresponding to the day
- If data is present, it is stored in the input fields

The input form will allow for value retention of input event data. It will do this by querying the database each time the input form is redirected to for the corresponding day and output the data, if present, as the values of the input fields. These values could then be changed/updated to allow the user to correct/add data which they previously entered which is a useful usability feature to implement and is helpful in displaying the data.

- The tag value is taken from the database
- The node in the calendar is coloured using the colour value associated with the tag

The calendar page will allow for colour coding as it was one of the essential features listed in the success criteria. To allow for this, I will query the database to retrieve a list of data regarding the tagged events (date of event and tag of event) which I will pass to the function creating the calendar nodes which can change the nodes' class/id to allow for colour coding. Implementing colour coding would satisfy a major user requirement and improve stakeholder satisfaction.

- The current date-time data is taken
- The earliest entry that has a date-time greater than or equal to the date time data is retrieved
- If the date-time data is the same, output "(Event name) is Today"
- If the date-time data is one less than the earliest retrieved value, output "(Event name) is Tomorrow"
- Else, output "(Event name) is on the (date time data of the earliest retrieved value)"

This outlines how a potential notifications system will work. It will fetch the data of the next event in the calendar and will output a message by comparing its date-time data with the current date and time. However, this feature will only be implemented if time constraints allow for it as I would like to perfect the other essential features to satisfy the stakeholders.

Skills page:

- Every record that has been tagged with one skill is found

- These are then sorted chronologically using the date-time data
- Each event name is displayed with a text box containing the stored description value underneath
- Next skill

The skills page uses the data stored in the database that relates to skills. For a given skill, it retrieves all the entries in the database that have been tagged with that skill. Then it will sort them chronologically and display them with a text input field underneath. Once all the entries have been displayed, the same will be done for the next skill. This will be repeated until all the entries for all the skills have been displayed.

The skills description feature is an important feature as it allows the user to provide evidence for the event data that they have input. Implementing this feature successfully allows for one of the main criteria of the project to be included and would also allow my project to become more robust and a good alternative to the current other options.

Timeline:

- Every record is retrieved and sorted chronologically
- The name of the event and the date-time data are displayed alongside a textbox containing the stored reflection value
- This is repeated until all the records have been displayed

The timeline is quite simple as it only requires a chronological sorting algorithm and displaying of the event name and date-time data in that chronological order. I will need to find a way to add text boxes after the event name is displayed as these text boxes will store the user reflections which was a feature requested in the user questionnaire in the analysis section. The event reflection values stored in the database will be retrieved and displayed on the timeline page to allow the users to view and edit their stored value. The value inserted into the database will be cleaned (which is a process that changes code which is entered into fields, rendering it useless) to prevent potentially malicious users from injecting code into the database.

- The div is converted into an image
- This image is then placed on a pdf with number of pages dependant on image size
- The pdf is downloaded on the user's computer

The save as PDF feature is an important desirable feature for the users. This feature would allow the user to have a copy of all of the events they have entered into the database which would be helpful for their reference and to attach to a cv. This will be implemented using jspdf and html2canvas which will allow me to convert a div into a pdf.

Subroutines

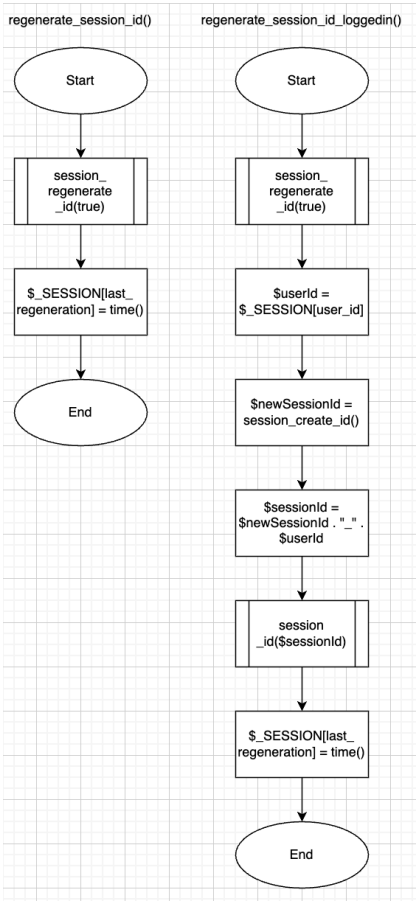
Now that the overall program has been planned, I will start planning the subroutines that I will be using. The program will be made using 4 types of files: HTML, CSS, PHP, and JavaScript. As the HTML and CSS files outline the structure and design of the website, they are not planned to contain any subroutines unless any issues arise during testing and development. Having separate JavaScript and PHP files would keep the code more organised as any HTML tags that need to be linked to any procedures can tagged with ids and the files can be imported using 'require' or by using 'script' tags.

Most of the unique functions necessary to be coded will be for the Login, Sign-up and Calendar pages as these require validation, storing dynamic generation algorithms. The input form page, skills page and timeline page will all require searching and sorting algorithms. These features are built into most DBMS systems; however, I will still plan backup algorithms in the case of errors in the linking of the website to the database.

Pseudocode and Flowcharts

I have included pseudocode for the primary functions of each page. I have included flowcharts for the complementary algorithms that have been created specifically for this program as they may be harder to follow.

Session Variables



The first algorithms I will plan will be the algorithms which create the session variables. I have designed these variables based on the 'PHP Guide for Beginners' by Dani Krossing linked in the bibliography.

First, the regenerate_session_id function calls the session_regenerate_id function with the parameter of true which updates the session id with a new one. The session variable of last_regeneration is then set to the current time. This function will be called when a time limit has passed. If the user is on the account creation pages (they are not logged in) this function is called which allows session variables to be created. This is useful as it will allow me to store the user id across all of the pages of the program which will allow me to retrieve the data associated with the user when they are attempting to login.

Next, the regenerate_session_id_loggedin function is a modular variant of the previous function. This function creates a more secure session id by generating the session id and appending the user id to it. This helps improve the security of the program as the session id will be harder to crack by malicious users while the user is logged in. This function has been implemented as further security is required while the user is logged in as the user will be inputting event data which could potentially be intercepted if the correct security measures aren't taken.

These functions allow me to have secure session ids which allows me to create session variables safely without the risk of interception of user data. They form a key part throughout the program in allowing me to store the user id across the pages for database querying without risking it.

Signup

I will implement two types of validation functions. The first functions will be made with JavaScript implementation in mind and have been designed based on the validation suggestions from the analysis stage. The alternative implementation will be based on the signup functions from video 28 from Dani Krossing's 'PHP Guide for Beginners' which goes over best practices when implementing a signup page. The initial functions will be implemented first and if issues occur, I will switch to the second iteration implementation.

Username Validation

Username validation will check the input username for certain special characters associated with SQL injection attacks and will prompt the user to enter a new username if their input username contains those characters.

```
procedure username_validation()
        username_valid = false
```

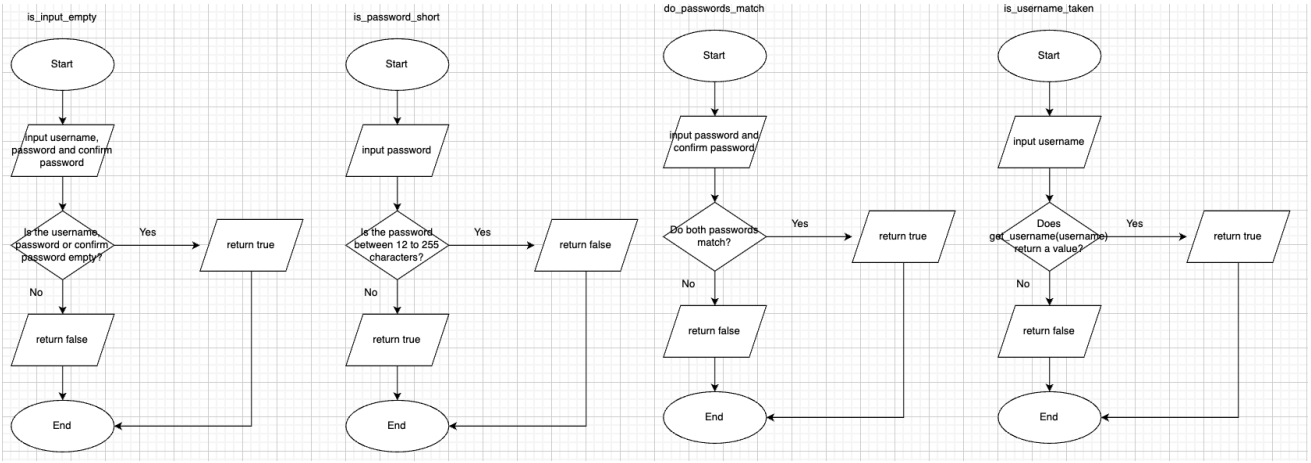
```
        special_characters = "-';/*"
        while username_valid != true
                count = 0
                inpUsername = input("Enter your username: ")
                for each letter in inpUsername
                        for each char in special_characters
                                if letter == char then
                                count = count + 1
                                endif
                        next char
                next letter
                if count > 0 then
                        print("Username should not contain special characters: - ' ; / *")
                else
                        username_valid = true
                        print("Username is valid")
                endif
        endwhile
endprocedure
```

Password Validation

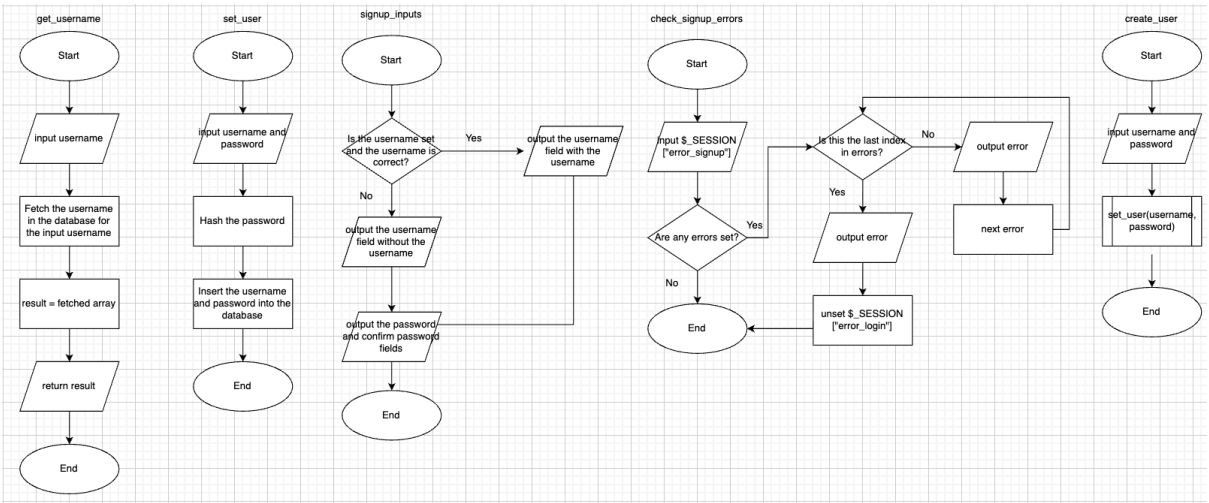
Password Validation will take two inputs, the sign-up password and confirmation password. Firstly, the password needs to be greater than 11 characters in length and if it is, both the password and the confirmation password need to match. If not, one of two messages will appear: “Password should be greater than 11 characters” or “Passwords do not match”.

```
procedure password_validation()
        password_valid = false
        while password_valid != true
                inpPassword = input("Enter your password: ")
                inpConfirmPassword = input("Confirm your password: ")
                if inpPassword.length >= 12 AND inpPassword.length <= 35 then
                        if inpPassword == inpConfirmPassword then
                                password_valid = true
                                print("Password is valid and both passwords match")
                        else
                                print("Passwords do not match")
                        endif
                else
                        print("Password should be between 12 and 35 characters")
                endif
        endwhile
endprocedure
```

Alternative functions



These functions are all modular as they all check for if an error has occurred and if it has, a boolean value is returned. The functions check for: if any of the inputs are empty; if the password is too short; if both the passwords match; and if the username is taken already. These are common checks in most login pages. The empty inputs, password length and passwords matching are common errors and the return value of the functions will be used to output errors. The username check is a helpful feature for the user and also helps prevent any clashes in the database based on the username.



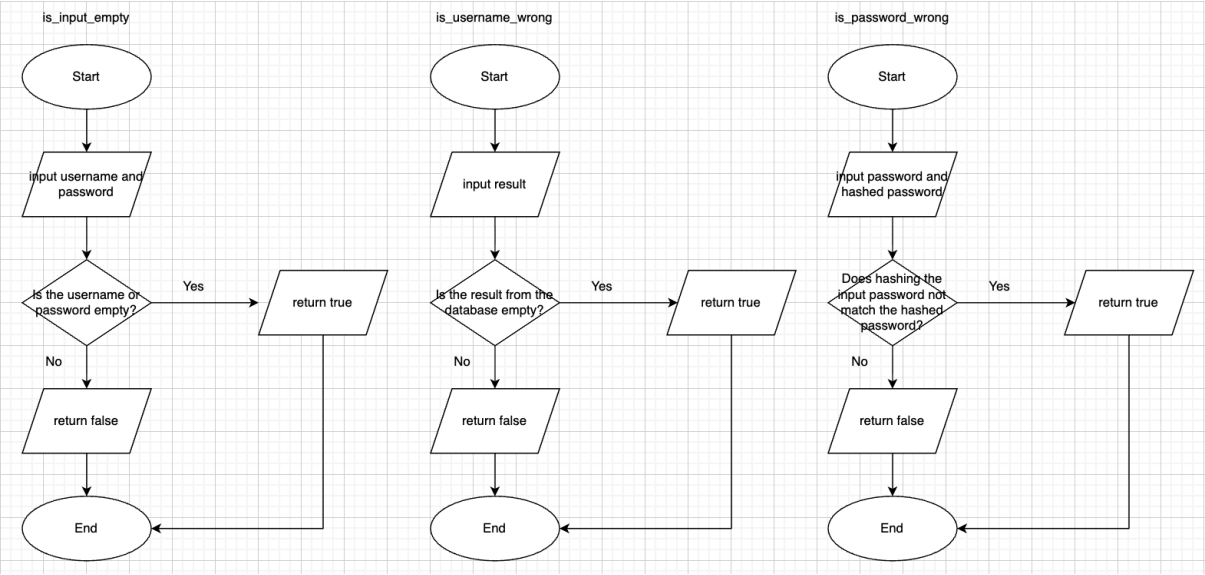
These functions deal with the interactions with the database and the outputs to the user. The get username function relates to the is_username_taken function and if there is a result when querying the database for the input username, true is returned in the function. The set_user function hashes the password to increase the security of the data and inputs the username and password in the database. This function is called by the create user function which is called when no errors are output in the program. The signup_inputs function helps implement a good usability feature. If the username was input and there were no issues with the username (however, there were other errors), the username field contains the username when it is output. This is helpful for the user as it allows them to know that the username is valid and it makes it so that the user

does not need to retype the password every time an error occurs. The check_signup_errors function takes in a session varible which stores all of the errors which were flagged by the previous functions and outputs them to the user. This allows the user to know exactly what error has occurred and allows my program to be more robust.

These functions allow me to implement a fully functional signup system with input validation, error message output, database insertion, hashing and a good usability feature. This would form part of the accounts system of the program, allowing the users to easily and securely create an account which they can use to log in to the system which gives the user peace of mind that their data will only be associated with their account.

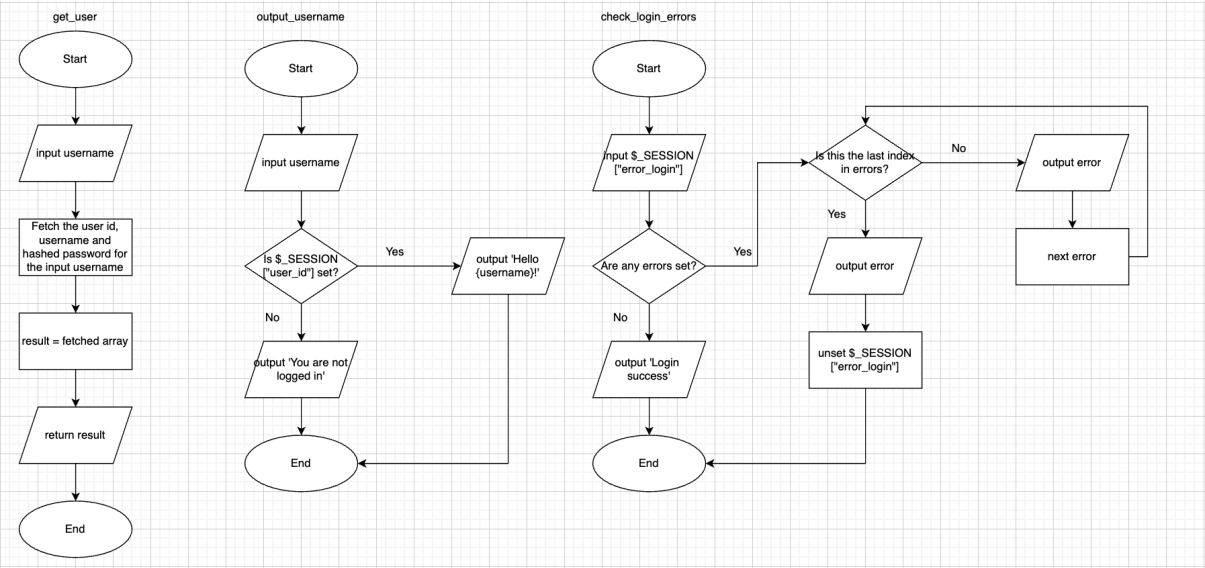
Login

The login system will be implemented with the reference of video 29 in the 'PHP Guide for Beginners' by Dani Krossing which will inform me of the best practices of the implementing a secure login system with PHP.



Firstly, these functions (similar to the error checking functions for the signup page) check if a certain error has occurred, if it has, true is returned. These functions check for: if any of the inputs are empty; if the username is wrong (this value is retrieved from get_user); or if the password is wrong. The return values of these functions will be used to inputs errors corresponding to the values in an array which will be passed to the check_login_errors function. The is_password_wrong function will use the password_verify function which is built into PHP which hashes the input password and compares it to the hashed password to see if it matches. These functions allow for

input validation which helps implement security for the system, making the program more robust.



Next, these are the functions which interact with the database or output messages to the user. The get_user function takes the username that the user input and checks if it is present in the database which allows for efficient username validation. The output_username function is a nice feature which makes the program feel more robust. If the user is logged in, it outputs a hello message which makes the program feel more responsive for the user. The check_login_errors function takes the errors which were set due to the return values of the error checking function and outputs them to the user. This helps the program feel more responsive and lets the user know which error has occurred.

These functions allow for input validation, error message output and responsive message output which helps create a functional and secure login system and helps complete the account creation pages. The message output helps the program feel more responsive and robust and be a nice feature to implement for the client.

Calendar

Suffix

```
function suffix(n)
        if n >= 11 and n <= 13 then
                return "th";
        elseif n % 10 === 1 then
                return "st";
        elseif n % 10 === 2 then
                return "nd";
        elseif n % 10 === 3 then
```

```
                return "rd";
        endif
        return "th";
endfunction
```

Firstly, this is a simple function which will help output the suffix value for each day cell in the calendar based on an input, n, which is the day. This helps make the code more modular as the suffixes for each month doesn't need to be manually set by implementing dictionary for example. The return value of this function will be passed to the create_day_cell function to add to the value on the day cell.

Create day cell

```
function create_day_cell(day, monthName)
        const td = document.createElement("td");
        td.className = "day";

        const form = document.createElement("form");
        form.method = "GET";
        form.action = "inputform.php"; /

        const hidden = document.createElement("input");
        hidden.type = "hidden";
        hidden.name = "date";
        hidden.value = day + suffix(day) + " of " + monthName;

        const submit = document.createElement("input");
        submit.type = "submit";
        submit.value = day + suffix(day);

        submit.className = "day-button";

        form.appendChild(hidden);
         form.appendChild(submit);

        td.appendChild(form);

        return td;
endfunction
```

This function returns a td value which contains the submit button and hidden value feature (which I researched on W3Schools) which allows each day in the calendar to be used as a redirect button to the input form with the date value of the button. This td value will be used to create each of the day cells for the calendar and will be called in the final calendar creation function. Using a hidden input prevents unnecessary clutter and allows the value of the day to be sent to the inputform page without the user

needing to manually do anything except for click on the button which is good for usability.

Calendar Creation

I will implement multiple functions to create each of the months dynamically. I have included the example for January. Each function contains a different heading content value, a different passed value to the create_day_cell function and a different rows array.

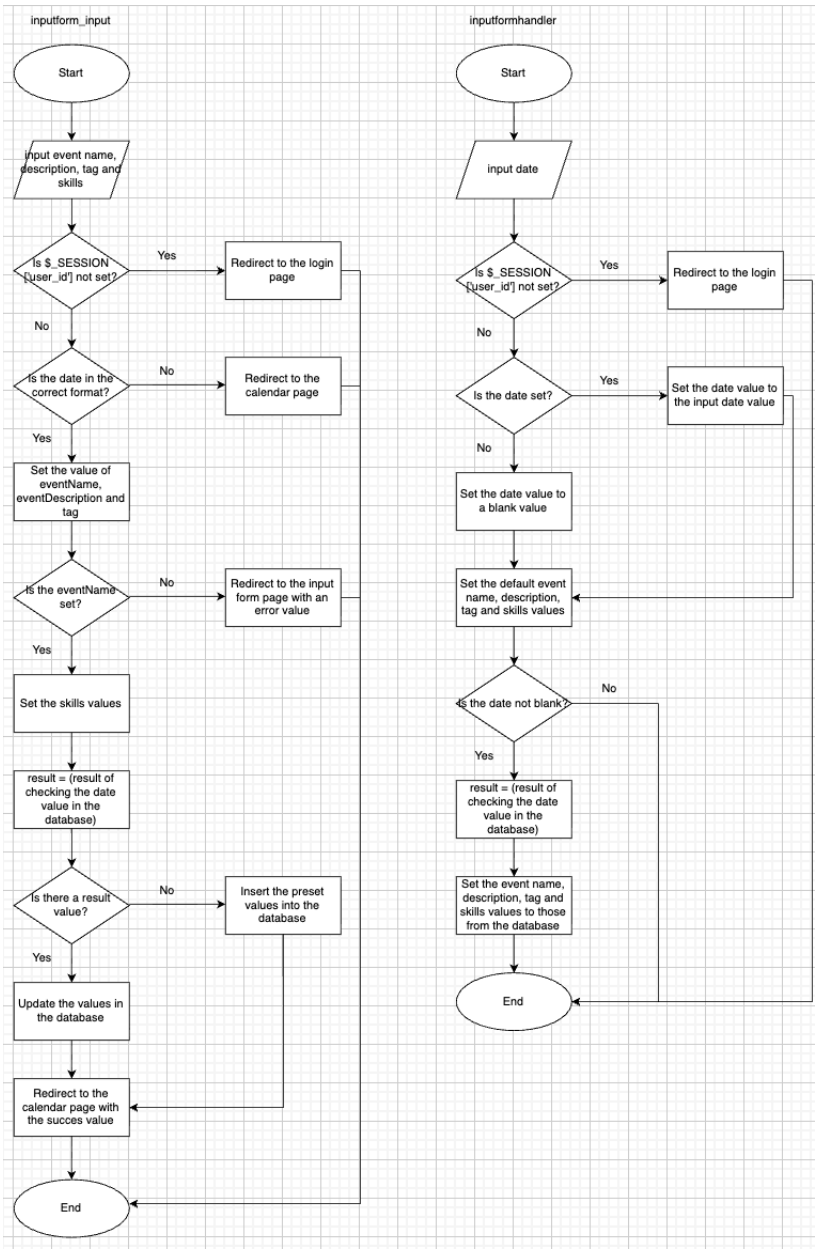
```
procedure replace_January()
        const container = document.getElementById("calendar-container");
         const heading = document.createElement("h2");
         heading.textContent = "January 2026";
        const table = document.createElement("table");
         table.className = "calendar-table";

        const rows = [ [null, null, null, 1, 2, 3, 4],
                       [5,6,7,8,9,10,11],
                       [12,13,14,15,16,17,18],
                       [19,20,21,22,23,24,25],
                       [26,27,28,29,30,31,null]];

        for i = 0 to rows.length

                const tr = document.createElement("tr");

               tr.className = "calendar-row";
                for j = 0 to rows[i].length
                        if rows[i][j] === null then
                                const td = document.createElement("td");
                                td.className = "calendar-cell";
                                tr.appendChild(td);
                        else
                                tr.appendChild(create_day_cell(rows[i][j], "January"));
                        endif
               next j
               table.appendChild(tr);
        next i
container.replaceChildren(heading, table);
endprocedure
```

The function helps dynamically create the calendar system row by row by calling the create_day_cell function for each of the days in the rows 2d array. The rows array contains the calendar data for each month formatted correctly which is used in a for loop to create each day cell. This function will be called each time the next/previous

button is pressed for the corresponding function and will allow the program to contain a dynamically generated, interactive calendar.

These programs help form the dynamic generation of month data for the calendar page. The calendar page was one of the 3 main pages identified in the analysis stage and will serve as the homepage for the program, therefore, implementing these functions allows the page to be generated correctly and successfully.

Input form



These functions are related to the storage and retrieval of event data to and from the database for the input form. The first function takes the input data as an input and performs validation on if the user is correctly logged in or if the date value is not

correctly formatted to prevent unwanted users from accessing the data for malicious reasons. After the validation checks are complete, the values of the variables to store the event details are set to the input values from the input form. The page performs a validation check to see if the event name was input, if not, an error message is output. Otherwise, the skill values are set, and the database is queried to see if a result already exists. If it does, the record is updated, if it doesn't the record is inserted into the database. The program then redirects the user to the calendar page with a success message. The second function is responsible for setting the values of the input fields. A validation check first occurs to check if the user is legitimately accessing the file, if not, they are redirected to the login page. Next, the date value is checked which is another form of validation. If the date is not set, values are not assigned to the input fields which prevents user data from being output if the page was accessed illegitimately. The default values are then set, and the database is queried for the set date to see if values are present. If they do, they are set as the new values, overwriting the default values. These values can then be accessed by the input form to be displayed on the page.

Radiobuttonoutput

```
procedure radiobuttonoutput(tags, x)
        for i = 0 to tags.length
                const title = document.createElement("label");
                const node = document.createTextNode(tags[i]);
                title.appendChild(node);
                const input = document.createElement("input");
                input.type = "radio";
                const element = document.getElementById(x);
                const br = document.createElement("br");
                element.appendChild(br);
                element.appendChild(title);
                element.appendChild(input);
        next i
endprocedure
```

This function allows for dynamic generation of radio buttons for event tags using an array of tags and the id of the tags title where they should be inserted under. This creates a label which outputs the tag name and a radio button which allows the tag to be selected allowing for colour coding.

Thes functions help form the complete input form page and allow for dynamic generation of tag radio buttons, input handling of form data, and input field value retention. These features allow the user to add events to the calendar and store them in the database and allow them to view the event details and colour code the event within the form. These functions help the user in populating the calendar and the database with events which will be accessed by the calendar and other pages.

Skills page

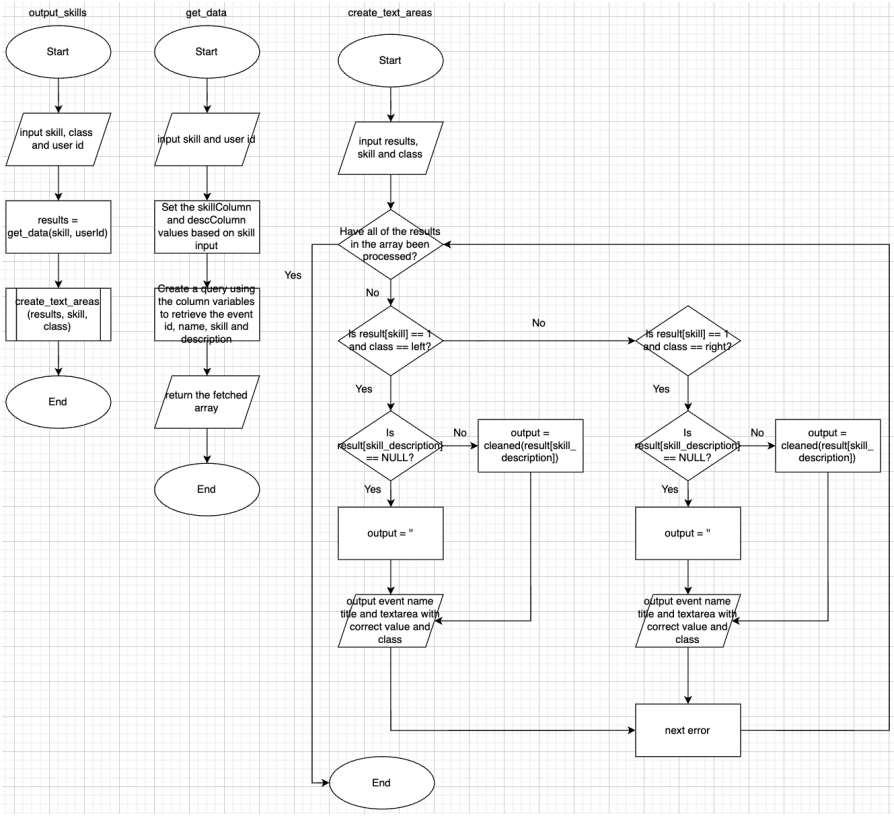
I will also implement two types of textbox output functions, one based on JavaScript and one based on PHP. I will initially implement the JavaScript function and will only implement the PHP functions if the JavaScript function does not pass the tests.

Textboxoutput

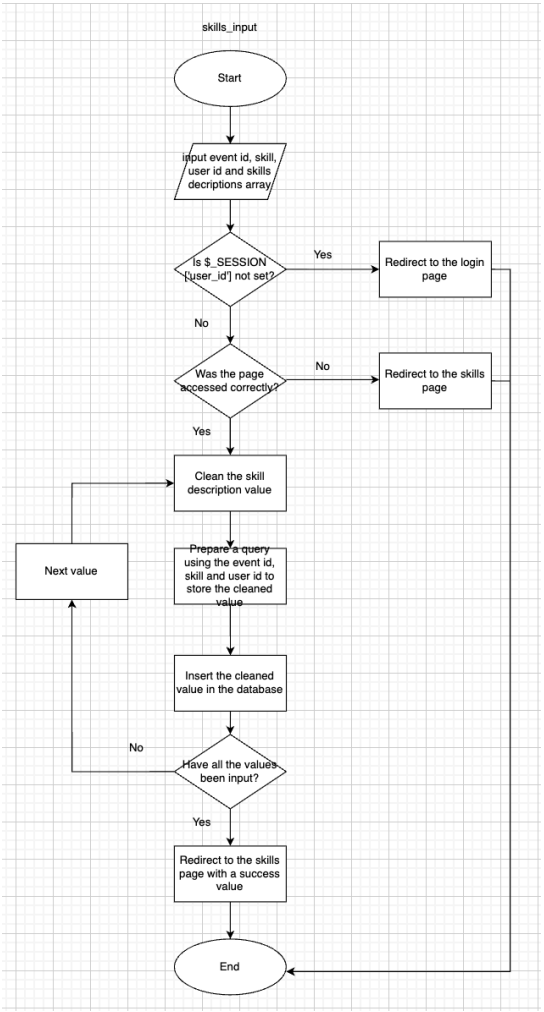
```
procedure textboxoutput(tagged_arr, x)
        for i = 0 to tagged_arr.length
                const title = document.createElement("h3");
                const node = document.createTextNode(tagged_arr[i]);
                title.appendChild(node);
                const text = document.createElement("textarea");
                const element = document.getElementById(x);
                element.appendChild(title);
                element.appendChild(text);
        next i
endprocedure
```

This function takes an array of events tagged with skills and the id of the skill title as an input. The function then creates a title element for the event name and a textbox for the skills description which is output underneath the skill. This helps in the dynamic generation of the skills page as an array of the events tagged with a skill can be passed to the function which will generate textareas and titles for them even when more events are added to the database.

Alternative functions



An alternative implementation with PHP in mind would be using these functions. Firstly, the get data function fetches the skills and skills description data for the input skill and returns it. This return value is then assigned to a variable ‘results’ in output_skills which passes the array to the create textareas function. The function uses a for loop which checks if the skill was flagged as shown and checks for the class passed to the function for styling. I would like to implement an alternating design which will help make the program feel more robust and modern. The program then checks if the description value is blank (if so, outputting a blank textarea), if not, the value from the database is inserted into the textarea and the title and textarea are output with the correct styling.



This function outlines how the skills descriptions will be input in the database. The description values will be stored in an array and passed to the function alongside the skill value. The database will then be updated with the cleaned value of the description, and this will be repeated until all of the descriptions have been uploaded. Uploading the cleaned value of the descriptions prevents malicious users from uploading code to potentially cause a SQL injection attack as the code becomes harmless once cleaned. The user is then redirected to the skills page with a success value which can be used to output a success message which would make the program more responsive.

These functions allow the skills page to be dynamically generated allow the input values to be stored in the database. These functions help complete the skills page and allow the user to provide evidence for the skills they have tagged the events with which is a key feature of the program and one of the 3 main pages planned in the analysis stage.

Timeline page

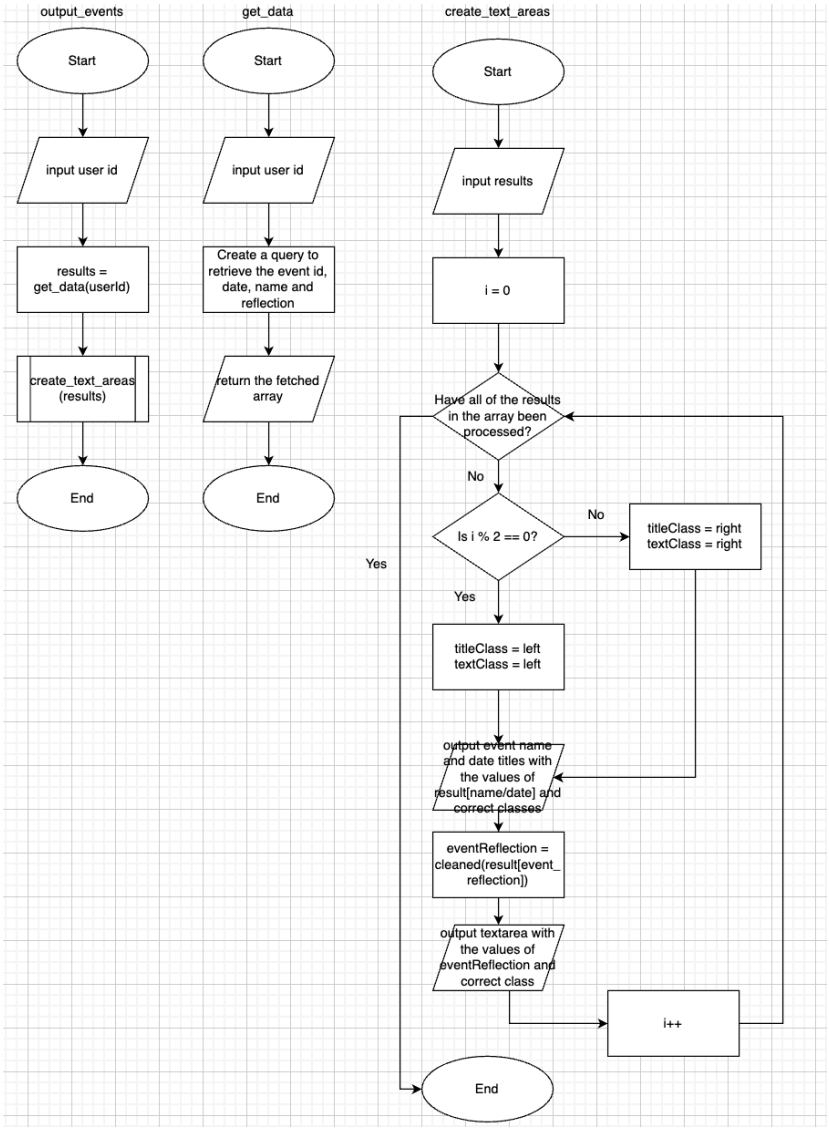
I will also implement two types of textbox output functions, one based on JavaScript and one based on PHP. I will initially implement the JavaScript function and will only implement the PHP functions if the JavaScript function does not pass the tests.

Textboxoutput

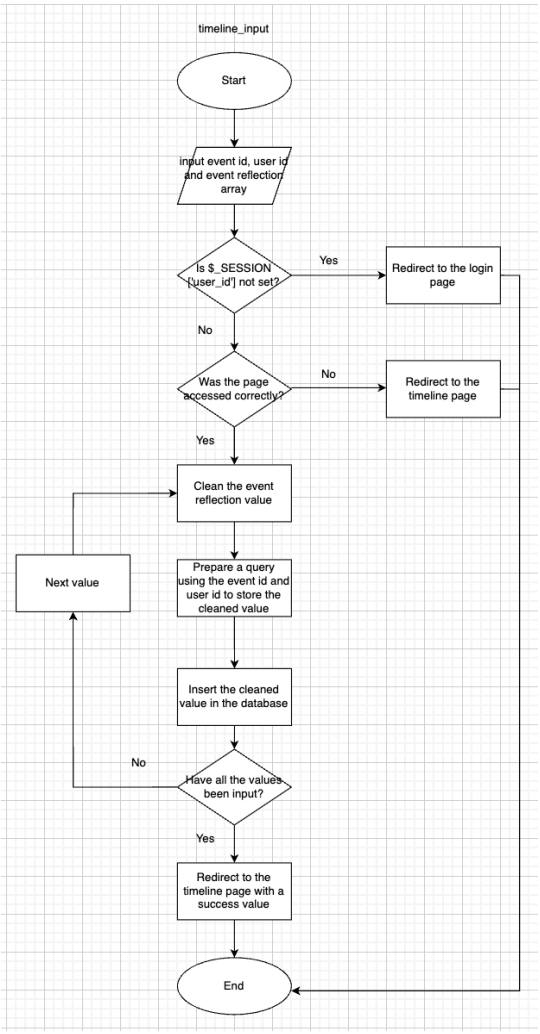
```
procedure textboxoutput(tagged_arr, x)
        for i = 0 to tagged_arr.length
                const title = document.createElement("h1");
                title.htmlFor = tagged_arr[i];
                const node = document.createTextNode(tagged_arr[i]);
                title.appendChild(node);
                const text = document.createElement("textarea");
                text.id = tagged_arr[i];
                text.name = tagged_arr[i];
                const br = document.createElement("br");
                const element = document.getElementById(x);
                element.appendChild(br);
                element.appendChild(title);
                element.appendChild(text);
        next i
endprocedure
```

This is the timeline textbox output function which is modularly based on the one present in the skills page but contains a few more additions such as adding breaks in between the events. As the timeline will contain every event in chronological order (passed to it by the tagged_arr array), I will need to space out each event to reduce the clutter of the page and improve usability. This function will allow me to dynamically output all of the events present in the database with their name and date (as stored in the array) as a title and the textbox in which the user can input their event reflections.

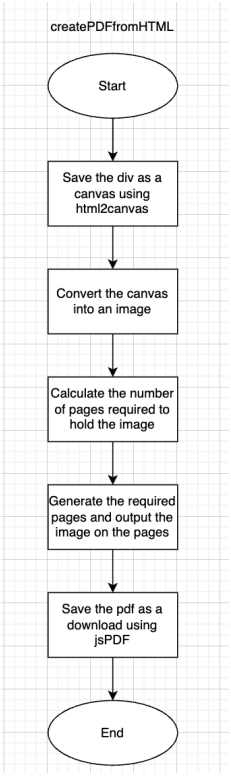
Alternative functions



Thes functions allow for an alternative implementation of the textboxoutput with PHP in mind. The reflection data is first fetched from the database by the get_data function which takes the user id as input and queries the database for the required values and stores them as an array. This array is assigned to variable results in the output_events function which then passes the results to the create_text_areas function. This function takes the results as input and implements an alternating design (similar to that of the skills page but each event alternates in sequence rather than being assigned a class. The event name, title, and event reflection textarea with the cleaned value are all output to the user with the correct styling and value.



This function outlines how the event refection values will be stored in the database after they are taken as input from the timeline page. The event id, user id and an array of event reflections are passed to the function. The function then performs two validation checks to see if the user is logged in (user id is set) or if the user accessed the page legitimately which prevents unwanted users from accessing the page. The value of the event reflection input is cleaned, then it is input in the database based on the event id and user id and then this is repeated for the other reflection values. Finally, the user is redirected to the timeline page with a success value which can be used to output a success message, making the program feel more responsive and robust.

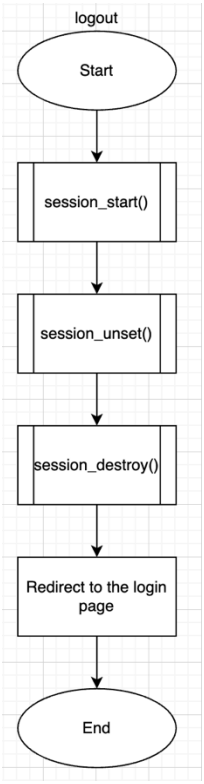


Finally, this function implements the desirable feature of saving the timeline as a PDF. This will be done using the html2canvas and jspdf modules which I have researched into on stack overflow. The function requires turning the div into a canvas using html2canvas, converting that canvas into an image, and pasting that image on a pdf with the required pages for it to be saved as a pdf using jspdf. Implementing this feature would be helpful for the user as it would allow them to have a reference of all of their events and would make the timeline page feel more robust.

These functions help me create a working timeline page where the events, dates and event reflection textareas are dynamically generated based on the data in the database; the reflection values can be updated/retained for the users to use to reflect on all of the events they have done; and the timeline can be saved as a pdf for the user. This also allows me to implement one of the 3 main pages planned in the analysis stage and allows me to implement two essential features from the success criteria and 1 desirable feature.

Navigation Bar

There is one algorithm that is present on the navigation bar and that is the logout algorithm.



The logout function first starts/resumes the session of the user when they are logged in to the program this is to prevent any issues with a user whose session expired or is currently running. The session variables are then unset to prevent access of these variables from malicious sources as I will use session variables to store the user id for retrieval of data from the database. Finally, all of the session data associated with the session is destroyed (as a form of security) and the user is redirected to the login page.

This is a valid logout algorithm as all of the user data which have been stored as session variables are securely unset/destroyed to prevent unnecessary access from malicious users. This function completes the program as a whole as it adds a logout feature to the program which coincides with the login feature of the program. Having a logout function allows the user to securely exit the website without needing to just close the browser tab. It also reassures the user that their data cannot be accessed by any other user of the same device.

This function allows me to implement a robust logout feature and also allows me to add further usage to the navigation bar which is an essential feature to the program as identified on the success criteria.

Backup Algorithms

These algorithms are efficient searching and sorting algorithms which will help with handling the data fetched from the database if there are any issues with data accessing. These functions will only be implemented if the returned result from database queries are in a format which require further searching and sorting.

Linear Search

Linear search is one of the searching algorithms I may implement if there are errors in the searching feature of the DBMS. It checks a list of items, one item at a time, and compares each item with the desired item. Once the desired item is found, its value and index are output. This will be useful when I am searching for an item by its name or by any non-integer attribute.

```
function linear_search(A, x)
        i = 0
        while i < A.length
                if A[i] == x then
                        return i
                else
                        i = i + 1
                endif
        endwhile
        return “Not found in data”
endfunction
```

In this case, A is a list, and x is the desired item.

Linear search works by checking each item in a list, one by one, from the beginning until it finds a match or reaches the end of the list. It starts at the first element, compares it to the item being searched for, and if it's not a match, moves to the next item and repeats the check.

Binary Search

Binary Search is the other searching algorithm I will be implementing in this project. It relies on divide and conquer and works by eliminating halves of a list by comparing the desired value with the middle value of the list. It is incredibly efficient; however, it is only easily implemented when searching by an integer attribute as the list passed to it needs to be arranged in ascending/descending order.

```
function binary_search(A , x)
low = 0
high = A.length - 1
        while low <= high
                mid = ( low + high ) / 2
```

```
                if A[mid] == x then
                        return mid
                elseif A[mid] > x then
                        high = mid - 1
                else
                        low = mid + 1
                endif
        endwhile
        return “Not found in data”
endfunction
```

In this case, A is the array of integers ordered in ascending order and x is the desired item.

Binary search compares the desired value to the middle element of the array. If they are not equal, the half in which the target cannot lie is eliminated and the search continues in the remaining half. The middle element is compared to the target value again, and this is repeated until the target value is found.

Bubble Sort

Like the searching algorithms, the sorting algorithms are only required if there are issues with the built-in sorting of the DBMS. I will plan them nonetheless as planning for potential failures is a key part of web development.

```
function bubble_sort(A)
for i = 0 to A.length - 1
for j = 0 to A.length - 2
                        if A[j] > A[j+1] then
                                temp = A[j]
                                A[j] = A[j+1]
                                A[j+1] = temp
                        endif
                next j
        next i
        return A
 endfunction
```

In this case, A is an array of integers (which may or may not be sorted).

Bubble sort works by repeatedly iterating through a list, comparing adjacent pairs of elements, and swapping them if they are in the wrong order. This repeats in “passes” until the entire list is sorted.

Insertion Sort

This is an alternative searching algorithm planned which performs similarly to bubble sort in terms of time complexity.

```
function insertion_sort(A)
        for i = 1 to A.length - 1
                elem = A[i]
                j = i –1
                while j >= 0 AND A[j]  > elem
                        A[j+1] = A[j]
                        j = j – 1
                endwhile
                A[j+1] = elem
        next i
endfunction
```

In this case, A is an array of integers (which may or may not be sorted).

Insertion sort works by creating a sorted sublist (starting from the left of the array) one element at a time, inserting each new element from the unsorted portion into its correct position within the sorted part.

Quick Sort

It is a sorting algorithm that implements recursion to sort an array and therefore its average case time complexity is better than Bubble and Insertion Sort. As JavaScript allows for recursion, I have chosen to implement this.

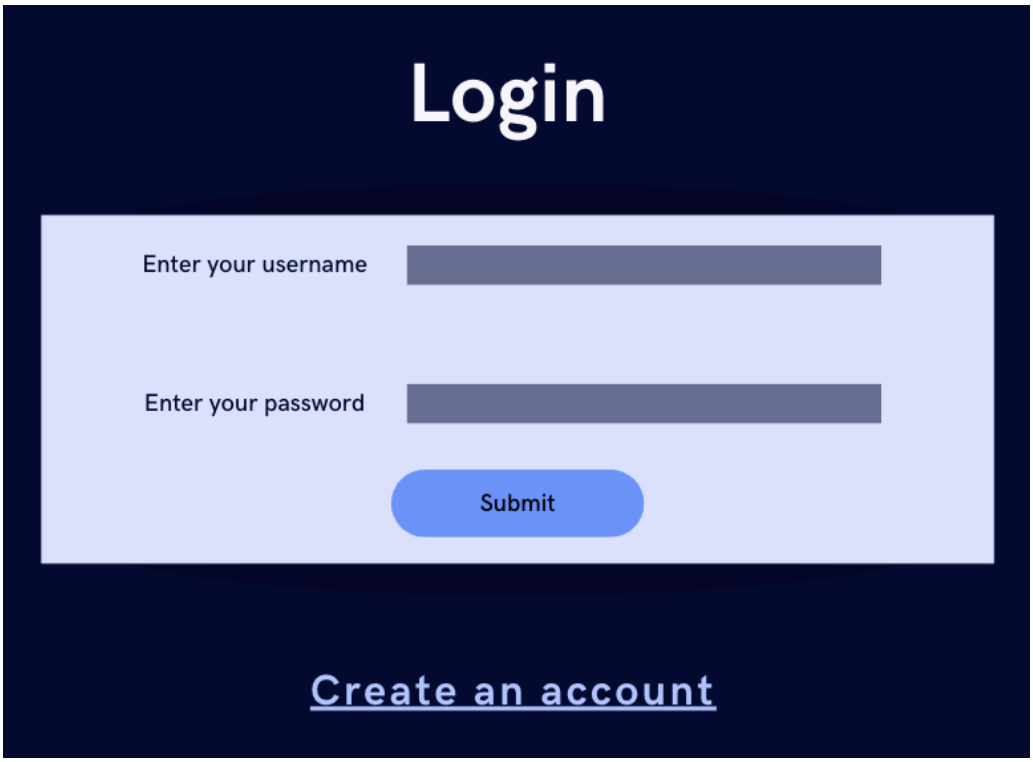
```
function quick_sort(A)
        if A.length <= 1 then
                return A
        endif
        pivot = A[0]
        left = []
        right = []
        for i = 1 to A.length - 1
                if A[i] <= pivot then
                        left.append(A[i])
                else
                        right.append(A[i])
                endif
        next i
        sorted_left = quick_sort(left)
        sorted_right = quick_sort(right)
        return sorted_left + [pivot] + sorted_right
endfunction
```

In this case, A is an array of integers (which may or may not be sorted).

Quick sort works by recursively selecting a pivot element and splitting the array around it, so that elements smaller than the pivot are to its left and elements larger are to its right. The process is repeated on the two smaller, unsorted sub-arrays (to the left and right of the pivot) until the entire array is sorted.

User Interface

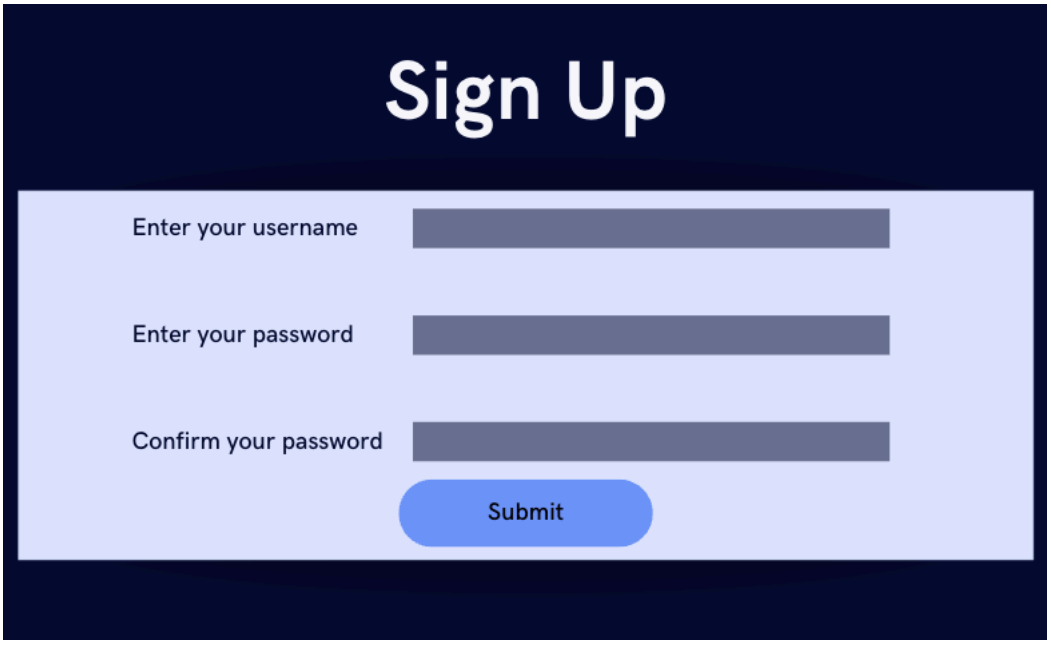
Login Page



As this program deals with the client's plans and calendars, I have chosen to implement a Login page. This allows the user to secure their data and allows me to group user data through creating a user-by-user storage database. This will be useful as the project is intended to be used by multiple clients, so maintaining separate accounts and grouping user data is key. Due to the scale of the project, it may be somewhat difficult to implement this validation system within the time limit. However, as data security is a key priority in web development and part of the Data Protection Act 1998, I will take the time to implement this.

I have chosen to make a large submit button and to colour the input fields to contrast from the background as quality-of-life features. As my project is targeted towards 6th form students, which is a very broad range, I cannot be sure if my users may have conditions such as colour blindness or other eyesight issues, therefore I have implemented these features to account for it.

Sign-Up

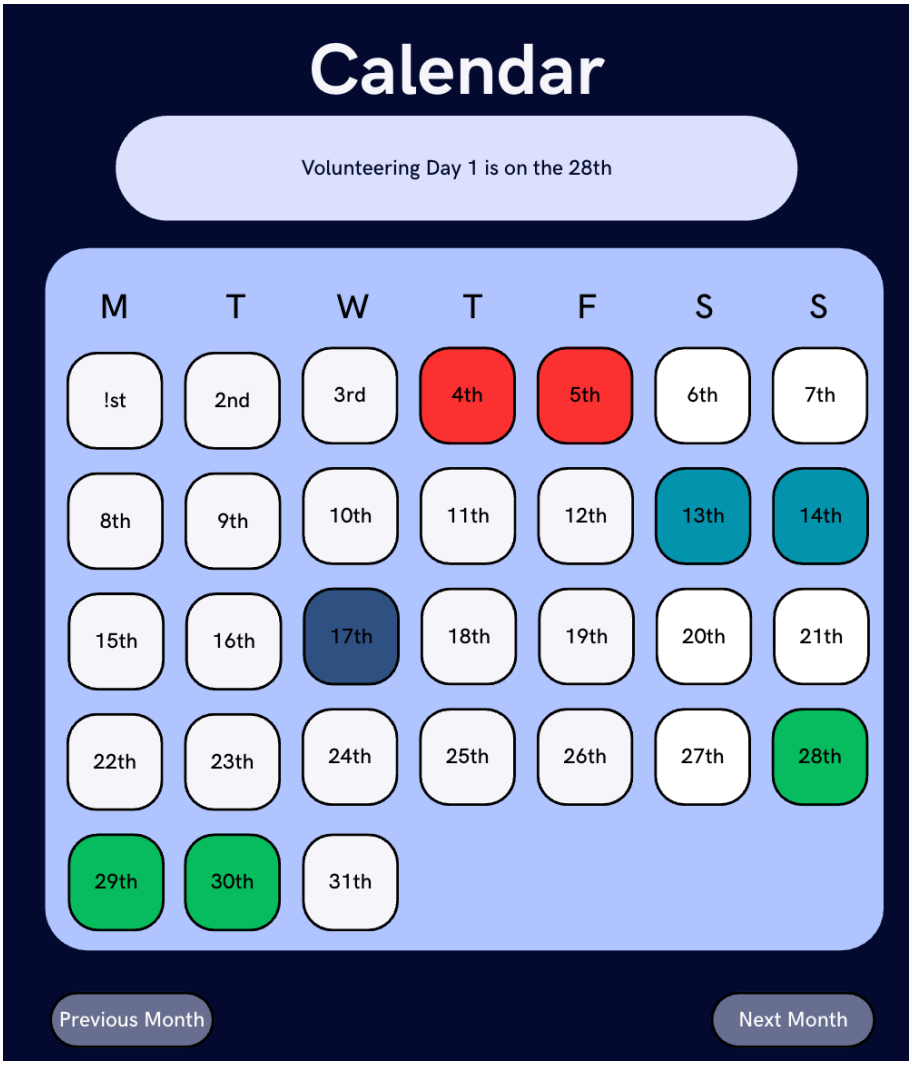


A sign-up page is key in any account-based system as it allows users who do not have accounts, to create them. This will be useful, again, as it allows the user's data to be grouped and stored by their username (and primary key e.g. user_id) as their data will be stored in a server-side database.

The username will be checked for certain characters as webpages that store data in a database face the risk of SQL injection attacks, therefore I will be implementing an algorithm for this purpose. The password input fields will not be checked as they will be hashed before they are stored in the database and I will not allow the banned characters to appear in the hashed password.

The password confirmation feature is a commonly used feature on most account creation menus as it allows the user to check for any unintentional mistakes that they may have made. It would allow the clients to have peace of mind as they set their passwords and is a useful quality-of-life feature which would be easy to implement.

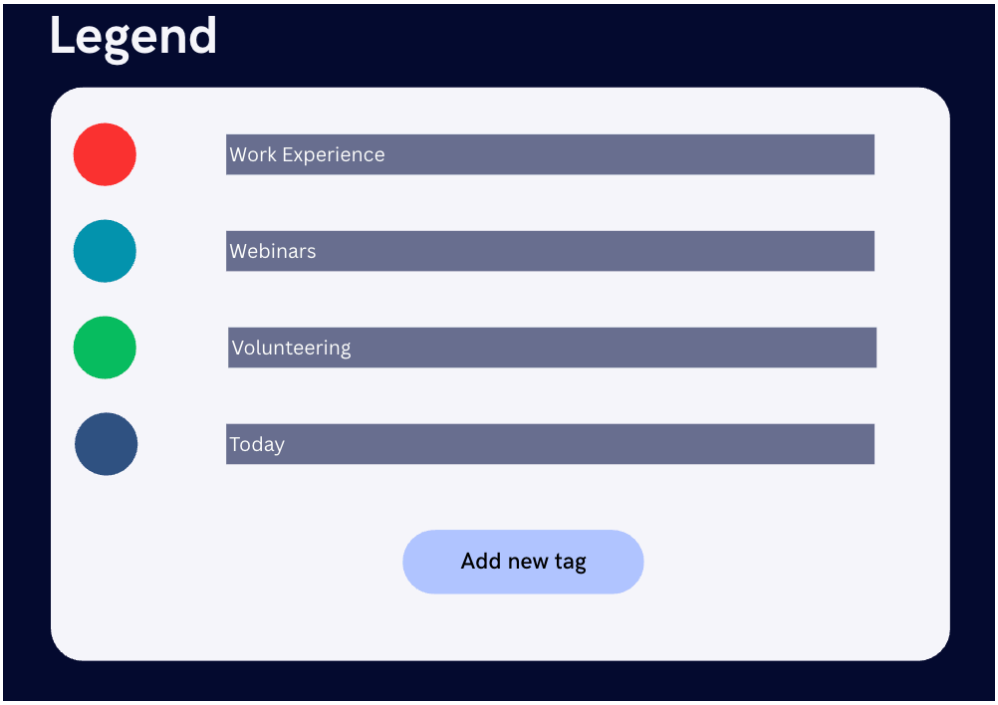
Calendar



The calendar is the main aspect of this program, as the other pages stem off the information input into this. The calendar is organised like most calendars: it includes indicators for days of the week, highlights the current day using date-time data, and allows for scrolling between months of the year.

I will try to implement a notifications system, as it was one of the suggested features by the clients in the first questionnaire, which will check if there is an event on the day, tomorrow, or later than that and display a message based on it. If an event is taking place on the day the user is on the site, the message will be 'Volunteering Day 1 is Today' (Volunteering Day 1 being an example name of an event). If there is an event tomorrow, the message will be 'Volunteering Day 1 is Tomorrow'. Or if the event is on a day other than those, it will appear as on the diagram including the date of the event.

Legend



The events in the calendar will be able to be colour coded and to allow the user to accurately remember what each colour means, I have implemented a ‘Legend’ feature on the bottom of the calendar page.

It will contain a colour which can be either user picked from a colour picker (if time permits) or will be selected from a set list of colours. Next to the colour will be an input field in which the user can write what each colour represents. I would like to implement this feature as it was a highly requested feature in the questionnaire and will allow the clients to structure and organise their calendar in a way that would help them easily remember what events are upcoming, even if the calendar doesn’t explicitly state it.

Input Field

This is an example overview of how the input field/display screen for one day in the calendar will look like. It will contain an input field which will store the event name; an input field to store general information about the event which may be helpful in remembering and describing the event; a checkbox set for the skills shown; and a radio button group for the colour coding tag.

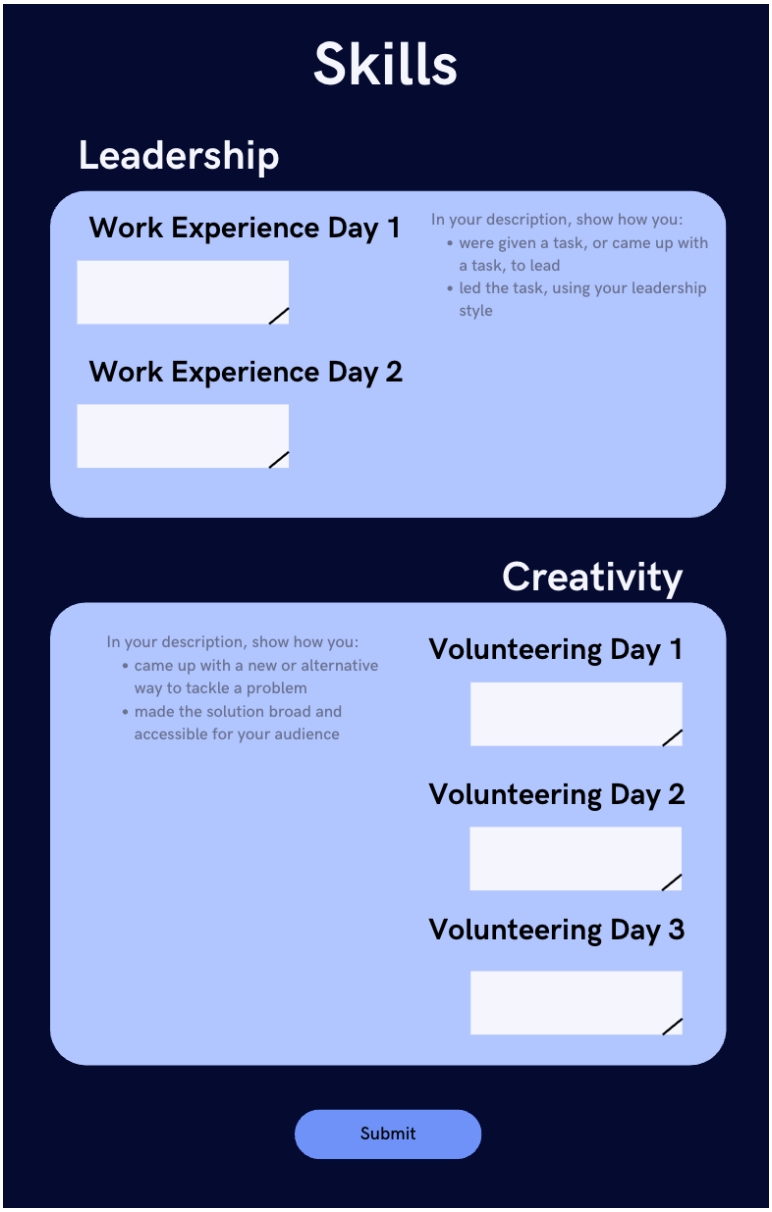
I have included a fixed sized input field for the event name as it promotes the user to only include the name of the event. This website is designed to help students during their university applications and interviews for jobs. During these processes, the interviewer/reviewer is less interested in details such as the full name of the event or where the event took place. There is a much bigger emphasis placed on skill demonstration and personal reflection and therefore, by limiting the input space, it reminds the user to focus less on trivial details.

I have still included a variable input field for these events as it directly targets some client's difficulty to remember past events. Certain details that may be important in reminding the client of the event can be included in this input field.

I have chosen to use checkboxes for the skills as the clients can display various skills during events they attend and therefore it is important to highlight how they have developed a variety of skills rather than focusing on one skill per event.

I chose to use radio buttons for the colour coding due to the scale of this project. Currently, it would be challenging to implement a colour coding feature where one event can be represented multiple colours due to the time constraints associated with coding the frontend in HTML, coding the backend in PHP/JS and learning how to link the HTML/CSS frontend to PHP/JS.

Skills

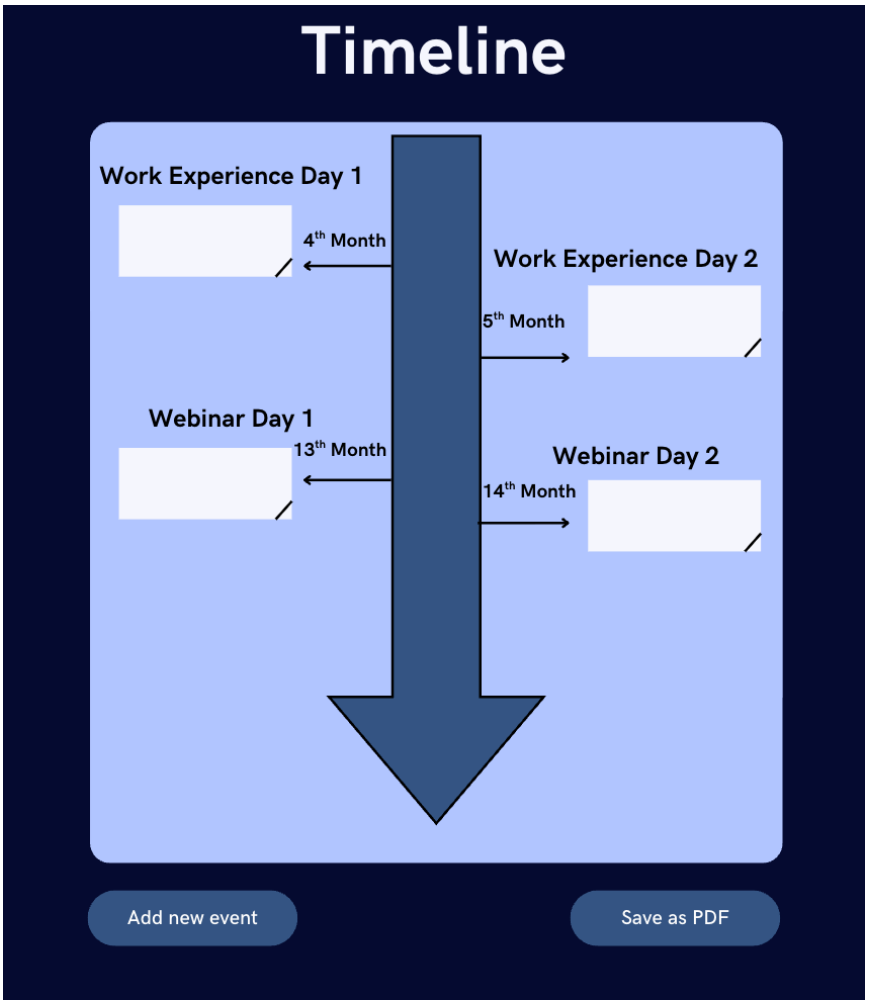


I chose to implement the 'Skills table' idea as more of a 'Skills page'. I have chosen to implement it like this as a table would require having a set word count for each skill

description input field. I feel that would be restrictive if the word count was too low or the page would be visually unappealing if the word count was too high. Implementing an input area would allow the user to type as much or as little as they would like without it being restrictive to them.

The skills page would check the calendar for events and which skills they have been tagged with. It would check from the oldest to latest inputted events. If an event was tagged with multiple skills, it would be added to all the skill areas that correspond to the event's tagged skills. This would allow the user to describe how they demonstrated each skill that they developed by attending the event. These descriptions could then be used during application processes or memorised for interviews.

Timeline



The timeline feature is the final main feature of the program which allows the user to have a visual overview of all the events logged in the calendar. It will search the calendar for events and display them in the form of oldest to latest which will provide the user with a visual reminder of all the activities they have completed. It will display

the name of the event, the date of the event, and an input field for a brief reflection on the event.

In the stakeholder questionnaire, there was a great emphasis on reflecting on the events the user has participated in and noting what went well and what could've been done better for future. The variable input field would allow the user a place of self-reflection, which is incredibly important during 6th form as it is a very important time for preparation before going to university or applying for an apprenticeship or job.

Usability Features

The usability features I have considered for this project help make this project easy to use for any of my potential users. I have chosen to make submit buttons a contrasting colour to the main background of the program to make them more distinguishable. Furthermore, I have chosen to implement a rounded shape for all buttons to further characterise them and make them easy to spot.

The text size must be big enough for all users to comfortably read it and therefore I have chosen to display details such as event names in a large font and to make the titles of each page in a very large font. Having the titles in a very large font, subtitles (such as the skill names or the legend) in a large font, and having event names in a medium font establishes a proper heading hierarchy which improves website readability for users.

Regarding the input fields, I have chosen to colour them differently based on whether they should be shorter or longer inputs. This variation in colour will be a helpful sign for the clients on how much they are expected to write alongside the size of the actual input field. Shorter inputs such as username and password will be coloured with grey backgrounds and white text, however text areas for skills descriptions and event reflections will be white with black text which would further help differentiate longer and shorter inputs visually.

I have also opted for a lighter-coloured box behind every important area of the webpage as it will help contrast the designated section of the webpage with the darker background and help draw the user's attention to these areas. I have chosen to do this to make the website more aesthetically pleasing and to specifically highlight key areas of the website.

Another usability feature that I have chosen to implement is the short example text in the skills page which helps tell the user what exactly they should write in each area. I chose to implement this as, without it, users may be confused on what kind of details they should include or what examples would be relevant to focus on in a skills description.

Stakeholder Feedback

As I now have a clear idea on how the website will look and function, I thought it would be useful to check this proposed design with the stakeholders to see if they are happy with the design or if they would like to recommend any changes.

My message to the stakeholders

“Hi,

Please find attached the proposed designs for the website that I am making.

The grey rectangles represent a fixed-size input field (fewer characters can be written) whilst the white rectangles with the lines in the bottom right represent input areas which can change in size (more characters can be written in them, such as paragraphs).

I have included the popular suggestions such as a notifications system and colour coding on the calendar page.

I have not included the navigation bar on the design as it will be on top of all pages and contain the name of each main page.

The page titled ‘28th of Month’ is meant to represent how adding an event to the calendar would look like when clicking on the specific date.

Also, due to the scale of this project I will only be able to add a finite number of skills. I am thinking of adding up to 15 different skills (most likely 10 unique skills). I have added ‘Leadership’, ‘Creativity’, ‘Teamwork’, and ‘Initiative’. Do you have any suggestions for changing the ones I have added or on any more I should add?

Thank you for your time.”

The replies

“The skills page looks useful for work experiences. I like the colour coding. Would it be better to put the colour coding on the same page as the calendar, such as in the top corner? The timeline would be very helpful for planning. So far, it looks like it is going well”

“For skills, you could add on analytical skills, communication skills and preparation skills (and maybe some academic skills like numerical or essay writing but to be honest I don’t think those skills are developed much in extracurriculars). The UI looks clean, if I had to change/add something I would 1) include the month and year on the calendar just to navigate between months easier 2) add more to the skills/creativity slide as I think there is some empty space that can be filled 3) for timeline, I personally would like the dates written in the DD/MM/YYYY form but that’s just my preference. Otherwise just

the month and year would be fine. 4) for the slide with 28th in the top, for the tag options I would add on another option as “other”.”

“This looks very modernistic and professional which I very much appreciate. I think it would be beneficial to get the colour coding and the calendar on the same page because the user may not want to constantly flick back and forth between pages”

“The UI looks good but slightly odd. Maybe try something more contemporary by removing the boxes and just having it on blank white space. Some skills to include would be organisation, problem solving, communication, practical skills, math, coding, cad, sketching, public speaking”

“I like the instructions on what to do. The layout is nice as well; I enjoy the use of round shapes. Having the different colours for boxes depending on word count is nice. Maybe including a word counter could be a good idea, so that ppl can see how much they write but also to see the limit for the boxes? For tags maybe add a ‘problem solving’ one? ‘Time management’ might also be a helpful one. I like the save as pdf option on the timeline.”

Review of feedback

Overall, there seems to be some confusion on the placement of the legend due to me providing separate images for the calendar page and the legend. The legend will be a part of the calendar page, and I was thinking of placing it underneath the calendar, however I will now have to compare placing it on the top right due to the feedback.

The feedback on the design seems to be positive. Only one user had suggested changing the UI entirely, however many users suggested some small changes that would be useful for the program which I will try and implement.

The clients suggested a variety of skills to include and I have decided on adding an additional 11 skills: problem solving, organisation, communication, public speaking, perseverance, time management, planning and preparation, analytical skills, practical skills (CAD, sketching, DT), academic skills (numeracy, essay writing), and computer skills (coding, AI literacy). This makes 15 total skills that can be selected from in the skill selection checkbox.

Variables and Data Structures

I have created this table to store, describe and justify variables and data structures that will be used in the program. The variable names may change slightly during development to fit the naming conventions present while I am coding the functions.

Name	Description	Justification
$_SESSION	This is the session superglobal present in PHP. This superglobal will allow me to create session variables which are temporary stores of data which can be used within a session (which will be created once the user logs in).	This will be used to store sensitive data which needs to be present across all pages of the website (such as the user id). As the user id needs to be used on most pages to help with fetching of data from the SQL database, I have chosen to use a session variable to do this.
$_GET	This is the get superglobal present in PHP. This superglobal will allow me to send form data between pages by embedding it in the URL of the pages.	This will be used in the day cells of the calendar which will redirect the user to the inputform with the correct date (which has been sent using hidden inputs). As the date selected is not sensitive in nature, using a get variable is sufficient as the data does not to be hidden upon redirection.
$_POST	This is the post superglobal present in PHP. This superglobal will allow me to send form data between pages securely without embedding it in the URL.	This will be used to send most data to validation algorithms and formhandlers (such as username, password or event details). This is because those inputs are sensitive in nature and cannot be visible in the URL of the page to allow for security.
$dbhost, $dbusername, $dbpassword, $dbname	These are the database credentials which will be stored as strings.	These credentials are necessary to allow me to connect to the database. They inform the database of the type of host, the username and password of the user and the name of the database I would like to access. Defining these correctly would prevent any issues with database access which would ruin my program.

$conn/$pdo	This is an object which allows the user to connect to the database using MySQLi or PDO.	This object allows me to instantiate a connection using the database credentials which will allow me to interact with the database which is the main backend of the program.
$e	This is a string which stores the error value after an issue with the database occurs.	This variable is integral to debugging and testing the code. Outputting this variable in the case of an error would be helpful in informing me about what the issue with my database access is and would reduce the debugging time.
$sql/$query	This is a string which stores the SQL queries which will be used when querying the database.	Having a simple variable name such as this would be helpful when debugging the code and would also help future developers of the program when it comes to maintenance of database querying.
$result/$results	This is an array which stores the result(s) of querying the database.	Naming the results of querying the database 'results' would be incredibly helpful when debugging the code for me and future developers as the array can be easily passed to functions which require the results of queries.
inpUsername/$username, inpPassword/$password, inpConfirmPassword/$cpassword	These variables store the values of the inputted username, password and confirmation password.	I deliberately chose to name them inp(name) to distinguish them from the real values of the username and password to be fetched from the database.
$hashed_password	This variable stores the value of the correct hashed password in the database.	Having a clear name such as hashed_password would prevent confusion of values as there are

		many password related variables such as the login password and the database password. Stating clearly that this is the hashed value of the password from the database would allow for easy implementation and debugging.
username_valid and password_valid	Boolean values which represent whether the username and password meet the defined criteria.	This was specifically done because I need to ensure that the username and password are both fully valid (length, special characters, etc.) before I can halt validation. I have chosen to make these local variables stored within the function to prevent invalid credentials from being stored in the database.
special_character	A string that holds the special characters that are banned from being a part of the username due to associations with SQL injection attacks.	This variable has been made local to prevent unauthorised characters from being used in usernames by editing the value of this variable.
$eventName, $date, $eventTag, $eventDescription, $skills	These are the event details which will be taken as input on the input form page. The event name, tag and description are strings; the date will be a PHP/JavaScript date object; and the skills will be 0 or 1.	Clearly defining the variables which will be stored in the database in the input form helps with debugging and preventing clashes with data types present in the database. In particular, the skills values will be 0s or 1s to reflect the checked/unchecked checkbox values as Booleans which can be input in the database.
$descriptionClean, $reflectionClean	These are strings which will be present in the skills page and the timeline page. They will be cleaned	Cleaning the values before inserting them in the database is best practice and a good validation method to prevent

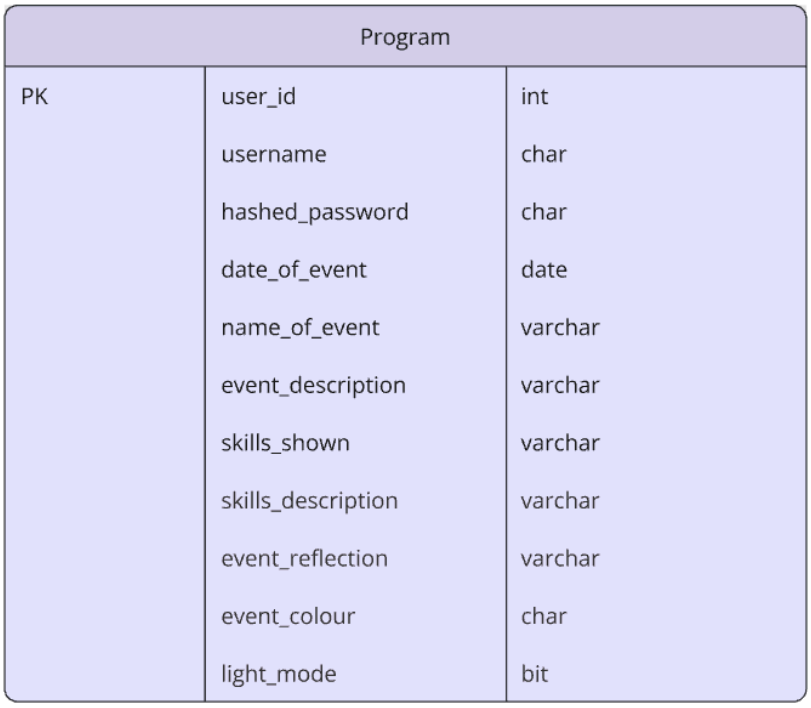
	values of the inputs using htmlspecialchars.	malicious users from inputting code in the database to attempt to perform SQL injection attacks.
rows	This is a 2D array which stores the calendar data for each month in each replace_(month name) function.	This is a useful 2D array as it stores the 2026 calendar data as an array with each sub array storing the data for each week. Empty days in the calendar are stored as null values to prevent accidental creation of month cells, and the days are ordered according to the respective calendar data for each function.
calendarData	This is a nested dictionary which stores the index of the first day of the month in terms of rows and the number of days in the month for each month.	This is a possible alternative implementation of rows if the replace_(month name) functions were to be implemented as one function. The starting index and number of days in a month are an incredibly simplified version of the rows 2d array, which are stored as a dictionary with the month name acting as the key value to that dictionary. This would allow me to implement the replace_month functions modularly as one function simplifying the code.
eventTags	This is a dictionary which stores the event tags associated with the dates of events in the database.	This dictionary will be a good way to implement a colour coding feature in the calendar by implementing a check within the create_day_cell function which could check if the date value is stored in this dictionary

		and to colour code the node if it is.
count	It is a counter which stores the number of times a letter of the username matches with a banned character.	A counter would be a simple way of checking if any banned characters appear as, if the value is greater than 1, there must be banned characters in the username.
i and j	They are incrementation counters for the loops contained within the algorithms.	These are commonly used and quick to type making for fast typing of array indices.
A	It is an array that contains retrieved data from the database. If it is an integer array, it is sorted and then searched for the desired value. If it is a string array, it can be searched using linear search.	It is useful as it allows for a temporary, changeable storage of retrieved data from the database and allows for the retrieval of linked data once the searching algorithm finds the index of the relevant data point.
low, mid, high	These are pointer values and represent indices in the array that the binary search algorithm is searching.	These are simple and easy to understand names which tell you that low is the bottom index, mid is the index of the middle value being compared to the desired value, and high is the upper index of the array.
temp	It is an integer that temporarily holds the value of an item in the array.	It stops the array value from being lost as it is being overwritten during the swapping process.
elem	This is an integer that stores the value of the item that is to be inserted into the correct position.	It is an easy to remember name that hasn't been previously used.
pivot, left, right	Pivot is the element in index 0 of the array. Left and right are lists that will store the separated values.	Pivot is the value used to compare the other elements in the array with and separates elements that are larger than it and smaller than it. Left stores the numbers that are smaller than or equal to it,

		right stores the number that are larger or equal to it.
sorted_left, sorted_right	Sorted_left and sorted_right are the results of using quick sort recursively on the left and right arrays.	Sorted_left and sorted_right allow for recursion in quick sort by performing quick sort on the left and right arrays which may not be sorted after being compared to the pivot value.

Entity Relationship Diagram

0NF



These are the attributes that I have identified to be key to the program's function.

Normalising this data would reduce data redundancy and improve data integrity, reducing the strain on the server. It would also allow for quick retrieval of data and increased security which would provide an improved user experience.

1NF

For a table to be in the first normal form, it must meet the following criteria:

- a single cell must not hold more than one value (atomicity)
- there must be a primary key for identification
- no duplicated rows or columns
- each column must have only one value for each row in the table

In the 0NF Database, the ‘skills_shown’ column would contain multiple skills the user at the event which would violate the first criteria. Therefore, this could be split into many individual columns which represent skills and hold a bit value if they have been shown or not, such as ‘leadership’, ‘creativity’, ‘teamwork’, etc. Alternatively, there could be a set number of columns for skills, and each skill could be added to that column, such as ‘skill1’ and ‘skill2’ varchar columns.

Program		
PK	user_id	int
	username	char
	hashed_password	char
	date_of_event	date
	name_of_event	varchar
	event_description	varchar
	leadership	bit
	creativity	bit
	skills_description	varchar
	event_reflection	varchar
	event_colour	char
	light_mode	bit

To maintain consistency in the code, I have chosen to implement the first option which would contain a column for each skill and hold a bit value (1 if shown, 0 if not) for each skill. This would allow for less emphasis on a set way of inputting the skills for a more user-friendly experience. However, as I need to define the number of columns required for this database, I will need to contact the stakeholders to agree on how many and which skills should be implemented.

This table already includes the primary key, ‘user_id’ and therefore that criteria has been met.

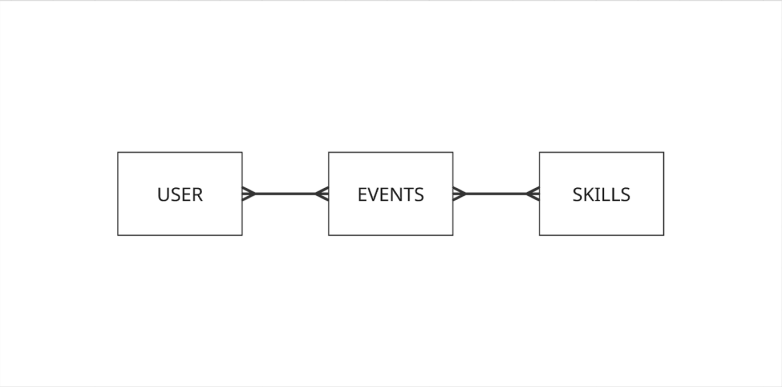
However, as many users can have many events and skills inputted, this table does not fulfil the third and fourth criteria.

2NF

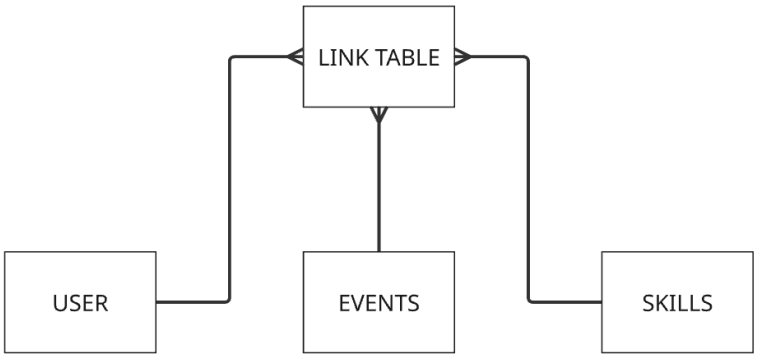
For a table to be in 2NF it must meet the following criteria:

- it's already in 1NF
- has no partial dependency, all non-key attributes are fully dependent on a primary key.

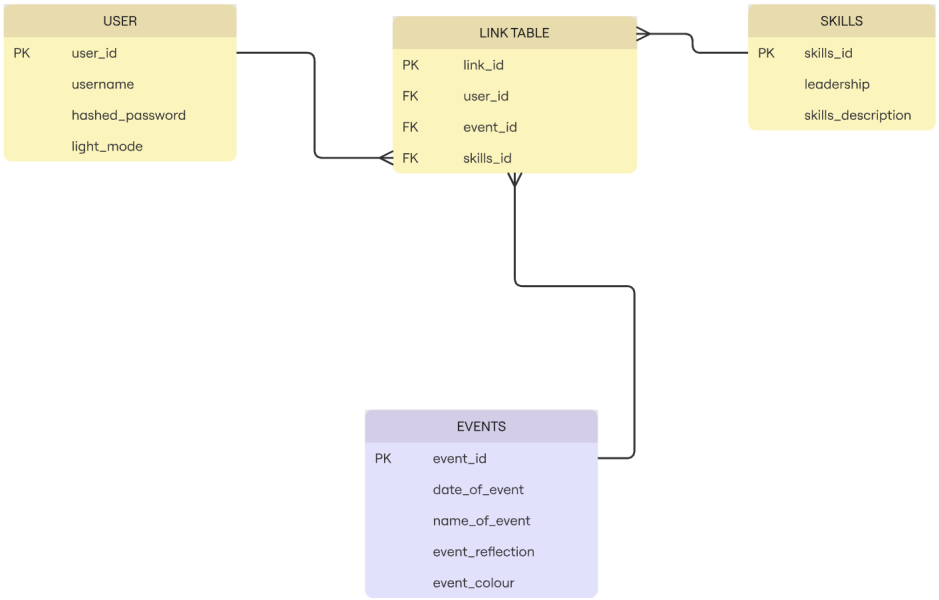
I planned on creating 3 main tables, the user table, the events table and the skills table. However, the relationships between these tables would cause an issue. As many users can have many events, and many events can be associated with many skills, this causes a many-to-many relationship between the tables.



I chose to split the table into 3 separate tables and include a link table to prevent the many-to-many relationship. This link table would have the primary key of 'link_id' to follow the 1NF criteria that all tables should have a primary key. Furthermore, it will include the primary keys of the user, events and skills tables as foreign keys so it can link users to events and events to skills without having many-to-many relationships.



The ERD of the table in 2NF would look like this. All attributes are dependent on the primary key, and every table is in 1NF. I have only included leadership in the skills table as a placeholder as I will discuss with the stakeholders how many and which skills I should include in the program.



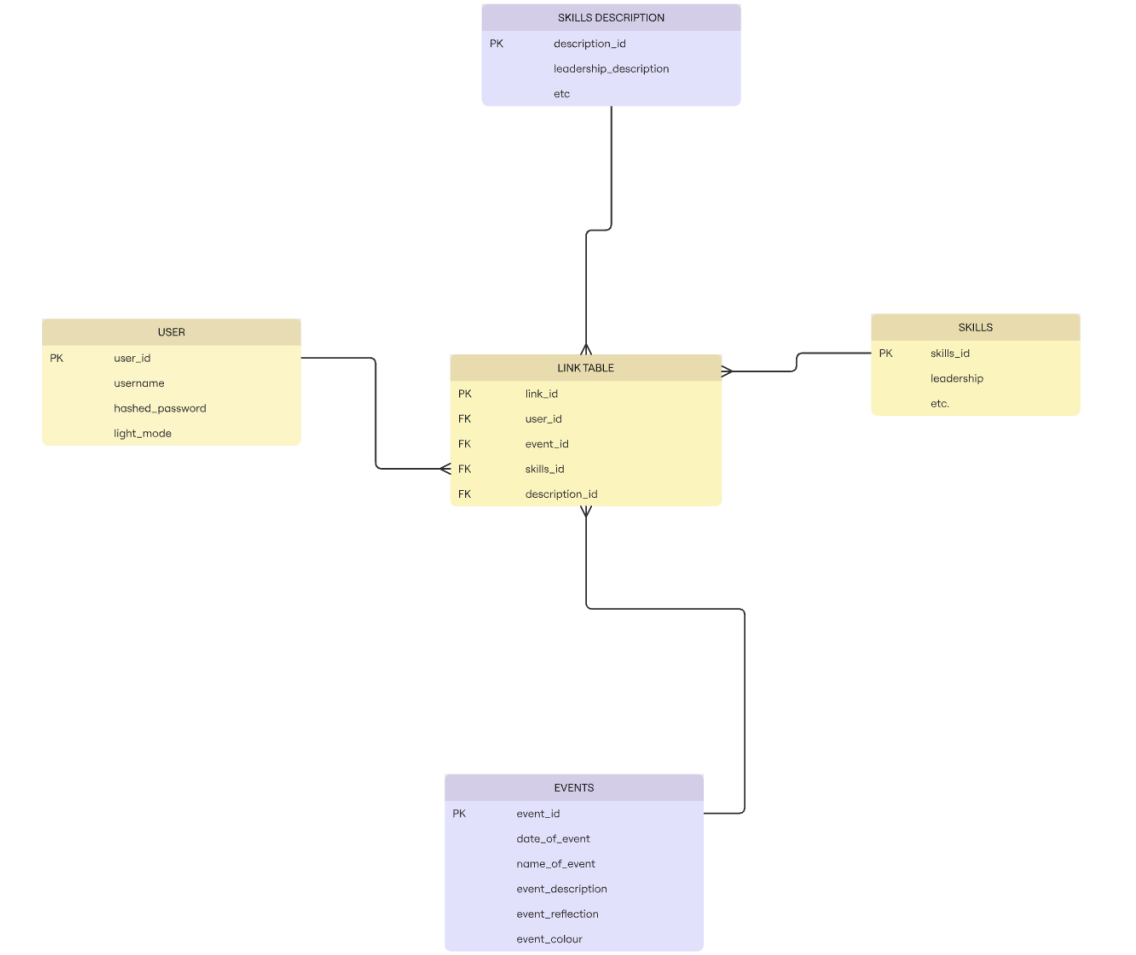
3NF

For a table to be in 3NF, it must:

- be in 2NF
- have no transitive partial dependency.

This means a non-prime attribute (an attribute that is not part of the candidate's key) is dependent on another non-prime attribute. This is what the third normal form (3NF) eliminates.

I realised that the skill description is unique to both the skill it is describing and the event it relates to creating a transitive partial dependency. Therefore, I decided to create a skills description table alongside the others and link it to the link table. It will contain 15 attributes excluding the primary key which will all be named (skillname)_description. If a skill value is 0, its corresponding description value will be Null.



This table is now in the 3NF and is ready to be tested and implemented.

Validation

Password Security

I will implement a validation algorithm that will prompt the user to create a strong username and password. I was initially considering prompting the user to ensure that their username and password should be between 8 to 20 characters and contain a mix of uppercase and lowercase letters, numbers, and special characters to prevent their password from being brute forced. However, after reading an article on the NIST (National Institute of Standards and Technology for the United States) Password Guidelines for 2020, I decided to change this requirement.

The article said that web developers should encourage users to use longer, memorable passphrases rather than shorter, complex passwords as increasing the length of a password, decreases the risk of it being guessed more than having a more complex password. For example, according to security.org, 'BLacKCofFEe15B!ttEr' would be

more memorable and would take a computer ‘5 hundred quadrillion years’ to crack compared to a shorter password with more special characters such as ‘S@£%&(‘ which would take a computer ‘1 hundred milliseconds’ to crack.

To implement this, I will prompt the user to input a password between 12-255 characters (255 is the maximum length of a varchar field in SQL) in length to ensure user password security.

SQL injection risks

The following information is from a Microsoft Article on SQL Server best practice:

“To minimize the risk of a SQL injection, consider the following items:

- Review any SQL process that constructs SQL statements for injection vulnerabilities.
- Construct dynamically generated SQL statements in a parameterized manner.
- Developers and security admins should review all code that calls EXECUTE, EXEC, or sp_executesql.
- Disallow the following input characters:
  - ; - Query delimiter
  - '- Character data string delimiter
  - -- - Single-line comment delimiter.
  - /* ... */ - Comment delimiters.
- Always validate user inputs and scrub error outputs from being spilled and exposed to the attacker.”

Therefore, to account for this, I will add a validation algorithm to the username input field to prevent those trying to breach the SQL server. It will check for the banned characters in the username and password and will ask the user to input a new username and password if the characters were detected.

The reason for not using the validation algorithm for the password field is because I will implement a backend hashing algorithm for the field. This is to prevent password compromise due to data interception. This hashing algorithm will not include any of the banned characters in any of the possible hashed passwords to prevent accidental breaching.

The reason as to why this validation algorithm will not be implemented for the other user input fields is due to the scale of this project. This project is aimed to be a working prototype of the idea; accurate validation flagging will be an issue if event names or descriptions contain the more commonly used banned characters such as an ' or ; for example. If I were to improve on this project in the future, I would implement a validation algorithm for all user inputs using logical flagging of event descriptions and names.

Testing Method

Test data to be used during development

The program functions entirely on the user inputs and outputs and therefore they must be clearly defined. When a user inputs an event name, for example, the event name will be saved and output on the skills page and timeline page. Therefore, I can easily plan the test data to be used to test these inputs and outputs.

Signup Page

Username Validation

The username validation functions check whether the username contains any special characters which are associated with SQL injection attacks or if the username is already taken. If errors are present, the user will be re-prompted to enter a username until it is valid.

Input	Expected Output	Justification
Usernames (excluding special characters) which are not already present in the database	"Username is Valid"/ "Account Created successfully" / Username remains in the username input if other errors present	A valid username with no special characters, which have not been taken already are valid and can be used for account creation. However, if there are other errors, such as an empty password field, the username will be retained in the username field which is a helpful usability feature.
Usernames including special characters	"Username should not contain special characters: - * / ; '"	This is a test output which I will only test for if I choose to implement the username validation function instead of the other error checking functions.
Usernames already present in the database	"Username already taken"	The program should output an error message if the username has already been taken in the database. This is to prevent any clashes when data is being retrieved from the database based on username which would cause errors.

Blank input	“Fill in all fields”	A blank input would provide no data to check against the database and therefore is deemed an incorrect username.

Password Validation

The password validation functions check whether the password is within the character limits set and whether it matches with the confirmation password. If both are true, then the password will be hashed and set.

Input	Expected Output	Justification
Passwords under 12 characters	"Password is too short"	This is to ensure security and to encourage the user to type a longer password as in the validation section, it was shown that a longer password is actually more secure than a shorter, complex password which would be decoded quicker.
Password and confirm password pairs don’t match	“Passwords do not match”	Implementing a confirmation password feature is helpful as it prevents the user from accidentally creating an account and forgetting the password. Also, a confirmation password feature is a feature present on most modern signup systems.
Password and confirm password pairs match and the password is greater than 12 characters	"Password is valid and both passwords match"/ “Account created successfully”	If both passwords match and are greater than 12 characters, the passwords are valid and the account can be created if the username is valid.
Blank input	“Fill in all fields”	A blank input would provide no data to check against the database and therefore is deemed an incorrect password.

Login Page

The login page checks the input username and password against the ones stored in the database to validate them; the program also checks for common mistakes like blank input fields; and the program will also output a hello message making the program feel more responsive.

Input	Expected Output	Justification
Blank input in either field	"Fill in all fields"	A blank input would provide no data to check against the database and therefore cannot be accepted by the login form.
Username or password do not match credentials	“Incorrect Login Info”	Incorrect username and password would both output the same message so any malicious users cannot attempt to brute force the credentials by waiting until the “Incorrect username”/ “Incorrect password” message changes. Having a general error message for both, reduces the risk of data breaches.
Both username and password match the database	“Login Success”	A simple success message upon login is a good feature which makes the program feel more responsive for the user.
Successful login to the website	“Hello (username)!”	Once the user enters the homepage after successfully logging in, a hello message will be output which will make the program feel more responsive for the user.

Calendar

The calendar system does not contain may inputs and outputs excluding the dynamic generation of the month data and the colour coding data therefore, I will test for these outputs.

The previous/next month buttons will be tested for if the correct previous/next month data will be output. For example:

Input	Expected Output	Justification
February is selected and 'next' is pressed	The data for March is output according to the 2026 calendar data.	The correct month data being dynamically output onclick will be helpful for users are implementing a full calendar on the homepage would clutter the page.
February is selected and 'previous' is pressed	The data for January is output according to the 2026 calendar data.	

Furthermore, the colour coding feature will also be tested to see if tagging an event in the input form outputs the correct styling for the day cell in the calendar. For example:

Input	Expected Output	Justification
Event is tagged on the input form (for example, tagged with 'Work Experience')	The day button is coloured with the colour specific to the tag on the calendar page (for example, red)	The colour coding feature was one of the most highly requested features by the stakeholders during the feedback sections. Implementing this correctly would allow me to satisfy stakeholder needs and to make my program more robust to match the researched alternatives.

Input form

Firstly, I will test if the input form for input data validation (if the event name was input) and if the input form allows for data retention as these are the main features of the input form.

Input	Expected Output	Justification
Correct event details without an event name input	"Event name is required"	This is to ensure that when an event is input in the form, if it is tagged with certain skills or for it to be output on the timeline page, the event name is required to be displayed. Therefore, implementing a required event name field

		would prevent accidental empty event names on the skills and timeline pages.
Correct event details with an event name input	"Event successfully stored/updated"	Having a message for successful input would make the program more responsive to the user. This also helps confirm to the user that the event data was successfully stored which is good for usability.
Clicking on a day cell with data previously input	The data from the database is displayed in the input form	This helps the user to view their input event data which they can then edit or use as a reference when looking at their stored skills.

Secondly, I will also test for the correct generation of radio buttons by the radiobuttonoutput function.

Input	Expected Output	Justification
Tags array is input in the function	Ordered output of the tag names as labels with the radio buttons on new lines with the correct styling. One button is selectable (as characteristic of radio buttons).	This allows for the dynamically generated tags to be output, matching the style of the rest of the program and also helps implement the colour coding feature efficiently regardless of the number of tags in the database.

Skills

First, I will test to see if the skills description textareas and titles are generated correctly by the functions as the main function of the skills page is to output the skills data.

Input	Expected Output	Justification
Events tagged with a certain skill passed to textbox output	Event titles and textareas output (textareas containing the stored description values)	This allows the user to view all of the events they have tagged with a certain skill and also allows them to add/update skills

		descriptions for each event which they can use to evidence the event skills data.

Next, I will test if inputting/changing the data in the textareas produces a suitable success message.

Input	Expected Output	Justification
Skills descriptions input in the textareas	"Skills Description successfully stored/updated"	This helps the program feel more responsive and allows the user to have a visual confirmation that the skills data that they input was input into the database correctly.

Timeline

First, I will test to see if the event reflection textareas and titles are generated correctly by the functions as the main function of the timeline page is chronological output of all of the events in the calendar.

Input	Expected Output	Justification
All of the events in the calendar/database	Event name and date titles and textareas output (textareas containing the stored reflection values) in chronological order with an alternating design	This allows the user to view all of the events they have in the calendar and also allows them to add/update event reflections for each event which they can use to make decisions on what to do better in future events they participate in.

Next, I will test if inputting/changing the data in the event reflection textareas produces a suitable success message.

Input	Expected Output	Justification
Event reflections input in the textareas	"Event Reflections successfully stored/updated"	This helps the program feel more responsive and allows the user to have a visual confirmation that their reflections were input

		into the database correctly.

Finally, I will test if the PDF of the timeline page is successfully downloaded with the CSS styling intact

Input	Expected Output	Justification
Save as PDF button is pressed	Timeline.pdf file downloaded on the user computer with the timeline div using the correct styling saved on it	This feature was one of the desirable features of the program and one of the features that I think would be helpful to implement for the stakeholders. Many stakeholders may need a copy of all of the events they have done for their own reference, so implementing this correctly with the correct formatting is essential.

Navigation Bar

The navigation bar allows the user to traverse between pages of the program efficiently and also allows them to logout of the program safely without directly closing their tab

I will test if all of the buttons correctly redirect the user to the specific pages they selected. Furthermore, I will also test if the logout button redirects the user to the login page.

Input	Expected Output	Justification
A button for the calendar page is pressed while the user is on the skills page (for example)	The user is redirected to the calendar page	This feature was one of the essential features of the program as it allows the user to travel between the main (non-account) related pages as there are no other planned redirects between pages. Implementing an unobtrusive navigation bar at the top of the page would be an efficient way of allowing for this.

The logout button is pressed	The user is redirected to the login page without access to the pages main program pages	Implementing a logout feature is necessary for all accounts-based systems as it allows the user to have peace of mind that their data cannot be accessed by any other users on the same device, therefore, this is necessary to test and implement.

Backup Algorithms

The backup algorithms will need to be tested so that they can be efficiently implemented in the case of database data being difficult to access.

Searching Algorithms

The searching algorithms will be tested in two cases, ones where the item being searched for is in the array, and one where the item being searched for isn't. I will also test linear search with an unsorted array as it is should be capable of finding the desired item even in an unsorted array. The algorithms should return the index of the item if it has been found or it should return "Not found in data".

Input	Expected Output	Justification
[1, 23, 134, 425, 3255, 6245, 7543, 9465, 10938] 10938	8	The searching algorithms will need to be able to efficiently locate individual pieces of data from the database array outputs and will also need to be able to output an error message if the data is not found, preventing any issues with functions such as textbox output outputting skills descriptions for events not in the database.
[1, 23, 134, 425, 3255, 6245, 7543, 9465, 10938] 10937	"Not found in data"	
[1, 6245, 10938, 23, 134, 7543, 9465, 425, 3255] 10938	2	
[1, 6245, 10938, 23, 134, 7543, 9465, 425, 3255] 10937	"Not found in data"	

Sorting Algorithms

The sorting algorithms will be tested with various types of integer arrays, some sorted to ensure they can handle all inputs.

Input	Expected Output	Justification
[132412412, 245, 2, 512451246, 356, 324512, 65327, 1255123, 5215, 2145, 55, 13, 21, 5]	[2, 5, 13, 21, 55, 245, 356, 2145, 5215, 65327, 324512, 1255123, 132412412, 512451246]	The sorting algorithms will need to be able to efficiently sort data in ascending order as the timeline and skills pages both require events to be output in chronological order.
[2, 5, 13, 21, 55, 245, 356, 2145, 5215, 65327, 324512, 1255123, 132412412, 512451246]	[2, 5, 13, 21, 55, 245, 356, 2145, 5215, 65327, 324512, 1255123, 132412412, 512451246]	

Further data to be used in post development

In post development, I will be testing the functionality of the program as a whole. For example, if the database doesn't correctly store or retrieve user data, it may cause issues with outputting incorrect event information on the input form which would be incredibly data-dependant system like this. It may lead to harmful consequences such as the user copying incorrect data from the website and attaching it to their CV or personal statement which may cause issues in those applications. Therefore, I will create a testing checklist which checks for all the key aspects of the program:

Aspect to test	Does it work?
The login page outputs correct error/success messages when logging in	
The signup page outputs correct error/success messages when creating an account	
The username is retained in the username field if it is valid but if there are other errors	
Program outputs a 'hello' message with the logged in user's username	
Correct calendar data is generated for each month (according to the 2026 calendar)	
Events tagged have the correct colour coding applied to them	
The legend displays the correct colour coding value for each tag	
If data is input into the inputform with an empty event name, error message is output	
If data is input into the inputform correctly, a success message is output	
Data is retained in the fields for previously stored data	
Skills page retains the textbox values and displays them correctly	
Skills page outputs a success message when the skills descriptions are updated/inserted	
Timeline page displays events chronologically with the retained event reflection values	

Timeline page outputs a success message when the event reflections are inserted/updated	
The save as pdf feature correctly saves the timeline div as pdf with styling	
The navigation bar correctly redirects the user to the pages selected	

This checklist tests if the program: validates inputs when necessary; implements helpful usability features (such as username retention); outputs correct error/success messages for responsiveness; allows for data to be input and retrieved correctly; and if all of the buttons work correctly.

If the program passes all of the test outlines by the testing checklist, the program will confidently be able to meet all of the essential criteria of the program and will also implement a desirable 'save as pdf' feature successfully. This will then allow me to be certain that the code has no errors and can be passed to the stakeholders for stakeholder testing.

Development and Testing

Stage 1: Database

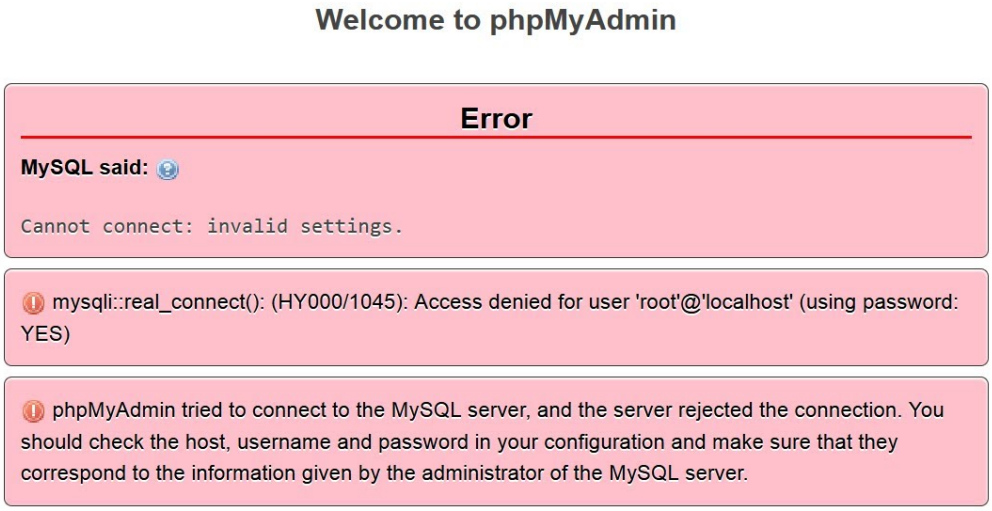
Firstly, I will start by creating the database for the program as I planned in the design section. I will implement this by following a video tutorial on how to connect to a MySQL database (linked in the bibliography).

Initial iteration implementation creating the database

```php
<?php
//Declare variables to be passed
$servername = "localhost";
$username = "root";
$password = "Aaryan";

//Create connection
$conn = new mysqli($servername, $username, $password);
// Check connection
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}

// Create database
$sql = "CREATE DATABASE NEA";
if ($conn->query($sql) === TRUE) {
  echo "Database created successfully<br>";
} else {
  echo "Error creating database: " . $conn->error ."<br>";
}

//Close connection
$conn->close();
?>
```

This piece of code will be initially used to connect to MySQL and create the NEA database.

I ran the code and when I checked phpMyAdmin on XAMPP to see if the database was created, I received this message.



I tried restarting the MySQL database, but the error was still not resolved.

I searched for a solution online and found articles from GeeksForGeeks and a post on Stack Overflow on this issue. Common solutions suggested by both articles were to open my 'config.inc.php' file in my XAMPP directory and to change '$cfg['Servers'][$i]['password'] = ' ';' to my password.

However, that did not solve the issue and therefore I checked my code to see if any errors were present.

In my code I discovered the error, I have declared the password to be "Aaryan".

```
$password = "Aaryan";
```

The default password for XAMPP MySQL is actually '' therefore the error was being caused because I was attempting to sign in with a password when no password was associated with the account.

```
$password = "";
```

Changing that line fixed the access issue and the database was created successfully.

Database created successfully

Then, I closed the previous connection and PHP tag and opened a new one to start creating the tables.

Second iteration implementation creating the USERS table

```php
<?php
//Declare variables to be passed
$db_server = "localhost";
$db_user = "root";
$db_pass = "";
$db_name = "NEA";
$conn = "";

//Connect to NEA database
try{
$conn = mysqli_connect($db_server, $db_user, $db_pass, $db_name);
}
catch(mysqli_sql_exception){
    echo "Could not connect" . $conn->connect_error ."<br>";
}
```

This was the code that connected to the database using the credentials and DB name.

If it connected to the database successfully, the screen would be white like this:

Next, I started working on creating the tables. An issue I noticed when creating the table is that, when testing the SQL statement in phpMyAdmin's SQL page to create the USER table, it was highlighting the word 'USER'.

```
CREATE TABLE USER (user_id INT(6) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
```

I found out that 'USER' is a reserved keyword in SQL and therefore naming my table USER may cause issues.

```
//Create User table
$sql = "CREATE TABLE USER (
user_id INT(6) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(30) NOT NULL,
hashed_password VARCHAR(30) NOT NULL
)";
```

There were two possible solutions to this issue:

1. Rename the USER table
2. Add backticks when declaring the name ``

To prevent further issues due to accidental omission of the backticks, I chose to use the first option, and I renamed the table to USERS.

```
$sql = "CREATE TABLE USERS
```

Furthermore, after realising that 30 characters could be too short for users who may want to implement longer, more secure passwords, I decided to change the character limits to 255 (the maximum value).

```
//Create User table
$sql = "CREATE TABLE USERS (
user_id INT(6) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(30) NOT NULL,
hashed_password VARCHAR(255) NOT NULL
)";
```

I also added a message that would inform me if the table was created successfully. This would be a helpful form of validation and will be helpful when testing the code.

```
//Check if table created successfully
if ($conn->query($sql) === TRUE){
    echo "Table USERS created successfully<br>";
} else{
    echo "Error creating table: ". $conn->error ."<br>";
}
```

In a similar way, I decided to implement the other tables.

Third iteration implementation creating the other tables

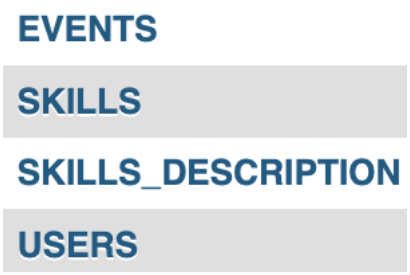
```
//Create Events table
$sql = "CREATE TABLE EVENTS (
event_id INT(6) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
date_of_event DATE NOT NULL,
name_of_event VARCHAR(255) NOT NULL,
event_description TEXT NOT NULL,
event_reflection TEXT,
event_tag VARCHAR(255)
)";

//Check if table created successfully
if ($conn->query($sql) === TRUE) {
  echo "Table EVENTS created successfully<br>";
} else {
  echo "Error creating table: " . $conn->error ."<br>";
}


//Create Skills table
$sql = "CREATE TABLE SKILLS (
skills_id INT(6) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
leadership BOOLEAN,
creativity BOOLEAN,
teamwork BOOLEAN,
initiative BOOLEAN,
problem_solving BOOLEAN,
organisation BOOLEAN,
communication BOOLEAN,
public_speaking BOOLEAN,
perseverance BOOLEAN,
time_management BOOLEAN,
planning_and_preparation BOOLEAN,
analytical_skills BOOLEAN,
practical_skills BOOLEAN,
academic_skills BOOLEAN,
computing_skills BOOLEAN
)";
```

```php
//Check if table created successfully
if ($conn->query($sql) === TRUE) {
  echo "Table SKILLS created successfully<br>";
} else {
  echo "Error creating table: " . $conn->error ."<br>";
}


//Create Skills Description table
$sql = "CREATE TABLE SKILLS_DESCRIPTION (
description_id INT(6) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
leadership_description TEXT,
creativity_description TEXT,
teamwork_description TEXT,
initiative_description TEXT,
problem_solving_description TEXT,
organisation_description TEXT,
communication_description TEXT,
public_speaking_description TEXT,
perseverance_description TEXT,
time_management_description TEXT,
planning_and_preparation_description TEXT,
analytical_skills_description TEXT,
practical_skills_description TEXT,
academic_skills_description TEXT,
computing_skills_description TEXT
)";

//Check if table created successfully
if ($conn->query($sql) === TRUE) {
  echo "Table SKILLS_DESCRIPTION created successfully<br>";
} else {
  echo "Error creating table: " . $conn->error ."<br>";
}

//Create Link table
$sql = "CREATE TABLE LINK_TABLE (
link_id INT(6) NOT NULL AUTO_INCREMENT PRIMARY KEY,
user_id INT(6) ,
event_id INT(6) ,
skills_id INT(6) ,
description_id INT(6) ,
FOREIGN KEY (user_id) REFERENCES USERS(user_id),
FOREIGN KEY (event_id) REFERENCES EVENTS(event_id),
FOREIGN KEY (skills_id) REFERENCES SKILLS(skills_id),
FOREIGN KEY (description_id) REFERENCES SKILLS_DESCRIPTION(description_id)
)";

//Check if table created successfully
if ($conn->query($sql) === TRUE) {
  echo "Table LINK_TABLE created successfully<br>";
} else {
  echo "Error creating table: " . $conn->error ."<br>";
}


// Close connection
$conn->close();
```

When I ran the code, however, it gave me this output:

Table USERS created successfully
Table EVENTS created successfully
Table SKILLS created successfully
Table SKILLS_DESCRIPTION created successfully

After checking the database, I saw that the 4 tables that output the success messages were created.



The LINK_TABLE was not being created. Therefore, I began working to fix this.

After searching up the issue online and looking at the W3Schools guide on foreign keys in SQL, I realised that is issue was that the type declarations of the foreign keys did not match the types of the primary keys. The primary keys were UNSIGNED, but the foreign keys weren't specified as UNSIGNED and therefore was causing issues. Therefore, I added UNSIGNED to each attribute and the table was created successfully.

```
//Create Link table
$sql = "CREATE TABLE LINK_TABLE (
link_id INT(6) NOT NULL AUTO_INCREMENT PRIMARY KEY,
user_id INT(6) UNSIGNED,
event_id INT(6) UNSIGNED,
skills_id INT(6) UNSIGNED,
description_id INT(6) UNSIGNED,
FOREIGN KEY (user_id) REFERENCES USERS(user_id),
FOREIGN KEY (event_id) REFERENCES EVENTS(event_id),
FOREIGN KEY (skills_id) REFERENCES SKILLS(skills_id),
FOREIGN KEY (description_id) REFERENCES SKILLS_DESCRIPTION(description_id)
)";
```

Table LINK_TABLE created successfully

I checked the database and it now showed all 5 tables

Therefore, the entire database was now created so now I could move on to creating the rest of the program.

Stage 1 Review

What has been done

I created a file called database.php which creates the NEA database. If the database was created successfully, a success message is output, if not an error message is output. Furthermore, the file also creates all of the tables (USERS, EVENTS, SKILLS, SKILLS_DESCRIPTION and LINK_TABLE) and outputs success messages based on whether they were successfully created.

How it has been tested

I have tested the code by running it and checking the output messages. Implementing if statements after executing every query allowed me to have visual representation of whether the tables and database were created.

The creation of the database and its tables were further tested by checking phpMyAdmin on XAMPP. It allowed me to have a visual representation after running the code on whether or not the database and tables were created.

Criteria met/being met

- Login Page with hashing algorithm
- Signup Page with hashing algorithm
- Calendar System for past, present and future events
- Colour coding of events
- Skills table that links to calendar
- Skills description feature
- Timeline page
- Reflection for each event on the timeline

How it meets the success criteria

The database is responsible for all of the data that is input or displayed on these pages. Creating the database allows for these pages to run the algorithms which store and retrieve data from the database.

Changes in the design that have resulted from this section

The first change was that the USER table has been renamed to the USERS table due to USER being a keyword in SQL which could have led to errors unless backticks were implemented in the code.

The second change was that the light_mode attribute from the USERS table was not implemented in the code. This is because I am conscious of whether or not I will be able to implement user customisation within the time constraints. Furthermore, the light_mode attribute could be added later in development if time allows.

The final change was that I renamed event_colour to event_tag as I believe that the tag should be stored in the database and the colour can be associated with the tag in the CSS styling.

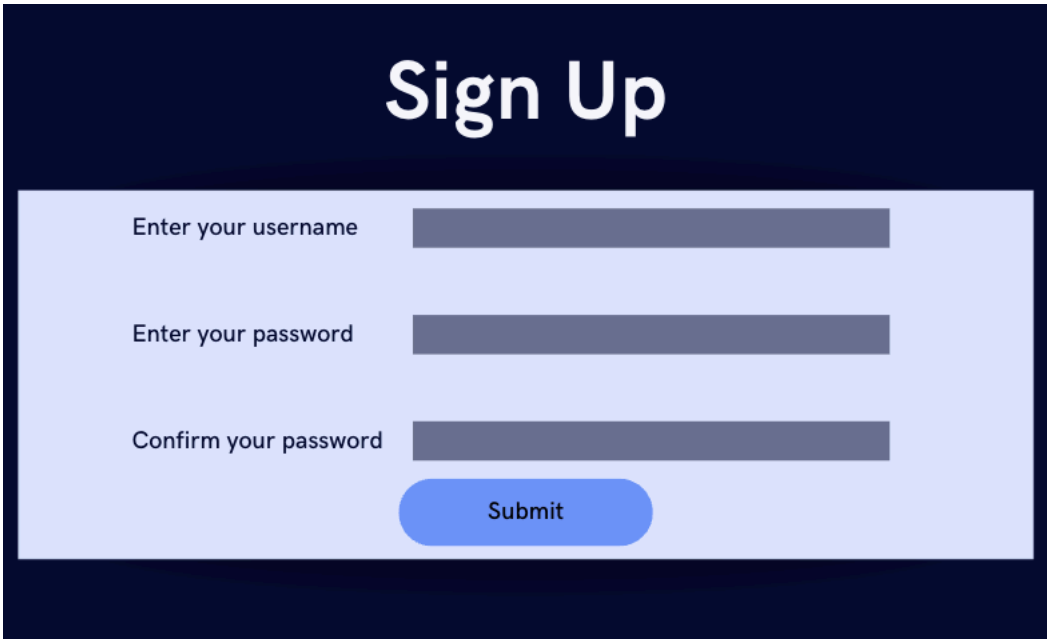
Summary of the whole project as a prototype at this stage

The database has been fully created including primary keys and relationships between tables using foreign keys. The project currently only contains this database in which data can be manually added using phpMyAdmin. It currently has no UI except for the page which displays the success messages when the database is created. However, this stage is necessary and serves as the backend foundation for the future pages of the program.

Stage 2: Signup Page

Creating an accounts system would allow for multiple users to use a server-side database which would be beneficial when scaling this project to be globally hosted rather than locally hosted. I would like to start by creating a signup system as it would allow me to implement my username and password validation algorithms that I designed in the design stage.

This is the design that I created for the signup page in the design stage.



The key aspects of the design are the dark grey input fields, the background div which highlights the input fields (the contrast improves usability) and the blue submit button.

I will first begin with creating the GUI for my program.

Creating the GUI of the Page

I decided to start working on my GUI before starting on my backend processing so that I have a user interface in which I can see the outputs when I test my code for errors.

signup.php

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Registration</title>
    <link rel="stylesheet" href="regstylesheet.css">

</head>
<body>

        <form>
            <br><br><b><label id="Reg">Sign Up</label></b><br><br><br><br>

            <div style="height: 300px;">
            <br>
            <label>Enter your username   </label><input type="text"><br><br><br>
            <label>Enter your password   </label><input type="text"><br><br><br>
            <label>Confirm your password   </label><input type="text"><br><br><br>
            <input type="submit" id="Submit">


        </div>
        </form>
        <br><br><br><br><br><br><br><br><br><br>
        <a href="login.php">Log in</a>
</body>
</html>
```

regstylesheet.css

```css
body{
  background-color: #070a2a;
  font-family: 'Trebuchet MS', 'Lucida Sans Unicode', 'Lucida Grande', 'Lucida Sans', Arial, sans-serif;
  text-align: center
}

form{
  color: #0b0e2e;
}

div{
  display: inline-block;
  margin: 10px auto;
  background-color: #dee2fb;
  height: 250px;
  width: 750px;
  font-size: large;
}

#Reg{
  color: aliceblue;
  font-size: 75px;
}

#Submit{
  background-color: #7c90ea;
  border-radius: 20px;
  font-size: large;
  border-color: #7c90ea;
  border-style: hidden;
  padding: 10px;
}

input[type="text"], textarea {
  background-color : #6c718c;
  border: 1px solid black;
  box-shadow: none;
}
```

```css
a{
  position: static;
  font-size: 40px;
}

a:link {
  color: #b5c4fa;
  background-color: transparent;
  text-decoration: underline;
}

a:visited {
  color: #b5c4fa;
  background-color: transparent;
  text-decoration: none;
}

a:hover {
  color: #7c90ea;
  background-color: transparent;
  text-decoration: underline;
}

a:active {
  color: white;
  background-color: transparent;
  text-decoration: underline;
}
```

I have created a basic signup page with a title, and an area to input the username, password and confirmation password. I have styled it using CSS with a stylesheet called ‘regstylesheet.css’ which I have called in the header. Using CSS, I tried to recreate the Canva design as best as I could while trying to maintain the simplistic design as this would not confuse the user and help with usability.

I chose to name the stylesheet ‘regstylesheet.css’ as I plan on using the same stylesheet for my login page as the design of the two pages are very similar. The only change I have made here is adding the 300px height attribute to the div containing the inputs. This is because the signup page contains 3 input fields (username, password, confirmation password) and therefore the div will need to be larger.

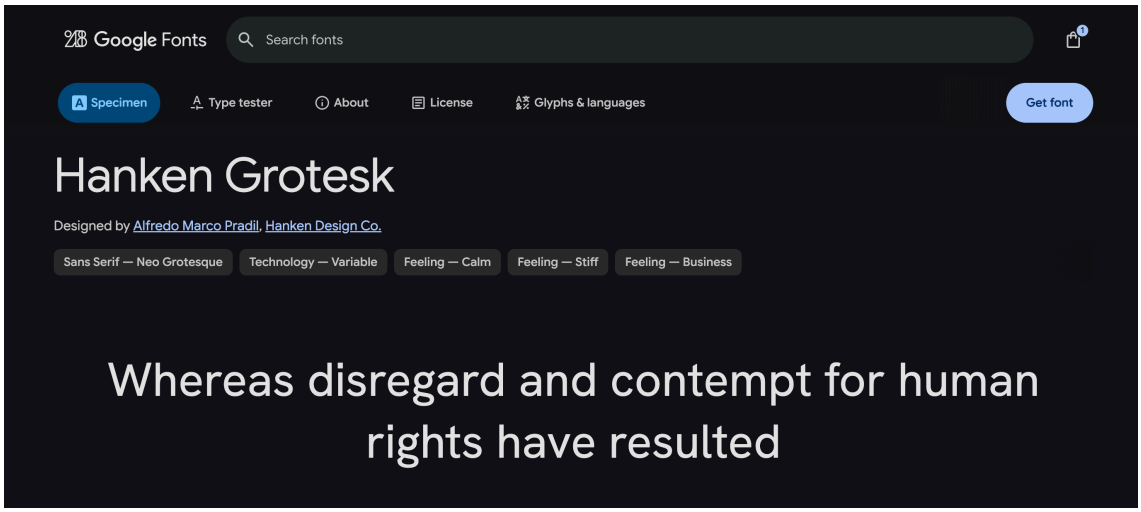
This is what the page looks like.:

However, when testing the page with login information, I have noticed 3 main issues visible in the image below.

Those issues are:

1. The font used doesn't match the font from the design.
2. The text in the input fields is black which is hard to read on the grey background.
3. The password fields are visible when typing.

Issue 1 is easy to fix as I can search for the font and try to import it into HTML. The font name is 'Hanken Grotesk' which I found on Google Fonts.



Clicking on 'Get font', lead me to this page where it gave me instructions on how to import the font into my stylesheet.

Embed code in the <head> of your html

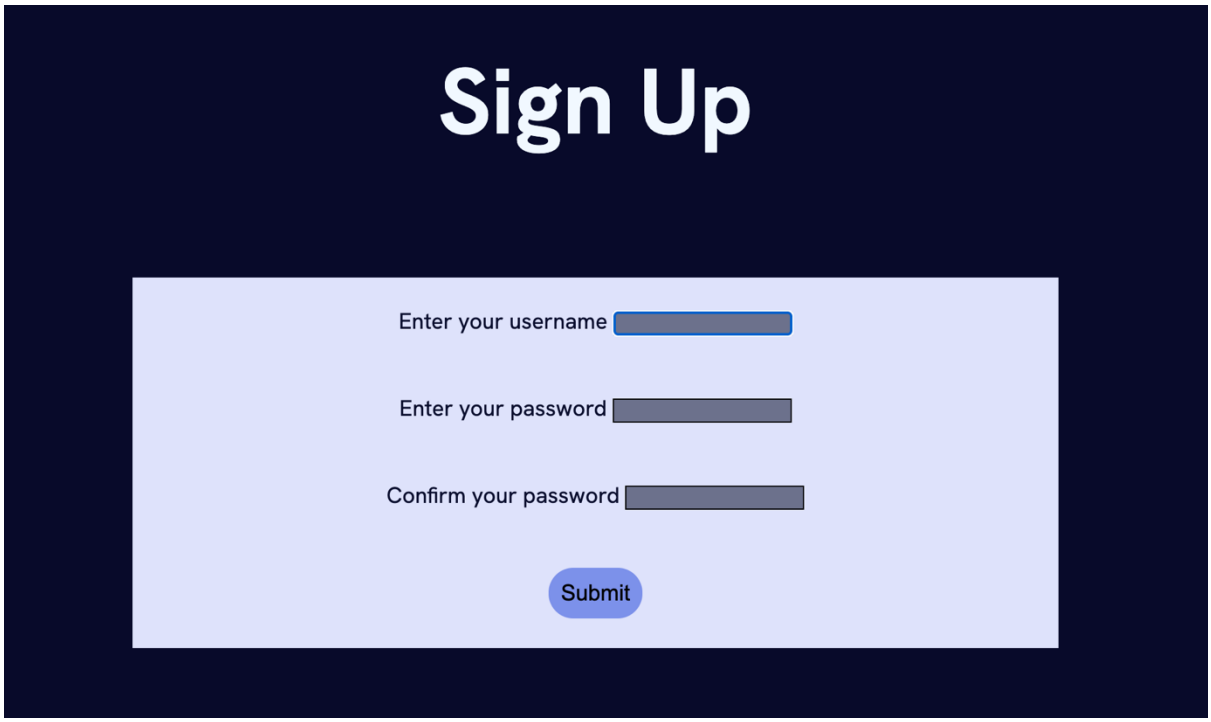
```
<style>
@import url('https://fonts.googleapis.com/css2?family=Hanken+Grotesk:ital,wght@0,100..
900;1,100..900&display=swap');
</style>
```

Copy code

I then added this code to the top of my stylesheet and changed the font-family attribute to this.

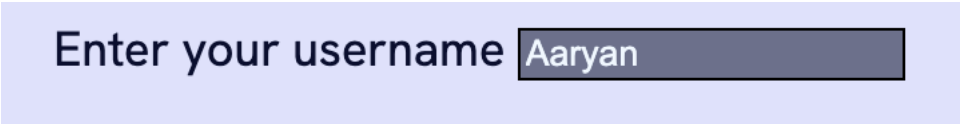
```
@import url('https://fonts.googleapis.com/css2?family=Hanken+Grotesk:ital,wght@0,100..900;1,100..900&display=swap');
/*Importing correct font (based on Canva design) from Google fonts*/

body{
  background-color: #070a2a;
  font-family: 'Hanken Grotesk', sans-serif;
  text-align: center
}
```

Which imported the correct font.



Issue 2 is also an easy fix. I changed the ‘color’ attribute in the input field styling to the same colour as the title which solved the issue.

```css
input[type="text"]{
  background-color : #6c718c;
  color: aliceblue;
  border: 1px solid black;
  box-shadow: none;
}
```



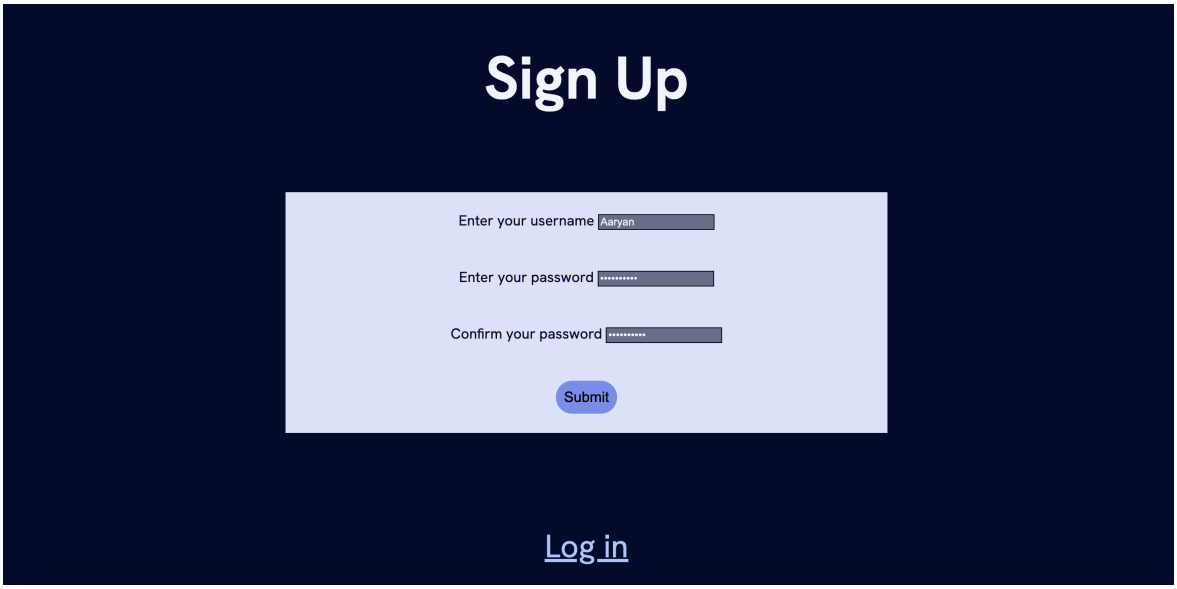
Finally, to solve the third issue, I had to change the type attribute of the password input fields to ‘password’ which hides the text input.

```
<label>Enter your password   </label><input type="password"><br><br><br>
<label>Confirm your password   </label><input type="password"><br><br><br>
```

Furthermore, as my CSS styles ‘text’ type inputs, I also had to copy the same styling for ‘password’ type inputs.

```css
input[type="text"]{
  background-color : #6c718c;
  color: aliceblue;
  border: 1px solid black;
  box-shadow: none;
}

input[type="password"]{
  background-color : #6c718c;
  color: aliceblue;
  border: 1px solid black;
  box-shadow: none;
}
```

Which lead to my completed GUI, which looks like this:



Now that my GUI is done, I will begin working on the backend processing of my webpage. I will start by creating a file that connects to the database. This file will be called 'dbh.inc.php' and will be stored in the 'includes' folder. I have decided that from now on, all the files relating to the backend PHP processing of the program will be stored in 'includes' for easier reference for future developers who may choose to follow up on this program.

Connecting to the database and creating session variables

dbh.inc.php

```php
//Host type (XAMPP is a localhost)
$host = 'localhost';
$dbname = 'NEA';
//Default credentials using XAMPP
$dbusername = 'root';
$dbpassword = '';

try {  //Tries connecting to the database using the host type, database name and database credentials
    $pdo = new PDO("mysql:host=$host;dbname=$dbname", $dbusername, $dbpassword);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {  //If an error occured
    die("Connection failed: " . $e->getMessage());  //End the program and display a 'Connection failed: ' message with the error message
}
```

I have created this after watching Dani Krossing's 'PHP Course for Beginners', therefore I will use PDO from now on when connecting to the database as I am more skilled using PDO after finishing the course.

Now that the database connection is done, I will create a file that allows me to regenerate session variables for the user which I can use to store the logged in user's user id to help when fetching data from the database. The file will be called 'config_session.inc.php' and will be in the 'includes' folder.

config_session.inc.php

```php
ini_set('session.use_only_cookies', 1);
/*Makes sure that the website only uses a session id that has been created by a server in our website
Also makes the session id more complex when created -> difficult for people to guess the session id*/
ini_set('session.use_strict_mode', 1);

session_set_cookie_params([  //Function that allows for changing of cookie attributes
    'lifetime' => 1800,  //Lifetime (seconds)
    'domain' => 'localhost',  //Domain (XAMPP)
    'path' => '/',  //Sub-paths (/ means works on any paths)
    'secure' => true,  //Only use this cookie in HTTPS
    'httponly' => true  //Avoids the user changing anything about the cookie
]);

session_start();

if (!isset($_SESSION["last_regeneration"])) {  //Checks if the last_regeneration session variable wasn't created before

    regenerate_session_id();  //If not, regenerate the session id

} else {  //If a session variable was created
    $interval = 60 * 30;  //Time until session id regeneration

    if(time() - $_SESSION["last_regeneration"] >= $interval) {  //Check if more than 30 mins have passed

        regenerate_session_id();  //Regenerate session id
    }
}

function regenerate_session_id(){
    session_regenerate_id();  //Regenerate session id

    $_SESSION["last_regeneration"] = time();  //Reset last_regeneration variable
}
```

This works well and allows me to create session variables which would be useful in helping me to check for errors when the user is signing up.

Now that the session variables have been created, I will now start to work on the initial iteration of the validation which uses JavaScript.

Initial iteration implementation using Javascipt

The initial iteration using JS will be stored in a file called ‘validation.js’.

username_validation

```javascript
function username_validation(){
    let username_valid = false; //Boolean value for username validity
    let special_characters = "-';/*";  //Forbidden special characters
    while (username_valid != true){  //Repeat until the input username is correct
        let count = 0;  //Count is the number of special characters present in the username
        let username = document.getElementById("username").value;  //Input username is taken and stored in variable username
        for(let i = 0; i < username.length - 1; i++){ //For loop iterates for the length of the username
            for(let j= 0; j < 5; j++){  //For loop iterates for the length of special_characters
                if (username[i] == special_characters[j]){  //Checks to see if a character of the username is a forbidden character
                    count ++;  //Count value increments
                }
            }
        }
        if (count > 0){  //Checks if a forbidden character is present in the username
            document.getElementById("usernamevalid").textContent = "Username should not contain special characters: - ' ; / *";  //Outputs an error message
            username_valid = false;  //Sets the boolean value to false
        }
        else{
            username_valid = true;  //Sets boolean value to true
            document.getElementById("usernamevalid").textContent = "Username is valid.";  //Outputs a success message
        }
    }

}
```

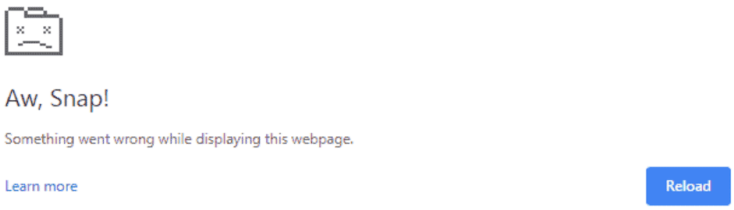
password_validation

```
function password_validation(){
    let password_valid = false;  //Boolean value for password validity
    while(password_valid != true){  //Repeats until the input password is correct
        let password = document.getElementById("password").value;  //Input password is taken and stored in variable password
        let confirmpassword = document.getElementById("confirmpassword").value;  //Input confirmation password is taken and stored in variable confirmpassword
        if(password.length > 12){  //Checks to see if the password fits the minimum length requirement
            if(password == confirmpassword){  //Checks to see if both input password and confirmpassword match
                password_valid = true;  //Sets boolean value to true
                document.getElementById("passwordvalid").textContent = "Password is valid and both passwords match.";  //Outputs a success message
            }
            else{
                document.getElementById("passwordvalid").textContent = "Passwords do not match.";  //Outputs an error message
            }
        }
        else{
            document.getElementById("passwordvalid").textContent = "Password should be greater than 12 characters.";  //Outputs an error message

        }
    }
}
```

I have coded these algorithms based on the pseudocode in my design section.

These algorithms are called when the submit button is pressed and are meant to repeatedly be called until a valid username, password and confirmation password are inputted.

However, when running the code, I was met with this screen which indicated that the program had frozen.



This is because the condition of looping until the username_valid and password_valid Boolean values were true, would not work as expected when implemented. This logic works in Python, where the code is implemented in the terminal, however, the JavaScript code is only run when the button is pressed. Therefore, the conditions never change to true, and the code loops infinitely.

This can be fixed by removing the Boolean condition and while loop by making it so that if the username and password inputs are valid, they are redirected, if not, they remain on the login page, and an error message is output.

However, I decided that this implementation of validation is not sufficient as it doesn't check if the username is already taken in the database. Furthermore, if the username and password were valid, sending the data to and storing the data in the database would be a challenge using JavaScript.

Due to time constraints and my current level of skill, implementing this check would require switching to PHP as it allows for easy database linkage, simpler function calling (using action and method when the submit button is pressed), and increased security (due to built-in hashing functions).

Second iteration implementation using PHP

Using PHP and by following video 28 in the 'PHP Guide for Beginners' by Dani Krossing (link in bibliography), I have implemented the signup system using 4 files in the folder named 'includes' in my project file as such:

signup.inc.php

```php
if($_SERVER["REQUEST_METHOD"] === "POST"){  //Check if page was accessed legitamately

    $username = $_POST["username"];  //Input username
    $password = $_POST["password"];  //Input password
    $cpassword = $_POST["cpassword"];  //Input confirmation password

    try{

    require_once 'dbh.inc.php';  //Allows for database linkage
    require_once 'signup_model.inc.php';  //Allows for the results of fetching from the database to be accessed by this file
    require_once 'signup_contr.inc.php';  //Allows for error-checking functions to be accessed by this file


    $errors = [];  //Storing any errors found in an array

    if (is_input_empty($username, $password, $cpassword)){  //Checks to see if any inputs are empty
    //If they are, create a key-pair value to store the "empty_input" error and the error message of "Fill in all fields"
        $errors["empty_input"] = "Fill in all fields";
    }
    if (is_password_short($password)){  //Checks to see if the password is too short
    //If it is, create a key-pair value to store the "short_password" error and the error message of "Password is too short"
        $errors["short_password"] = "Password is too short";
    }
    if (!do_passwords_match($password, $cpassword)){  //Checks to see if password and cpassword don't match
    //If they are, create a key-pair value to store the "invalid_password" error and the error message of "Passwords do not match"
        $errors["invalid_password"] = "Passwords do not match";
    }
    if (is_username_taken($pdo, $username)){  //Checks to see if the username is already taken
    //If they are, create a key-pair value to store the "username_taken" error and the error message of "Username already taken"
        $errors["username_taken"] = "Username already taken";
    }

    require_once 'config_session.inc.php';  //Allows for session variables to be created/regenerated

        if($errors){  //If errors are present
            $_SESSION["error_signup"] = $errors;  //Store them as a session variable

            $signupData = [  //Create an associative array called signupData to store the username
                "username" => $username
            ];
            $_SESSION["signup_data"] = $signupData;  //Store it as a session variable


            header("Location: ../signup.php");  //Redirect back to the signup page
            die();  //End the program
        }

        create_user($pdo, $username, $password);  //Calls the create user function from signup_contr.inc.php

        header("Location: ../signup.php?signup=success");  //Redirect to the signup page with 'success'

        $pdo = null;  //Unset database variables (best practice)
        $stmt = null;  //Unset database variables (best practice)

        die();  //End the program
        } catch (PDOException $e) {  //If it fails, get the error
        die("Query failed: " . $e->getMessage());  //End the program and output the error
    }

    }else {
        header("Location: ../signup.php");  //If accessed incorrectly, redirect to the signup page
        die();  //End the program
    }
```

This file manages the main functions of the signup page and calls functions from the other related signup files, the database connection file and the session variable file. It first checks if the file was accessed by submitting the form correctly. If not, it redirects the user back to the signup page. This is a form of validation and prevents users from accidentally or intentionally accessing this file without correctly filling in the form. Next it retrieves the input data from the input fields and stores them as variables. It then runs a try catch block which validates the data and then submits it. If there was an error with the try block, the catch block will output an error message. The try block first accesses the files from which it calls the required functions. It then creates an array called 'errors' which checks if the called functions are flagging errors, if they are, it stores the error message with the error name as the key value. The file then accesses 'config_session.inc.php' to allow for session variables to be created. If errors are present, the array is stored as a session variable. Another associative array is created to store the username and then that array is stores as a session variable. The file then redirects the user back to the signup page and kills the program. If there weren't any errors, the create_user function is called, and the user information is stored in the database. The user is then redirected to the same success page with a get variable of signup=success which can be used to output a success message on the signup page. Finally, the pdo object and stmt variable are set to null before the program ends.

This file contains the input validation of the signup page and is created in a modular way such that the function names can be changed for the same code to be implemented for the login system.

signup_contr.inc.php

```php
function is_input_empty(string $username, string $password, string $cpassword){
    if (empty($username) || empty($password) || empty($cpassword)){  //Checks if any input fields are empty
        return true;
    } else {
        return false;
    }
}

function is_password_short(string $password){
    if (strlen($password) > 12 && strlen($password) < 255){  //Checks if password is within the length requirement
        return false;
    } else {
        return true;
    }
}

function do_passwords_match(string $password, string $cpassword){
    if ($password === $cpassword){  //Checks if both passwords match
        return true;
    } else {
        return false;
    }
}

/*The $pdo object is passed to all functions that interact with the database as it allows for database connection*/
function is_username_taken(object $pdo, string $username){
    if (get_username($pdo, $username)){  //Checks if the get_username() function from signup_model.inc.php returned an output
        return true;
    } else {
        return false;
    }
}

function create_user(object $pdo, string $username, string $password){
    set_user($pdo, $username, $password);  //Calls the set_user() function from signup_model.inc.php and inputs the user data into the database
}
```

This file contains the definitions of all the error checking functions and the user creation function. Firstly, the error checking functions check for: if any of the input fields are empty; if the password is too short; if both input passwords match; and if the username has already been stored in the database. This allows for account validation before account creation. Finally, the create_user function calls the set_user function to store the user's credentials in the database if they were valid.

The functions are modular in nature as the main difference in the code is the if statement that they check for validation, this allows for more error checking functions to be implemented in this style in further iterations of the program, and the code has been annotated to help in the maintenance of the program.

signup_model.inc.php

```php
declare(strict_types=1);  //Best practice to prevent errors in data types

function get_username(object $pdo, string $username){
    //Using prepared statements is best practice to avoid errors and to enforce security when accessing the database
    $query = "SELECT username FROM USERS WHERE username = :username;";
    $stmt = $pdo->prepare($query);  //Query is prepared
    $stmt->bindParam(":username", $username);  //Username parameter is bound
    $stmt->execute();  //Statement is executed

    $result = $stmt->fetch(PDO::FETCH_ASSOC);  //The username is fetched from the database in an associative array
    return $result;  //The array containing the username is returned
}

function set_user(object $pdo, string $username, string $password){
    //Using prepared statements is best practice to avoid errors and to enforce security when accessing the database
    $query = "INSERT INTO USERS (username, hashed_password) VALUES (:username, :hashed_password);";
    $stmt = $pdo->prepare($query);  //Query is prepared

    $options = [
        'cost' => 12  //Time taken to load to prevent hackers from brute-forcing the signup system
    ];

    $hashed_password = password_hash($password, PASSWORD_BCRYPT, $options);  //Hashing the password with the built-in hashing function in PHP

    $stmt->bindParam(":username", $username);  //Username parameter is bound
    $stmt->bindParam(":hashed_password", $hashed_password);  //Hashed password parameter is bound
    $stmt->execute();  //Statement is executed
}
```

This file contains the functions which access the database. Separating the database interaction to one file allows for better debugging and modularity. Errors relating to the database can be located and fixed and additional functions regarding database interaction for the signup page can be added to this PHP file in further iterations of the program.

The get_username function gets the first example of where the entered username is present in the database and returns the result. If there was a result, it would be output as an associative array. If there was no result, a null value would be output.

The set_user function hashes the input password with a specified cost value which increases the hashing time which increases the time for the credentials to be uploaded to the database which reduces the number of brute-force attempts a user could maliciously make and acts as validation. After the password is hashed, the username and hashed password are input in the database to create the user's account.

signup_view.inc.php

```php
declare(strict_types=1);  //Best practice to prevent errors in data types

function signup_inputs(){
    //If a username was input that wasn't already taken
    if (isset($_SESSION["signup_data"]["username"]) && !isset($_SESSION["error_signup"]["username_taken"])){
        echo '<label>Enter your username   </label>';  //Output the label
        //Output the username input field with the username in it (usability)
        echo '<input type="text" name="username" id="username" value="' . $_SESSION["signup_data"]["username"] . '"><br><br><br>';

    } else{  //Otherwise
        echo '<label>Enter your username   </label>';  //Output the label
        echo '<input type="text" name="username" id="username"><br><br><br>';  //Output the username empty input field

    }

    echo '<label>Enter your password   </label>';  //Output the label
    echo '<input type="password" name="password" id="password"><br><br><br>';  //Output the password input field
    echo '<label>Confirm your password   </label>';  //Output the label
    echo '<input type="password" name="cpassword" id="cpassword"><br><br><br>';  //Output the confirm password input field
}

function check_signup_errors(){
    if(isset($_SESSION["error_signup"])){  //If errors are stored as a session variable
        $errors = $_SESSION["error_signup"];  //Create a local variable to hold the errors

        echo "<br>";  //Start a new line

        foreach($errors as $error){  //For every error in the array
            echo '<p id="form-error"><b><u>' . $error . '</u></b></p>';  //Output the error
        }

        unset($_SESSION["error_signup"]);  //Unset the session variable
    }
}
```

This file contains the functions which output data to the user. I have separated the outputting functions into a separate file to further help with debugging and modularity as it will allow for easier error location and additions to the code.

The signup_inputs function creates the input field for the user. It allows me to implement a useful usability feature where if the username was already previously input and if the error redirecting the user was not a username_taken error, the username remains in the username input field. This prevents the user from having to retype the username multiple times if there were no errors with it.

The check_signup_errors function checks if any errors occurred previously when the user tried signing up. If any errors occurred, they are stored in a local variable and for every error that occurred, it is output as a p tag with an id of form-error. This allows the user to have a clear error message to inform them of any input issues, which also is helpful for usability.

I have also made changes to the main ‘signup.php’ page to access these files, call the signup_inputs and check_signup_errors functions, and to add a success message.

signup.php

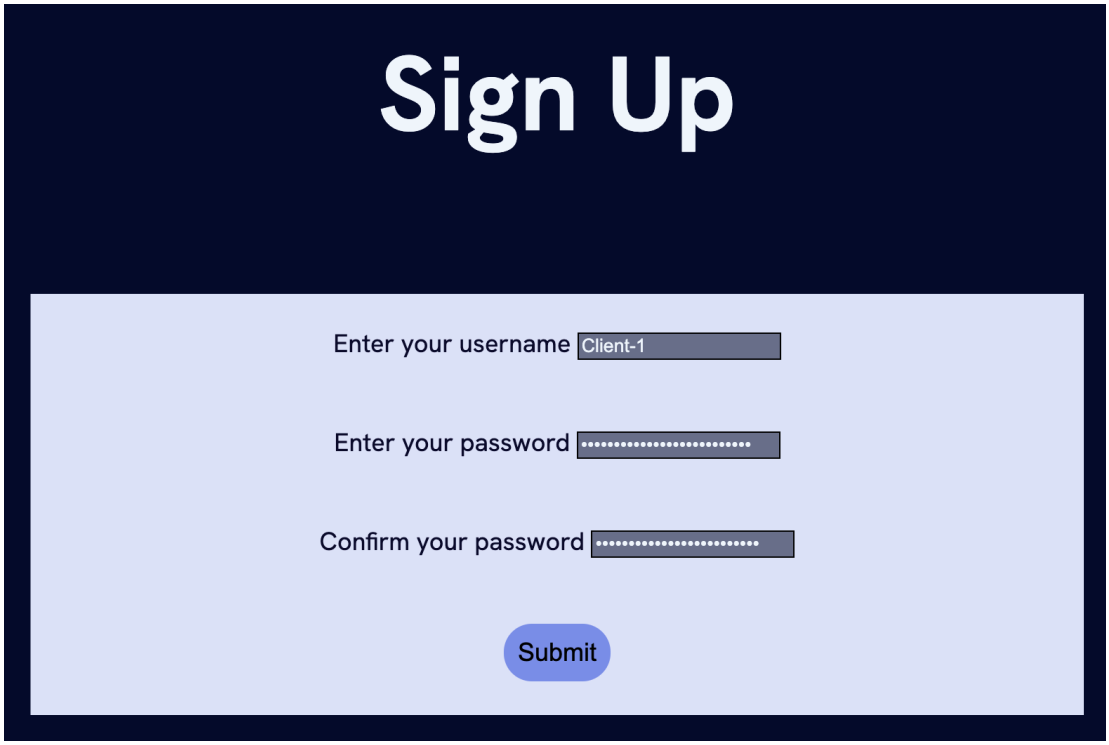
```
<body>
    <!--This declares the form element and specifies where (signup.inc.php) and using which method the data will be transferred
    (this data is sent using post as it is sensitive data)-->
    <form action="includes/signup.inc.php" method="post">
        <br><br><b><label id="Reg">Sign Up</label></b><br><br><br><br><br>

        <!--Style is individually declared as there are 3 inputs for the signup page compared to the 2 for the login page-->
        <div style="height: 300px;">
            <br>

            <?php
            /*Calls the function that if there is an error with the passwords but the username is not taken,
            the username will stay in the username field (usability feature) otherwise all of the fields will be empty
            so the username doesn't need to retype username if it was valid*/
                signup_inputs();
            ?>

            <input type="submit" id="Submit">
        </div>
    </form>

    <?php
    check_signup_errors(); //Calls the function which checks for any errors in signup such as the username being taken or blank fields

    if(isset($_GET['signup']) && $_GET['signup']==='success'){ //Checks to see if the signup session variable is set and if it was a success
        echo '<p id="form-success"><u>Account created successfully.</u></p>'; //If it was, output a success message
    }
    ?>

    <br><br><br><br><br>
    <!--A redirect link which takes you to the login page-->
    <a href="login.php">Log in</a>
</body>
</html>
```

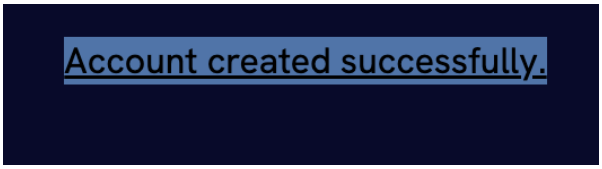
I have accessed the config_session and signup_view files at the top of the page to allow for session variables to be regenerated and to output the input form fields.

The form now sends the data using post to the signup.inc.php file to error check the code and to allow for error messages to be output. The input fields have now been replaced with the signup_inputs function to allow for username retention if the username is valid, but the passwords are not. Finally, I have added a block of code which checks if the page was redirected after a successful signup, and if so, a success message is output to the user with the id of form-success.

This solved the issue of the program repeatedly crashing and error messages were being output whenever the user made an error/successfully created their account.

I will now begin testing the program. First, I began by creating an account with valid credentials which lead to this message being output:

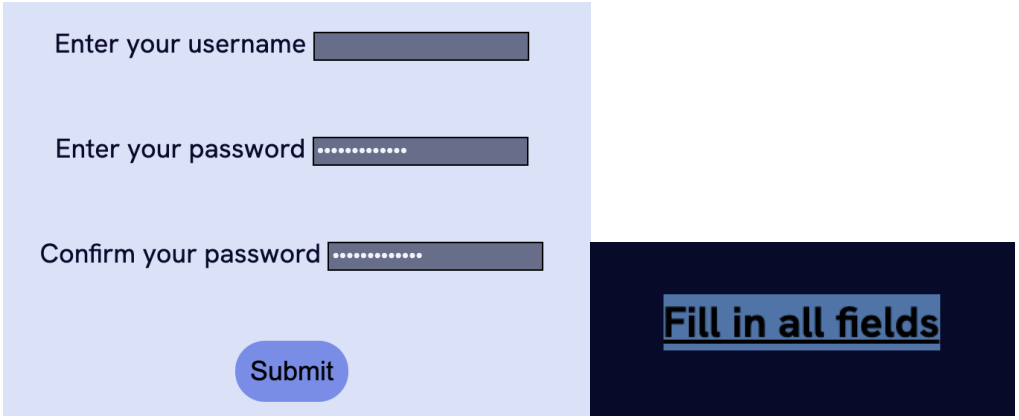




The logic was correct and the error checking didn't flag any errors. This was the expected output message. I then checked the database to see if the account was added to the USERS table successfully, which it did.

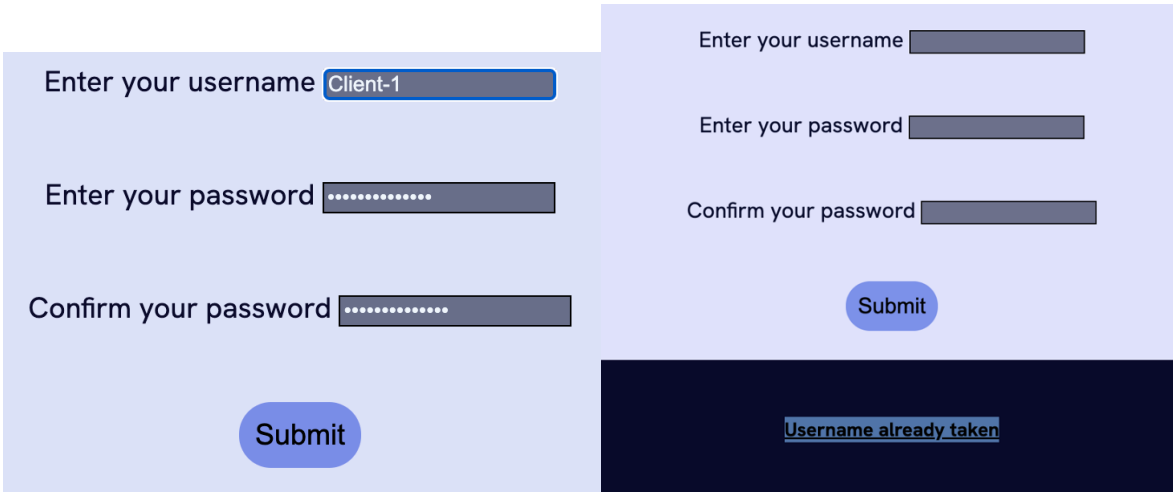
Client-1	$2y$12$AZI1eZhkVj4qCp4Fd9pNmuIWbIGfSOQLFgVu12JkIk1...

Next, I tested if leaving the username input field blank would flag any errors which lead to this output:



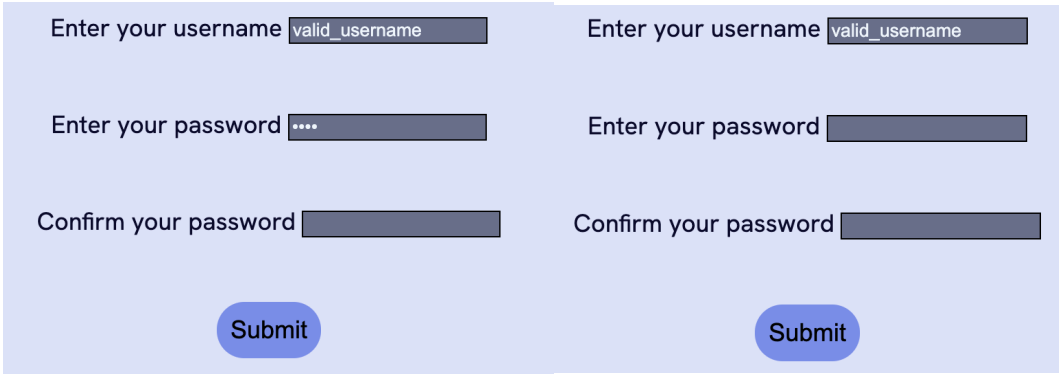
The logic was again correct, and the correct error message was output.

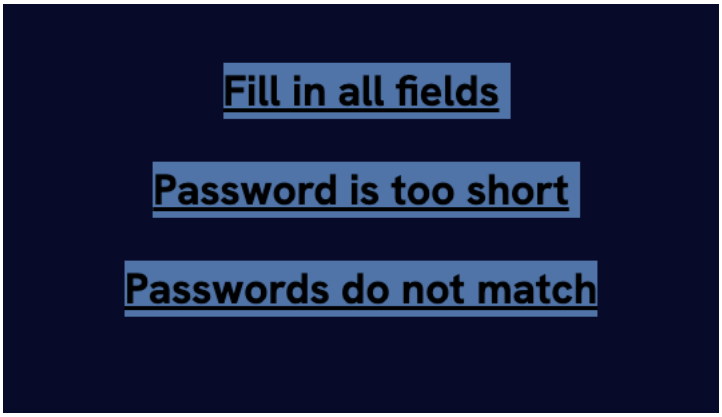
I then tested if entering a taken username outputs the correct error message and removes all entries from the input field even if the passwords are valid.



This outputted the correct error message whilst also removing all of the inputs although the passwords were valid.

Then, I tested if leaving one password input field blank with the other password field being too short whilst entering a valid username would prompt the correct output messages:





The correct messages were being output, and the username remained in the username input field. This confirms that the logic of the error checking functions and all of the other functions relating to the signup inputs had correct logic.

However, there were some issues that were present when testing:

1. Error messages weren't being correctly styled
2. Success message wasn't being correctly styled

Both issues were caused by confusing classes and ids. In the PHP code, I have set form-success and form-error as ids however, in the CSS I styled them as classes.

```php
foreach($errors as $error){  //For every error in the array
    echo '<p id="form-error"><b><u>' . $error . '</u></b></p>';  //Output the error
}
```

```php
if(isset($_GET['signup']) && $_GET['signup']==='success'){ //Checks to see if the signup session variable is set and if it was a success
    echo '<p id="form-success"><u>Account created successfully.</u></p>'; //If it was, output a success message
}
?>
```

```css
.form-error{
  color: aliceblue;
  font-size: 35px;
}

.form-success{
  color: aliceblue;
  font-size: 50px;
}
```

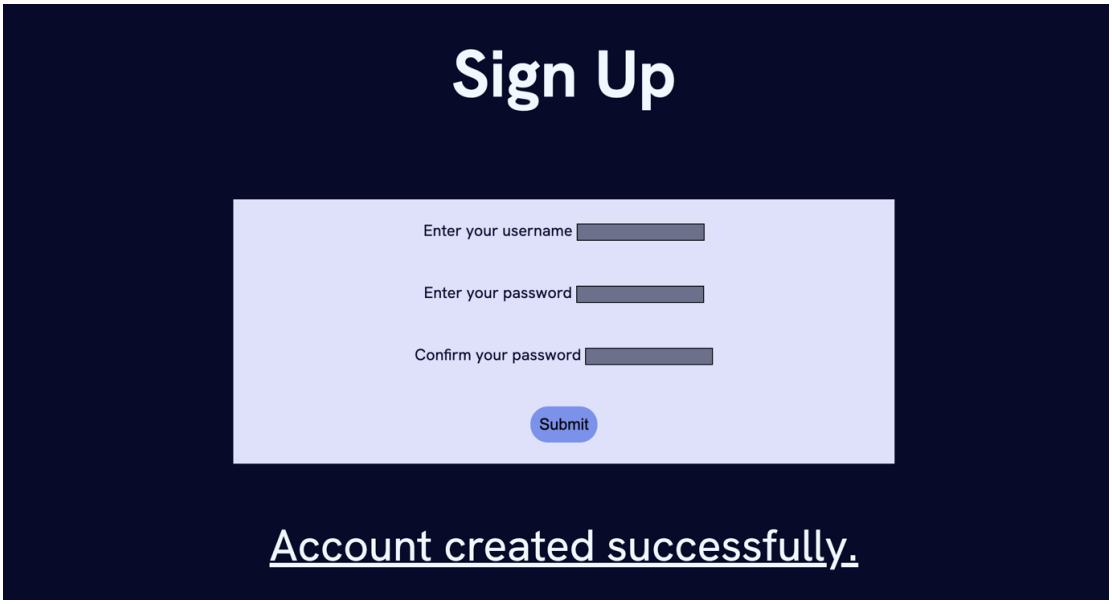
Leading to these outputs without correct styling:

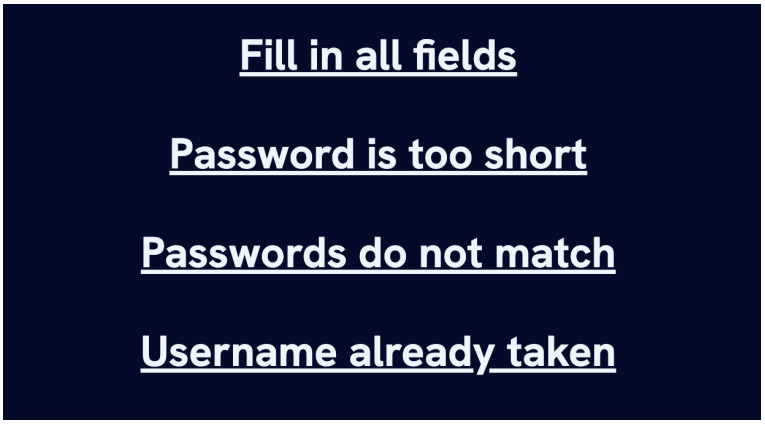
I was able to solve this by changing the echo strings:

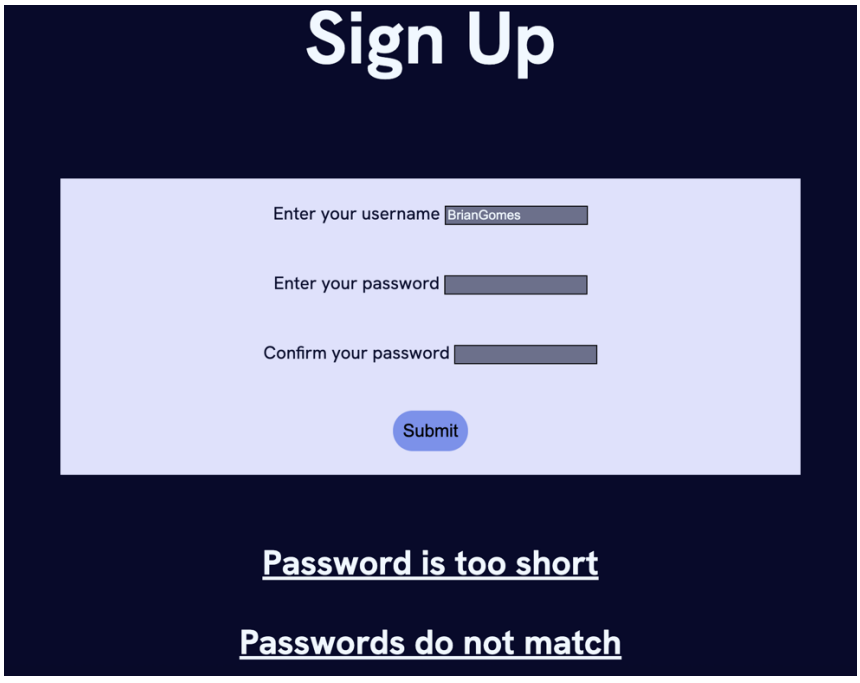
```php
foreach($errors as $error){  //For every error in the array
    echo '<p class="form-error"><b><u>' . $error . '</u></b></p>';  //Output the error
}
```

```php
if(isset($_GET['signup']) && $_GET['signup']==='success'){ //Checks to see if the signup session variable is set and if it was a success
    echo '<p class="form-success"><u>Account created successfully.</u></p>'; //If it was, output a success message
}
?>
```

Now, the messages are styled correctly.







Therefore, the session variables and signup system now worked as intended and now I can move on to the login system.

Stage 2 Review

What has been done

The signup page has been fully implemented with a GUI that almost fully matches the initial design. A file has been created which would allow me to create session variables which will be useful in storing the user's user id when querying the database. The input fields for the signup system are generated by an algorithm (signup_inputs) which retains the username entered by the user if it was valid which is a useful usability feature. Furthermore, validation algorithms for the input data have been implemented which allow for the input data to be checked for length or compared against the database for example and which output error messages when the input data is invalid. Furthermore, if the credentials are valid, a success message is output, and the data is stored in the database.

How it has been tested

I have tested each input field with types of valid and invalid data to see if the correct error/success messages were output. I also tested the submit button which correctly sent the data to signup.inc.php.

The data input in the input field was checked for: whether any input field was empty; if the two input passwords matched, if the password was too short; or if the username was already taken. I also tested to see if the password was being hashed before being sent to the database, which it correctly was.

Criteria met/being met

- Signup Page with hashing algorithm

How it meets the success criteria and user expectations

The signup page functions as intended with input validations and a good usability feature. Furthermore, the input data is hashed before it is sent to the database which I checked for using phpMyAdmin.

Changes in the design that have resulted from this section

The UI of the signup page is slightly different from the design, however, that is because I am creating this website with the intent for it to be used on a laptop or computer. The aspect ratio of the laptop requires more space on the sides of the div; stretching the div to fit the sides of the screen would make the page look badly designed and ruin the minimal look.

Furthermore, I have chosen to implement multiple validation algorithms in PHP in signup_contr.inc.php rather than the two username_validation and password_validation functions as the JS implementations were causing the page to freeze. After following the PHP guide for beginners, I was able to develop my skills in PHP which led me to switch from using JavaScript to PHP for input validation. PHP also

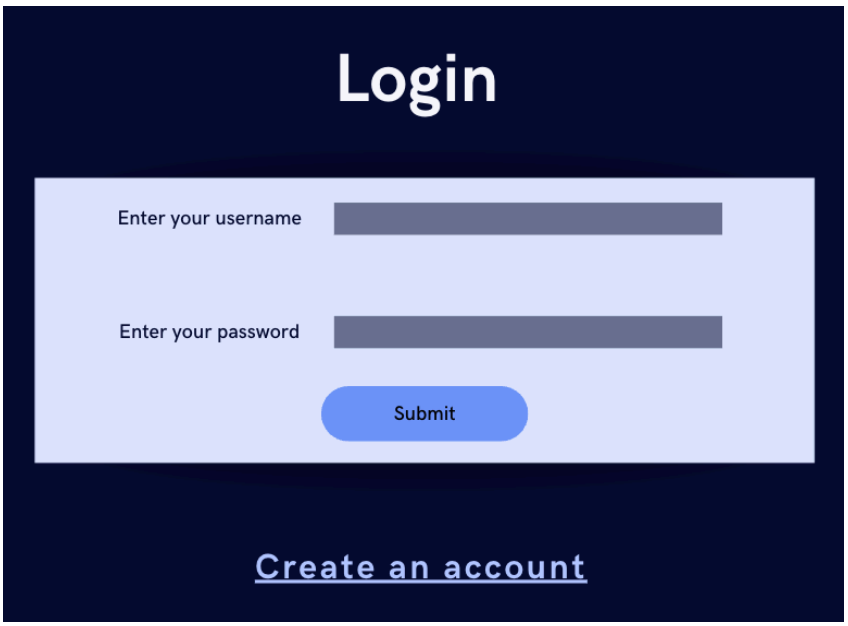
allows for session variables to be created which would allow for error flags in the errors array to be stored securely minimising the risk of data interception and manipulation.

Summary of the whole project as a prototype at this stage

The project currently allows for many users to create their accounts to which the data can be linked, their credentials can all be stored in the USERS table in the NEA database. The program also validates the credentials and hashes them before storage.

Stage 3: Login Page

This is the design that I created for the login page in the design stage.



The key aspects of the design are similar to the ones for the signup page. A lighter coloured div to highlight the input fields, a blue submit button, dark grey input fields and a hyperlink which redirects to the signup page.

I will first begin with creating the GUI for the page.

Creating the GUI of the Page

Similarly to stage 1, I will start by creating the GUI for the page before working on the backend so that I can see the output of any errors.

login.php

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Registration</title>
    <link rel="stylesheet" href="regstylesheet.css">
    <script src="validation.js"></script>

</head>
<body>

        <form>
            <br>
            <b>
                <h1>
                    <label id="Reg">Login</label>
                </h1>
            </b>
            <br><br><br><br>

            <div>
                <br>
                <label>Enter your username   </label><input type="text" id="submituser">
                <br><br><br><br>
                <label>Enter your password   </label><input type="text" id="submitpassword">
                <br><br><br>
                <button onclick=""></button>
            </div>

        </form>

        <br><br><br><br><br><br><br><br><br><br>
        <a href="signup.php">Create An Account</a>

</body>
</html>
```

This page also accesses the stylesheet called ‘regstylesheet.css’ which is shared between both the login and signup pages.

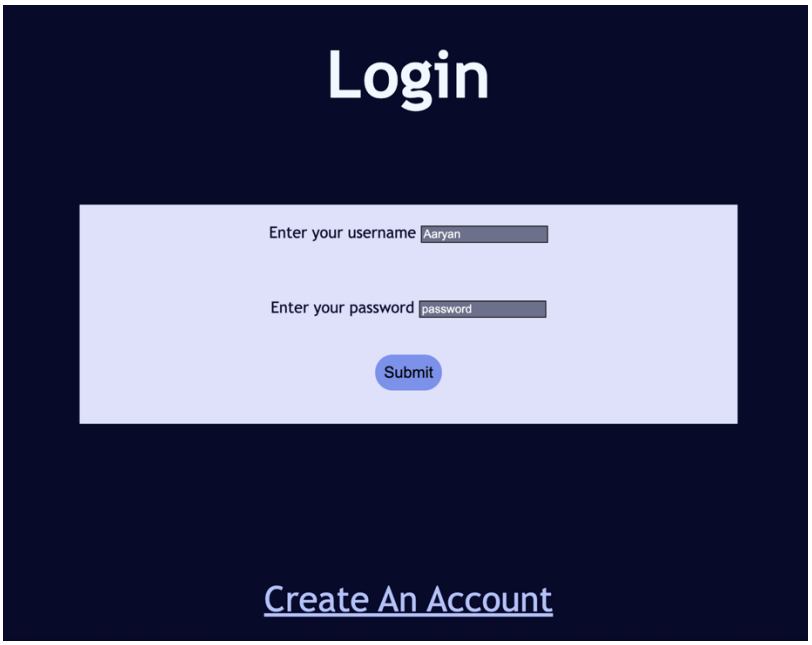
regstylesheet.css

```
body{
  background-color: #070a2a;
  font-family: 'Hanken Grotesk', sans-serif;
  text-align: center
}

form{
  color: #0b0e2e;
}

div{
  display: inline-block;
  margin: 10px auto;
  background-color: #dee2fb;
  height: 250px;
  width: 750px;
  font-size: large;
}

#Reg{
  color: aliceblue;
  font-size: 75px;
}

#Submit{
  background-color: #7c90ea;
  border-radius: 20px;
  font-size: large;
  border-color: #7c90ea;
  border-style: hidden;
  padding: 10px;
}

input[type="text"]{
  background-color : #6c718c;
  color: aliceblue;
  border: 1px solid black;
  box-shadow: none;
}
```

The stylesheet is the same one used by the signup page as the designs are very similar, however one main change is that the div size of the login page has not been overwritten

in the PHP file and it has the div size specified in the stylesheet as it contains fewer input fields

Accessing the styling leads to this output:



The password field does not hide the input password and therefore I will need to change this. Furthermore, currently, the form isn't sending the data to any file for validation and therefore I will need to add action and method variables to the form similarly to how it was with the signup page. I will need to implement a login input handler similar to the signup page and I will do this by following the guide in video 29 of the 'PHP Course for Beginners' YouTube playlist by Dani Krossing, however, by implementing the validation algorithms planned in the design stage instead of the ones in the video.

Initial iteration implementation – updating the form attributes and fields

These are the changes I made to the form tag which solves both of those issues:

```
<!--This declares the form element and specifies where (login.inc.php) and using which method the data will be transferred
(this data is sent using post as it is sensitive data)-->
<form action="includes/login.inc.php" method="post">
<br><br>
    <div>
        <br>
        <label for="username">Enter your username   </label><input id="username" type="text" name="username" autocomplete="on">
        <br><br><br><br>
        <label for="password">Enter your password   </label><input id="password" type="password" name="password" autocomplete="on">
        <br><br><br>
        <input type="submit" id="Submit">
    </div>
</form>
```

I decided to make the form send the data to a file names 'login.inc.php' which will be similar in nature to 'signup.inc.php'. I also added autocomplete and name attributes to the inputs.

Including autocomplete attributes allows for better usability as the user can log in without needing to remind themselves of their password. This makes the program simpler and easier to use for the user.

Adding name attributes to the input field would allow the backend processing files to access the data input in those fields using the POST superglobal.

Furthermore, to style the password input fields, I used the same styling as the one used for the username input fields which I previously implemented in the signup page.

```css
input[type="password"]{
  background-color : #6c718c;
  color: aliceblue;
  border: 1px solid black;
  box-shadow: none;
}
```

Now I will begin working on the backend of the login system to add functionality to the program

Second iteration implementation – implementing the formhandler

config_session.inc.php

```php
if(isset($_SESSION["user_id"])){  //Checks if user_id was stored as a session variable
    if (!isset($_SESSION['last_regeneration'])) {  //If it was, checks if a session variable wasn't created

        regenerate_session_id_loggedin();  //If not, regenerate the logged in session id

    } else {  //If a session variable was created
        $interval = 60 * 30;  //Time until session id regeneration

        if(time() - $_SESSION['last_regeneration'] >= $interval){  //Check if more than 30 mins have passed

            regenerate_session_id_loggedin();  //Regenerate a logged in session id
        }
    }
}else{
    if (!isset($_SESSION['last_regeneration'])) {  //If it wasn't, checks if a session variable wasn't created

        regenerate_session_id();  //If not, regenerate session id

    } else {
        $interval = 60 * 30;  //Time until session id regeneration

        if(time() - $_SESSION['last_regeneration'] >= $interval){  //Check if more than 30 mins have passed

            regenerate_session_id();  //Regenerate session id
        }
    }
}
```

```php
function regenerate_session_id(){
    session_regenerate_id(true); //Regenerate session id

    $_SESSION['last_regeneration'] = time(); //Reset last_regeneration variable
}

function regenerate_session_id_loggedin(){
    session_regenerate_id(true); //Regenerate session id

    $userId = $_SESSION["user_id"];  //Takes the user_id session variable
    $newSessionId = session_create_id();  //Creates a new session id
    $sessionId = $newSessionId . "_" . $userId;  //Concatenate user_id to session variable (creating a more complex compound session id)
    session_id($sessionId);  //Set this as the session id

    $_SESSION['last_regeneration'] = time(); //Reset last_regeneration variable
}
```

I have made some changes to the config_session.inc.php file. It now first checks if the user is logged in (by checking if the 'user_id' session variable has been created). If it has, a different function known as 'regenerate_session_id_loggedin' is called. This function regenerates the session id but also concatenates the user id to it, which increases the session security as it makes it harder for hackers to try to brute force the session id as it is now associated with a user's unique id. This increased security will be vital when the user is logged in as that is when they will be entering their private event data which will need to remain secure, which this facilitates.

login.inc.php

```php
<?php

if($_SERVER["REQUEST_METHOD"] === "POST"){  //Check if page was accessed legitamately
    $username = $_POST["username"];  //Input username
    $password = $_POST["password"];  //Input password

    try {
        require_once 'dbh.inc.php';  //Allows for database linkage
        require_once 'login_model.inc.php';  //Allows for the results of fetching from the database to be accessed by this file
        require_once 'login_contr.inc.php';  //Allows for error-checking functions to be accessed by this file


        $errors = [];  //Storing any errors found in an array

        if (is_input_empty($username, $password)){  //Checks to see if any inputs are empty
         //If they are, create a key-pair value to store the "empty_input" error and the error message of "Fill in all fields"
            $errors["empty_input"] = "Fill in all fields";
        }

        $result = get_user($pdo, $username);  //Store the reasults of querying the database for user credentials in the result variable

        if(is_username_wrong($result)){  //Checks to see if the username was not found in the database
        //If it wasn't, create a key-pair value to store the "login_incorrect" error and the error message of "Incorrect login info"
            $errors["login_incorrect"] = "Incorrect login info";
        }

        //Checks to see if the username was found and if the input password doesn't match the stored password
        if(!is_username_wrong($result) && is_password_wrong($password, $result["hashed_password"])){
        //If it wasn't, create a key-pair value to store the "login_incorrect" error and the error message of "Incorrect login info"
            $errors["login_incorrect"] = "Incorrect login info";
        }

        require_once 'config_session.inc.php';  //Allows for session variables to be created/regenerated

        if($errors){  //If errors are present
            $_SESSION["error_login"] = $errors;  //Store them as a session variable

            header("Location: ../login.php");  //Redirect back to the login page
            die();  //End the program
        }
```

```
        $newSessionId = session_create_id();  //Generates a new session id
        $sessionId = $newSessionId . "_" . $result["user_id"];  //Concatenates the session id with the user id (increased security)
        session_id($sessionId);

        $_SESSION["user_id"] = $result["user_id"];  //The user id is stored as a session variable
        $_SESSION["user_username"] = htmlspecialchars($result["username"]);  //The username is stored as a session variable

        $_SESSION['last_regeneration'] = time();  //The last regeneration time is set to the current time

        header("Location: ../login.php?login=success");  //Redirect to the login page with 'success'
        $pdo = null;  //Unset database variables (best practice)
        $stmt = null;  //Unset database variables (best practice)
        die();  //
    } catch (PDOException $e) {  //If it fails, get the error
    die("Query failed: " . $e->getMessage());  //End the program and output the error
}
}
else{
    header("Location: ../login.php");  //If accessed incorrectly, redirect to the login page
    die();  //End the program
}
```

This file manages the main functions of the login page and calls functions from the other related login files, the database connection file and the session variable file.

It first checks if the file was accessed by submitting the form correctly. If not, it redirects the user back to the login page. This is a type of validation to prevents users from accidentally or intentionally accessing this file without correctly filling in the form.

Next it retrieves the input data from the input fields and stores them as variables. It then runs a try catch block which compares the input data to the stored data. If there was an error with the try block, the catch block will output an error message.

The try block first accesses the files from which it calls the required functions. It then creates an array called ‘errors’ which stores flagged errors, if errors are present, it stores the error message with the error name as the key value.

The program first checks if any input fields are empty and stores the error if they are. Next, the results of checking the user credentials against the database are stored in a variable called ‘result’. This result is passed to the is_username_wrong and is_password_wrong functions to validate the username and password. If either of them are incorrect, the ‘login_incorrect’ key and its corresponding error message is stored in the array.

The file then accesses ‘config_session.inc.php’ to allow for session variables to be created.

If errors are present, the array is stored as a session variable. The file then redirects the user back to the login page and kills the program.

If there weren’t any errors, a new session id is created. This id is then concatenated with the user id of the user to increase the security of the program. The user id and a cleaned (htmlspecialchars makes injected code unusable) username are stored as session variables and the last regeneration session variable is updated with the current time.

The program then redirects back to the login page with login=success which allows for a successful login message to be output. The success messages will be output on the calendar once that has been implemented however, for testing purposes I will currently be implementing them on the login page.

Finally, the pdo object and stmt variable are set to null before the program ends.

Now that the formhandler is complete, I will begin implementing the functions planned in the design stage and accessed by the form handler page.

Third iteration implementation – implementing the functions

login_contr.inc.php

```php
<?php

declare(strict_types=1);  //Best practice to prevent errors in data types

function is_input_empty(string $username, string $password){
    if(empty($username) || empty($password)){  //Checks if either of the input fields are empty
        return true;
    } else {
        return false;
    }
}

function is_username_wrong(bool|array $result){  //To prevent data type errors if the result is false
    if(!$result){  //If there isn't a result
        return true;
    }else {
        return false;
    }
}

function is_password_wrong(string $password, string $hashed_password){
//Built in php function to check if an input variable is the same as another hashed variable
    if(!password_verify($password, $hashed_password)){  //If they are not the same
        return true;
    }else {
        return false;
    }
}
```

This file contains the validation and error checking functions of the program.

The first function checks if any input fields are empty. The second function checks if there was a result when querying the database for the input username. If there was no result, a true is output. The final function checks if hashing the input password matches the hashed password stored in the database. If they do not match, a true is output.

login_model.inc.php

```php
<?php

declare(strict_types=1);  //Best practice to prevent errors in data types

function get_user(object $pdo, string $username){
    //Returns the user_id, username and hashed_password where the input username matches the stored username
    $query = "SELECT * FROM USERS WHERE username = :username;";
    //Using prepared statements is best practice to avoid errors and to enforce security when accessing the database
    $stmt = $pdo->prepare($query);  //Query is prepared
    $stmt->bindParam(":username", $username);  //Username parameter is bound
    $stmt->execute();  //Statement is executed

    $result = $stmt->fetch(PDO::FETCH_ASSOC);  //The username is fetched from the database in an associative array
    return $result;  //The array containing the data is returned
}
```

This file contains a function that interacts with the database.

The database is queried for any records from the USERS table where the username in the database is the same as the input username. This is done using prepared statements which first query the database and then send the parameters to prevent SQL injection attempts. The first matching result is then retrieved and returned.

login_view.inc.php

```php
<?php

declare(strict_types=1);  //Best practice to prevent errors in data types

function output_username(){
    if(isset($_SESSION["user_id"])){  //Checks if the user id is set
        echo "<p class='login-info'><b>Hello " . $_SESSION['user_username'] . "!</b></p>";  //If it is, output a hello message
    }else{
        echo "<p class='login-info'>You are not logged in</p>";  //If it isn't, say you are not logged in
    }
}

function check_login_errors(){
    if(isset($_SESSION["error_login"])){  //If errors are stored as a session variable
        $errors = $_SESSION["error_login"];  //Create a local variable to hold the errors

        echo "<br>";  //Start a new line

        foreach($errors as $error){  //For every error in the array
            echo '<p class="form-error"><b><u>' . $error . '</u></b></p>';  //Output the error
        }

        unset($_SESSION["error_login"]);  //Unset the session variable

    }
    else if(isset($_GET['login']) && $_GET['login'] === "success"){  //If the user logged in successfully
        echo '<br>';
        echo '<p class="form-success"><b><u>Login success</u></b></p>';  //Output a success message
    }
}
```

This file contains the functions which output messages to the user.

The first function is a QoL usability feature which makes the program feel more responsive. Upon a successful login (if the user id is set) a hello message is output to the user. Otherwise ‘You are not logged in’ is output to the user.

The second function returns any errors to the user. It checks if any errors were stored as a session variable and if so, outputs them with the class of form error. If there are no errors a success message is output with the class of form-success. I have fixed the id/class confusion in stage 1 and have remembered to include classes in the styling of the messages.

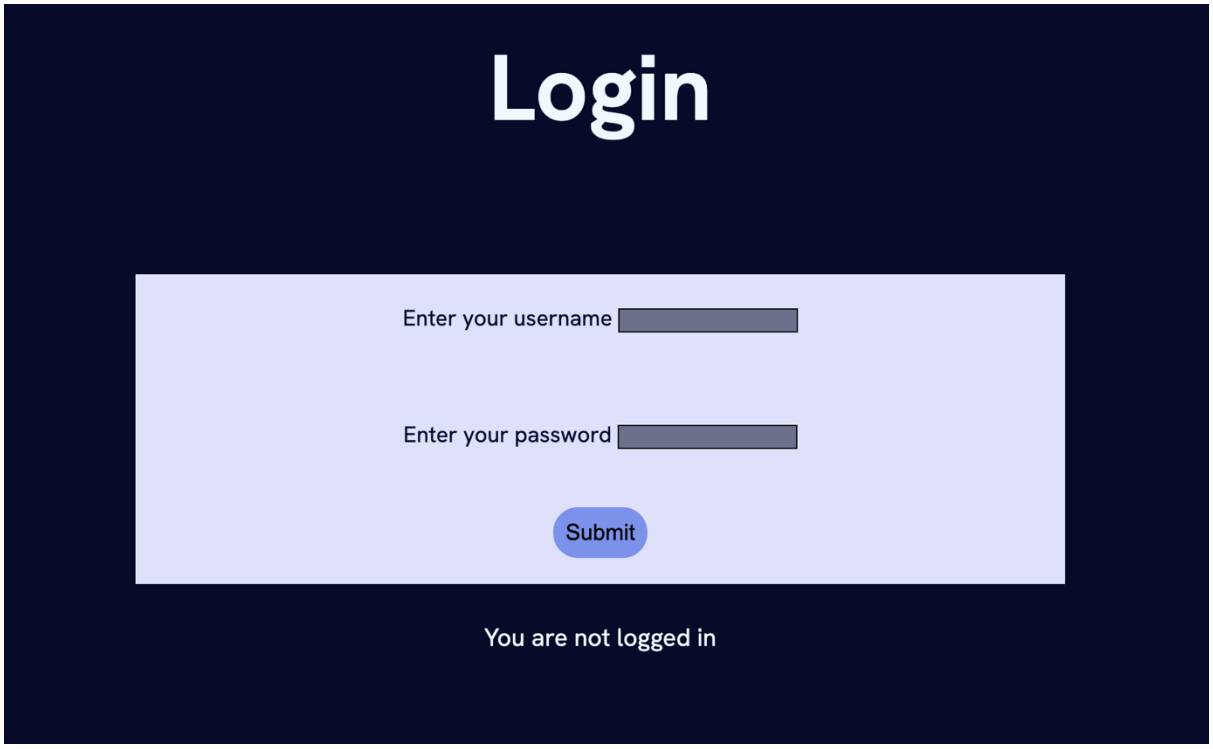
Furthermore, I have deliberately made the incorrect username and incorrect password messages the same as having different messages may allow hackers to brute force the program easier. They could have been able to brute force it until the username error message disappeared and then continued to brute force the program until the password was also correct.

login.php

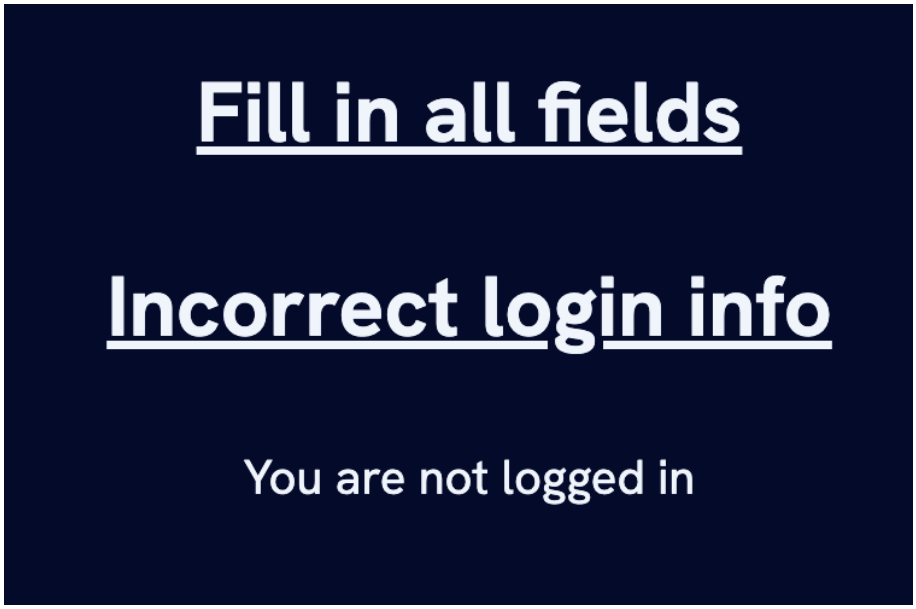
```php
<?php
require_once 'includes/config_session.inc.php';  //Allows for session variable regeneration (added security)
require_once 'includes/login_view.inc.php';  //Allows for check_login_errors() to be accessed
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Registration</title>
    <!--Allows for styling to be accessed-->
    <link rel="stylesheet" href="regstylesheet.css">
</head>

<body>
    <b><h1 id="Reg">Login</h1></b>

    <!--This declares the form element and specifies where (login.inc.php) and using which method the data will be transferred
    (this data is sent using post as it is sensitive data)-->
    <form action="includes/login.inc.php" method="post">
    <br><br>
        <div>
            <br>
            <label for="username">Enter your username   </label><input id="username" type="text" name="username" autocomplete="on">
            <br><br><br><br>
            <label for="password">Enter your password   </label><input id="password" type="password" name="password" autocomplete="on">
            <br><br><br>
            <input type="submit" id="Submit">
        </div>
    </form>

    <?php
        check_login_errors();  //Calls the function which checks for any errors in login such as incorrect login information or blank fields
        output_username();  //Output a hello message if the user logged in successfully
    ?>

    <br><br><br><br>
    <!--A redirect link which takes you to the signup page-->
    <a href="signup.php">Create An Account</a>
</body>
</html>
```

I made some minor changes to the login page. I have accessed the config_session and login_view files at the top of the page to allow for session variables to be regenerated and to output error messages. Furthermore, I have called the check_login_errors and output_username functions underneath the form div to output the messages to the user. The output_username function will be in the calendar page as a message that appears upon login for the user when the page is implemented.

Now I will begin testing the program. First, I will test by inputting an empty username and password field:

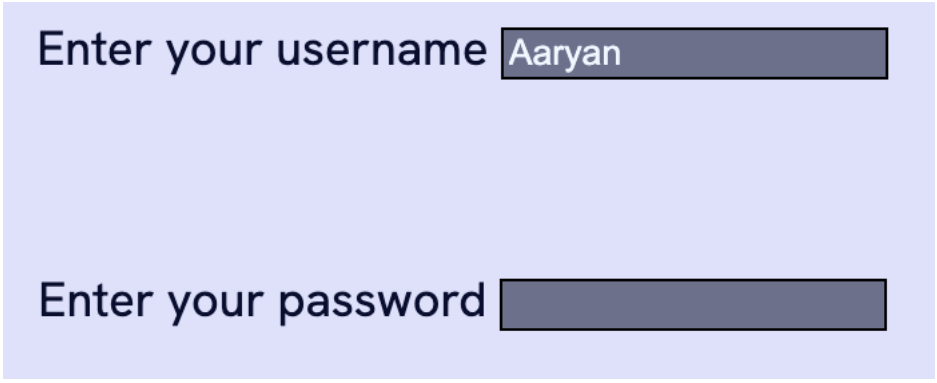


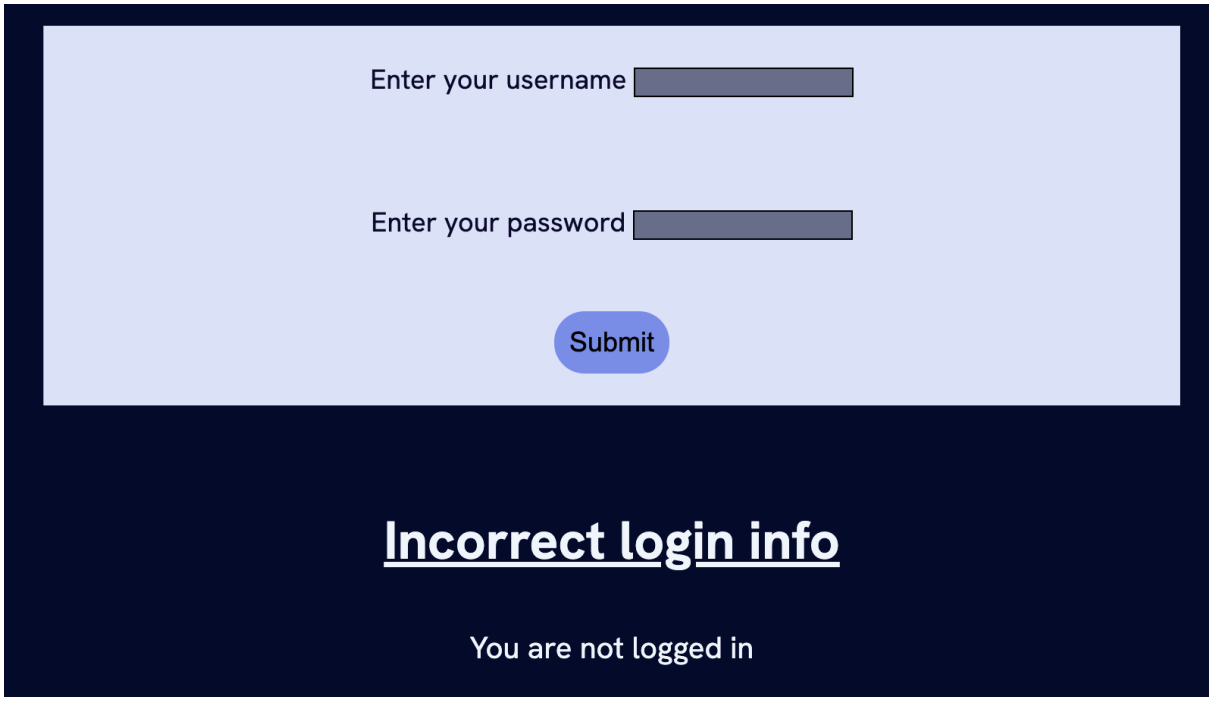
The ‘you are not logged in’ message successfully appeared before entering the credentials.



This outputs the correct validation message, and the login status does not change.

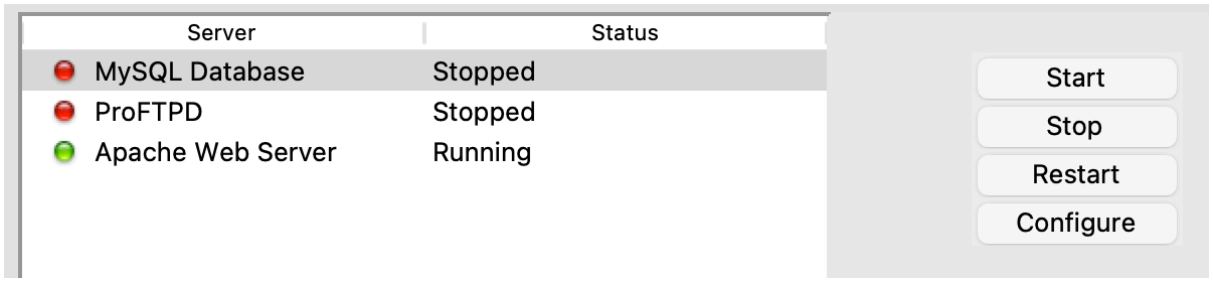
Now I will try with a correct username and an incorrect password field:





The correct error message was displayed, and the login status hasn't changed.

In between tests, I had left XAMPP running on my laptop and when I returned, the MySQL database connection appeared as stopped. I tried starting the connection to the database again, however it wouldn't start even after displaying the 'starting' message for some time.



After researching this issue online, I came across a stack overflow article which stated that this issue is caused due to the SQL database being run in the background and the

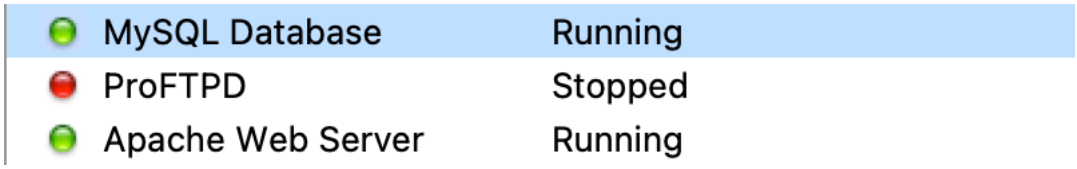
process not being fully stopped. This leads to the live server being unable to establish a new connection to the MySQL database as the previous connection has not yet been stopped. Therefore, to solve this issue, I had to run this line of code in the terminal of my laptop.

```
. sudo killall mysqld
```

This piece of code in the terminal stops all instances of a MySQL database running in the background and therefore allows the connection to be restarted in XAMPP without the live server encountering a collision. Running that line of code and entering my password didn't output any further error message.

```
aaryannair@Aaryans-MacBook-Air ~ % sudo killall mysqld
Password:
aaryannair@Aaryans-MacBook-Air ~ %
```

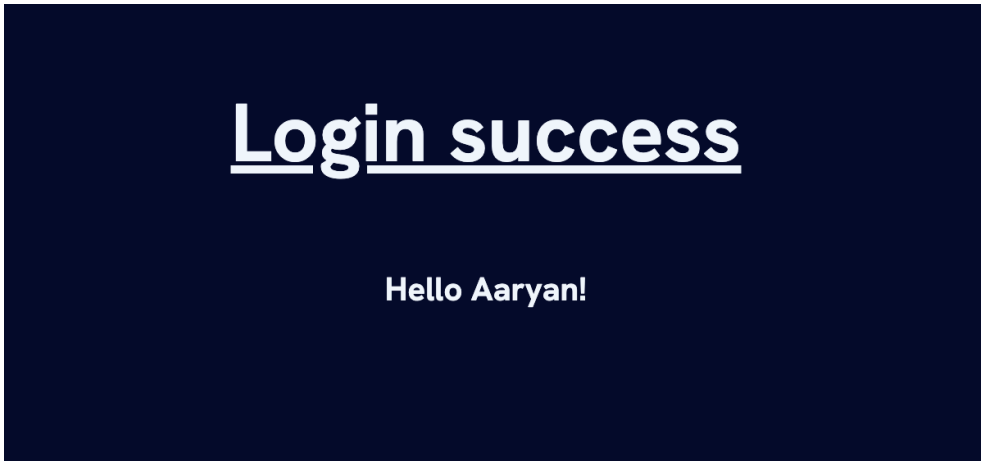
Therefore, I tried starting the database once more and it started running correctly.



This allows me to move on to last test of the login page.

I will now test the page with valid credentials to see if the hello message is output and if the login status changes.

These were the credentials I entered into the input fields; this account will be the main account in which I will test the features.



The login success message and the hello message were correctly output onto the page. This shows that the program can correctly validate the user credentials and output a successful login message when the login is a success.

I will now remove the output_username function from the login.php file as this message will be implemented on the calendar page which will be the home page upon login for the user. I called the function here to test whether it would output the correct message upon a valid login which it was able to. Furthermore, I will also change the successful redirect from 'login.php' to 'calendar.php' as redirecting the user to the home page upon login would be a necessary usability feature for the clients.

```
<?php
    check_login_errors();  //Calls the function which checks for any errors in login such as incorrect login information or blank fields
?>

header("Location: ../calendar.php?login=success");  //Redirect to the calendar page with 'success'
```

The login page has now been successfully created, so I can now begin working on the calendar system.

Stage 3 Review

What has been done

The login page has now been fully implemented with a GUI which almost exactly matches the design created during the design stage. The login page can validate the input data once the submit button is clicked, in the login.inc.php file. There, various validation functions check if the input credentials are valid or not and output corresponding success/error messages. I have also implemented a function which outputs a hello message upon successful login which is a nice usability feature. I have also identified a database connection error which is caused by leaving XAMPP running in the background of the computer without terminating the database connection.

How it has been tested

Each input field in the login system has been tested with valid and invalid data to check if the correct error/success messages were being output. The submit button was also tested each time the data was input in the field to ensure that it sent data to login.inc.php.

I tested the data in the input field for: if the username was not found in the database; if any input field was blank; or if the username was found in the database but hashing the input password didn't match the stored hashed password.

Criteria met/being met

- Login Page with hashing algorithm

How it meets the success criteria and user expectations

The login page contains the correct validation algorithms and good user responsiveness in the form of error and success messages. Furthermore, a built-in hashing algorithm checks if an unhashed input is the same as a hashed input (this is to validate the password).

Changes in the design that have resulted from this section

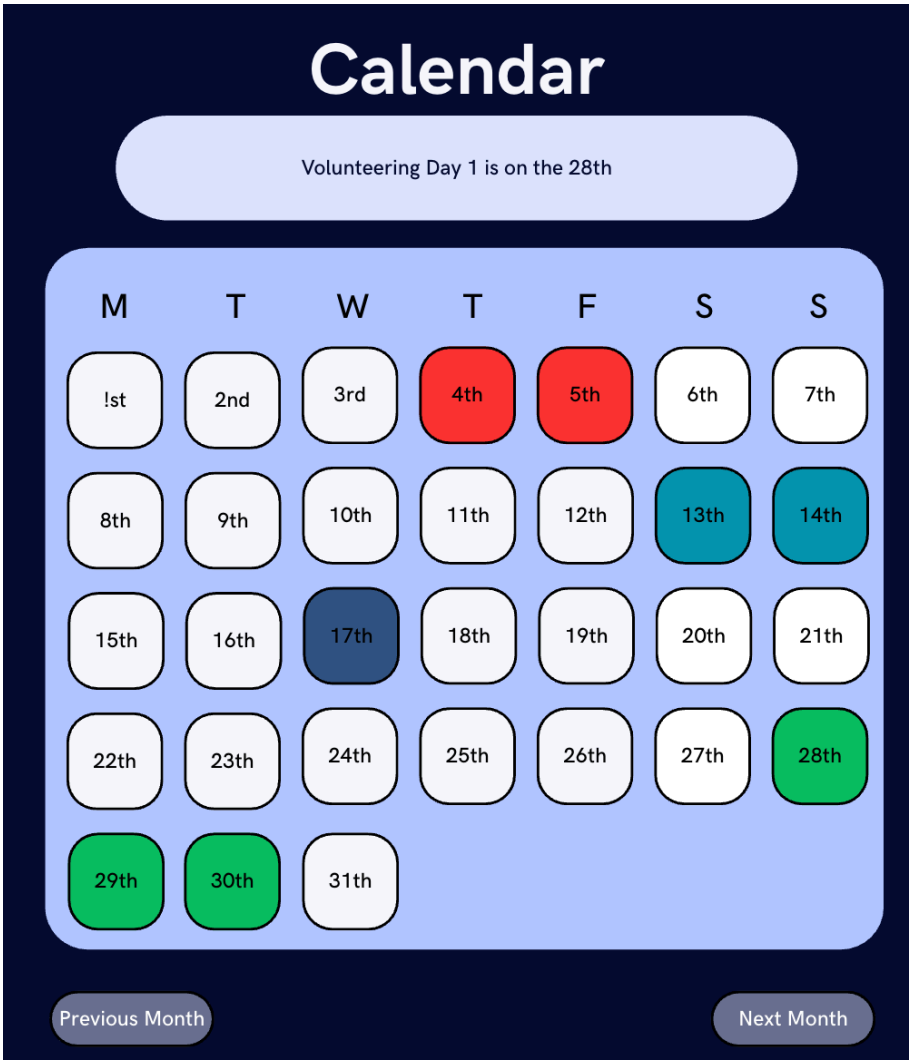
The login page UI has very slightly changed the initial design planned in the design stage. The div is slightly smaller than the planned design which is due to the aspect ratio of the laptop. Furthermore, the submit button is slightly smaller to be less obtrusive in the design.

Summary of the whole project as a prototype at this stage

The project now contains both of the user account related pages and therefore, the accounts system of the program is fully functional. Each input field is validated before the credentials are sent to the database and both pages can successfully query the database created in stage 1. The login page, upon successful login, now redirects to a page called 'calendar.php' which doesn't currently exist but will be implemented in the next stage.

Stage 4: Calendar Page

This is the design that I created in the design stage.



While I was reviewing the design for implementation, I was confused as to how I could implement previous and next month buttons. I could do it using a month session variable which could store the index of the current month. However, due to time constraints I will implement it as a month selector which calls a function to generate the html of the specified month. It will replace the elements in the div with the month which would allow for months to be overwritten.

For this page I will begin by creating a prototype of how I would like to implement the calendar buttons.

```
<form action="inputform.php" method="GET">
<input type="hidden" id="date" name="date" value="25th of July">
<input type="submit" value="25th">
```

25th

I would like each day to have its own form element and hidden input. The submit button would serve as the day button which would send the the date information of the hidden input to the input form page. The title of the input form page would be the date that is sent using GET to the page. These buttons can be arranged in a grid format to form the calendar system.

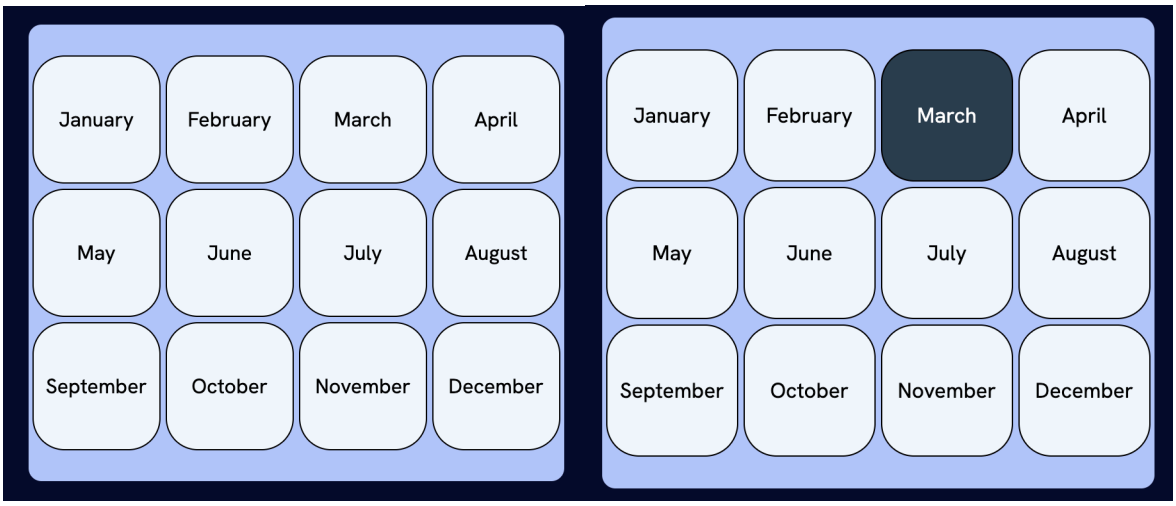
First, I will implement the month selector of the calendar. It will be a 4 by 3 grid which contains the names of each month. Each month button will have an onclick attribute that calls the replace_(month name) function for the specific month and outputs the calendar into a div element known as calendar-container. This is the html for the month selector:

```
<div id="calendar-grid">
        <br>
        <table>
            <tr class="calendar-grid-row">
                <td><button onclick="replace_January()" class="calendar-grid-month">January</button></td>
                <td><button onclick="replace_February()" class="calendar-grid-month">February</button></td>
                <td><button onclick="replace_March()" class="calendar-grid-month">March</button></td>
                <td><button onclick="replace_April()" class="calendar-grid-month">April</button></td>
            </tr>

            <tr class="calendar-grid-row">
                <td><button onclick="replace_May()" class="calendar-grid-month">May</button></td>
                <td><button onclick="replace_June()" class="calendar-grid-month">June</button></td>
                <td><button onclick="replace_July()" class="calendar-grid-month">July</button></td>
                <td><button onclick="replace_August()" class="calendar-grid-month">August</button></td>
            </tr>

            <tr class="calendar-grid-row">
                <td><button onclick="replace_September()" class="calendar-grid-month">September</button></td>
                <td><button onclick="replace_October()" class="calendar-grid-month">October</button></td>
                <td><button onclick="replace_November()" class="calendar-grid-month">November</button></td>
                <td><button onclick="replace_December()" class="calendar-grid-month">December</button></td>
            </tr>
        </table>
        <br>
    </div>
```

I have styled it using CSS to make the design of the selector as similar to the rest of the program as possible. I have used specific margins and padding to make the month selector easy to view and in the centre of the page. I have also made the buttons large, with a white background colour and a border to allow for better readability and usability.

```
#calendar-grid{
    background-color: #b5c4fa;
    margin-left: 35.75%;
    margin-right: 35.5%;
    border-radius: 10px;
    padding: 10px, 10px, 10px, 10px;
}

.calendar-grid-row{
    padding-bottom: 100px;
}

.calendar-grid-month{
    font-family: 'Hanken Grotesk', sans-serif;
    width: 100px;                    /*Full cell width*/
    height: 100px;                   /*Full cell height*/
    background-color: aliceblue;   /*Blueish White background*/
    border: 1px black solid;
    border-radius: 25%;
    cursor: pointer;                 /*Pointer on hover*/
    font-size: medium;
    transition: background-color 0.2s;

}

.calendar-grid-month:hover{
    background-color: #2c3e50;   /*Blue on hover*/
    color: white;
}
```

This design led to this output:



I have added styling for when the button is being hovered over and for when the button is selected to make the program feel more responsive and more user-friendly. Furthermore, the buttons are large and well contrasted from the background which is great for usability.

I will now implement the functions to generate the calendar underneath the calendar selector. These functions will be declared and stored in the calendargeneration.js file and will then be called by calendar.php.

Initial iteration implementation of replace_(month name)

```
//Returns suffix for a day for the nodes on the calendar page
function suffix(n) {
    if (n >= 11 && n <= 13) return "th";  //Special case of dates
    if (n % 10 === 1) return "st";  //Checks if final digit is 1, if it is, return st
    if (n % 10 === 2) return "nd";  //Checks if final digit is 2, if it is, return nd
    if (n % 10 === 3) return "rd";  //Checks if final digit is 3, if it is, return rd
    return "th";
}
```

This function is a simple function which generates the corresponding suffix for each date. This prevents the need for manual date suffix declaration and makes the program more modular. It checks for special cases first, and then for usual cases and returns the correct suffix for each date. This suffix will be displayed on the submit buttons and will form part of the GET value sent to the input form.

```
// Function to create a td containing a clickable day button
function create_day_cell(day, monthName) {
    //Create a table cell td to hold the day
    const td = document.createElement("td");
    td.className = "calendar-cell"; // Add a CSS class to td for styling

    //Create a form element for this day
    const form = document.createElement("form");
    form.method = "GET";           // Use GET method
    form.action = "inputform.php"; // Submit to server

    //Create a hidden input to store the full date
    const hidden = document.createElement("input");
    hidden.type = "hidden";                 // Hidden from user
    hidden.name = "date";                   // Form field name
    hidden.value = day + suffix(day) + " of " + monthName;

    //Create the visible submit button for this day
    const submit = document.createElement("input");
    submit.type = "submit";                  // Make it clickable
    submit.value = day + suffix(day);       // Button text

    //Add a CSS class to the button for styling
    submit.className = "calendar-day";       // Assign class "calendar-day"

    //Add hidden input and button to the form
    form.appendChild(hidden);
    form.appendChild(submit);

    //Add the form to the table cell
    td.appendChild(form);

    //Return the completed table cell
    return td;
}
```

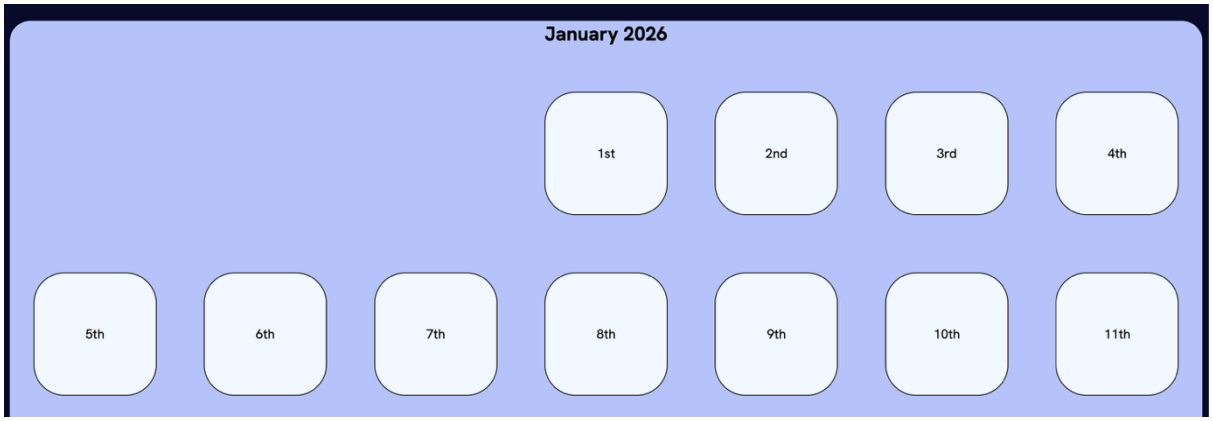
This is the create_day_cell function which creates the prototype hidden input form element in the form of a td element as I will be using tables to implement the calendar as a grid. A td element is first created, and a class is added to it for styling. Then, the form element which redirects to the input form is created for each individual input. This ensures that only one get value is sent to the input form when a submit button is clicked on the calendar (as there are hidden inputs for each day in the calendar). The hidden input is then declared with the name ‘date’ and with the value of a string which states the date. Next, the submit button is created which has the day and the suffix on it as the value. A class is added to the submit button for styling purposes. The input elements are appended to the form, the form is appended to the td element, and the td is returned.

```
function replace_January() {
    const container = document.getElementById("calendar-container"); // Select container
    const heading = document.createElement("h2");   // Create month heading
    heading.textContent = "January 2026";

    const table = document.createElement("table");
    table.className = "calendar-table";            // CSS class for table

    const rows = [
        [null, null, null, 1, 2, 3, 4],
        [5,6,7,8,9,10,11],
        [12,13,14,15,16,17,18],
        [19,20,21,22,23,24,25],
        [26,27,28,29,30,31,null]
    ];

    for (let i = 0; i < rows.length; i++) {
        const tr = document.createElement("tr");
        tr.className = "calendar-row";            // CSS class for row

        for (let j = 0; j < rows[i].length; j++) {
            if (rows[i][j] === null) {
                const td = document.createElement("td");
                td.className = "calendar-cell";   // CSS class for empty cell
                tr.appendChild(td);
            } else {
                tr.appendChild(create_day_cell(rows[i][j], "January")); // Day cell already has class
            }
        }
        table.appendChild(tr);
    }

    container.replaceChildren(heading, table);    // Replace old content
}
```

This is one of the 12 replace_(month name) functions. Each function has a different textContent for the heading, a different rows const which relates to the 2026 calendar, and a different passed value for the create_day_cell append line.

Firstly, the div in which the calendar will be generated is retrieved by its id, calendar-container. Next, the heading for the month is declared as a h2 (to ensure consistent heading hierarchy) and the month name and year are stored as its textContent. The table element is created which will hold each month in a regular grid format and it is given a class for styling. The rows const which is a 2d array which has the calendar data for the month for 2026 is declared. Next, a for loop is created for each row in the rows array, a tr element is created with class name calendar-row. Then, for each value in the sub array in rows, it is checked if the value is null. If it is, a td element is created with the class calendar-cell (for it to be styled as a blank cell) and it is appended to the row. If the value was not null, the td value returned by create_day_cell when passing the date and month to it is appended to the row. Then, the row is appended to the table. Once all rows have been appended, the div content is replaced with the heading and the table.

Implementing these functions, adding styling for each class, and clicking on January in the month selector leads to this output:



Furthermore, hovering over the month buttons gives this output:

This output is correct and is a good prototype for the calendar. The buttons are large and well contrasted, it is accurate to the 2026 calendar, and the suffixes are correct for each button. The calendar also fills almost the entire screen width which is also good for usability. When hovered over, the buttons transition to a different colour which is another helpful usability feature.

I similarly tested the remaining month selector buttons, and each button generated the data correct for its month. The program passes the initial tests and now I will move on to the next iteration of the program.

Second iteration implementation – adding the header row

However, comparing this output to the design, this test does show an error. There is no heading row which states the day of the week. That heading row is a useful usability feature as it makes the days of the week easy and clear for the user's reference, and it is also helpful in the user's planning. Therefore, I now need to fix this issue.

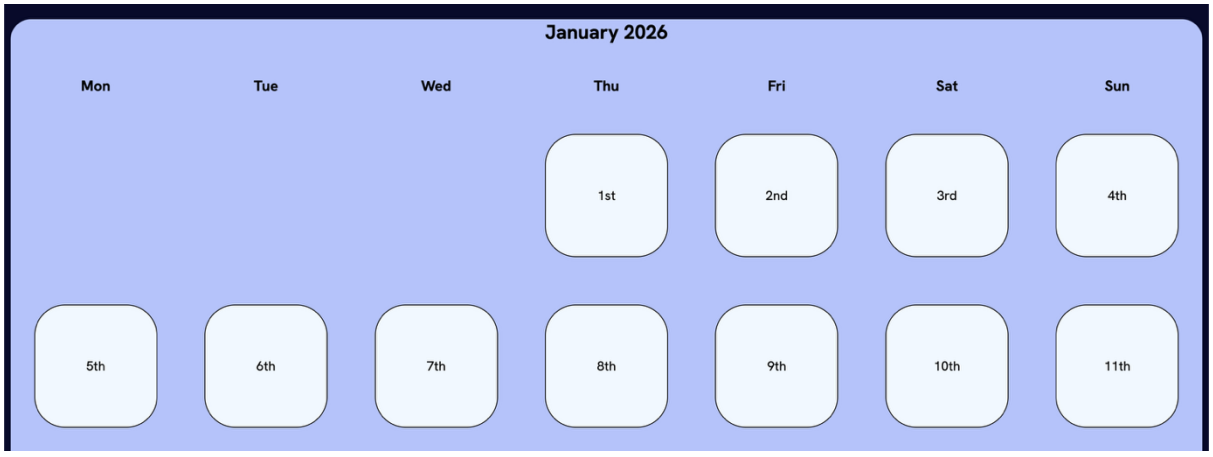
To solve this, I have chosen to add this block of code to each of the replace_(month name) functions before the row declaration:

```
//Weekday header
const headerRow = document.createElement("tr");  //A tr element is created to hold the day indicators
const daysOfWeek = ["Mon","Tue","Wed","Thu","Fri","Sat","Sun"];  //Array storing abbreviated days of the week
for (let d of daysOfWeek) {  //For every day in the array
    const th = document.createElement("th");  //Create a th element
    th.className = "calendar-cell calendar-weekday";  //Add a class for styling purposes
    th.textContent = d;  //Make the text content the day abbreviation
    headerRow.appendChild(th);  //Append the th to the tr
}
table.appendChild(headerRow);  //Append the tr to the table
```

This piece of code first creates a tr element to hold the day values. It then creates an array of the abbreviated day values called daysOfWeek. Then, for each day in the daysOfWeek array, a table heading element is created with the classes calendar-cell and calendar-weekday (for the background to be like an empty cell but for the text to be styled). The text content of the th is set to the day and it is appended to the header. Once all of the days have been added, the header row is appended to the table.

This code is modular in nature as the daysOfWeek array can be edited for parts of the world which may prefer to use ‘Sun’ as the starting day rather than ‘Mon’. This modularity allows for further iterations of the program made for specific regions of the world to be easily modified and changed to suit their cultural preferences.

After implementing this line of code and refreshing the page, the program gave this output:



This output is now almost perfect as the user now has a helpful indicator for the days of the week which correspond to the days in the calendar. This is a good usability feature for the user, and it has been implemented in a way which is modular and can be easily changed for future iterations of the program.

Third iteration implementation – fixing the date value formatting

I now began planning how to add the colour coding feature to the calendar and so I started rereading the code. When reviewing the code, I realised that the hidden value is

currently stored in a human-readable format which may cause issues when I implement SQL connection to the input form if the declaration doesn't match. After further research on the official format (linked in bibliography), I realised that the format I am currently using for the date values is incorrect. The dates should be formatted as YYYY-MM-DD, which is the SQL format of storing the dates but currently it is stored as such:

```
hidden.value = day + suffix(day) + " of " + monthName;
```

Therefore, I will need to add another section in the create_day_cell function which will format the date before it is associated as the hidden input's value. This is the code:

```
//Format date as YYYY-MM-DD to match database format
const date = new Date(month + " " + day + ", " + year);
let monthNum = date.getMonth() + 1;
let dayNum = date.getDate();
if (monthNum < 10){
    monthNum = "0" + monthNum;
}
if (dayNum < 10){
    dayNum = "0" + dayNum;
}
const fullDate = year + "-" + monthNum + "-" + dayNum;
```

I have first created a date element which is formatted using the built-in JavaScript date object. Then I declared the monthNum variable which stores the month number in the correct format using the built in getMonth method. As JavaScript indexes months starting from 0, 1 is added to the result of getMonth to ensure the correct month value is stored. Next, I declared the dayNum variable which uses the getDate method to return the date in the correct format. Next, if monthNum or dayNum are single digit numbers, leading zeros are added to ensure the format matches the SQL date format (YYYY-MM-DD). This will allow for correct date inputs in the input form for the database for event dates. Finally, the full date is formatted using the year, monthNum and dayNum separated by hyphens (for the exact SQL format).

```
hidden.value = fullDate;  //The value of the input field is the date formatted correctly for the database
```

Finally, I also changed the hidden input value to fullDate to solve the issue of formatting.

Fourth iteration implementation – making the code more modular

While I was reading the code, I also noticed that having unique replace_(month name) functions for each month with only three major changes causes redundancy and inflates the code size. Therefore, I chose to generalise the replace_(month name)

functions to one replace_month function. This allows the code to be modular in nature and further reduces code redundancy.

To do this, I will need to implement the 3 changes which were present in the functions in different ways. The main change between functions was the rows array. I identified the fact that the starting index of the month and the number of total days are the important aspects of the rows array and therefore I chose to implement a dictionary called calendarData to store this information for each of the months as a nested dictionary.

```
//Calendar Data for 2026 stored in nested dictionaries
//If startDay: 1 = Monday, 7 = Sunday etc for days of the week
//totalDays: number of days in month
const calendarData = {
    "January":   { startDay: 4, totalDays: 31 },
    "February":  { startDay: 7, totalDays: 28 },
    "March":     { startDay: 7, totalDays: 31 },
    "April":     { startDay: 3, totalDays: 30 },
    "May":       { startDay: 5, totalDays: 31 },
    "June":      { startDay: 1, totalDays: 30 },
    "July":      { startDay: 3, totalDays: 31 },
    "August":    { startDay: 6, totalDays: 31 },
    "September": { startDay: 2, totalDays: 30 },
    "October":   { startDay: 4, totalDays: 31 },
    "November":  { startDay: 7, totalDays: 30 },
    "December":  { startDay: 2, totalDays: 31 }
};
```

The month data can be accessed by using the key value associated with the month. Blank cells can be implemented before the start day and day cells can be implemented using a while loop by calling the create day cell function until a counter value is equal to the total days value.

```
//Replace month function
function replace_month(month, year = 2026) {  //The year value has been set to 2026 (*This can be changed to allow for more years*)
    const container = document.getElementById("calendar-container");  //Retrieves the div to hold the calendar data
    container.innerHTML = ""; //Clears previous content

    //Month heading
    const heading = document.createElement("h2");  //Creates a h2 element
    heading.textContent = `${month} ${year}`;  //The text will be the month and the year
    heading.className = "calendar-header";  //Class added for styling purposes

    //Table setup
    const table = document.createElement("table");  //Table is created to hold days in a grid format
    table.className = "calendar-table";  //Class for styling purposes

    //Weekday header
    const headerRow = document.createElement("tr");  //A tr element is created to hold the day indicators
    const daysOfWeek = ["Mon","Tue","Wed","Thu","Fri","Sat","Sun"];  //Array storing abbreviated days of the week
    for (let d of daysOfWeek) {  //For every day in the array
        const th = document.createElement("th");  //Create a th element
        th.className = "calendar-cell calendar-weekday";  //Add a class for styling purposes
        th.textContent = d;  //Make the text content the day abbreviation
        headerRow.appendChild(th);  //Append the th to the tr
    }
    table.appendChild(headerRow);  //Append the tr to the table

    //(*calendarData[month] can be replaced with calendarData[year][month] to allow for more years*)
    const { startDay, totalDays } = calendarData[month];
    let day = 1;
    let tr = document.createElement("tr");

    //Add empty cells before first day of the month
    for (let i = 1; i < startDay; i++) {
        const td = document.createElement("td");
        td.className = "calendar-cell";
        tr.appendChild(td);
    }
```

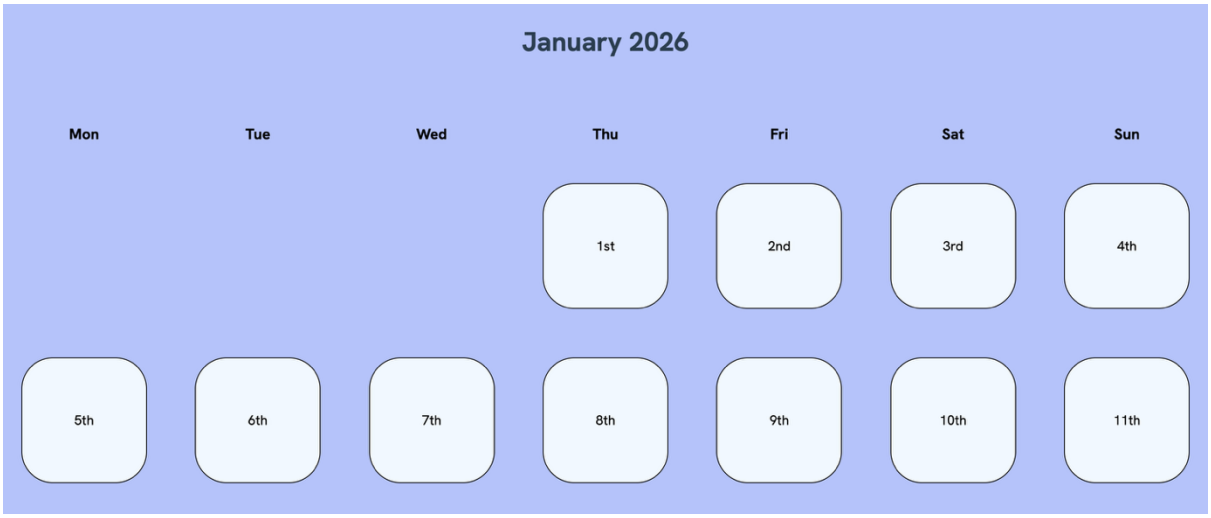
```
    //Add the days to the calendar
    while (day <= totalDays) {
        tr.appendChild(create_day_cell(day, month, year));
        day++;

        if (tr.children.length === 7) {
            table.appendChild(tr);
            tr = document.createElement("tr");
        }
    }

    //Add remaining empty cells in last row
    while (tr.children.length > 0 && tr.children.length < 7) {  //While there are empty spaces in the last row
        const td = document.createElement("td");  //Create a td element
        td.className = "calendar-cell";  //Add a class for styling
        tr.appendChild(td);  //Append it to the row
    }
    //Add last row to calendar if it has values
    if (tr.children.length > 0){  //If the last row has at least one value
        table.appendChild(tr);  //Add it to the table
    }

    container.appendChild(heading);  //Add the heading to the div
    container.appendChild(table);  //Add the table to the div
}
```

The first change I made to the code is that the function takes the month name as a parameter to be specific for the month. The next major change is after the header row. A const is declared which stores the startDay and totalDays for the passed month. A day incrementor and a tr row element is declared. Then, for every day before the start day, blank cells are created (td elements with the calendar-cell class). After that, while the day is less than or equal to the totalDays, the day cells are created for each day, and the day value is incremented. If the row is 7 elements long, the row is appended to the table and a new row is created. Once all day nodes have been created, if there is any leftover row with less than 7 elements but at least one day element, it is added to the table. Finally, the heading and table are appended to the div.

These changes have allowed my code to be more modular in nature as the month names can be passed to a singular function rather than creating multiple functions with minimal differences. Furthermore, implementing the rows 2d array as one calendar data dictionary instead further simplified the code and allows for future maintenance as the calendar data dictionary can either be duplicated for a different year's data or another layer of nesting can be implemented with a key value of the year to allow for more years in the calendar.

```
    <!--Month selector for calendar (implementing next month/previous month functionality
    was challenging within the time constraints and with my current skills so I chose to implement a selector)-->
<div id="calendar-grid">
    <br>
    <table>
        <tr class="calendar-grid-row">
            <td><button onclick="replace_month('January')" class="calendar-grid-month">January</button></td>
            <td><button onclick="replace_month('February')" class="calendar-grid-month">February</button></td>
            <td><button onclick="replace_month('March')" class="calendar-grid-month">March</button></td>
            <td><button onclick="replace_month('April')" class="calendar-grid-month">April</button></td>
        </tr>

        <tr class="calendar-grid-row">
            <td><button onclick="replace_month('May')" class="calendar-grid-month">May</button></td>
            <td><button onclick="replace_month('June')" class="calendar-grid-month">June</button></td>
            <td><button onclick="replace_month('July')" class="calendar-grid-month">July</button></td>
            <td><button onclick="replace_month('August')" class="calendar-grid-month">August</button></td>
        </tr>

        <tr class="calendar-grid-row">
            <td><button onclick="replace_month('September')" class="calendar-grid-month">September</button></td>
            <td><button onclick="replace_month('October')" class="calendar-grid-month">October</button></td>
            <td><button onclick="replace_month('November')" class="calendar-grid-month">November</button></td>
            <td><button onclick="replace_month('December')" class="calendar-grid-month">December</button></td>
        </tr>
    </table>
    <br>
</div>

<br><br>

<!--This is where the selected calendar month HTML will be inserted-->
<div id="calendar-container">
    <h2 id="selection-message">Select a month</h2>
</div>
```

I have also changed the month selector to call the correct functions onclick and have passed the month data for each button which led to this output onclick:



I also fixed the styling of the title to be the correct heading size, and the program has passed the test. The calendar page as a prototype is almost fully complete, I will now try to implement the final colour coding feature

Fifth iteration implementation – adding colour coding

To add colour coding to the days, I will need to add a statement to the create_day_cell function which checks if the date has a tag associated with it in a dictionary called event tags which stores the date and tag data from the database as key pair values.

Firstly, I will add the code to create day cell to check for colour coding.

```
    //Color-code cell based on eventTags
    if (eventTags[fullDate]) {  //If the date has a colour coding value associated with it in eventTags
    submit.classList.add(eventTags[fullDate]);  //Add the tag from eventTags to the class list of the button
}
```

This statement checks if the date for the day cell has a tag set in the dictionary eventTags. If so, another class is added to the submit button which allows for it to be colour coded. This is added as a final check before the button is appended to the td which serves as validation as to whether or not the day has a colour coding value associated with it. Furthermore, implementing the check as an if set allows for modularity and scalability as it isn't dependant on the current tags in the database and therefore the code doesn't need to be changed when more tags are added.

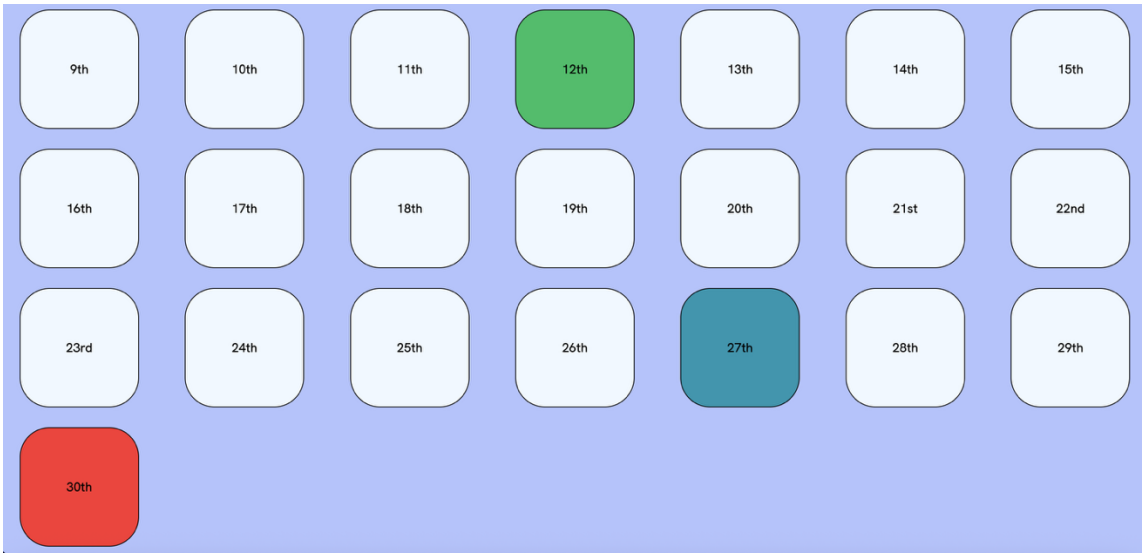
```
$userId = $_SESSION['user_id'];

/*Retrieves relevant event information (date and tag) for the logged in user*/
$stmt = $pdo->prepare("SELECT EVENTS.date_of_event, EVENTS.event_tag FROM EVENTS INNER JOIN LINK_TABLE
ON EVENTS.event_id = LINK_TABLE.event_id WHERE LINK_TABLE.user_id = :user_id;");
$stmt->bindParam(":user_id", $userId);  //Prepared statement parameters are passed (user_id)
$stmt->execute();
$events = $stmt->fetchAll(PDO::FETCH_ASSOC);  //Fetch all to retrieve every entry that matches the SQL query
```

Next, I added this to the top of the calendar page which retrieves the user id and then queries the database for the event data and tag using a prepared statement. The fetched value is stored as an associative array and stored as variable events.

```
<!--Creating a const in JS which is accessible by the replaceMonth() functions which stored the date and tag of
tagged events for colour coding-->
<script>
    const eventTags = {
        <?php
        $total = count($events);
        $i = 0;
        foreach ($events as $event) {  //Loops through associative array of event tags and dates fetched from database
            $i++;
            $date = $event['date_of_event'];  // Must be in format 'YYYY-MM-DD'
            $tag = $event['event_tag'];  // The tag string stored in DB
            echo "'$date':'$tag'";  // Adds them as key-value pairs into eventTags
            if ($i < $total) echo ",";  // Adds commas between pairs for correct syntax
        }
        ?>
    };
</script>
```

I defined this dictionary at the end of the calendar page and can be accessed by the month selector and create_day_cell functions. Each row in the array events is converted into key pair values for the dictionary eventTags which stores all of the events. This allows for colour coding to work in the program and can be tested after the input form is implemented.

However, to manually test if the colour coding CSS worked, I manually added the colour coding to some events using inspect element which led to this output:



This is shows that the colours are being properly implemented and styled and therefore now the only test remaining is if these can be dynamically implemented in the next stage.

Sixth iteration implementation – adding the legend

I will now add a legend div which will serve as a reference for the user and will be a good usability feature. I will implement this underneath the calendar and in a modular two column layout so when more tags are added, they can be added to the columns.

```
<!--Legend which acts as reference for colour coding-->
<h2 id="subtitle">Legend</h2>

<!--Two column layout to show a circle with the colour-coding colour and the tag name-->
<div class="row" id="legend">
    <div class="column">
        <div class="circle-work-experience"></div>
        <div class="circle-course"></div>
        <div class="circle-competition"></div>
    </div>
    <div class="legend-tags">
        <h1>Work experience</h1>
        <h1 class="legend-tags-below">Course</h1>
        <h1 class="legend-tags-below">Competition</h1>
    </div>
</div>
```

I have implemented a circular div which displays the colour next to a heading element which states the tag name. I have styled it in a two-column layout which makes the div more readable. Furthermore, to improve readability, I have made the background colour of this div white which is different to the other sections (calendar, month selector etc) to highlight to the user that the contents of this div are not interactable which is a useful usability feature.

After styling the html appropriately to implement to correct colours and margins, the program produced this output:

This is the correct output as it allows the user to have a reference for their colour coding. The headings are aligned correctly, and the colours correspond to the CSS styling for the tagged events.

Furthermore, the legend was implemented on the same page as the calendar as this was one of the requested features by the stakeholders when I sent them the mock designs.

Finally, I called the output_username function on the calendar page underneath the title (as planned in the login stage). This feature serves as a good usability feature and makes the program feel more responsive upon login.

```
output_username();  //Outputs a 'Hello (username) !' for the user who is currently signed in
```

Which produced this output:

The message was correctly displayed and formatted.

The calendar page is now complete with correct usability features (large buttons for month selector, light background div colours for contrast, header row etc), colour coding and the legend div. Now I will implement the input form to allow for dynamic colour coding and insertion of data into the database.

Stage 4 Review

What has been done

The calendar page is now complete and functional. A month selector has been implemented which outputs the month data to a div underneath the selector. The month data is correctly output according to the 2026 calendar with a heading, a header row which states the day of the week, and the day buttons. Each button is a submit

button for an individual form element and hidden input which redirects to the input form with the date as a get value. The date value has been correctly formatted for each button so that the value is of a suitable format for use with the MySQL database. Furthermore, the code allows for checking for events with colour coding for which it adds classes to the day cell for it to be styled. The multiple replace_(month name) functions have been replaced with a single replace_month function and a dictionary called calendarData which allows for modularity and scalability as data for more years can be stored using further nesting of dictionaries. The areas which would need to be changed for future maintenance have been highlighted with the changes stated so that future developers can expand the calendar data.

How it has been tested

The program has been tested for usability features such as testing button styling (large size, light background colour etc) and header rows indicating the day of the week associated with a day in the calendar.

The program has also been tested for important areas for future pages such as for correct formatting of dates to match the MySQL formatting to allow for database linkage and for the correct styling of colour coded days which will be further tested in the next stage once events can be tagged in the input form.

Criteria met/being met

- Calendar System for past, present and future events
- Colour coding of events

How it meets the success criteria and user expectations

The calendar system showcases calendar data for the year of 2026. This page displays the earlier, middle, and later months of 2026 which makes the calendar system for past, present and future events. This implementation of the calendar page is limited; however, it is a useful initial implementation of the feature which can be developed further in future iterations of the program by adding more data to the calendarData dictionary. This has been clearly annotated in the calendargeneration.js file which will allow the code to aid future maintenance of the program.

The calendar system has also implemented a check for colour coding which uses a dictionary known as eventTags to store the event date and tag information fetched from the database. This dictionary is then checked for in the create_day_cell function which adds a CSS class to the day cell to allow it to be colour coded. This feature will be further tested once tagging is implemented in the input form.

Changes in the design that have resulted from this section

There have been a few changes in the design in this stage of development. The main change that has occurred is the implementation of a month selector instead of previous

and next month buttons. This change was implemented due to time and skill constraints; however, I believe that the change was a beneficial one. Including a month selector with large buttons and contrasting colours to the background instead of previous and next month buttons is better for usability. The selector has been styled for it to contrast with the darker colour background and for it to be easily visible to the user upon login. Previous and next month buttons were planned to be implemented underneath the month which may have caused confusion to some users as the buttons were not immediately visible. Therefore, I believe that implementing a month selector was a justified change from the design.

The second change from the design are the changes to the replace_(monthname) and create_day_cell functions. The replace_(monthname) functions have bene generalised into one replace_month function which allows the code to be more modular and easier for maintenance in future iterations of the program. Furthermore, the create_day_cell function was changed in two major ways. The function's hidden value has been changed to a format which matches the SQL database (otherwise it would cause issues in further stages of development). Furthermore, an if statement was added which checks to see if the event has a colour coding value associated with it to add colour coding to day cells. This change was necessary to implement the colour coding feature which is an essential feature of the program and a feature that was highly requested by the clients.

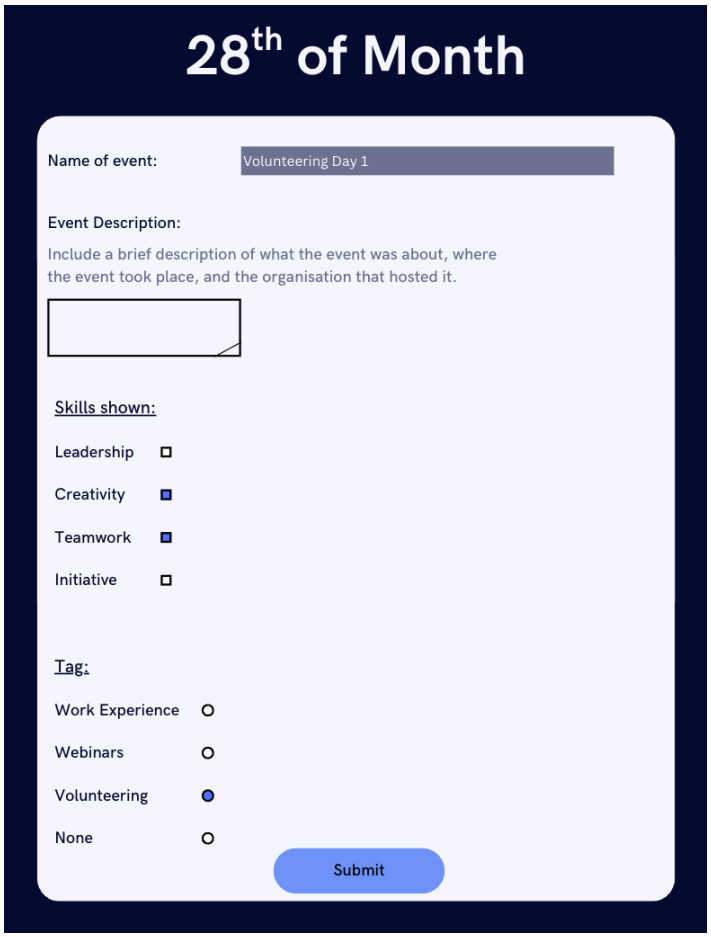
The final change in the design is that the calendar now fits the entire screen width of the page instead of a set screen width. I believe that this is better for user readability and usability as the calendar days are more visible to them and the buttons are more spaced out which helps reinforce the minimal design of the project. This reduces any 'cluttering' that the use may feel if the buttons were too close together and therefore helps with usability.

Summary of the whole project as a prototype at this stage

The project as a whole, now contains a working account creation and login system with input validation and usability features (username retention), and a homepage where the user is redirected upon successful login. The homepage is a working calendar system which dynamically outputs the calendar data for a month selected using the month selector. This calendar data is formatted and styled for user readability and usability and the backend code for the calendar creation is modular and allows for scalability for more years in future iterations of the program. The calendar also has functionality for colour coded events to be styled and underneath the calendar, a legend section helps the user to determine the event tag based on the colour coding.

Stage 5: Input Form Page

This is the design that I created for the input form in the design stage.



The input form must contain areas for the event name, event description, skills shown, and the event tag. I will first begin by creating a prototype GUI for my program.

Creating the GUI of the Page

I decided to start working on a prototype of the GUI of this page first so that I can then test and view the outputs of my tests effectively. I am only implementing a prototype of the GUI so that any stylistic errors can be identified initially and resolved alongside the final iteration implementation of the backend.

I will first create the inputform.php file with the form and its submit elements.

inputform.php

```
<b><p id="title"><?php
$title = htmlspecialchars($_GET["date"]);
echo $title;
?></p></b>
<div id="div1"> <!--Creating a div element to style the background according to the design (white background colour and rounded edges)-->
    <form>
        <!--Input fields with labels so they can be accessed by the php file and sent to the database-->

        <!--This section of the input form is for the event name.-->
        <label for="eventname">Name of event:</label>
        <input type="text" name="eventname" placeholder="Event Name">
        <br><br><br><br>

        <!--This section of the input form is for the event description.-->
        <label>Event Description:</label><br>
        <input type="textarea">
        <br><br><br><br>

        <!--This section of the input form is for the skills the event has been tagged with.-->
        <label>Skills Shown:</label>
        <br><br>
        <label>Leadership</label>
        <input type="checkbox">
        <br><br>
        <label>Creativity</label>
        <input type="checkbox">
        <br><br>
        <label>Teamwork</label>
        <input type="checkbox">
        <br><br>
        <label>Initiative</label>
        <input type="checkbox">
        <br><br>
        <label>Problem Solving</label>
        <input type="checkbox">

<label>Organisation</label>
<input type="checkbox">
<br><br>
<label>Communication</label>
<input type="checkbox">
<br><br>
<label>Public Speaking</label>
<input type="checkbox">
<br><br>
<label>Perseverance</label>
<input type="checkbox">
<br><br>
<label>Time Management</label>
<input type="checkbox">
<br><br>
<label>Planning and Preparation</label>
<input type="checkbox">
<br><br>
<label>Analytical Skills</label>
<input type="checkbox">
<br><br>
<label>Practical Skills (sketching, DT)</label>
<input type="checkbox">
<br><br>
<label>Academic Skills (numeracy, essay writing)</label>
<input type="checkbox">
<br><br>
<label>Computer Skills (coding, CAD)</label>
<input type="checkbox">
<br><br><br><br>

<!--This section of the input form is for the tags the event has been tagged with. It calls the radiobuttonoutput function
to generate a label and radio button pair for the webpage based on the event_tag data stored in the database-->
<u><label id="tag">Tag:</label></u>
<script>radiobuttonoutput(['1', '2', '3', '4'], "tag");</script>
<br><br><br><br>
```

This file contains a prototype of the input form which currently doesn't contain database linkage to allow for the event data to be output. Currently, the title is the GET value of the date which is sent from the day cell submit buttons once pressed and a htmlspecialchars version of the date is output as the title. This is a form of validation as if the GET value is compromised to contain JavaScript or SQL code, it is cleaned/sanitised before it is output to the user.

In the form element, I have included a text input for the event name and a text area for the event description. This showcases to the user that they have to write more text for the event description than for the event name which is a helpful usability feature. Furthermore, I have included checkboxes for the skills the user could've shown and a tag section which calls the radiobuttonoutput function which dynamically generates tags based on the data passed to the function. For the current test (as the function isn't currently implemented yet) I will comment out the line before testing it.

inputformstylesheet.css

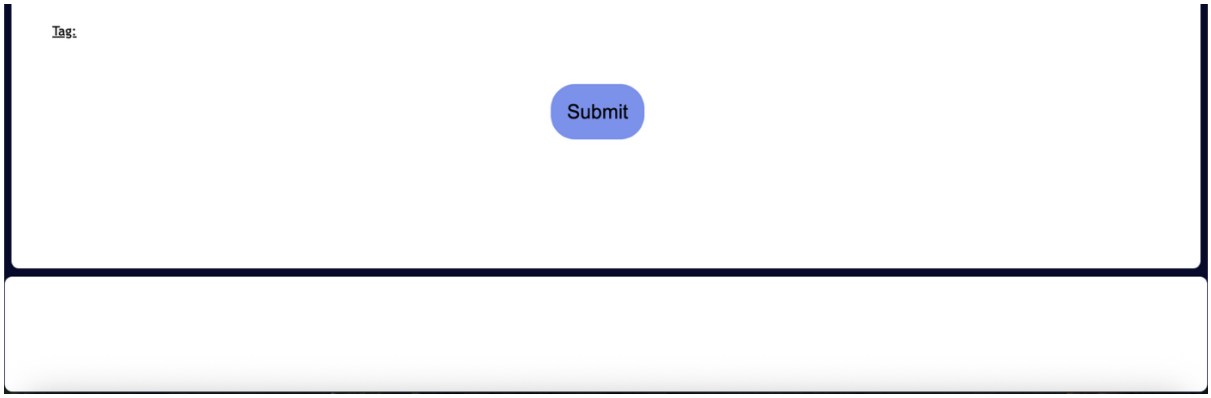
```
body{
    color: black;
    background-color: #070a2a;
    font-family: 'Trebuchet MS', 'Lucida Sans Unicode', 'Lucida Grande', 'Lucida Sans', Arial, sans-serif;
    text-align: center;
}
div{
    text-align: left;
    background-color: white;
    height: 1050px;
    padding-left: 50px;
    padding-right: 50px;
    padding-bottom: 65px;
    padding-top: 75px;
    border: solid 1px #060a2e;
    border-collapse: separate;
    border-spacing: 0 10px;
    border-radius: 10px;
    margin: -10px, 20px, 0px, 20px;
}
footer {
    margin-top: -1500px;
}
#title{
    color: white;
    font-size: xx-large;
}
#Submit{
    background-color: #7c90ea;
    border-radius: 30px;
    font-size: x-large;
    border-color: #7c90ea;
    border-style: hidden;
    padding: 20px;
    margin-left: 45%;
}
```

This is the styling for the page. The submit button has been styled in a way which is similar to the submit button on the login page but larger as the input form is a larger page to navigate than the login page.

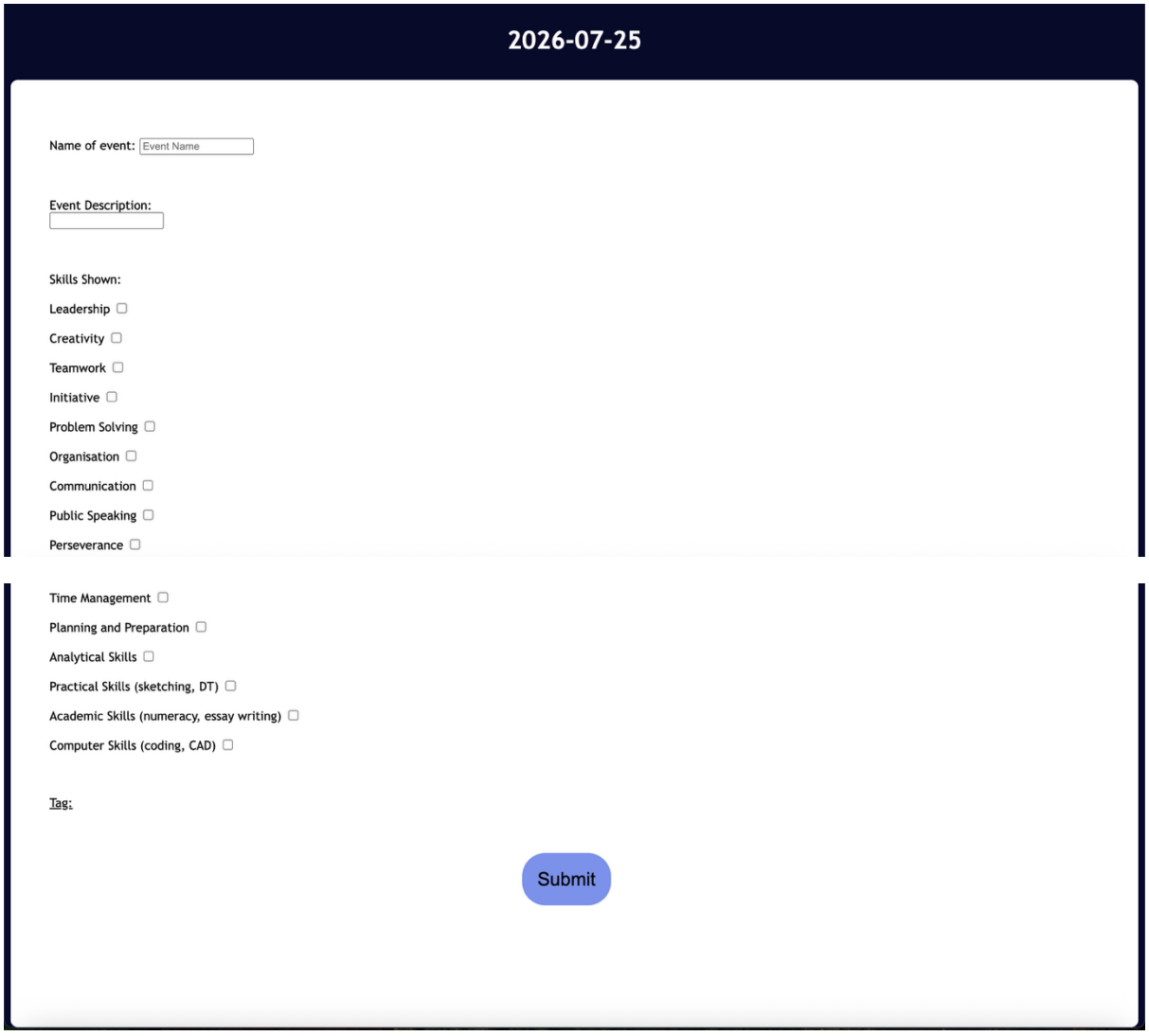
Furthermore, the footer has -1500 padding specified as it removed unnecessary padding which I found the solution to on stack overflow (linked in bibliography).

```
footer {
    margin-top: -1500px;
}
```

Otherwise, it causes padding on the bottom of the page like this, which ruins the design:



Implementing the design and styling produces this output when the 25th of July button is pressed:



This output is a good prototype as all of the inputs excluding the radio buttons (which will be dynamically generated) have been created. However, there is one main issue with the inputs. The event description input field has not been created as a textarea. This is an issue as it doesn't make it clear to the user that they should enter a large amount of text. Furthermore, the text input cannot be resized and therefore, it would

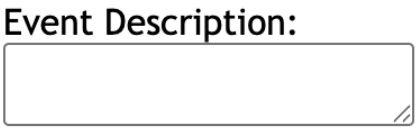
cut off a large amount of text when the user is typing which would be bad for usability. To solve this, I replaced this line:

```
<input type="textarea">
```

With this line, which is the correct declaration of a textarea:

```
<textarea name="event_description" id="event_description"></textarea>
```

This leads to the correct resizable textarea being output:



This solves the immediate issue with the input type. However, there are other issues with the program to do with functionality and style.

The main functionality issue in the program is that the form currently has no action and method attribute. The form doesn't redirect to any file which can handle the input data and insert/update the data in the database. Therefore, I will create a file called inputform_input.inc.php which the form will POST the data to as the event data is private information and shouldn't be visible within the URL.

Furthermore, I will need to implement the radiobuttonoutput function and a way in which the data stored in the database is reflected on the input form. I will implement this using a file called inputformhandler.inc.php which will check if there are values in the database associated with the date. If there are, those values will be used, otherwise, placeholder values are used.

A stylistic issue is that I need to implement underlining for all of the input names as this would help differentiate the names of the inputs from the options which would help with usability as there are multiple skills options which may confuse the users.

Another stylistic issue is that the font is incorrect on this page which can be easily solved by adding an import line to the top of the stylesheet which would import the font from google fonts.

A final stylistic issue is that there is too much empty space after the submit button which negatively impacts the design as the user must unnecessarily scroll further down. Furthermore, implementing the submit button inside of the white input form div I believe, ruins the layout and design of the page as the submit button should be outside of the white div to maintain a consistent design. Therefore, I will implement these changes in the next iterations of the program.

Initial iteration implementation – implementing radiobuttonoutput

First, I will begin by implementing the radiobuttonoutput function which will dynamically output labels and radio buttons based on an array passed to it.

```
/*This function takes an array of tag names as input and the id of the 'Tag:' label
to append the tag names as labels and radio buttons for each tag*/
function radiobuttonoutput(tags, x){
        for(let i = 0; i < tags.length; i++){  //Loop through the array of tag names
        const title = document.createElement("label");  //Create a label element for the name of tag
        const node = document.createTextNode(tags[i]);  //Create a text node for the tag name
        title.appendChild(node);  //Append the text node to the label element

        const input = document.createElement("input");  //Create an input element for the tags
        input.type = "radio";  //Specify the type to be a radio button (each event can only have one tag for colour coding)
        const element = document.getElementById(x);  //Retrieve the 'Tag:' label based on its id

        const br = document.createElement("br");  //Create a br element so that each radio button and label pair is on a new line

        element.appendChild(br);  //Append the break to the 'Tag:' label
        element.appendChild(title);  //Append the tag label to the 'Tag:' label
        element.appendChild(input);  //Append the radio button to the 'Tag:' label
        }
}
```

The function takes an array of event name and date strings and the id of the ‘Tags:’ title. For each item in the array, a label element is created with the text content of the event name and date from the array. A radio button input and line break are also created for each label, so each label button pair starts on a new line. Finally, the line break, label and button are appended to the ‘Tags:’ title.

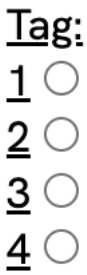
To test the code, I un-commented the line containing the script call in the main page. I removed the underline element from the ‘Tag:’ title and tried selecting multiple options and it gave me this output:

This shouldn’t be possible as radio buttons do not allow for multiple to be selected unlike checkboxes. I will perform a further test while keeping the underline tag. After I import the correct font.

I added the correct font to the stylesheet by adding an import line importing from google fonts which solves one of the stylistic issues.

```
@import url('https://fonts.googleapis.com/css2?family=Hanken+Grotesk:ital,wght@0,100..900;1,100..900&display=swap');
/*Importing correct font (based on Canva design) from Google fonts*/

body{
    color: black;
    background-color: #070a2a;
    font-family: 'Hanken Grotesk', sans-serif;
    text-align: center;
}
```

I performed a second test of the code by keeping the underline tag which led to this output:



There are two main issues with the test outputs:

- The styling of the 'Tag:' title is being inherited by the label elements
- Multiple radio buttons can be selected which shouldn't be possible

The inheritance in styling is an issue as it would mean that any tag names which I implement will be underlined which would be detrimental to usability as the underlined titles represent the input names. Underlining the tag options would unnecessarily confuse the user and complicate the design therefore I will need to find a way to implement this alternatively.

The main issue is the fact that multiple radio buttons can be selected. This should not be possible as multiple radio buttons cannot be selected in one form and can lead to issues when the inputs are being sent to the inputform_input file. Therefore, this further highlights that I will need to implement the radio buttons in an alternative way using inputformhandler and PHP.

Second iteration implementation – implementing inputformhandler

I will first begin by implementing the inputformhandler file which will contain the values that will be set for each form element. After implementing this, I will make the necessary changes to the form elements to allow them to retain values.

inputformhandler.inc.php

```php
require_once 'dbh.inc.php';  //Allows for database linkage
require_once 'config_session.inc.php';  //Allows for session variable regeneration (added security)

//Check user is logged in
if (!isset($_SESSION['user_id'])) {  //If the user id is not set
    header("Location: ../login.php");  //Redirect back to the login page
    exit();
}

$userId = $_SESSION['user_id'];  //Otherwise, retrieve user id

//Get date value from the url
if (isset($_GET['date'])) {  //If the date is set
    $date = $_GET['date'];  //Retrieve it as a variable date
} else {
    $date = '';  //Otherwise use an empty string
}

//Default values of input fields
$eventName = '';
$eventDescription = '';
$eventTag = '';

$leadership = 0;
$creativity = 0;
$teamwork = 0;
$initiative = 0;
$problem_solving = 0;
$organisation = 0;
$communication = 0;
$public_speaking = 0;
$perseverance = 0;
$time_management = 0;
$planning_and_preparation = 0;
$analytical_skills = 0;
$practical_skills = 0;
$academic_skills = 0;
$computing_skills = 0;
```

This is the first half of the inputformhandler file. This half is responsible for setting the default values of the form before checking if there are records in the database. Firstly, the required files for database connection and session variable generation are imported. Then, validation occurs to see if the user is correctly logged in (user id session variable is set). If they aren't logged in (or if the session variable isn't set), the user is redirected to the login page and the program ends. Otherwise, the file continues.

The file first checks if the get value of the date is present. If it is, it is stored as a variable which will be used to query the database to check for records. Otherwise, an empty string is set. This is another layer of validation as it prevents the file from querying the database if the page was accessed illegitimately to minimise the risks of SQL injection attacks and database breaches.

Next, the default values for the field values are declared. Event name, description and tag are empty string values (as they are stored as strings/chars in the database) and the skill values are set to 0 (as they are implemented as Boolean values in the database).

```
/*Load event if it exists*/
if ($date != '') {  //If the date has been set

    //Retrieve the values from the database (limit 1 prevents accidental data collision)
    $sql = "SELECT EVENTS.name_of_event, EVENTS.event_description, EVENTS.event_tag, SKILLS.*
            FROM EVENTS
            INNER JOIN LINK_TABLE ON EVENTS.event_id = LINK_TABLE.event_id
            INNER JOIN SKILLS ON LINK_TABLE.skills_id = SKILLS.skills_id
            WHERE EVENTS.date_of_event = ?
            AND LINK_TABLE.user_id = ?
            LIMIT 1
    ";

    $stmt = $pdo->prepare($sql);  //The statement is prepared
    $stmt->execute([$date, $userId]);  //The parameters are passed and statement is executed
    $result = $stmt->fetch(PDO::FETCH_ASSOC);  //The retrieved values are stored as result

    if ($result) {  //If there was a result in the database
        //Set the parameters to the database values
        $eventName = $result['name_of_event'];
        $eventDescription = $result['event_description'];
        $eventTag = $result['event_tag'];

        $leadership = $result['leadership'];
        $creativity = $result['creativity'];
        $teamwork = $result['teamwork'];
        $initiative = $result['initiative'];
        $problem_solving = $result['problem_solving'];
        $organisation = $result['organisation'];
        $communication = $result['communication'];
        $public_speaking = $result['public_speaking'];
        $perseverance = $result['perseverance'];
        $time_management = $result['time_management'];
        $planning_and_preparation = $result['planning_and_preparation'];
        $analytical_skills = $result['analytical_skills'];
        $practical_skills = $result['practical_skills'];
        $academic_skills = $result['academic_skills'];
        $computing_skills = $result['computing_skills'];
    }
}
```

If the date value was set correctly, the database is queried for the event name, description, tag and all of the skills values for the user's user id and the date specified in the GET value. The statement is prepared and executed, and the result is stored as an associative array 'result'.

Next, the file checks if a result was returned from the database. If it wasn't, the default values don't change and are passed to the fields. If there was a result, the values of the variables are set to the values found in the database. These variables now retain these values which can be accessed by the input form to set the values of the input elements.

inputform.php

The input elements of the form have been updated to allow for the values set in inputformhandler to be displayed.

```
<u><label for="eventname">Name of event:</label></u><br>

<!--This section of the input form is for the event name. If the an event has been stored in the database for this date, the input field
will show a cleaned value (htmlspecialchars prevents malicious JS code from being displayed on the page) of the event name-->

<input type="text" name="eventname" id="eventname" value="<?php echo htmlspecialchars($eventName); ?>"><br><br><br>

<u><p>Event Description:</p></u>

<!--This section of the input form is for the event description. If the event has a description in the database, the textarea
will show a cleaned value (htmlspecialchars prevents malicious JS code from being displayed on the page) of the event description-->

<textarea name="event_description" id="event_description"><?php echo htmlspecialchars($eventDescription); ?></textarea><br><br><br>

<u><label>Skills Shown:</label></u><br><br>

<!--This section of the input form is for the skills the event has been tagged with. If the event has been tagged in the database
with a certain skill, the checkbox button will already be checked when viewing the page-->

<label for="leadership">Leadership</label>
<input type="checkbox" name="leadership" id="leadership" value="1" <?php if ($leadership == 1) echo 'checked'; ?>><br><br>
```

The input form elements have been assigned name, id and value attributes. The name attribute will be helpful in accessing the data once it has been sent to inputform_input. The id links to the for attribute of the label tag corresponding to the input field which is bets practice in HTML forms. Finally, the value attribute allows the values set in inputformhandler to be cleaned/sanitised using htmlspecialchars (which is a form of validation) before they are displayed to the user.

To allow for the preset value for the event name input to be displayed, the value attribute is assigned the htmlspecialchars (cleaned) version of the eventName variable set in the inputformhandler file.

To allow for the event description preset value to be displayed, a value attribute is not required. The htmlspecialchars version of the eventDescription variable can be declared in between the textarea tags to output the value in the textarea.

For the skills checkboxes, I have implemented a check where, if the value of the associated attribute is 1 (which means that the event has been tagged with that skill), the checked attribute is echoed in the tag. Therefore, the preset value will be displayed as a pre-checked box. This has been repeated for the other 14 skills checkboxes.

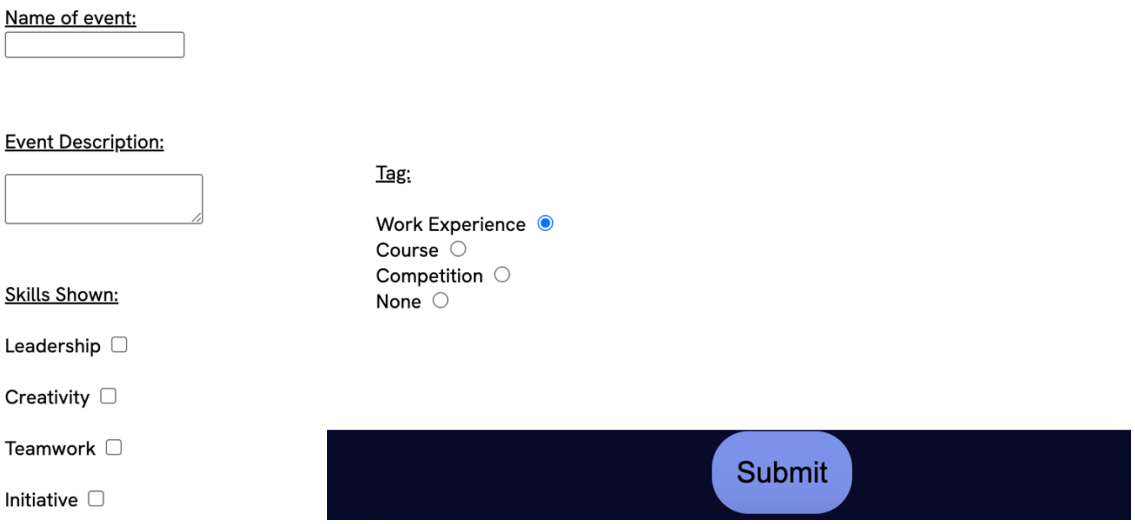
```
<u><label>Tag:</label></u><br><br>

<!--This section of the input form is for the tags the event has been tagged with. If the event has been tagged in the database with a ce
the radio button will already be checked when viewing the page-->

<label for="work-experience">Work Experience</label>
<input type="radio" name="tag" id="work-experience" value="work-experience" <?php if ($eventTag == 'work-experience') echo 'checked'; ?>>

<label for="course">Course</label>
<input type="radio" name="tag" id="course" value="course" <?php if ($eventTag == 'course') echo 'checked'; ?>><br>

<label for="competition">Competition</label>
<input type="radio" name="tag" id="competition" value="competition" <?php if ($eventTag == 'competition') echo 'checked'; ?>><br>

<label for="none">None</label>
<input type="radio" name="tag" id="none" value="none" <?php if ($eventTag == NULL) echo 'checked'; ?>><br><br>
```

To implement the tags, I decided that the simplest way of solving the inheritance of styling and the multiple selection of the radio buttons was to implement them statically instead of implementing the tags dynamically. Each tag will check if the eventTag attribute is the same as their tag name. If it is, the radio button will already be checked. As the database only allows for one tag per event, this solves the issue of multiple buttons being pre-selected. Furthermore, implementing the tags statically allows for me to implement the underline tag around the 'Tag:' label which will not be inherited by the tag labels.

```
</div>

<input type="submit" id="submit" value="Submit">
```

Furthermore, I removed the submit button from the div to solve one of the initial stylistic issues of maintaining a consistent design. This would mean that it is outside of the white background which would be better for the design of the form.

Implementing this code, I checked the output of the input form which displayed this:



The first stylistic issue has been solved as all of the input area names have been underlined. This helps with readability of the program as the user is able to easily identify the areas which are to be filled in. This is a good usability feature of the program. Another stylistic issue of the submit button being within the form div has been removed and now the code has a more consistent design which further improves usability.

Furthermore, for the tags section, this is the correct output as only one tag is able to be selected at once. Furthermore, there are no inheritance issues with the labels of the tag names. However, there is a stylistic issue from before which I need to solve, the gap between the end of the tag section to the end of the form needs to be reduced.

To implement this, I changed the padding-bottom attribute to 20px to decrease the space between the tags and the submit button.

```
padding-bottom: 20px;
```

Refreshing the page after implementing this change displayed this output.

The final stylistic issue has now been resolved and now, the only aspect of the form left is to implement the action and method attributes and the inputform_input file to handle the form inputs.

Third iteration implementation – implementing inputform_input

First, I will begin by adding the action and method attributes to the form element.

```
<form action="includes/inputform_input.inc.php?date=<?php echo htmlspecialchars($date); ?>" method="post">
```

I have declared the action attribute to be the inputform_input file with the date as a GET value as the file will need this to locate the record that needs to be updated/inserted. The form data, however, will be sent using POST as it is sensitive and making the data visible in the URL may cause issues when the program is being used in public.

inputform_input.inc.php

```php
require_once 'dbh.inc.php';  //Allows for database linkage
require_once 'config_session.inc.php';  //Allows for session variable regeneration (added security)

//Check user is logged in
if (!isset($_SESSION['user_id'])) {
    header("Location: ../login.php");
    exit();
}
$userId = $_SESSION['user_id'];  //User id is stored

//Get date from GET and convert to MySQL format
if (isset($_GET['date'])) {  //If a date was set (page accessed legitamately)
    $timestamp = strtotime($_GET['date']);  //It is converted into a timestamp data structure
    if ($timestamp !== false) {  //If the date was correctly formatted
        $date = date('Y-m-d', $timestamp); //Date is stored in the MySQL format
    } else {  //Otherwise
        die("Invalid date format");  //End the program and output an error
    }
} else {  //Otherwise
    header("Location: ../calendar.php");  //Redirect back to the calendar page
    exit();
}

//Get form data safely
//If the event name is set, use it, otherwise use NULL
$eventName = isset($_POST['eventname']) ? $_POST['eventname'] : NULL;
//If the event description is set, use it, otherwise use NULL
$eventDescription = isset($_POST['event_description']) ? $_POST['event_description'] : NULL;
//If the tag is set, use it, otherwise use NULL
$tag = isset($_POST['tag']) ? $_POST['tag'] : NULL;

//Validate required field
if (empty($eventName)) {
    //Redirect back to the form with an error message
    header("Location: ../inputform.php?error=missing_event_name&date=" . urlencode($_GET['date']));  //URL encodes the date
    exit();
}
```

Firstly, the file imports the files required for establishing a database connection and for accessing session variables. It then performs a validation check to see if the user is logged in. If not, they are redirected to the login page. This prevents the file from being accessed by those with malicious intent as is a good validation check to implement for when the file is scaled and globally hosted. The user id is then stored as a variable for use in database querying. The file first checks if the file was accessed legitimately by clicking on a day node from the calendar (if the date GET attribute was set). If not, the user is redirected to the calendar page. If the date was set, it is converted into a timestamp using the built-in PHP strtotime function. If the timestamp was not invalid, the date in the SQL format is accessed from the timestamp. Otherwise, an error is output and the program ends.

Next, the eventName, eventDescription and tag variables are set using ternary operators. These operators check if the values have been set in POST. If they have, the variable stores the POST value. If they haven't, the variable stores NULL.

I have implemented validation for the event name input element as the user should not be able to input data for an event without stating the event name. Therefore, using the built in empty function which I used on the login pages, If the event name is empty, the user is redirected back to the input form with a get value of 'error'='missing_event_name'. I have implemented a check for this on the input form

page which will check if the error get value has been set and if it was a missing event name error. If it was, an error message is displayed.

```
/*This piece of code checks to see if an event has been input without an event name*/
    if (isset($_GET['error']) && $_GET['error'] === 'missing_event_name') {  //If an event was input without a name
        echo '<p id="form-error"><u>Event name is required.</u></p>';  //Return an error
    }
```

This is the code that I have implemented in the input form underneath the title. It checks if an error was set and if the error set was a ‘missing_event_name’ error. It then outputs an error message with the form-error styling. The code acts as a responsive validation block which is good for user usability as it outputs a precise error message.

This piece of code is modular in nature as it checks if the specific error occurred, not only if an error occurred. This means that in future iterations of the project, more error validation statements can be implemented by changing the error value being checked for and the output error.

```
//Skills (checkboxes values will be 1 if checked, 0 if not)
$skills = [
    'leadership' => isset($_POST['leadership']) ? 1 : NULL,
    'creativity' => isset($_POST['creativity']) ? 1 : NULL,
    'teamwork' => isset($_POST['teamwork']) ? 1 : NULL,
    'initiative' => isset($_POST['initiative']) ? 1 : NULL,
    'problem_solving' => isset($_POST['problem_solving']) ? 1 : NULL,
    'organisation' => isset($_POST['organisation']) ? 1 : NULL,
    'communication' => isset($_POST['communication']) ? 1 : NULL,
    'public_speaking' => isset($_POST['public_speaking']) ? 1 : NULL,
    'perseverance' => isset($_POST['perseverance']) ? 1 : NULL,
    'time_management' => isset($_POST['time_management']) ? 1 : NULL,
    'planning_and_preparation' => isset($_POST['planning_and_preparation']) ? 1 : NULL,
    'analytical_skills' => isset($_POST['analytical_skills']) ? 1 : NULL,
    'practical_skills' => isset($_POST['practical_skills']) ? 1 : NULL,
    'academic_skills' => isset($_POST['academic_skills']) ? 1 : NULL,
    'computing_skills' => isset($_POST['computing_skills']) ? 1 : NULL
];

//Check if event already exists for this user & date
$sql = "SELECT EVENTS.event_id, LINK_TABLE.skills_id
        FROM EVENTS
        INNER JOIN LINK_TABLE ON EVENTS.event_id = LINK_TABLE.event_id
        WHERE EVENTS.date_of_event = ?
        AND LINK_TABLE.user_id = ?
        LIMIT 1
";

$stmt = $pdo->prepare($sql);  //Query is prepared
$stmt->execute([$date, $userId]);  //Date and user id parameters are bound and the statement is executed
$result = $stmt->fetch(PDO::FETCH_ASSOC);  //All of the data is fetched and stored as result
```

More ternary operators are used for the skill values based on the POST values and these skill values are stored as an associative array to be passed to the statement.

Next, the SQL statement which retrieves the event id and skills id for the logged in user for the date GET value. The statement is prepared and then executed, and the result is stored in an array named result.

```php
if ($result) {  //If a result was fetched
    //Update existing event data
    $updateEvent = $pdo->prepare("UPDATE EVENTS SET name_of_event = ?, event_description = ?, event_tag = ? WHERE event_id = ?
    ");
    $updateEvent->execute([$eventName, $eventDescription, $tag, $result['event_id']]);  //Parameters are bound and the statement is executed

    //Update existing skills data
    $updateSkills = $pdo->prepare("UPDATE SKILLS SET
            leadership = ?, creativity = ?, teamwork = ?, initiative = ?, problem_solving = ?,
            organisation = ?, communication = ?, public_speaking = ?, perseverance = ?,
            time_management = ?, planning_and_preparation = ?, analytical_skills = ?,
            practical_skills = ?, academic_skills = ?, computing_skills = ?
        WHERE skills_id = ?
    ");
    $updateSkills->execute([  //Parameters are bound and the statement is executed
        $skills['leadership'], $skills['creativity'], $skills['teamwork'], $skills['initiative'], $skills['problem_solving'],
        $skills['organisation'], $skills['communication'], $skills['public_speaking'], $skills['perseverance'],
        $skills['time_management'], $skills['planning_and_preparation'], $skills['analytical_skills'],
        $skills['practical_skills'], $skills['academic_skills'], $skills['computing_skills'],
        $result['skills_id']
    ]);

} else {  //If no result was fetched
    //Insert event into database
    $insertEvent = $pdo->prepare("INSERT INTO EVENTS (date_of_event, name_of_event, event_description, event_tag) VALUES (?, ?, ?, ?)");
    $insertEvent->execute([$date, $eventName, $eventDescription, $tag]);  //Parameters are bound and the statement is executed
    $eventId = $pdo->lastInsertId();  //The event id of the last insertion is retrieved

    //Insert empty SKILLS_DESCRIPTION row
    $insertDesc = $pdo->prepare("INSERT INTO SKILLS_DESCRIPTION () VALUES ()"); //Empty row (values can be added later)
    $insertDesc->execute();  //Statement is executed
    $descId = $pdo->lastInsertId();  //The description id of the last insertion is retrieved

    //Insert skills values into database
    $insertSkills = $pdo->prepare("INSERT INTO SKILLS (leadership, creativity, teamwork, initiative, problem_solving, organisation,
    communication, public_speaking, perseverance, time_management, planning_and_preparation, analytical_skills, practical_skills,
    academic_skills, computing_skills) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)");
    $insertSkills->execute(array_values($skills));  //Parameters are bound and the statement is executed
    $skillsId = $pdo->lastInsertId();  //The skills id of the last insertion is retrieved


        //Link table is updated
        $insertLink = $pdo->prepare("INSERT INTO LINK_TABLE (user_id, event_id, skills_id, description_id) VALUES (?, ?, ?, ?)");
        $insertLink->execute([$userId, $eventId, $skillsId, $descId]);  //Parameters are bound and the statement is executed

    }

    //Redirect after success
    header("Location: ../calendar.php?save=success");
    exit();
```

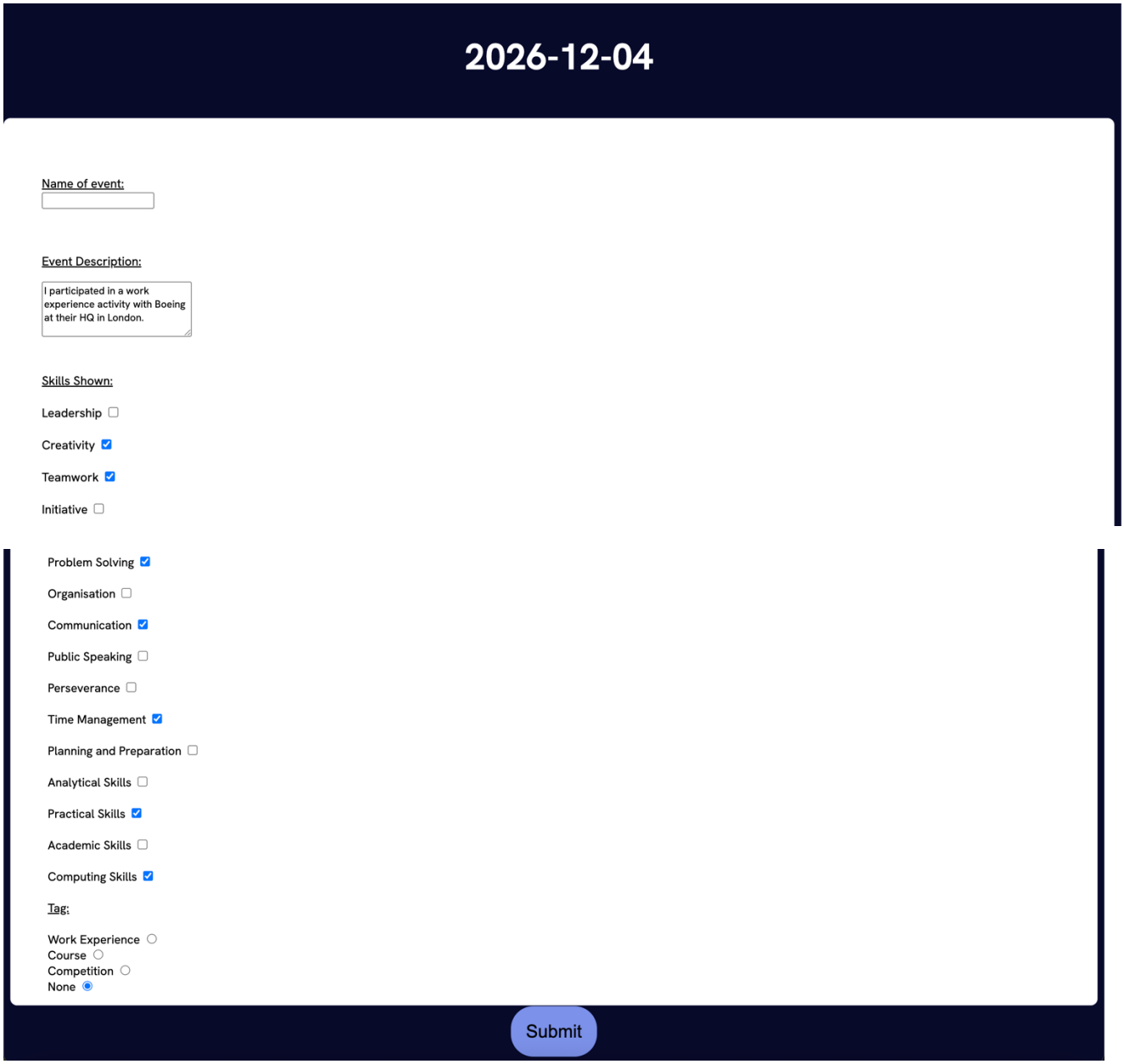
The program then checks if a result was fetched. If a result was fetched, that means that a record already exists and therefore the record is updated. If no record was fetched, this means that there is no event associated with the date and therefore the event must be inserted instead of updated. The parameters are passed in both cases and the statements are executed.

After the insertion is done, the file redirects to the calendar page with a save=success GET value. I have done this to implement a success message which will help the program feel more responsive and improve usability.

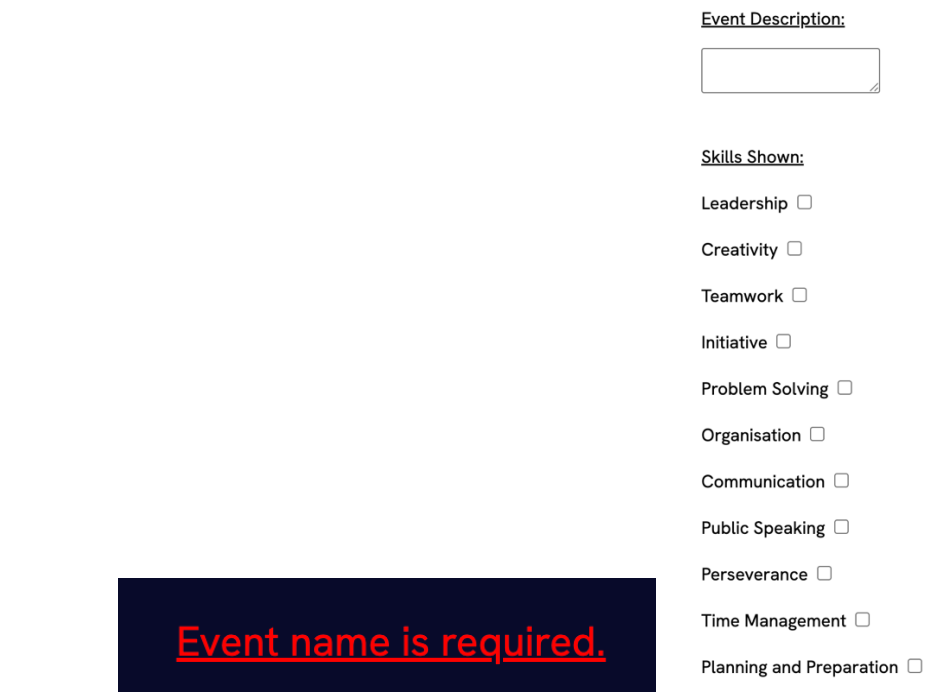
```php
//Checks to see if the calendar page was redirected from the inputform and if the saving was successful or not
if(isset($_GET['save']) && $_GET['save']==='success'){
    echo '<p id="form-success"><u>Event successfully stored/updated.</u></p>';
}
```

I have added this line of code alongside the output_username function in the calendar page which checks if the GET value was set and if the value was success. If so, a success message is output. This is implemented similarly to the message for an empty event name and therefore allows for the same level of modularity when scaling the project.

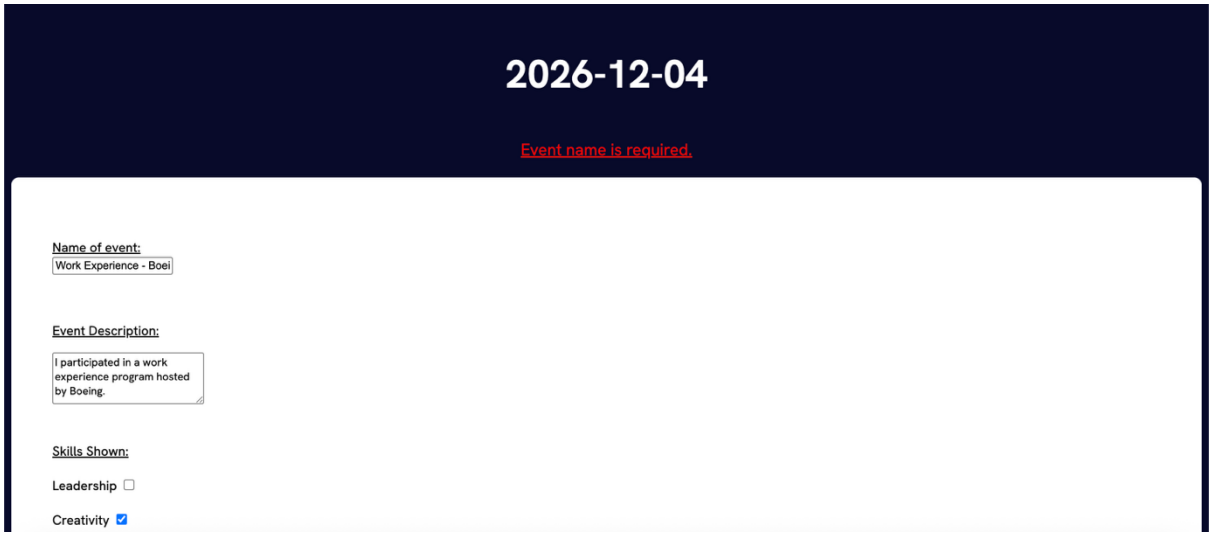
Now that all of the code is complete, I will now test the program. First, I will test the input form using an empty event name and no tag selected, therefore, the error message should be output:



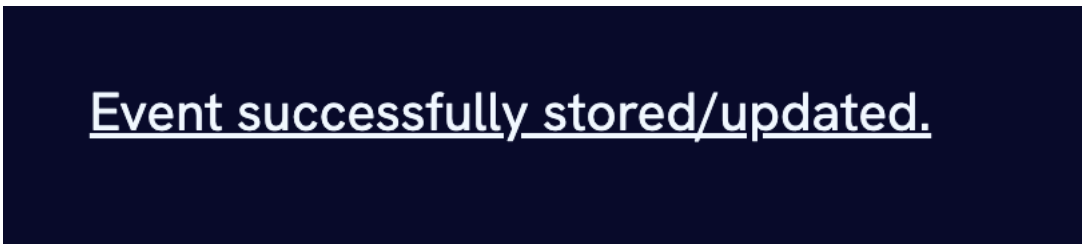
This was the data I input in the form. I have left the name of the event field empty to check if the input validation works correctly. The rest of the form should be blank once the submit button is pressed as data will not be retained in the form if there is an event name validation issue. This is the output I got after I submitted the form:



The error message appeared with the correct styling, and all of the input data was removed from the input field. This confirms that the validation aspect of the program is functional. Now, I will test for a valid input with no colour coding to see if the success message is output and if the data is retained in the form.

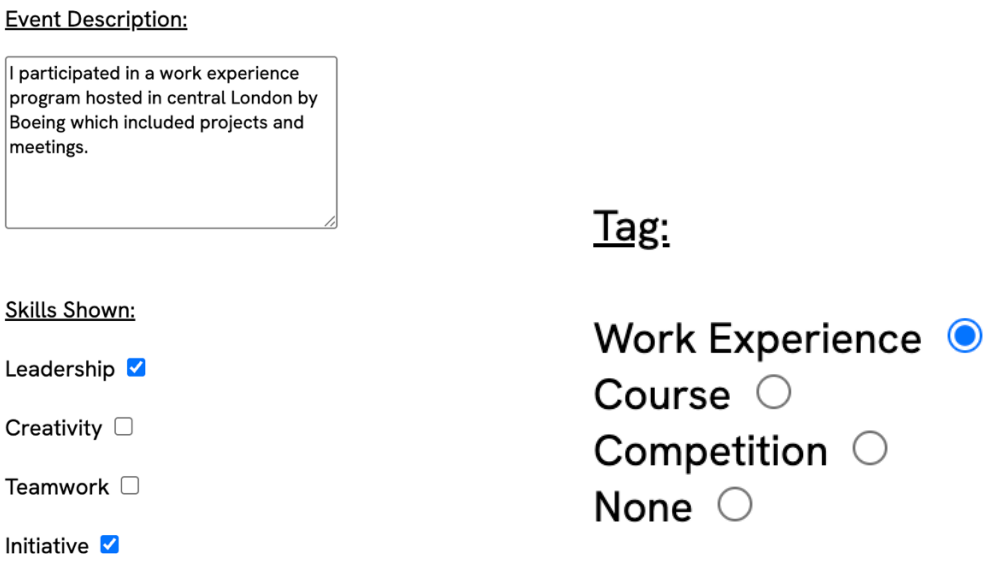


This is the data I input in the form. I input a valid event name, resized the textbox so that the entire description was visible, checked some skills boxes and didn't tag the event with a tag. I am expecting the form to redirect me to the calendar page and output the success message. Furthermore, the day node in the calendar should not be colour coded and the data should be retained in the input form page. Submitting the code gave me this output:



The form redirected me to the calendar page with the correct success message being output. The message is correctly styled and therefore the success validation of the form is working correctly.

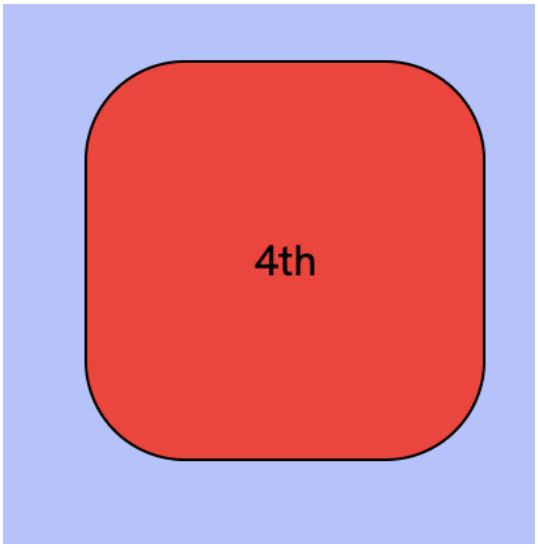
Checking the day cell, no colour coding has been applied to it which is correct as I did not tag the event with an event tag. Clicking on the node, redirected me to the input form where the data was correctly retained in the form. This shows that the data retention aspect of the program correctly works and that there are no issues with the inputform_input or inputformhandler files when it comes to data insertion and retrieval. I will now test the code to see if the data can be updated successfully by changing the event description, the skills shown and by adding a tag. I will also test if the correct colour coding is implemented for an event which is tagged with a certain skill:



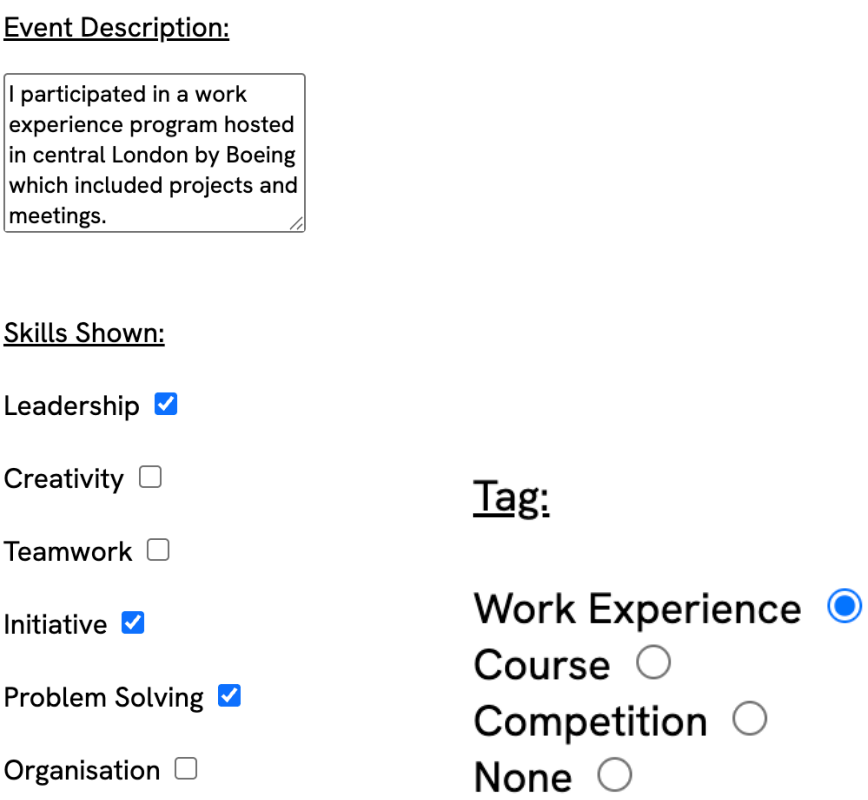
These are the major changes I made to the input data. I added some more details to the event description, changed some of the skills shown, and added the 'Work Experience' tag to the event as I am inputting test data based on a work experience program.

Clicking the submit button lead to this output:

I was redirected to the calendar page, and the success message was correctly output which is correct as there were no errors with the input data. I then checked the day node to see if the colour coding was updated.



This was the output. The day cell had been correctly styled with the colour of red which corresponds to the Work Experience tag as stated in the legends section of the calendar page. This shows that the colour coding feature was successfully implemented in the previous stage and in this stage which means that a core part of the program has been completed. I will now click on the day cell to check if the changes I made to the data was retained and if the database was correctly updated. Clicking on the day cell gave me this output:



The changes I made to the input form data have been correctly retained. The event description is the updated description, the skills shown reflect the skills I updated before clicking submit, and furthermore, the tag of work experience has been retained.

This proves that the inputform_input and inputformhandler files (and the changes to the input form and calendar page) were correctly implemented and that the input form is now complete. I will now move on to the skills page.

Stage 5 Review

What has been done

The input form has been fully implemented. Each input has been declared according to the design (input field for the event name, textarea for event description, checkboxes for skills and radio buttons for tags). The input field successfully sends data to the 'inputform_input.inc.php' file which performs validation on the input data. An error message is output if the event name field is blank, but a success message is output if the input data was inserted into the database successfully. The 'inputformhandler.inc.php' file allows for the values of the input fields to retain the values stored in the database when the day cell is clicked, and the values can be updated upon the submit button being pressed. Furthermore, the page has been designed to match the planned design from the design stage of the project using CSS.

How it has been tested

I have tested the program for the empty event name validation to see if the error message was successfully output if the event name was blank. I also tested if adding an event to the input form successfully outputs the success message and if the values were retained in the form. Finally, I also checked if updating the values saved the changes and I tested if tagging an event produced the successful colour coding. All of these tests were successful and produced the correct output.

Criteria met/being met

- Calendar System for past, present and future events
- Colour coding of events

How it meets the success criteria and user expectations

The input form allows the user to add event data to the database from the calendar. By clicking on the day node, the user is redirected to the input form which allows them to add/update event data for events they would like to store in the calendar. The inputform has been tested successfully which showed that data was being correctly input into the database. Furthermore, the input form was also retaining values which would allow the user to view the stored event data allowing them to interact with the calendar successfully.

The colour coding feature has also been successfully implemented. In the previous stage, I added the backend capability for tagged events to be assigned a colour coding class. In this stage, the inputform allows for events to be tagged and I have tested that it allows for the colour coding to be successfully implemented. This is good for usability, and it was one of the primary requested features by the user.

Changes in the design that have resulted from this section

There have been a few changes in the design in this section for this stage of the program. The three changes have been:

The removal of the submit button from the input form div. This is a purely stylistic choice as upon running the code, the submit button felt obtrusive and the design felt cluttered. Separating the submit button from the input form div is better for usability.

The tags have been changed from the planned tags in the design stage. I have replaced ‘Webinars’ and ‘Volunteering’ with ‘Course’ and ‘Competition’ as I believe that the planned tags were too specific and not broad enough. As I am only implementing 3 tags currently (which can be updated in further iterations of the program as the code is modular), I would like them to be as broad as possible as that would be helpful for usability. Specific tags such as the tags planned in the design stage can be implemented in further iterations of the program and can be created based on further stakeholder feedback.

The guidance for the event description field has been removed. This was to simplify the design as I believe that the title is explanatory enough. This reduces clutter on the inputform and will improve the usability of the form as the user does not need to scroll as much. This feature can be implemented in future iterations of the program if it is an issue identified in the stakeholder feedback.

Summary of the whole project as a prototype at this stage

The project has a working accounts system with input validation, the calendar page which serves as the home page of the program which generates the months allowing the user to input past, present and future events in the calendar. The database has also been created and now, the input forms for each of the calendar day nodes has been implemented. Event data can be input into the database, and the data can be retrieved to be displayed on the form and allow for colour coding on the calendar page. The program now allows for event data to be added which will be helpful for the next stages of development where I will implement the skills and timeline page which will require events to be stored in the database to correctly function.

Stage 6: Skills Page

This is the design that I created for the skills page in the design stage.

The key aspects to the design of the skills page are the alternating design, the rounded sections for each skill, the example descriptions and the submit button.

I will begin by implementing a prototype of the GUI which I will use to test the outputs of functions and improve in further iterations of the program.

Creating a prototype GUI of the page

skills.php

```
<h1 id="skills">Skills</h1>

<form>
    <table>
        <tr>
            <td><h2 id="title1" class="lefttitle">Leadership</h2>
            <script>textboxoutput(['1', '2', '3', '4'], "title1");</script></td>

            <td><p id="description" class="rightp">In your description, show how you:
                <br>were given a task, or came up with a task, to lead
                <br>led the task, using your leadership style</p></td>
        </tr>



        <tr>
            <td><p id="description" class="leftp">In your description, show how you:
                <br>were given a task, or came up with a task, to lead
                <br>led the task, using your leadership style</p></td>
            <td><h2 id="title2" class="righttitle">Creativity</h2>

            <script>textboxoutput(['1', '2', '3', '4'], "title2");</script></td>
        </tr>
```

This is the prototype design of the skills page. It will contain the title of the skill, the output of the textboxoutput function (which will be implemented next), and the example description which will alternate in arrangement. These have been implemented as a table as it would allow me to efficiently organise the layout of the webpage.

I have currently included placeholder text for all of the example descriptions as I would first like to check if the styling of the webpage (rounding the divs, the alternating design, etc) is being correctly implemented first. The code is incredibly modular in nature as the rest of the skills have been implemented in the exact same way as the first two using alternating designs and therefore, more skills can be added in future iterations of the program without worrying about affecting the current design of the page.

skillsstylesheet.css

```css
p{
    color: #6c718c;
}

table {
    border-collapse: separate;
    border-spacing: 0 10px;
    margin: -10px, 20px, 0px, 20px;
    width: 100%;
}

td {
    border: solid 1px #060a2e;
    border-style: solid none;
    padding: 1px;
    background-color: #b5c4fa;

}
td:first-child {
    border-left-style: solid;
    border-top-left-radius: 10px;
    border-bottom-left-radius: 10px;
}
td:last-child {
    border-right-style: solid;
    border-bottom-right-radius: 10px;
    border-top-right-radius: 10px;
}
```

```css
.lefttitle{
    font-size: x-large;
    text-align: left;
    padding-left: 20px;
}
.righttitle{
    font-size: x-large;
    text-align: right;
    padding-right: 20px;
}
.leftp{
    text-align: left;
    padding-left: 20px;
}
.rightp{
    text-align: right;
    padding-right: 20px;
}
```

This is the code that styles the website. The skills description colour has been set to light grey, similar to the design. Using a guide on stack overflow (linked in bibliography), I was able to implement the rounded corners for the table rows to implement the style from the design in a grid format. The border collapse attribute will need to be set as separate to allow for the corners to be rounded using the td first-child and td last-child elements which rounds the corners to the design. This is a good usability feature as it highlights and separates the skills

Furthermore, the alternating design is implemented using the left/righttitle and left/rightp classes. Instances where the title is aligned to the left (using lefttitle) the example description will be aligned to the right (using rightp) and vice versa. This allows for the alternating design which is integral to the planned design of the page.

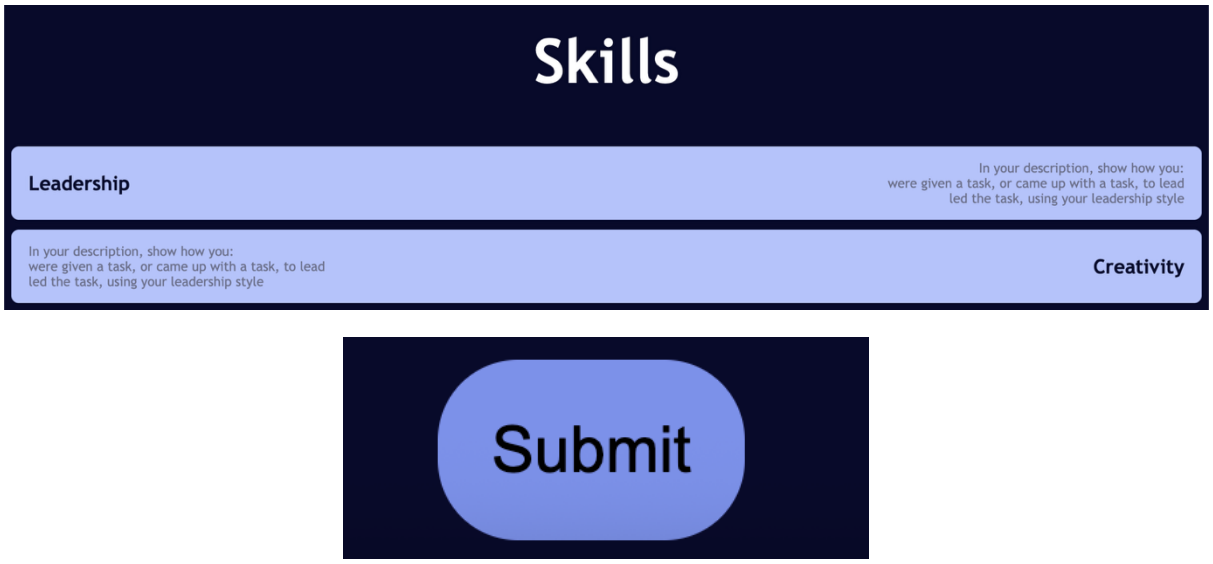
I have also included a submit button at the bottom of the page which will allow the form data to be submitted and sent to a formhandler.

```html
<input type="submit" id="submit">
```

```css
#submit{
    background-color: #7c90ea;
    border-radius: 30px;
    font-size: x-large;
    border-color: #7c90ea;
    border-style: hidden;
    padding: 20px;
}
```

I have implemented the same styling as for the input form. This consistent design would be helpful for usability and make the program feel like a complete professional site.

Implementing the styling and running the code has led to this output:



The skills page has the correct rounding of each skills area, the example descriptions have been correctly coloured, and the alternating design is present throughout the program. The title for the page and the skill titles are large and well contrasted with the background of the page which is good for usability. Furthermore, the submit button has been styled in the same way as the input form which helps maintain a consistent design for good usability.

However, there are a few issues with the program currently. The main issue currently is that I have used placeholder text for the skills apart from leadership. I will need to change this in the next iteration of the GUI. Furthermore, another minor issue is that the correct font has not been implemented, therefore, I will need to import it in the stylesheet.

Second iteration of the GUI

Firstly, I will add the specific example descriptions for each skill. Below, is an example of the example description which I implemented for the skill ‘Creativity’.

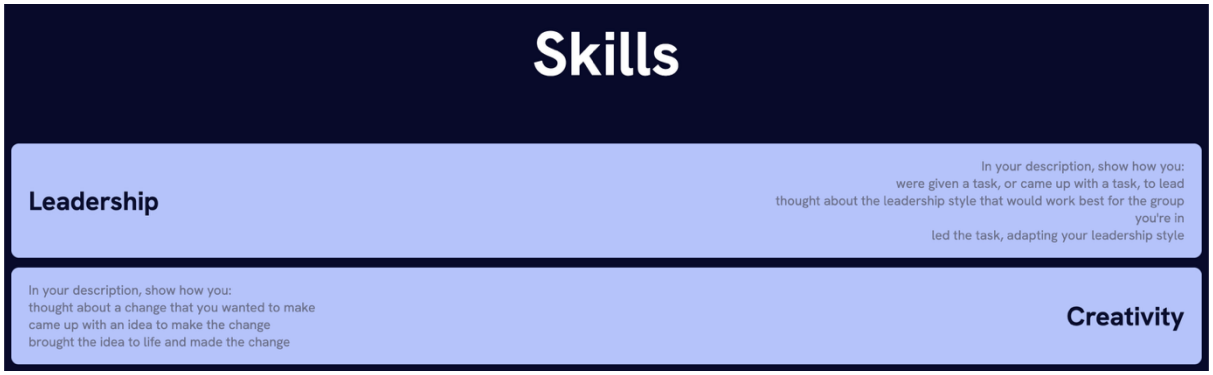
```
<tr>
    <td><p id="description" class="leftp">In your description, show how you:
        <br>thought about a change that you wanted to make
        <br>came up with an idea to make the change
        <br>brought the idea to life and made the change</p></td>
    <td><h2 class="right-title-header">Creativity</h2>
```

I have similarly implemented this for all of the remaining skills and have included between 2-4 pointers for each example description to maintain consistency and usability for the user.

Next, I imported the font from google fonts and included it as the font for the entire body of the website, which will use the correct font for the entire page.

```css
@import url('https://fonts.googleapis.com/css2?family=Hanken+Grotesk:ital,wght@0,100..900;1,100..900&display=swap');
/*Importing correct font (based on Canva design) from Google fonts*/

body{
    color: #0b0e2e;
    background-color: #070a2a;
    font-family: 'Hanken Grotesk', sans-serif;
    text-align: center;
}
```

Running this code leads to this output:



The design of the skills page is now complete excluding the textareas. The titles have been correctly styled to contrast the background, specific example descriptions which have been formatted correctly have been added for each skill and the rounded design for each row has been implemented.

Now, I will begin by implementing the textboxoutput function from the design stage.

Initial iteration implementation using JavaScript

```javascript
/*This function takes an array of event names tagged with a certain skill as input
and the id of the skill title (e.g. Leadership) to append the names of events in the
array as titles and textareas for event descriptions*/
function textboxoutput(tagged_arr, x){
        for(let i = 0; i < tagged_arr.length; i++){  //Loop through the array of event names tagged with the skill
        const title = document.createElement("h3");  //Create a title element for the name of event
        const node = document.createTextNode(tagged_arr[i]);  //Create a text node for the name of the event
        title.appendChild(node);  //Append the text node to the title element

        const text = document.createElement("textarea");  //Create the input textarea for skills descriptions
        const element = document.getElementById(x);  //Retrieve the skill title based on its id

        element.appendChild(title);  //Append the name of the event
        element.appendChild(text);   //Append the description textarea
        }
}
```

The function takes an array of event names and the id of the skill title to append the textboxes and event names to. For each name in the array, a h3 element is created with the event name as its text value. A textarea is also created and the title of the event name and textareas are appended to the skill title name. This function allows for scalability as the function does not depend on the number of events passed through it. Furthermore, the code is modular in nature as this function can be called for each skill without any changes required.

After calling the function for each event name, this is the output that was produced:

The titles and textboxes are being created correctly in terms of formatting and alignment. The event names and textboxes for the skills on the right are aligned correctly and same for the ones on the left.

However, there are some issues with this output:

- I have declared the event names as h3 elements however they are being styled the same as the h1 event title elements. The event names are inheriting the formatting of the skills titles and therefore I will need to find an alternative way of implementing these textboxes.
- Furthermore, these textboxes currently don't send data to any formhandler and therefore I will need to add an action and method attribute to the form.
- Finally, I will need to implement a way for the textareas to retain the values sent from the database and allow for the values to be updated/inserted.

I will work on these issues in the next iterations of the code.

Second iteration implementation using PHP for textbox output with values

First, I will implement the functions from the design stage which will retrieve the data from the database, create the textareas with the values from the database and a composite function which will call both functions and can be called in skills.php.

```php
declare(strict_types=1);  //Best practice to prevent errors in data types

require_once 'dbh.inc.php';  //Allows for database linkage

    function get_data(object $pdo, string $skill, int $userId): array {  //Function returns an array therefore it has been declared
    //Allows for data for the specified skill to be returned
    $skillColumn = "SKILLS.$skill";
    $descColumn = "SKILLS_DESCRIPTION.{$skill}_description";

    //Returns the event_id, name_of_event, skill value and skill description for the logged in user in ascending order
    $query = "SELECT EVENTS.event_id, EVENTS.name_of_event, $skillColumn AS skill, $descColumn AS skill_description
              FROM EVENTS
              INNER JOIN LINK_TABLE ON EVENTS.event_id = LINK_TABLE.event_id
              INNER JOIN SKILLS ON LINK_TABLE.skills_id = SKILLS.skills_id
              LEFT JOIN SKILLS_DESCRIPTION ON LINK_TABLE.description_id = SKILLS_DESCRIPTION.description_id
              WHERE LINK_TABLE.user_id = :userid
              ORDER BY EVENTS.date_of_event ASC;";

    $stmt = $pdo->prepare($query);  //Query is prepared
    $stmt->bindParam(':userid', $userId, PDO::PARAM_INT);  //User id parameter is bound
    $stmt->execute();  //Statement is executed

    return $stmt->fetchAll(PDO::FETCH_ASSOC);  //All of the data is fetched and returned
}
```

The first function is the get_data function (I have included the ‘: array’ declaration to specify the return value which is best practice using strict types). It first sets the skills and skills description column values which will be used in the query using the passed skill name. Then the query is declared which retrieves the event id, name of event, skill value and description value from the database for the logged in user ordered in ascending order. This query is prepared; the parameters are passed. The results from the database are returned.

```php
function create_text_areas(array $results, string $skill, string $class){
    foreach ($results as $result) {  //For each returned event data from the database

        if ($result['skill'] == 1 && $class == 'left-title') {  //If the skill has been set and if the title should be aligned to the left

            if ($result['skill_description'] === NULL){  //If there is no skill description set
                $output = '';  //Output a blank string
            } else {
                $output = htmlspecialchars($result['skill_description'], ENT_QUOTES, 'UTF-8');  //If there is, output a cleaned version of it
            }

            //The name of event is output with the specified class
            echo '<h3 class="' . $class . '">' . htmlspecialchars($result['name_of_event'], ENT_QUOTES, 'UTF-8') . '</h3><br>';
            //The text area is output with the left align class, a name containing the event id and skill name, and the value of the output variable
            echo '<textarea class="text-area-left" name="event_' . (int)$result['event_id'] ."_". $skill . '">'. $output . '</textarea><br><br><br>';

        }
        elseif ($result['skill'] == 1 && $class == 'right-title') {  //If the skill has been set and if the title should be aligned to the right

            if ($result['skill_description'] === NULL){  //If there is no skill description set
                $output = '';  //Output a blank string
            } else {
                $output = htmlspecialchars($result['skill_description'], ENT_QUOTES, 'UTF-8');  //If there is, output a cleaned version of it
            }

            //The name of event is output with the specified class
            echo '<h3 class="' . $class . '">' . htmlspecialchars($result['name_of_event'], ENT_QUOTES, 'UTF-8') . '</h3><br>';
            //The text area is output with the right align class, a name containing the event id and skill name, and the value of the output variable
            echo '<textarea class="text-area-right" name="event_' . (int)$result['event_id'] ."_". $skill . '">'. $output . '</textarea><br><br><br>';
        }
    }
}

//This function combines both functions into one that can be called in the skills page
function output_skills(object $pdo, string $skill, string $class, int $userId){
    $results = get_data($pdo, $skill, $userId);  //The return value of the get_data function is stored as results

    create_text_areas($results, $skill, $class);  //The data is passed to the create_text_area function
}
```

The next function is the create_text_areas function which takes the results fetched from the database and the class for alignment. For each class and result, it checks if the skill description is blank in the database, the output variable is a blank string. If there is a value, the value is cleaned (htmlspecialchars) and then output is set to the cleaned value. The name of the event is cleaned and output as a h3 element with the passed class. This function allows for the data values to be retrieved from the database and retained in the textareas; however, I will be able to test this after the next iteration of the program where I implement the formhandler and action and method attributes. Furthermore, this function is also modular in nature as it does not require changes to the code when the program is scaled with more skills. The new skill name can be passed to the function without the need for changing anything as the retrieval system will work as intended.

Finally, the output_skills function gets the value returned by get_data stored as results and the result is passed to the create_text_areas function which generated the textareas and titles. This function simplifies the code to a function which can be called

in the skills page which further allows for modularity in code as declaring the lines as a function means that in each skills area, only the function needs to be called and the results do not need to be set each time and passed to create_text_areas.

```
<tr>
    <td><h2 class="left-title-header">Leadership</h2>
    <?php output_skills($pdo, 'leadership', 'left-title', $_SESSION['user_id']); ?>

    <td><p id="description" class="rightp">In your description, show how you:
        <br>were given a task, or came up with a task, to lead
        <br>thought about the leadership style that would work best for the group you're in
        <br>led the task, adapting your leadership style</p></td>
</tr>
```

Lastly, I have called the output_skills function for each skills area with the correct skill, class and user id session variable passed to it. This code has been similarly implemented for each skill using the corresponding classes for the correct alternating alignment.

Implementing the output_skills function and running the code leads to this output:

This is the correct output. The events which I added to the database using the input form have been correctly displayed underneath the skills title they have been tagged

with. The styling of each event name header is correct without it inheriting the styling of the skills title. The textboxes are resizable; they are separated equally and styled correctly. The alternating design is also present by passing the correct classes to the function when it is called for each skill which correctly implemented the design from the design stage.

Now that the textboxes are being output correctly, I will now begin working on the formhandler which will allow me to insert/update the database with the values input into the textareas.

Third iteration implementation – formhandler

First, I will add the action and method attributes to the form containing the input fields where the data will be sent using POST for insertion/update in the database.

```
<form action="includes/skills_input.inc.php" method="post">
```

I have added the action attribute to skills_input.inc.php which is named similarly to inputform_input.inc.php to maintain consistency in code which makes the program more robust and easier to navigate for future developers. Furthermore, I believe that the data should be sent using post (as skills description data is private data) as the data will not be visible in the URL when redirecting.

skills_input.inc.php

```
declare(strict_types=1);  //Best practice to prevent errors in data types

require_once 'dbh.inc.php';  //Allows for database linkage
require_once 'config_session.inc.php';  //Allows for session variables to be created/regenerated

//Ensure user is logged in
if (!isset($_SESSION['user_id'])) {
    header('Location: ../login.php');
    exit();
}

$userId = $_SESSION['user_id'];  //Retrieve the user id from the session variable

if ($_SERVER['REQUEST_METHOD'] === 'POST') {  //If the page was correctly accessed

    //Loop through submitted textareas
    foreach ($_POST as $textareaName => $descriptionText) {

    if (substr($textareaName, 0, 6) === 'event_') {  //If the textareaname begins with event_

        $withoutPrefix = substr($textareaName, 6);  //Remove event_ from the name
        $firstUnderscore = strpos($withoutPrefix, '_');  //Stores the index position of the first underscore in the text

        if ($firstUnderscore === false) {  //If the underscore wasn't found
            continue;  //The input is skipped as the format is invalid (validation)
        }

        $eventId = (int)substr($withoutPrefix, 0, $firstUnderscore);  //Event id is retrieved from the name without prefix
        $skill = substr($withoutPrefix, $firstUnderscore + 1);  //The skill name is retrieved

        //If the descriptionText is already set, use it, otherwise use "" (ternary operator)
        $descriptionClean = isset($descriptionText) ? $descriptionText : "";

        $descColumn = "SKILLS_DESCRIPTION.{$skill}_description";  //The column is specified based on skill
```

The program first runs a validation check to see if the user has logged in legitimately, if not, they are redirected to the login page. Otherwise, for each value sent from the skills

page, the textarea name prefix is removed and the index of the underscore is retrieved to allow for the event id and skill name to be retrieved from it. If the format of the name was incorrect, the value is skipped and the next value is used. This is to prevent unauthorised users from sending POST data to the input form as the textarea name would need to be in the exact format to be processed. The eventId and skill are taken from the textarea name with no prefix based on the index of the first underscore. The description value is then assigned using a ternary operator which checks if the description was set. If it was, that value is used, if not, a blank string is used. Then, the description column is declared before the query to update the data in the database is run.

```
        //Updates the skills description for the logged in user for the specific event id
        $query = "UPDATE EVENTS
                  INNER JOIN LINK_TABLE ON EVENTS.event_id = LINK_TABLE.event_id
                  INNER JOIN SKILLS_DESCRIPTION ON SKILLS_DESCRIPTION.description_id = LINK_TABLE.description_id
                  SET $descColumn = :skill_description
                  WHERE EVENTS.event_id = :event_id
                  AND LINK_TABLE.user_id = :user_id";

        $stmt = $pdo->prepare($query);  //Query is prepared
        $stmt->bindParam(':skill_description', $descriptionClean, PDO::PARAM_STR);  //Skills description parameter is bound
        $stmt->bindParam(':event_id', $eventId, PDO::PARAM_INT);  //Event id parameter is bound
        $stmt->bindParam(':user_id', $userId, PDO::PARAM_INT);  //User id parameter is bound
        $stmt->execute();  //Statement is executed
    }
}


    //Redirect back to skills page after update
    header('Location: ../skills.php?update=success');
    exit();
} else {  //Otherwise
    header('Location: ../skills.php');  //Redirect back to the skills page
    exit();
}
```

The SQL query updates the skills descriptions where the event id and user id match the textarea details and the logged in user's credentials. The query is then prepared, parameters are passed and the query is executed. Prepared statements are a form of validation as if the incorrect data type is input into the variables, the database is queried before the variables are passed. Finally, the program is redirected back to the skills page with the GET value of update=success and the program ends. If the page was incorrectly accessed, the user is redirected to the skills page.

```
if(isset($_GET['update']) && $_GET['update']==='success'){
    echo '<p id="form-success">Skills Descriptions successfully stored/updated.</p>';
}
```

This is a statement I added underneath the Skills title on the skills page. The GET value in the URL is used to output a validation statement which will confirm to the user that the description was successfully stored/updated. This is a helpful usability feature and makes the program feel more responsive. The statement has been implemented similarly to the redirects in the input form which will make the code easier to debug for future users.

After implementing these changes, I reloaded the page which produced these outputs:

First, I added some description text to a textarea in the leadership row based on an event I previously input into the calendar. I resized the textarea which allowed me to view the amount of text I input into the form:

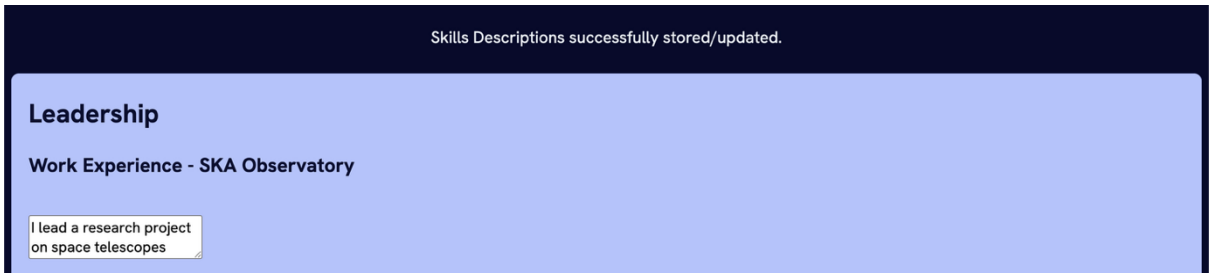
Pressing the submit button refreshed the page and displayed this success message under the skills descriptions title.

Scrolling down, I checked to see if the value was retained in the textarea.

The value was successfully stored in the textarea and therefore, skills.inc.php and skills_input.inc.php are successfully retaining the values in the textareas and the values are being input in the database. This showed that data is being correctly inserted into the database but now I will test if fields can be updated successfully.

I changed the text in the textarea entirely to check if records can be updated in the database alongside being inserted in the database.

Pressing the submit button led to this output:



The success message successfully appeared underneath the title, and the changed values were present in the input fields.

This means that the skills page is now functional and has passed all of the functionality tests. The page successfully inserts/updates descriptions in the databases and retains their values in the textareas after the submit button is pressed. The page outputs a success message once the database interaction is successful and the page has been styled almost exactly like the planned design in the design stage. However, there is still currently an issue with the program.

Fourth iteration implementation – navigation bar

Currently the only way to change pages is based on redirects. To go from the input form page to the calendar page, for example, unless the user inputs an event successfully in the form the user needs to use the back button. This issue is especially prevalent when the user wants to travel between the input form and the skills page/calendar page where there are no redirects available. The user would need to type in the URL of the page to access it. To solve this issue and to implement an essential feature from the success criteria, I would like to implement a navigation bar which would allow for traversal between the pages.

This is the code for the navigation bar which I will include at the top of every page except for the account creation pages.

```
<!--This is the navigation bar which allows the user to traverse between pages and logout from the website-->
<nav>
<ul>
<li><a href="calendar.php">Calendar</a></li>
<li><a href="skills.php">Skills</a></li>
<li><a href="timeline.php">Timeline</a></li>
<li><a href="includes/logout.inc.php">Logout</a></li>
</ul>
</nav>
<br>
```

The navigation bar has been implemented using a nav tag and a ul tag. These tags are important as, using these tags, I will be able to style the navigation bar horizontally and in line with the minimal design I established in the design stage. The bar has links to all 3 of the main pages (excluding the input form as an empty date get value would result in errors). This would allow the user to switch between pages easily which is helpful for usability. The navigation bar is modular in design, as when more pages are added to the program in future iterations, the page links can be added by adding another li and a element before the logout option.

Furthermore, I have also included a button which would log out the user. This was a feature that I would like to implement as there is no current way for the user to logout after they login. This would be a useful usability feature which would prevent the need for the user to close the window to logout.

```
nav{
position: left;
}

nav ul{
list-style: none;
background-color: #070a2a;
overflow: hidden;
margin: 0;
padding:0;
}

nav ul li{
float: left;
}

nav ul li a{
display: block;
color: aliceblue;
text-align: center;
padding: 14px 20px;
text-decoration: none;
border-color: black;
}

nav ul li a:hover {
background-color:  #b5c4fa;
}
```

This is the styling the navigation bar I have used. It aligns the navigation bar to the top left of the page, and the bar will be a horizontal selector rather than vertical. I have chosen this design to fit the minimal aesthetic and to be reminiscent of navigation bars found in modern websites. The default colour of the bar will be the same as the

background colour to prevent the design from feeling obtrusive when using the website. Furthermore, to reduce obtrusiveness, I have set the position attribute to left rather than fixed as using the fixed attribute would mean that the navigation bar may cover up elements of the website which may hamper user experience. Furthermore, when the use hovers over a button in the bar, it will change colour to the colour for the divs used for interactive areas of the website which is a useful usability feature. The design choices have been made with usability and robustness in mind.

Saving the code and running it produced this output:

The navigation bar has been correctly styled with the correct font, horizontal alignment and background colour. Now I will test the program by hovering over one of the buttons on the bar:

The page name has successfully changed colour which contrasts well with the other pages which is good for usability as it helps contrast the buttons well with the rest of the design. Now I will test if the redirects work as intended (I will not be able to test the redirects for the timeline page or the logout feature as they have not been implemented yet). Clicking on the calendar button on navigation bar while being on the skills page produced this output:

The bar successfully redirected me to the calendar page while I was previously on the skills page and therefore the redirection feature of the navigation bar works as intended.

Fifth iteration implementation - logout

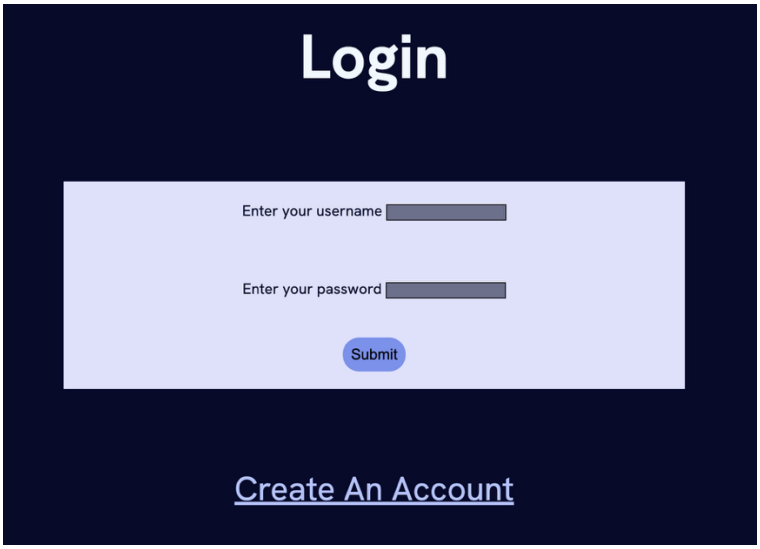
Now, I will begin working on the logout system as planned in the design stage.

```
session_start();  //Session is started
session_unset();  //Session variables are unset
session_destroy();  //The session is ended
header("Location: ../login.php");  //The program is redirected to the login page
die();  //The code ends
```

The code for the logout feature is simple. It starts a session, unsets all of the previously set session variables, ends the current session and redirects the user to the login page. The code is simple and easy for future developers to debug in future iterations of the project if a feature that needs session variables which remain after logout to be implemented. The logout feature is modular as the session_unset line can be replaced with specific session variables that need to be unset if some session variables are to be kept after logout.

I will now test the code to see if I am logged out successfully:

Clicking on the logout button on the navigation bar on the calendar page gave me this output:



I was successfully redirected to the login page which proves that the code has been successfully implemented and that the skills page and navigation bar are now complete.

Stage 6 Review

What has been done

The skills page has been implemented according to the planned design. The rounded divs and separation by skills have been implemented using a table as a grid. The example descriptions have been implemented for each skill, and the alternating design is present throughout the page. The page creates the textareas and each textarea retains the data inserted in the database using the output_skills function which combines the functions which retrieve the data from the database and create the textareas based on the data. This function can be called in the skills page which makes the design modular in nature as any changes to the previous functions can be implemented in the output_skills function without needing to change anything on the skills page. Furthermore, the formhandler has been implemented which stores the input data in the database and produces a success message. This is helpful for usability as the user has a clear message informing them of a successful input. Finally, a navigation bar has been implemented which will be included on all pages except for the account creation pages to allow the user to traverse between pages of the program easily which is a good usability feature. The navigation bar also contains a button for logout which unsets all of the session variables and redirects the user to the login page. This allows the user to logout of the website without needing to close the tab. This makes the program feel more robust and similar to other modern websites.

How it has been tested

The page has been tested for styling: the alternating design is present when creating the titles and textareas, the divs have been styled according to the planned design and the submit button has been styled consistently to the input form to maintain a robust design of the program. The program has implemented this using CSS, and any style inheritance issues have been solved by implementing the textboxoutput using PHP rather than JS.

The page has also been tested for functionality: the textboxes are resizable, the textbox values are successfully inserted/updated upon submission, and the values are retained in the database. Furthermore, only the events with the correct skills tag are output in the skills areas. The program has passed these tests successfully and I will further test the program using the testing checklist in the final testing stage.

Criteria met/being met

- Skills table that links to calendar
- Skills description feature
- Example description of skills and extracurriculars
- Navigation bar

How it meets the success criteria and user expectations

The skills page was successfully styled and implemented using HTML and CSS. The separation and rounding of skills areas have been implemented to match the planned design. The alternating design is also present which helps the program feel more robust. The skills page outputs titles of event names tagged with the skill correctly under each skill section. This means that the design of the program matches the planned design and therefore user expectations.

Furthermore, the skills description feature has been implemented using the output_skills function. Skills descriptions can be inserted/updated in the database by submitting the page which sends the data to a formhandler. The formhandler validates the data and then submits it in the database. Stored values are cleaned (for validation) and output to the user to allow them to view the values they have input in the database.

The example descriptions have been implemented for each skill. The example description ranges between 2-4 pointers for each skill which guides the user on how to write their skills descriptions. They have also been styled according to the planned design which helps it match user expectations.

The navigation bar has also been implemented. This allows the user to traverse the pages easily and the bar has been styled to minimise obtrusiveness in design. The bar also allows the user to logout properly without needing to close the tab entirely. This feature as a whole has bene designed to improve usability of the program.

Changes in the design that have resulted from this section

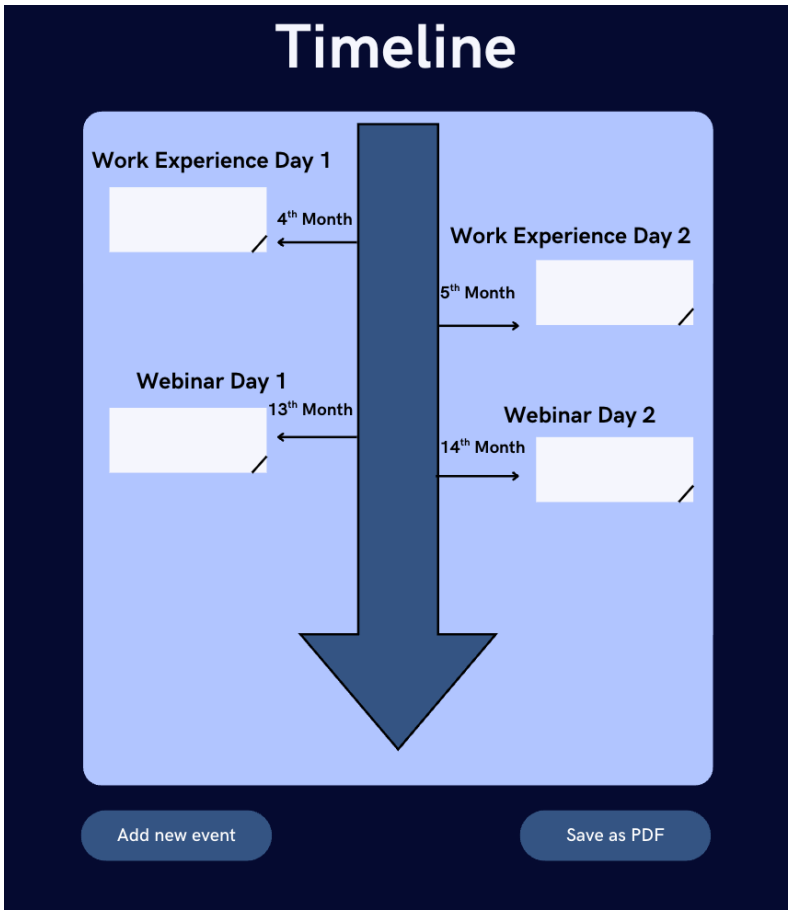
There has only been one change in the design in this section. The skills titles have been included within the divs rather than outside of it. As I am implementing the skills page as a table, implementing the titles outside of the divs would increase the amount of scrolling the user would need to do and, as the skills page is already a lengthy page, this would negatively impact the usability of the program. The rest of the program has been implemented exactly as planned in the design stage which ensures that user expectations will be met.

Summary of the whole project as a prototype at this stage

The program as a whole contains an account system with input validation, a calendar homepage where events can be added and colour coded using the inputform and now the skills page. The skills page allows the user to explain how they have demonstrated the skill they have tagged the event with. To allow for this, the textareas are created which contain the values stored in the database. These values can be inserted/changed using the formhandler file. The navigation bar has also been implemented which allows the user to traverse between pages easily. This feature has not been implemented on the account creation pages as it would allow the user to access the homepages without the proper validation which could increase risks of data breaches.

Stage 7: Timeline Page

This is the design that I created for the timeline page in the design stage.



The key aspects of the design are the arrow image present in the background of the div, the event name, date and reflection textboxes ordered chronologically and the buttons to add an event or save as PDF. I will change the 'add new event' button to a submit button for the event reflections as adding new events can be done using the calendar and the input form.

I will begin by first creating a prototype of the GUI of the webpage so I can see the outputs of the functions which I will implement on the page.

Creating a prototype GUI of the page

timeline.php

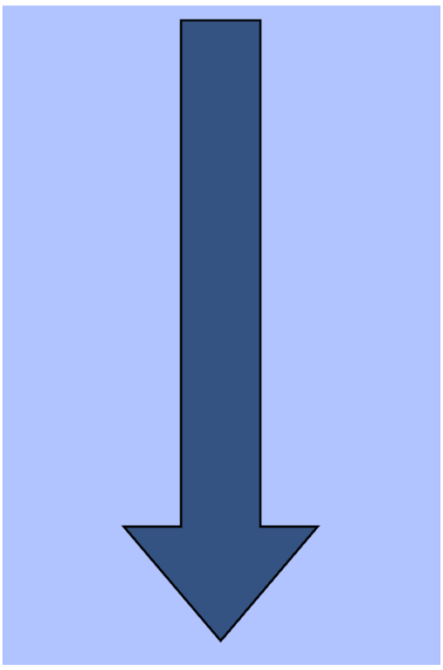
```
<form action="includes/timeline_input.inc.php" method="post">
    <div id="timeline" class="html-content">
        <script>textboxoutput(['1', '2', '3', '4'], "timeline");</script>
    </div>

    <div class="button-container">
        <input type="submit" id="submit" value="Submit">
        <button id="download" type="button" onclick="CreatePDFfromHTML()">Save as PDF</button>
    </div>
</form>
```

This is the form element which makes up the majority of the timeline page. Taking previous stages of development into account, I have already added action and method attributes to the form (however, I will implement timeline_input.inc.php after implementing the function to create the event reflection textareas). The timeline div has an id and class; the id will be used to style the div with the arrow image and the class will be used to save the timeline as a pdf in a later iteration of implementation. The timeline form also contains another div with the class 'button-container' which will store the submit and save as pdf buttons and the class will be used to style them according to the design.

timelinestylesheet.css

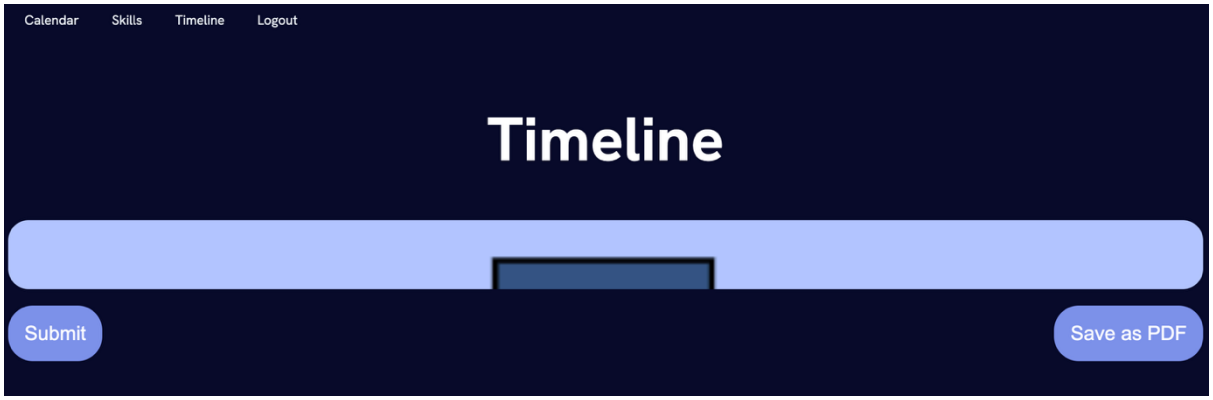
```css
@import url('https://fonts.googleapis.com/css2?family=Hanken+Grotesk:ital,wght@0,100..900;1,100..900&display=swap');
/*Importing correct font (based on Canva design) from Google fonts*/

body{
    color: ■#0b0e2e;
    background-color: ■#070a2a;
    font-family: 'Hanken Grotesk', sans-serif;
    text-align: center;
}

#title{
    color: □white;
    font-size: 75px;
}

#timeline{
    background-image: url('timeline_arrow_new.jpg');
    background-repeat: no-repeat;
    background-size: cover;
    border-radius: 25px;
    text-align: left;
    padding-left: 15px;
    padding-right: 15px;
}
```

timeline_arrow_new.jpg ×



The stylesheet is responsible for making the website look as close to the design as possible. Learning from previous stages of development, I have imported and included the font correctly in the styling of the page. Furthermore, the title has been designed to match the style of the other page titles which is good for usability as it maintains a consistent design across pages of the website. The timeline div has been styled to include the image of the arrow as the background image. The arrow will be one consistent image which will cover the div and more of the arrow will be revealed as more events are added. I believe that this is a good usability feature as it encourages users to add more events to the calendar to see the full timeline arrow.

Implementing code leads to this output:



The div is correctly generated with the start of the arrow and the corners are rounded similar to the design. The submit buttons have been implemented as a mix of the design from the previous pages and the planned design to ensure consistency with the rest of the website. The buttons are large and easily identifiable which is good for usability.

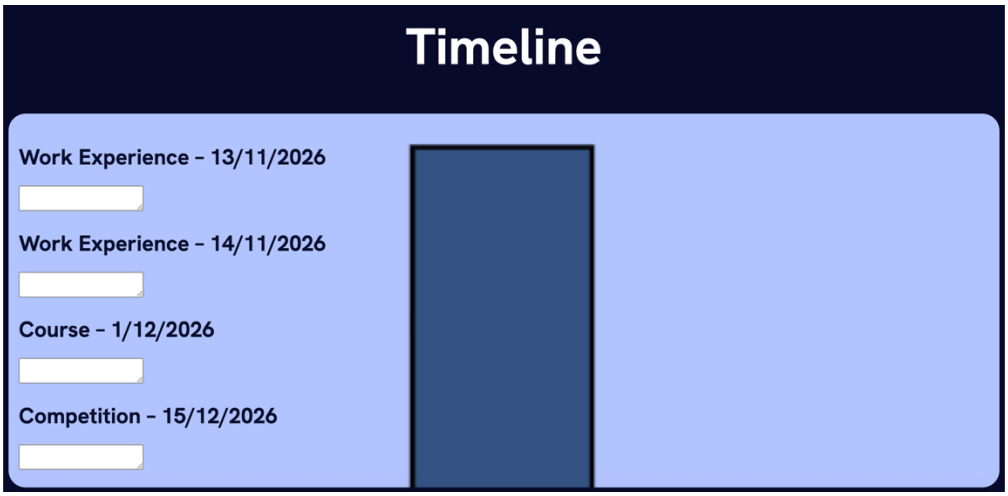
Now that a prototype of the GUI has been created, I will now implement the textboxoutput function planned in the design stage for the timeline page which will generate the event reflection textareas.

Initial iteration implementation using JavaScript

```
/*This function takes an array of event names and dates (stored as one string) as input
and the id of the timeline page div to append the names and dates of events in the
array as titles and textareas for event reflections*/
function textboxoutput(tagged_arr, x){
        for(let i = 0; i < tagged_arr.length; i++){  //Loop through the array of event names tagged with the skill
        const title = document.createElement("h1");  //Create a title element for the name and date of event
        title.htmlFor = tagged_arr[i];  //Add a for attribute for correct formatting
        const node = document.createTextNode(tagged_arr[i]);  //Create a text node for the name and date of the event
        title.appendChild(node);    //Append the text node to the title element

        const text = document.createElement("textarea");  //Create the input textarea for event reflections
        text.id = tagged_arr[i];  //Add an id for styling purposes
        text.name = tagged_arr[i];  //Add a name so the data can be retrieved when the form is submitted

        const br = document.createElement("br");  //Create a break

        const element = document.getElementById(x);  //Retrieve the timeline div area based on its id
        element.appendChild(br);  //Append a break
        element.appendChild(title);  //Append the name and date of the event
        element.appendChild(text);  //Append the event reflection textarea
        }
}
```

This function takes an array of all of the event names and dates formatted as 'Work Experience – 13/11/2026' and creates titles and textarea elements for them. Each title is stylised as h1 elements for better readability and usability on the page. Furthermore, the textareas have been given id and name attributes which correspond to the event names and dates. This allows for the values of the textareas to be accessed once the form is submitted. The titles and textareas are appended to the div with breaks in between subsequent titles and textareas. This function is modular as it is similar to the textboxouptut function used in the skills page with a few changes. The changes being adding attributes to the title and textarea elements to allow for access from the formhandler.

After implementing the function and reloading the webpage, this was the output:



The textboxes are being output correctly, however, there are two main issues with the output which will need to be fixed in further iterations of the page:

- The alternating design cannot be implemented using the current function
- The function has no way of displaying the stored event reflections

To implement these features, I will create a file called timeline.inc.php (similar to skills.inc.php from the skills page) with similar functions which were designed in the design stage. This file will query the database and create textareas and titles with the event reflection value present and will output them with the correct styling on the timeline page to allow for the planned design to be implemented.

Second iteration implementation using PHP – timeline.inc.php

timeline.inc.php

The file as a whole is based on the principle of modularity. The file contains almost the same functions as the skills.inc.php file, however, with changes which allow for them to be implemented on the timeline page. This was planned in the design stage and done deliberately to further improve the modularity of my code.

```php
declare(strict_types=1);  //Best practice to prevent errors in data types

require_once 'dbh.inc.php';  //Allows for database linkage

function get_data(object $pdo, int $userId): array {  //Function returns an array therefore it has been declared

    //Returns the event_id, date_of_event, name_of_event and event_reflection for the logged in user in ascending order
    $query = "SELECT EVENTS.event_id, EVENTS.date_of_event, EVENTS.name_of_event, EVENTS.event_reflection FROM EVENTS
    INNER JOIN LINK_TABLE ON EVENTS.event_id = LINK_TABLE.event_id
    INNER JOIN USERS ON LINK_TABLE.user_id = USERS.user_id
    WHERE USERS.user_id = :user_id
    ORDER BY EVENTS.date_of_event ASC";

    $stmt = $pdo->prepare($query);  //Query is prepared
    $stmt->bindParam(':user_id', $userId, PDO::PARAM_INT);  //User id parameter is bound
    $stmt->execute();  //Statement is executed

    return $stmt->fetchAll(PDO::FETCH_ASSOC);  //All of the data is fetched and returned
}
```

The first function of the file is the get_data function. This function retrieves the event id, date of event, name of event and event reflection from the database in ascending order (as the timeline is chronological) for the logged in user. All of the data is retrieved as the purpose of the timeline page (as outlined in the breakdown of the problem) is to display all of the events stored in the database chronologically. The user id is passed to the statement and the query is executed. The results of the query are returned to the function. This function highlights the modularity of the code as it is similar to the function in the skills page, however, it takes fewer parameters as the skill does not need to be specified for the timeline page.

```php
function create_text_areas(array $results){
    $i = 0;
    foreach ($results as $result) {  //For each returned event data from the database
        if ($i % 2 === 0) {  //If i is even
            $titleClass = 'left-title';  //The title is aligned to the left
            $textClass = 'left-text-area';  //The text area is aligned to the left
        } else {
            $titleClass = 'right-title';  //The title is aligned to the right
            $textClass = 'right-text-area';  //The text area is aligned to the right
        }

        // Output event name and date safely
        //The name of event is output with the specified class
        echo '<h3 class="' . $titleClass . '">' . htmlspecialchars($result['name_of_event']) . '</h3>';
        //The date of event is output with the specified class
        echo '<h3 class="' . $titleClass . '">' . htmlspecialchars($result['date_of_event']) . '</h3><br>';

        // Textarea with name = event_EVENTID, contains sanitized reflection
        //If there is a reflection, output the cleaned value, otherwise output ''
        $eventReflection = htmlspecialchars(isset($result['event_reflection']) ? $result['event_reflection'] : '');
        //The text area is output with the correct class, a name containing the event id, and the value of the eventReflection variable
        echo '<textarea class="' . $textClass . '" name="event_' . (int)$result['event_id'] . '">' . $eventReflection . '</textarea><br><br><br>';

        $i++; //i is incremented
    }
}

//This function combines both functions into one that can be called in the timeline page
function output_events(object $pdo, int $userId) {
    $results = get_data($pdo, $userId);  //The return value of the get_data function is stored as results

    create_text_areas($results);  //The data is passed to the create_text_areas function
}
```

The second function is the create_text_areas function which outputs the event name and date with the textarea. The function loops through the array of results and uses a counter value to set the class values. If the value is even, the elements are aligned to the left, if the value is odd, the elements are aligned to the right. This is a simple way of

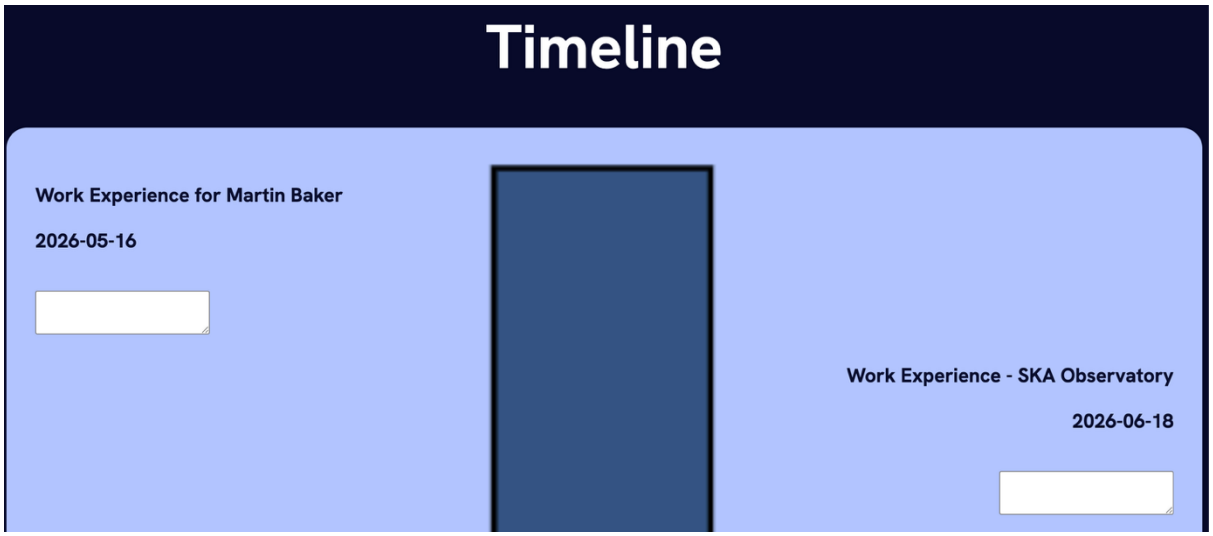
implementing the alternating design, fulfilling user expectations and making the program more robust. After the classes are set, cleaned values of the event name and date are used to create h3 title elements. I have chosen to switch to using h3 elements to maintain consistency with the skills page and as I have chosen to implement the date and name on different lines. Implementing them using larger title elements may unnecessarily increase the length of the page which may hinder usability. After the titles are created, the value of the textarea is determined using a ternary operator. If there was a value fetched from the database, a cleaned version of the value is output (validation) if not, a blank string is output. Then, the textarea is created with the correct class, name and preset value.

Finally, the output_events function combines the get_data function and create_text_areas function into one function which can be called on the timeline page to produce an output. The function is designed for modularity as if any changes need to be made to the other functions, the output_events function can also be updated without the need to change anything on the timeline page.

Finally, I called the output events function on the timeline page.

```php
<?php
    output_events($pdo, $_SESSION['user_id']);
?>
```

After accessing the file and calling it in the timeline page, it produced this output:



This is the correct output. The titles are correctly formatted with the date underneath the name to maintain a minimal, user-friendly design. The titles have been formatted similarly to the skills page to maintain consistency across pages and to help with usability. Finally, the textareas have been correctly implemented to be resizable as the event reflections will require a large input field.

There are two main pending features of the program, the formhandler and the save as pdf feature. In the next iteration, I will begin implementing the formhandler which will allow me to test if the data retention works.

Third iteration implementation using PHP – timeline_input.inc.php

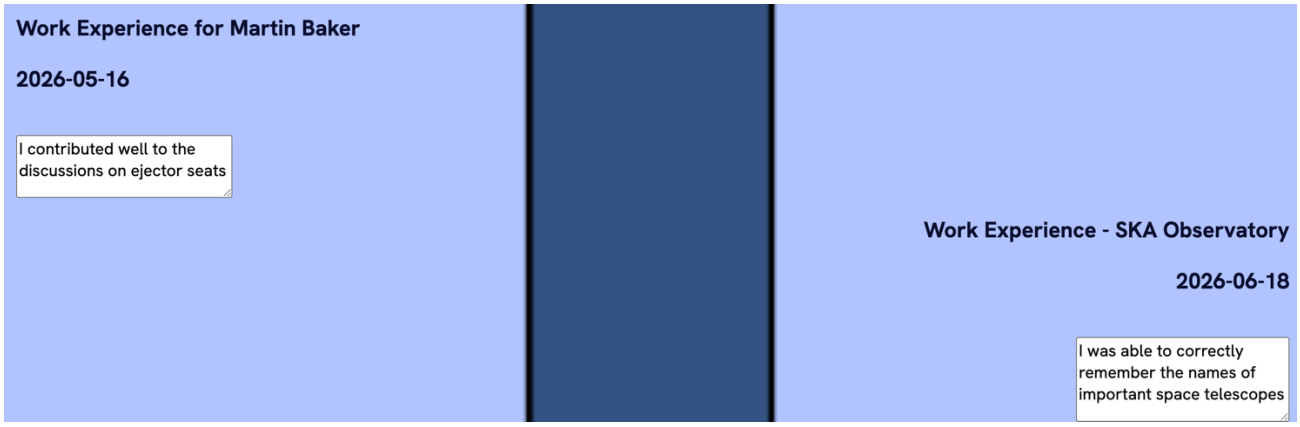
timeline_input.inc.php

```php
declare(strict_types=1);  //Best practice to prevent errors in data types

require_once 'dbh.inc.php';  //Allows for database linkage
require_once 'config_session.inc.php';  //Allows for session variables to be created/regenerated

//Ensure user is logged in
if (!isset($_SESSION['user_id'])) {
    header('Location: ../login.php');
    exit();
}

$userId = $_SESSION['user_id'];  //Retrieve the user id from the session variable

if ($_SERVER['REQUEST_METHOD'] === 'POST') {  //If the page was correctly accessed

    //Loop through submitted textareas
    foreach ($_POST as $textareaName => $reflectionText) {

        if (substr($textareaName, 0, 6) === 'event_') {  //If the textareaname begins with event_

            $eventId = (int) str_replace('event_', '', $textareaName);  //Event id is retrieved from the name without prefix
            $reflectionClean = htmlspecialchars($reflectionText, ENT_QUOTES, 'UTF-8');  //Cleaned reflection is retrieved

            //Updates the event reflection for the logged in user for the specific event id
            $query = "UPDATE EVENTS
                    INNER JOIN LINK_TABLE
                    ON EVENTS.event_id = LINK_TABLE.event_id
                    SET EVENTS.event_reflection = :reflection
                    WHERE EVENTS.event_id = :event_id
                    AND LINK_TABLE.user_id = :user_id";

            $stmt = $pdo->prepare($query);  //Query is prepared
            $stmt->bindParam(':reflection', $reflectionClean, PDO::PARAM_STR);  //Event reflection parameter is bound
            $stmt->bindParam(':event_id', $eventId, PDO::PARAM_INT);  //Event id parameter is bound
            $stmt->bindParam(':user_id', $userId, PDO::PARAM_INT);  //User id parameter is bound
            $stmt->execute();  //Statement is executed
        }
    }

    //Redirect back to timeline after update
    header('Location: ../timeline.php?update=success');
    exit();
} else {  //Otherwise
    header('Location: ../timeline.php');  //Redirect back to the timeline page
    exit();
}
```

This is the formhandler which I have implemented. It first runs a validation check to see if the is logged in to the site (if the user id is set) if not, they are redirected to the login page. This prevents malicious users from attempting to perform SQL injection attacks by attempting to directly access formhandler files. The user id is then retrieved, and the file performs another check to see if the user accessed the formhandler successfully which adds another layer of validation to the program to further protect the database. If the page was incorrectly accessed, the user is redirected back to the timeline page. If the user accessed the page successfully, for each textarea, a validation check for the name attribute is done. If the name doesn't match the correct format, the data isn't inserted into the database. This adds a further layer of validation which prevents any incorrectly formatted POST values which may try to maliciously access the database from being input/updated. If the name format is correct, the event id is retrieved from the name value, and a cleaned version of the reflection text is taken. The database is

updated with the cleaned event reflection for the event id and logged in user. The statement is prepared, parameters are passed and the statement is executed. This is repeated for each textarea value until all of the values have been checked and finally, the user is redirected to the timeline page with the update=success GET value to output a success message which makes the program feel more responsive and usable.

Running the code, I will test the timeline page by adding values to the event reflection textareas:



This is a screenshot of the input that I am entering into the event reflection field. After pressing submit, this was the output:

Checking the input fields to see if the reflections remained, I saw this:

This is the correct output and the test was successful. The textboxes retained the correct values entered into the input field and the success message was correctly output. This shows that the functions in timeline.inc.php are retrieving the data from the database correctly, and furthermore, it shows that the timeline_input.inc.php formhandler is successfully updating the form data in the database upon submission.

Now that the timeline page is functional, I will add the save as pdf functionality in the next iteration of the program. This is a useful desirable feature to have and was an important part of the planned design.

Fourth iteration implementation using JS – save as pdf feature

To implement this feature, I looked at a stack overflow thread which discussed this topic. After reading multiple answers which all had varying levels of success (some couldn't save CSS styling properly, etc), I created this function using the answers from the thread.

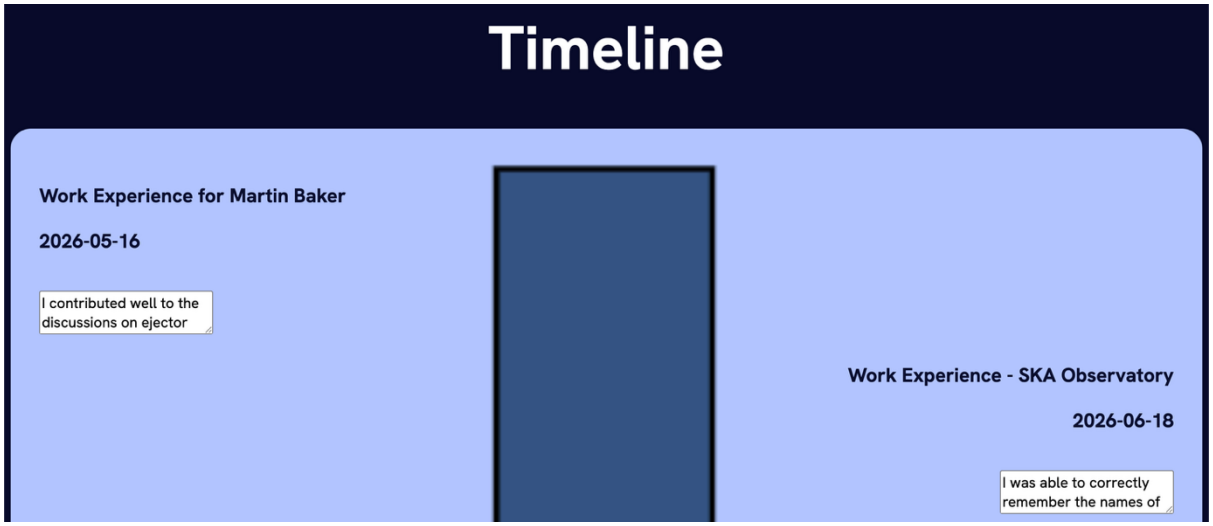
```
/*Function that converts HTML content into a downloadable PDF*/
function CreatePDFfromHTML() {
    var HTML_Width = $(".html-content").width();  //Get the width of the HTML element
    var HTML_Height = $(".html-content").height();  //Get the height of the HTML element
    var top_left_margin = 15;  //Margin size for the PDF (gap between the edges of the div to the page of the pdf)
    var PDF_Width = HTML_Width + (top_left_margin * 2);  //Calculate the total PDF width (including margins)

    var PDF_Height = (PDF_Width * 1.5) + (top_left_margin * 2);  //Calculate the PDF height (1.5 ratio helps maintain a page-like aspect ratio)
    var canvas_image_width = HTML_Width;  //Width of the image that will be placed into the PDF
    var canvas_image_height = HTML_Height;  //Height of the image that will be placed into the PDF

    //Calculate how many additional PDF pages are needed
    var totalPDFPages = Math.ceil(HTML_Height / PDF_Height) - 1;  //Math.ceil rounds up to ensure all content fits

    html2canvas($(".html-content")[0]).then(function (canvas) {  //html2canvas converts the HTML element into a canvas image
        var imgData = canvas.toDataURL("image/jpeg", 1.0);  //Convert the canvas into a JPEG image
        /*Create a new PDF document using jsPDF
        'p' = portrait orientation
        'pt' = unit of measurement (points)
        [PDF_Width, PDF_Height] = custom page size*/
        var pdf = new jsPDF('p', 'pt', [PDF_Width, PDF_Height]);

        //Add the image of the HTML content to the first page
        pdf.addImage(imgData, 'JPG', top_left_margin, top_left_margin, canvas_image_width, canvas_image_height);

        //Loop to create extra pages if the content is taller than one page
        for (var i = 1; i <= totalPDFPages; i++) {
            pdf.addPage(PDF_Width, PDF_Height);  //Add a new page to the PDF
            //Add the same image but shift it upward so the next section appears
            pdf.addImage(imgData, 'JPG', top_left_margin, -(PDF_Height*i)+(top_left_margin*4),canvas_image_width,canvas_image_height);
        }
        //Save the generated PDF file
        pdf.save("Timeline.pdf");
        $(".html-content").hide();
    });
}
```

This function relies on jspdf and html2canvas to create the pdf by turning the div into a canvas using html2canvas width and height elements. The program then converts the created canvas into an image which is then stored on the jspdf pages. If the image requires multiple pages, multiple pages are added to the pdf to store the full image. The pdf is then saved as timeline.pdf.

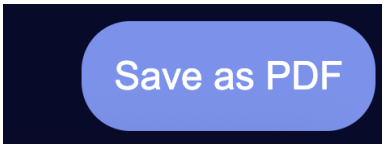
The thread stated that I would need to add the following script declarations to the header of the timeline page to access the jspdf and html2canvas functions.

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/1.3.3/jspdf.min.js"></script>
<script src="https://html2canvas.hertzen.com/dist/html2canvas.js"></script>
```

I saved the code and ran it, and it produced this output:



This was the timeline page before I clicked the save as pdf button.



Clicking on the button, however, lead to no change occurring.

I checked the downloads folder of my browser; however, nothing was downloaded and there was no pdf present. Therefore, I checked the code to see if any errors were present or to see if I called the function correctly in the onclick, however, the code was correct.

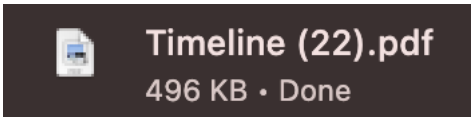
After researching the issue online, I found a website (linked in the bibliography) which also showed how to create a pdf using html canvas and in their header file they said to include the following declaration alongside the other previous declarations.

```
<script src="https://code.jquery.com/jquery-1.12.4.js"></script>
```

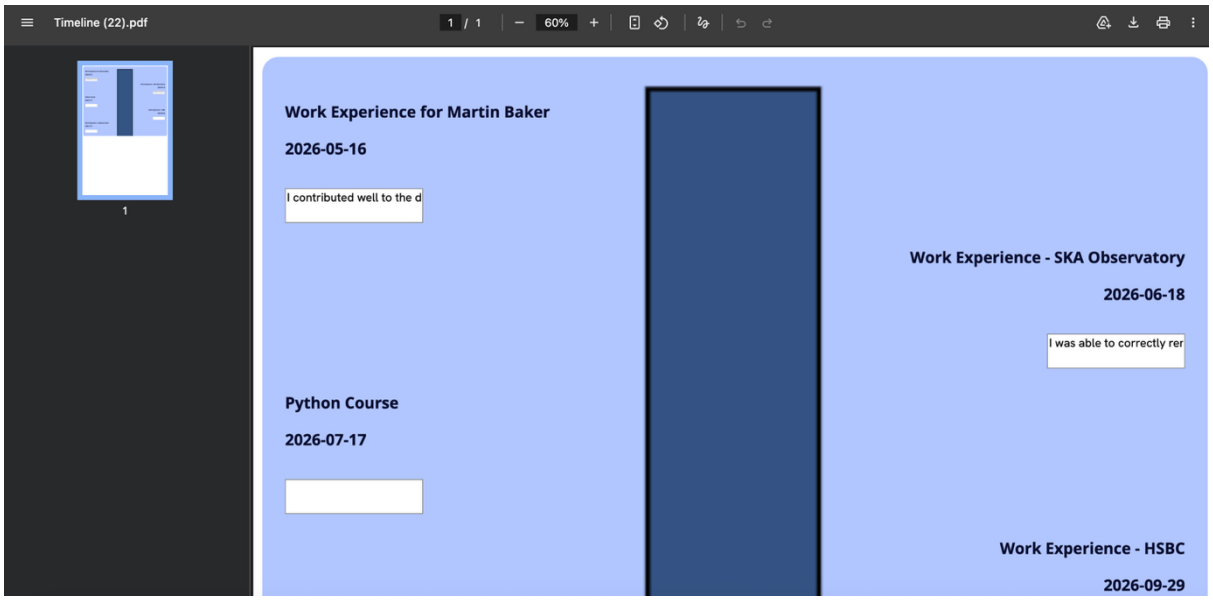
The CreatePDFfromHTML function was not able to import the jquery correctly.

Adding that line, saving the code and clicking on the save as PDF button produced this output:

The timeline div got removed from the page which is not the correct output. However, the pdf file was downloaded.



Opening the file produced this screen:



The function successfully saved the timeline page as a PDF while preserving the styling of the website. However, the html content of the page was disappearing therefore, I will need to check my code to resolve this.

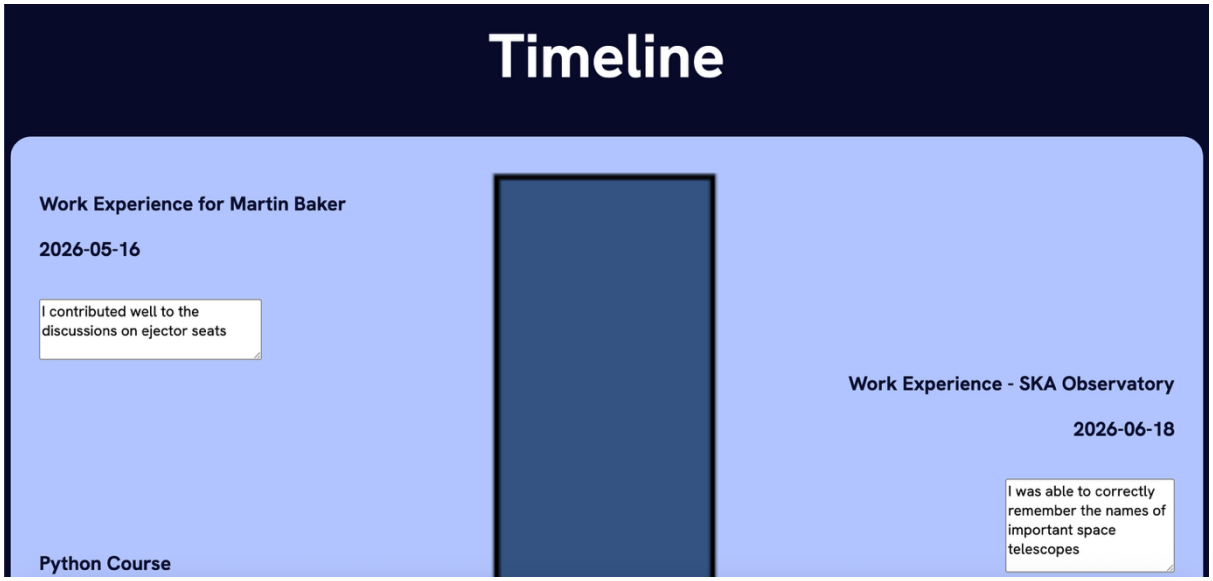
When checking my code, I identified this line which I implemented following one of the functions in the thread. It turns out that the hide function hides the contents of the timeline div with the class ‘html-content’.

```
$(".html-content").hide();
```

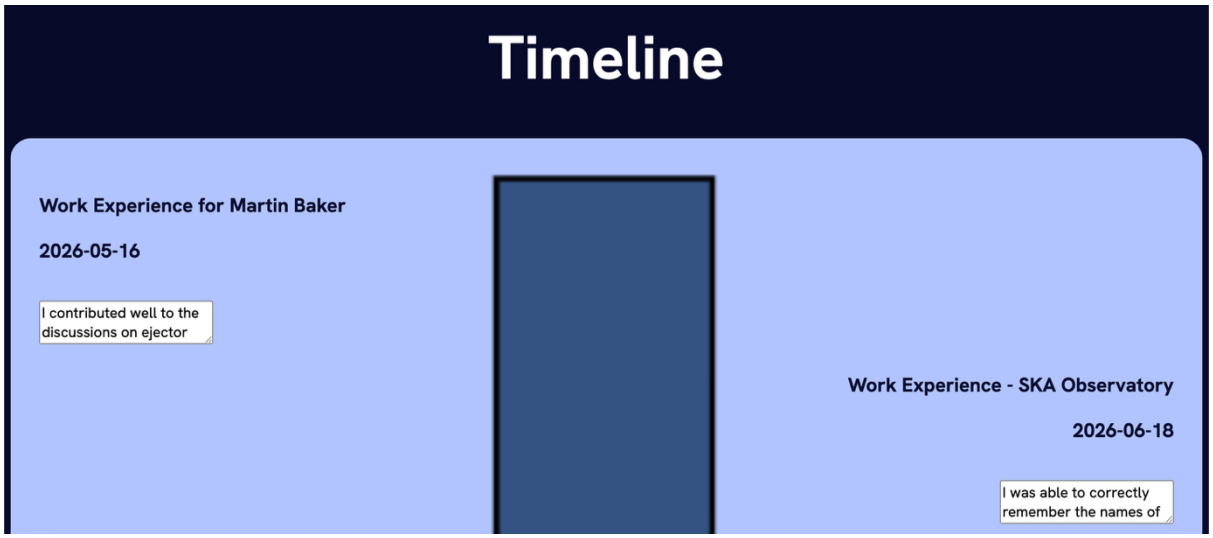
Therefore, I commented out the line from the function declaration.

```
//$(".html-content").hide();
```

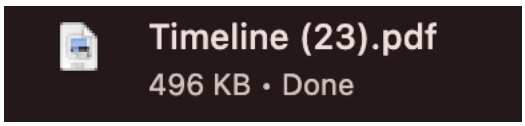
Saving the changes and running the website once again, I checked the timeline page. This was the page before I pressed the save as pdf button:



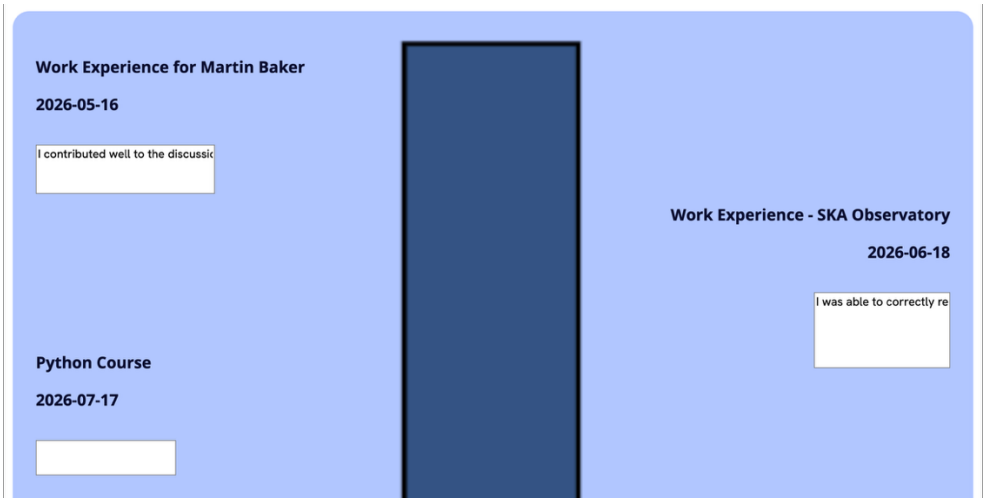
After pressing the submit button, this was the page:



The html content was no longer being hidden, the output is correct. However, I will now check to see if the timeline pdf is still being correctly created.

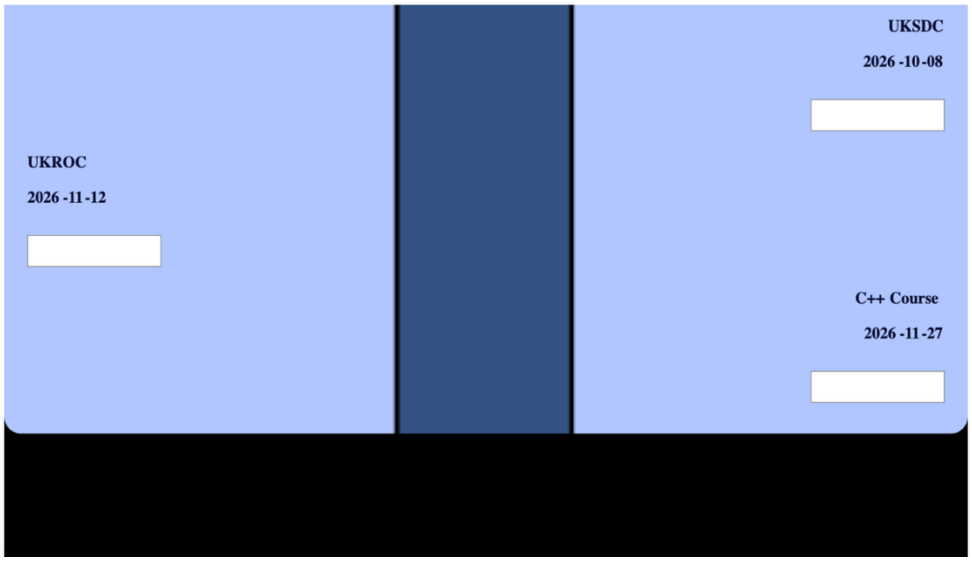


The file was correctly downloaded and opening it produced this output:



The timeline div was correctly stored as a pdf and therefore the program is now fully functional.

One failed test that I would like to include, as it will help inform the limitations of the program, was the following test. I was checking to see if the save as pdf feature was working and if the feature could be used after adding more events to the calendar. When I clicked the button, my internet connection disconnected. This was due to WIFI issues at the time; however, a file was downloaded. Opening the saved PDF gave me this output:



The fonts were not correctly loaded and there was a black background underneath the div which didn't show the remaining events on the page which means that there was an issue with the html2canvas functions. This test shows that one limitation of the program is that it requires a stable internet connection.

Now that all of the required features of the program have been implemented (alongside the save as pdf desirable feature) I will move on to the final testing of the program before I send the program to the stakeholders for stakeholder testing.

Stage 7 Review

What has been done

The timeline page has been fully implemented. The GUI of the page has been correctly styled according to the design present in the design stage of the program using the same arrow image used in the initial design. All of the event names and dates are output as titles alongside textareas for event reflections using the output_events function in timeline.inc.php. This file as a whole is modular in nature as it uses the same create_text_areas and get_data functions present in skills.inc.php but with changes which allow the program to be suitable for the timeline page. The event reflection textareas contain the values inserted in them in the database upon creation. These values can be inserted/updated in the database by changing the values and pressing the submit button which sends the data to a formhandler which validates the data before it is sent to the database. The save as pdf desirable feature has also been implemented which allows the user to have a copy of the timeline page to attach to their cv or use as reference.

How it has been tested

The program has been tested for styling: the GUI contains the correct arrow image to visually show the chronological order and the submit and save as pdf buttons have been styled using the text colour from the design but generalising the background colour to that present in other submit buttons on other pages of the program (for better usability). The alternating design of the events has also been implemented in the create_text_areas function using a counter value which ensures that user expectations are met.

The program has also been tested for functionality: the event reflection areas can be input into and the values are inserted/updated and a success message is output once the submit button has been pressed, the values are retained in the textareas after submission for user reference and to allow them to change the values and the save as pdf feature works as intended when the button is pressed.

Criteria met/being met

- Timeline page
- Reflection for each event on the timeline
- ‘Save as PDF’ feature on the timeline page

How it meets the success criteria and user expectations

The timeline page has been created according to the user design. The alternating design, the background arrow image, the button styling has all been implemented successfully using HTML and CSS. The page contains large text size and buttons as present in the proposed design which helps the program meet user expectations.

The event reflections feature has been implemented successfully. The textareas are created with the stored values and these values can be updated and submitted into the database by pressing the submit button. They are ordered in an alternating style as planned in the design stage and the functionality has been implemented as a modular, adapted version of that of the skills page as both contain creating textareas which retain values which can be inserted into the database. This allows the event reflections page to be successfully implemented according to user expectations.

Finally, the save as pdf desirable feature has been implemented by referencing a stack overflow discussion thread. The pdf is saved successfully as 'Timeline.pdf' and the styling of the arrow and the alternating design is correctly preserved.

Changes in the design that have resulted from this section

There have been three main changes compared to the design section:

The 'add new event' button has been replaced with a submit button. This is because, adding events to the calendar is the function of the input form. Adding a second button on the timeline page would make the calendar-to-input form redirect redundant and would also cause issues with date data being added. Therefore, to make the code feel more robust, I replaced it with a submit button for the event reflection textareas.

The dates have been implemented underneath the event name rather than to the side of it. This is to reduce the clutter in design and the improve usability of the program. Implementing the event name, date and reflection inline is easier for users to read and follow the timeline especially when more events are added to it.

The arrows pointing to the event names from the main arrow have been removed. This is for a similar reason to the date formatting change. Adding arrows for each event would unnecessarily clutter the design of the timeline page and therefore; to implement a simple, minimal design and to improve the usability of the program, I have decided to remove it.

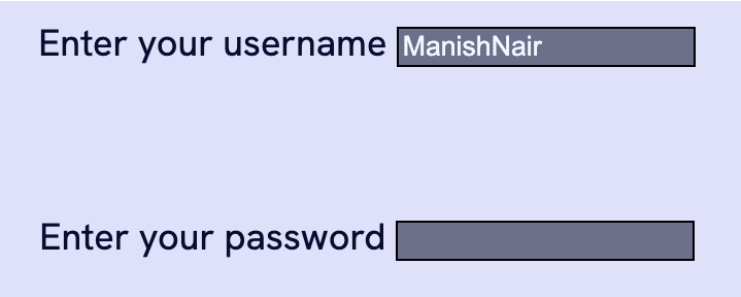
Summary of the whole project as a prototype at this stage

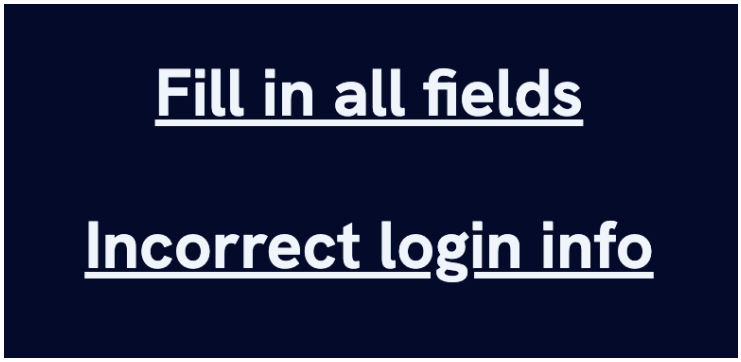
The program as a whole contains a database to store the data; an account creation system with input validation algorithms; a calendar home page which generates calendar data for 2026 dynamically; an input form which allows event data to be viewed inserted into the database based on the date information of the day cell; a skills page which allows the user to write how they demonstrated the skills they tagged the event with; a navigation bar which can be used to traverse between pages easily and to logout from the website; and a timeline page, where the user can write event reflections on all of the events they have stored in the database and they can save the page as a PDF for their reference and future use. The program is now fully complete as all of the required features and one of the desirable features have been implemented. I will now move on to the final testing of the project using the test data planned in the design stage.

Stage 8: Final Testing

Now I will begin with the final testing of the program according to the checklist. I have included full testing videos in the supporting materials folder and will only include examples of tests which match the checklist here.

Firstly, I will test the login page for error message output. I will begin by including a correct username field with an empty password field. The expected outputs are the 'Incorrect login info' and 'Fill in all fields' error messages.

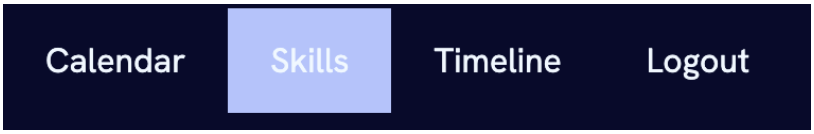




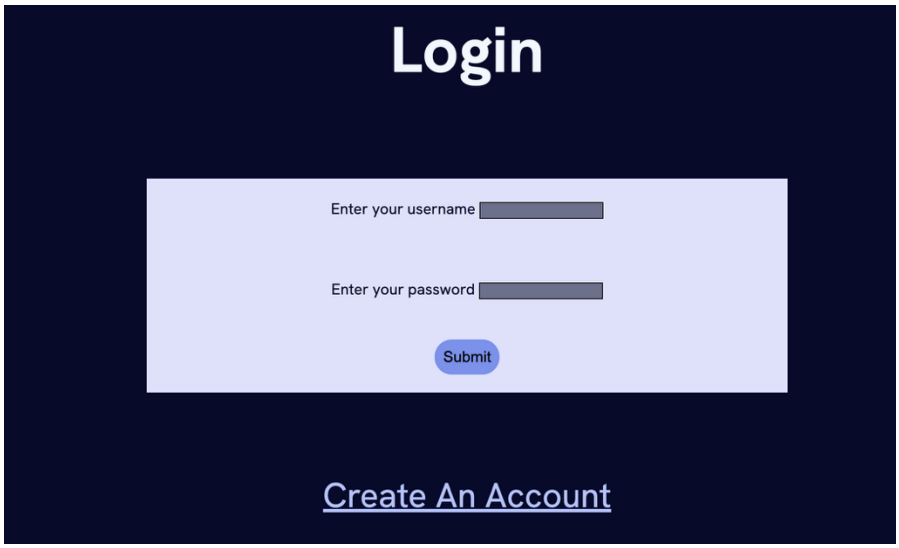
The login page successfully output the correct error message. Now I will test if it outputs a success message if the credentials are correct.

I was redirected to the calendar (home) page, and the hello message was output. The login page passed the tests successfully and the hello message was also correctly output for the logged in username.

Now I will test the navigation bar. I will first test if it successfully redirects me to the skills page.

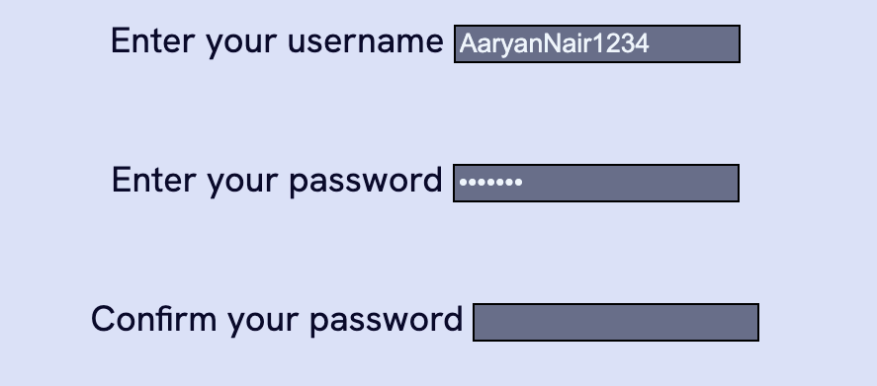


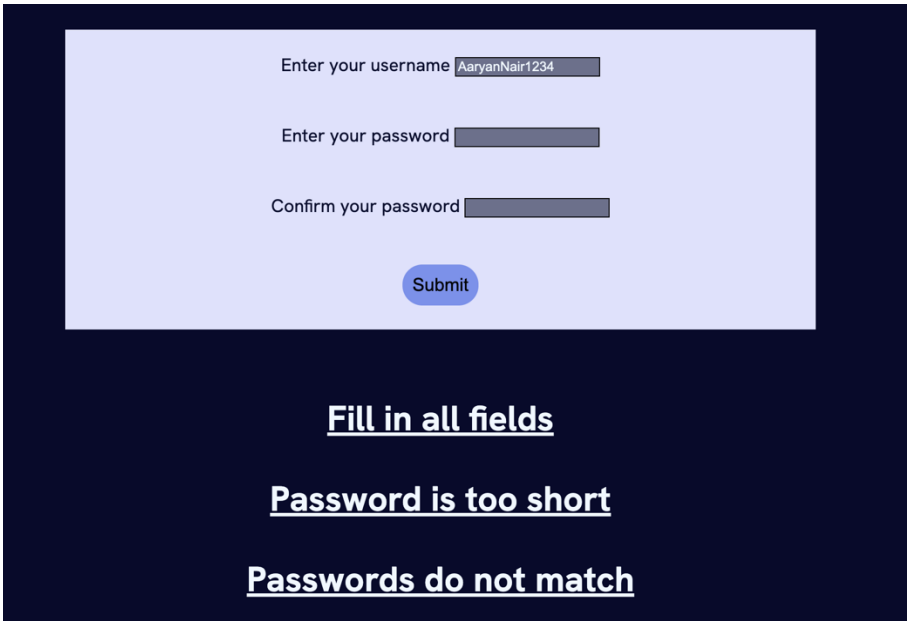
I was successfully redirected to the skills page. I will now test if the logout button redirects me to the login page.



This is the correct output and now I have fully tested the navigation bar.

I will now test the signup system with a unique username, but short password and empty confirm password field. I am expecting the username to be retained in the username field and for the ‘Password is too short’, ‘Passwords do not match’ and ‘Fill in all fields’ messages to be output.

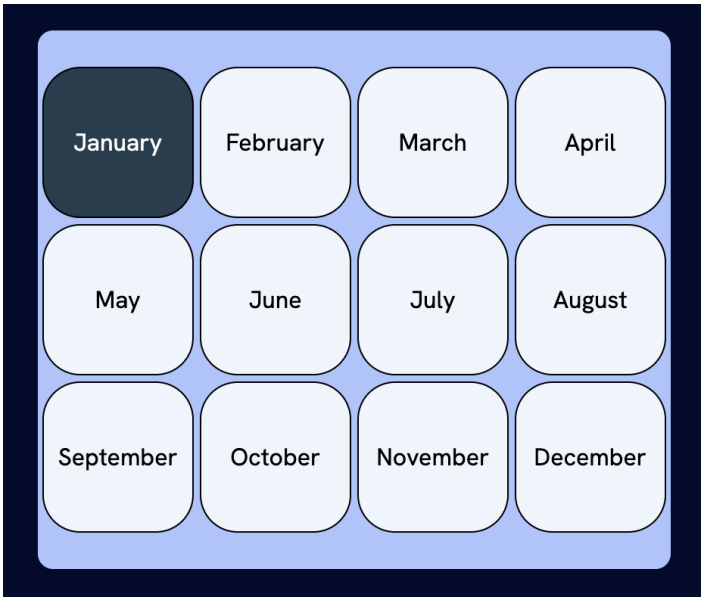


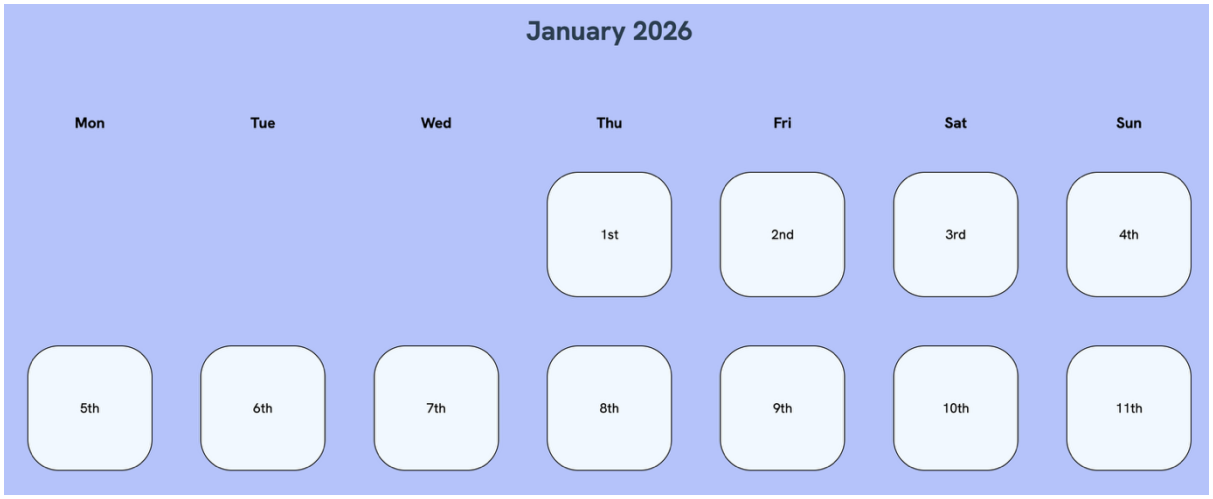


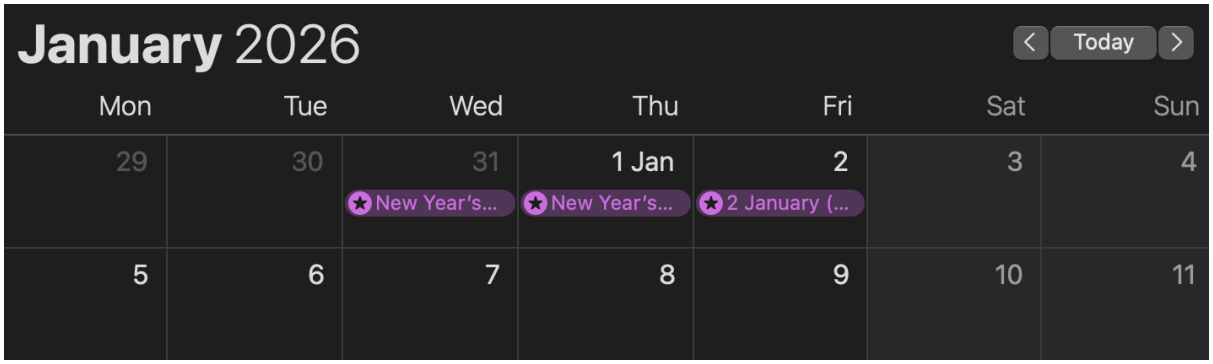
The program produced the correct output and therefore, the signup system has been tested for usability features and error checking. I will now check if inputting valid credentials outputs the 'Account created successfully' success message.

The success message was correctly output which shows that the signup page has been successfully implemented with error/success messages and usability features.

I will now log back in and test the calendar page. I will select the January month in the calendar selector and expect the month data for January to be successfully output.



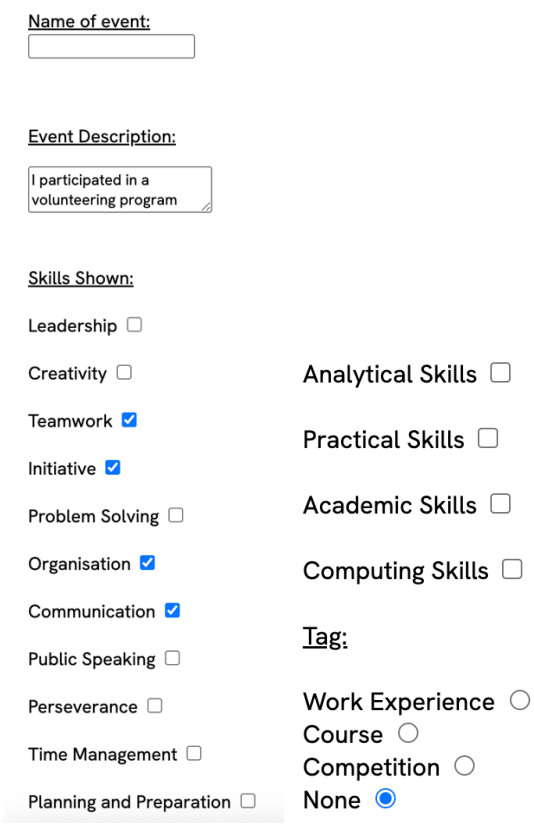




The data is correctly output and in the same layout as that of the calendar data. This shows that the calendar data can be dynamically and correctly output using the month selector.

I will now test the input form by adding an event for the 3rd of January before I test for the correct colour coding of the day cells.

Firstly, I will test the input form by adding event data with no event name which should output an error message.

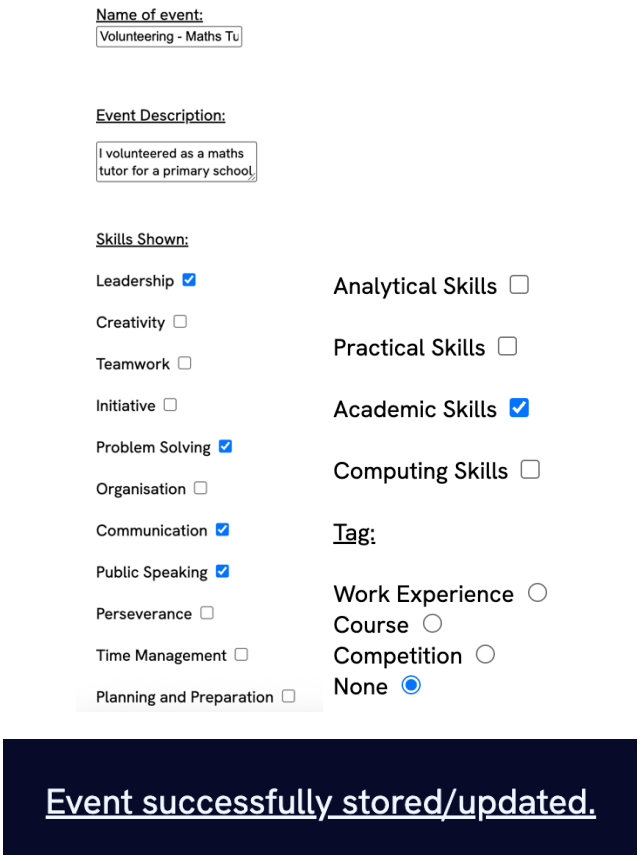


The error message was successfully output with the correct colour and styling. However, to maintain a consistent design across all pages I am choosing to change the colour of the error messages on the accounts pages to red to match this error message. This would be helpful for usability as it maintains consistency across the design.

```
.form-error{
  color: red;
  font-size: 35px;
}
```

Implementing the changes produces this output.

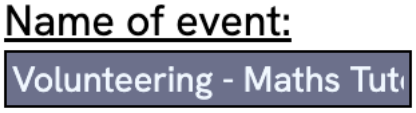
Now, there is a consistent design across all pages which improves the usability of the program and makes it more robust. I will now test the input form by inputting valid event details and the expected output is a success message.



The success message was correctly output, which shows that the input form has the correct error/success checking and message outputs.

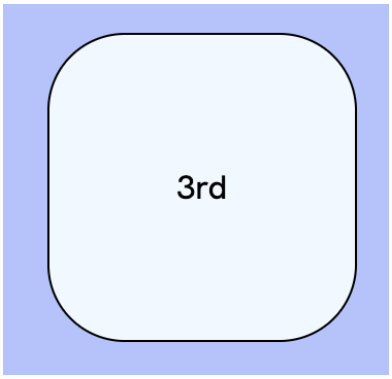
However, event name input has some errors. The font in the field is not correct and furthermore, it hasn't been styled to match the login fields as planned in the design. Therefore, I will change this to maintain a consistent design, match user expectations and to improve usability.

```
#eventname{
    font-family: 'Hanken Grotesk', sans-serif;
    background-color : #6c718c;
    color: aliceblue;
    border: 1px solid black;
    box-shadow: none;
}
```



This is the correct output, and the design has now been fixed to match user expectations.

I will now test the input form data retention and colour coding by tagging this event with the ‘Work Experience’ tag which should colour it as red as shown in the legend.



Name of event:
Volunteering - Maths Tut

Event Description:

I volunteered as a maths tutor for a primary school

Skills Shown:

Leadership ☑
Creativity ☐
Teamwork ☐
Initiative ☐
Problem Solving ☑
Organisation ☐
Communication ☑
Public Speaking ☑
Perseverance ☐
Time Management ☐
Planning and Preparation ☐

Analytical Skills ☐
Practical Skills ☐
Academic Skills ☑
Computing Skills ☐

Tag:

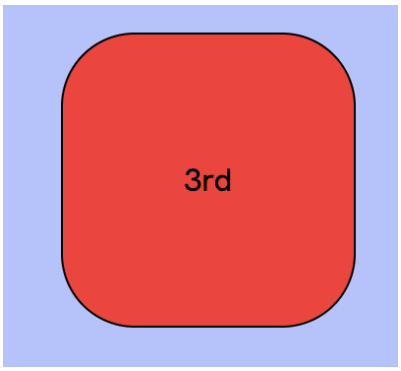
Work Experience ○
Course ○
Competition ○
None ◉

The data is correctly retained in the input form. Now I will add a tag to implement colour coding.

Tag:

Work Experience ◉
Course ○
Competition ○
None ○

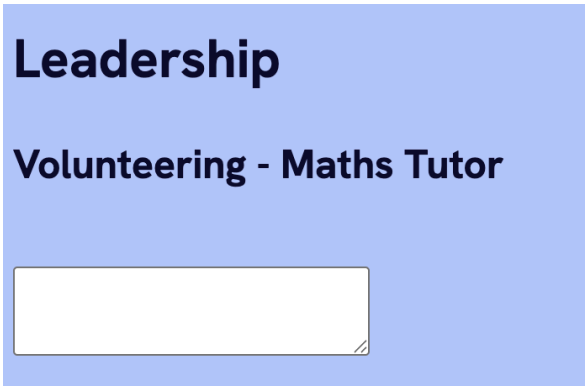
Clicking on the month selector for January produced this output.



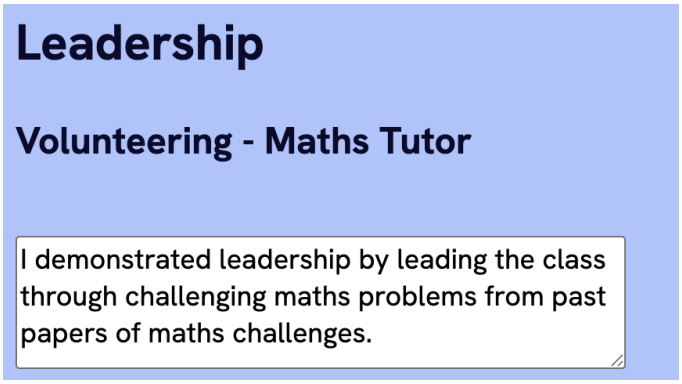
The button has been colour coded with red. I will now check this with the legend to see if this matches the colour stated for the 'Work Experience' tag.

The legend correctly shows that the colour coding matches the event tag which proves that the colour coding feature works, and that the legend displays the correct colour coding values associated with the tags.

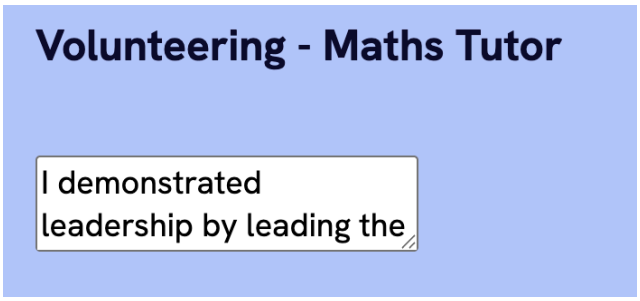
I will now test the skills page to see if this event appears under the correct skills (e.g. 'Leadership').



The event correctly appears underneath the leadership title with an empty description value as there is no stored value. I will now add a description to see if the success message is output and to see if the value is retained in the textarea. I will also test to see if the textareas are resizable which is a helpful usability feature.



This was the input data, the textarea was resizable which proves that the page has helpful usability features. Submitting the page gave me this output.



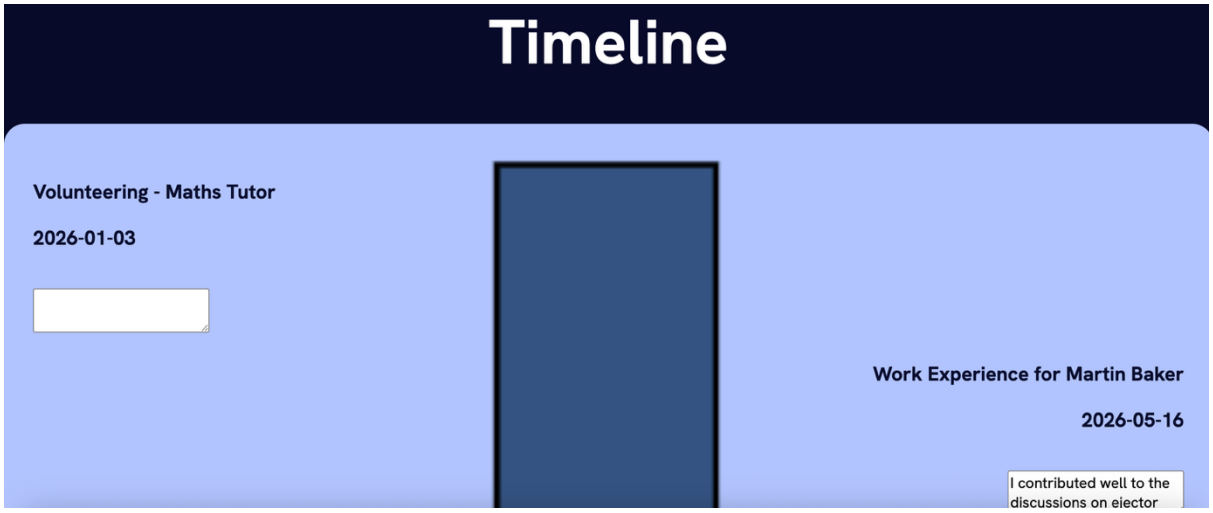
The correct success message was output, and the description value was retained (although the textarea reset to its initial size). This proves that the skills page can retain values correctly, however, there is an issue with the success message.

The success message is not underlined, which does not match the other success messages in the program and therefore, I will change this to improve the usability of the program.

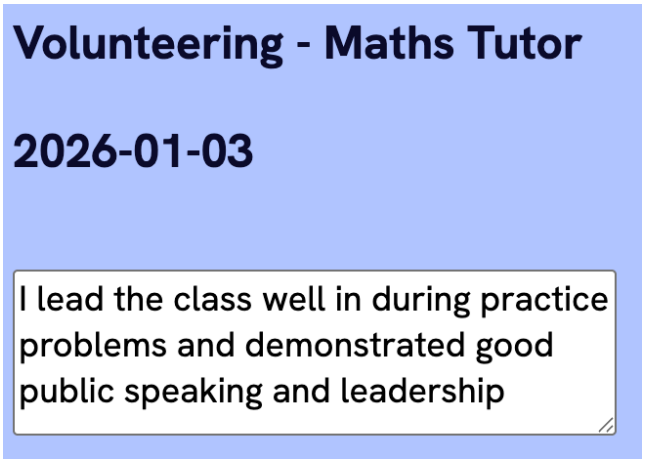
```
echo '<p id="form-success"><u>Skills Descriptions successfully stored/updated.</u></p>';
```

This proves that the skills page outputs the correct success message, styled in a way to match the other pages, making the program more robust and usable for the stakeholders. The skills page has now been fully tested so now I will move on to testing the timeline page.

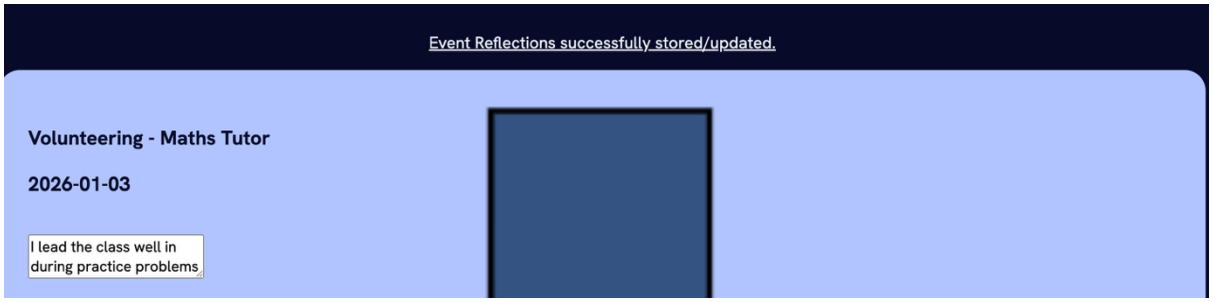
I will first test the timeline page by seeing if the volunteering event is output chronologically with the correct date and empty textarea.



This output proves that the timeline correctly outputs events chronologically and correctly (event name and date titles and reflection textarea with the correct alternating styling). Now I will test if inputting a reflection value outputs the correct success message (styled correctly) and retains the value in the textarea.



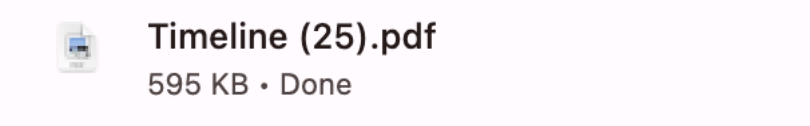
This proves that the textareas are resizable for better usability. Submitting this value outputs this result.



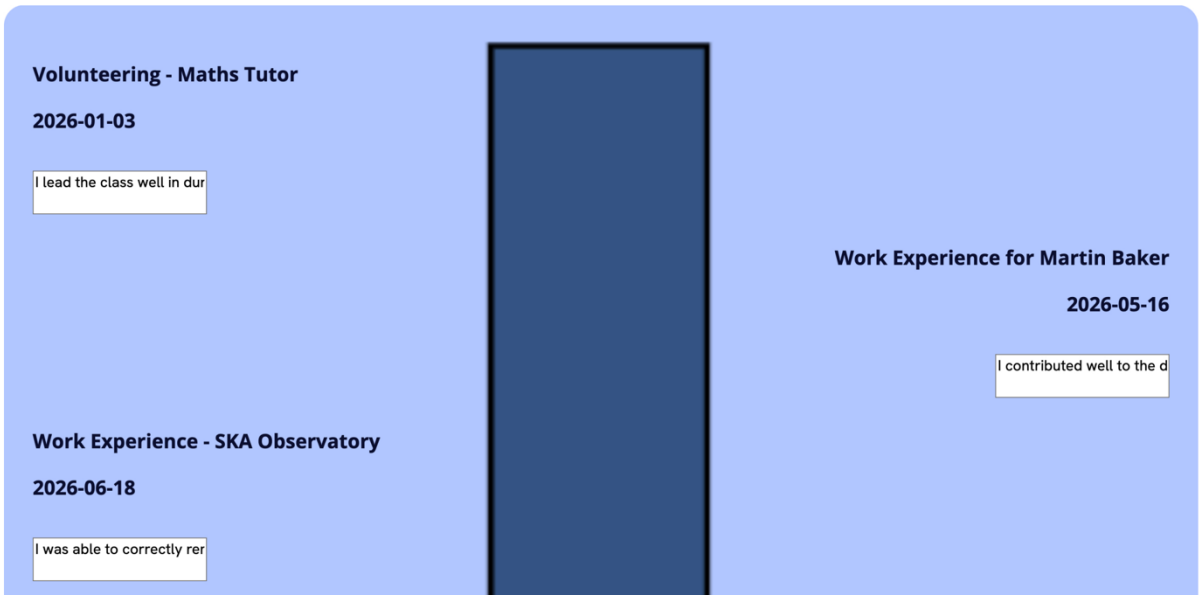
The success message is correctly output, and the value is retained in the timeline page. This shows that the reflection value retention works correctly and that the timeline outputs success messages correctly after submission.

Finally, I will test if the save as pdf feature works correctly.

Pressing the ‘save as pdf’ button led to this output.



The file was successfully downloaded. Now I will check if the CSS was correctly implemented in the saved pdf.



This is part of the saved PDF which correctly shows the events chronologically, with some of the event reflections with the CSS implemented. This shows that the save as PDF feature works successfully.

I have tested the program fully according to the testing checklist and it has passed all of the tests. Any issues with design and usability have been fixed and implemented and the program is now error-free.

Hardware and Software

This program has been tested on devices with different hardware as specified in the hardware specifications. The program worked as intended on each of the different machines as long as an active internet connection was present and the computer was powerful enough. Using different OS (supported by XAMPP) also didn’t produce any issues when the database was being created or queried which is good as my users are using devices with varying OS (Mac, Windows and Linux). Furthermore, the code ran well regardless of the browser used, which is another important difference between my stakeholders. I tested the code for use on the main popular browsers used and there were no issues with dynamic generation of data or styling of the program.

Checklist

Aspect to test	Does it work?
The login page outputs correct error/success messages when logging in	Yes
The signup page outputs correct error/success messages when creating an account	Yes
The username is retained in the username field if it is valid but if there are other errors	Yes
Program outputs a 'hello' message with the logged in user's username	Yes
Correct calendar data is generated for each month (according to the 2026 calendar)	Yes
Events tagged have the correct colour coding applied to them	Yes
The legend displays the correct colour coding value for each tag	Yes
If data is input into the inputform with an empty event name, error message is output	Yes
If data is input into the inputform correctly, a success message is output	Yes
Data is retained in the fields for previously stored data	Yes
Skills page retains the textbox values and displays them correctly	Yes
Skills page outputs a success message when the skills descriptions are updated/inserted	Yes
Timeline page displays events chronologically with the retained event reflection values	Yes
Timeline page outputs a success message when the event reflections are inserted/updated	Yes
The save as pdf feature correctly saves the timeline div as pdf with styling	Yes
The navigation bar correctly redirects the user to the pages selected	Yes

The checklist has been fully tested for and all of the points on the checklist have been met as shown in the testing in this stage. I can now confidently state that the code is error free, and that I can now send it to the stakeholders for them to test it and for me to gather feedback which I can use to evaluate the program.

Stage 9: Stakeholder Testing

For the stakeholder testing, I sent the stakeholders a copy of the code and the following questionnaire to ask for their feedback on the project.

These were the questions:

1. **How simple was the account creation process?**
2. **How intuitive was the process of adding events to the calendar?**

3. **Are you satisfied with the current colour coding options? If not, what would you like to change/add?**
4. **Are the skills description and event reflection tools helpful? Do they meet your expectations?**
5. **Are the example skills descriptions helpful in guiding you on what to write?**
6. **Is the 'save as pdf' feature on the timeline page useful and would you use it in the future?**
7. **How responsive was the software for you? Were you satisfied with the messages that were output to the user on signup, login and input?**
8. **In a further iteration of the project, what changes or additional features would you like to see?**

I received the following responses:

1. **How simple was the account creation process?**

"The account creation process was simple and easy to use. I really liked the password length validation feature in particular, as it made the process feel like most other websites."

"The process was very simple. It couldn't have been simpler, in my opinion."

"I found the process simple, straightforward and clear"

"It was pretty simple; however, I kept having to retype my password multiple times because I kept forgetting it and it wasn't visible."

2. **How intuitive was the process of adding events to the calendar?**

"It was pretty intuitive; it was simple and effective."

"Adding events to the calendar was very intuitive and pretty self-explanatory. I didn't have any issues with it."

"It was pretty simple."

"It was quite intuitive, it just felt like I was scrolling a lot because of the gaps between sections."

3. **Are you satisfied with the current colour coding options? If not, what would you like to change/add?**

“I am satisfied with the current colour coding features, but I would like the ability to add more user defined colour coding tags.”

“Happy with the current features. However, implementing different colours for in person vs virtual work experiences could be good. Separating the competition and course tags by the type/subject (e.g. maths, physics) would also be good”

“I am satisfied overall, but I think specifying the type of course (such as school course) and adding colour coding options for general school timetable subjects would be helpful.”

“I am partially satisfied with the current options. I think letting the users choose which colours are associated with each tag would be good. Also, adding custom tags specified by the user would also be good.”

4. **Are the skills description and event reflection tools helpful? Do they meet your expectations?**

“Tools are useful overall; however, adding brief information paragraphs to tell the user to put reflections on timeline page would be good as it is a bit unclear currently. They met my expectations.”

“The layout for the skills page is perfect. I really like the way it’s presented (the alternating aspect), however, formatting for the example event descriptions can be better.”

“I am satisfied with the current tools. The features are pretty straightforward to use, but others may need slightly more guidance on what to write on the timeline page.”

“These features meet my expectations. However, some indications that the boxes are resizable would be good and a tutorial on how to resize them could be good.”

5. **Are the example skills descriptions helpful in guiding you on what to write?**

“The example descriptions are good, but they could be formatted slightly better (using bullet points etc).”

“I like how each skill has a minimum of three bullet points. I find them quite useful in guiding me on how to write. However, some could be longer and formatted better, and a written example would also be nice.”

“The formatting could be improved but the feature is helpful.”

“The text colour could be slightly darker. The formatting causes a lot of movement for the eyes, so it might be good to put the descriptions and textboxes all on one side.”

6. **Is the ‘save as pdf’ feature on the timeline page useful and would you use it in the future?**

“I thought that the save to pdf feature was useful as I could print them out and have a hard copy with me.”

“When using my laptop and when I want to save it somewhere it’s nice to have but not essential.”

“It is a good feature; however, some text does get cut off for longer descriptions unless the textareas are resized before saving.”

“It’s useful but better the more events you have stored.”

7. **How responsive was the software for you? Were you satisfied with the messages that were output to the user on signup, login and input?**

“I was satisfied with the output messages. It was great that it let me know that I had stored an event successfully.”

“Very responsive. No issues with it. The hello message is a good touch.”

“Pretty responsive. They were all fine.”

“I am satisfied. The software is very responsive.”

8. **In a further iteration of the project, what changes or additional features would you like to see?**

“Add a click to reveal password feature. Also, in each month button, show how many of each event you have stored for that month. So, a mini blue circle with a 5 next to it could represent 5 courses being stored for that month. Finally, add a feature that allows for more than one event to be stored per day.”

“A forgot password feature would be good. Customisable themes would also be a really nice feature to have. Have various types of screens such as purple and pink or green and turquoise. Personally, I think that the legend should be moved to the left of the month selector. Also, add options to input events for past and future days. You could also include advice on how to find work experience e.g. SpringPod or Sutton Trust.””

“Adding a time feature per day would be good for making revision timetables. Furthermore, you should be able to add multiple events per day.”

“A reveal password feature would be good. I think allowing the user to create their own tags and to choose the colour they use for colour coding would be really helpful in helping make plans suited to them.”

Stage 9 Review

All of the stakeholders found the program simple and intuitive to use and were mostly satisfied with the current features. In particular, they liked how simple it was to create an account and add events to the calendar and were happy with the skills descriptions and event reflections features.

The stakeholders did highlight areas of improvement with the example descriptions and the colour coding which can be implemented in future iterations of the project. However, they did state that the current implementations of those features were good and satisfactory. They suggested many new features which could be added such as user-defined colour coding, a ‘click to reveal password’ feature, a ‘forgot password’ feature and allowing for multiple events to be input per day.

Evaluation

Success Criteria Review

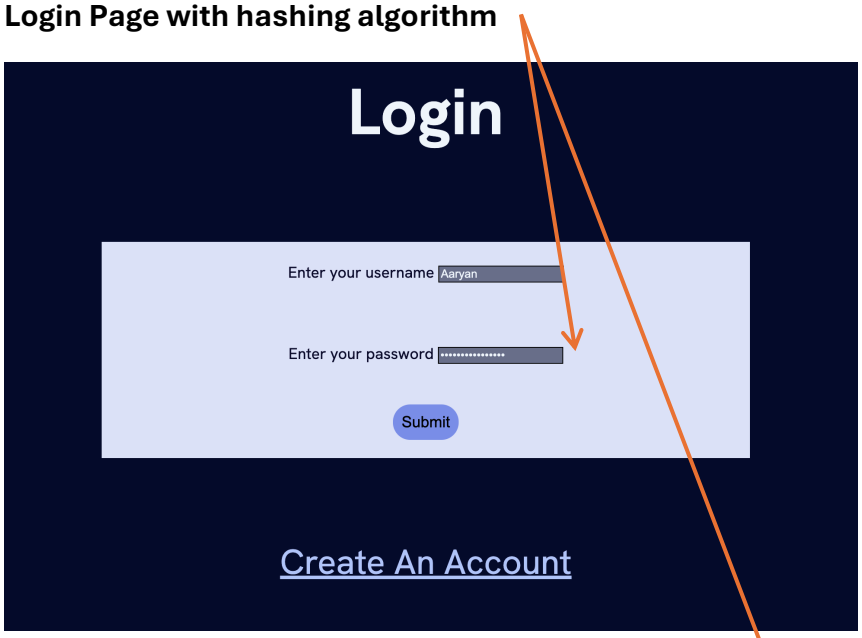
Success Criteria

Essential Features	*Met?*
Login Page with hashing algorithm	Yes
Signup Page with hashing algorithm	Yes
Calendar System for past, present and future events	Partially
Colour coding of events	Partially
Skills table that links to calendar	Yes
Skills description feature	Yes
Example description of skills and extracurriculars	Yes
Timeline page	Yes
Reflection for each event on the timeline	Yes
Navigation bar	Yes

Desirable Features	*Met?*
‘Save as PDF’ feature on the timeline page	Yes
Notifications system	No
Light/Dark mode toggling	No
Skills Generator	No

Evidence

Login Page with hashing algorithm

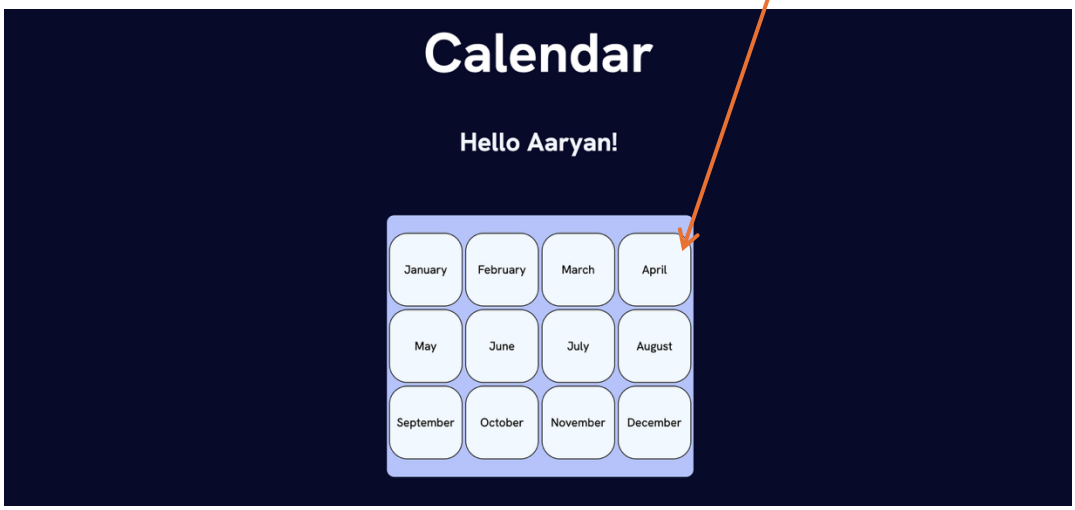


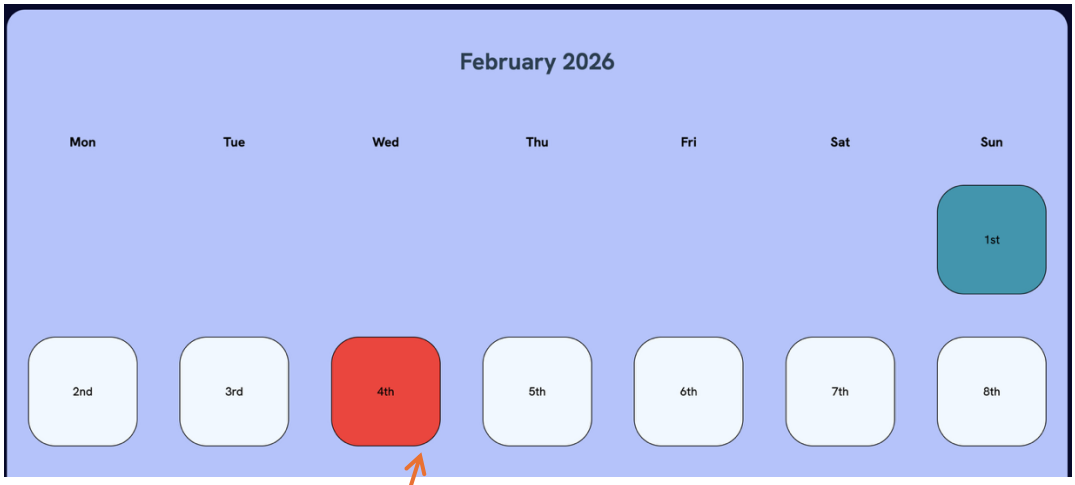
Signup Page with hashing algorithm

1	AaryanNair	$2y$12$lhzmBGbezDys/fHNcmw0n./csMdTzwH9F7zuA1TUuKa...
2	client_1	$2y$12$j7o9Km.V.7/uXwC.MwFs0.92QiCu0hUL7vD.D5lfYko...
3	client_2	$2y$12$joxnGECy7MbDmMWIUF4Fnel7lisTfisCGul4PNZUVXx...

This is a screenshot of the data in the SQL database which shows that the passwords are being stored after being hashed by the hashing algorithm.

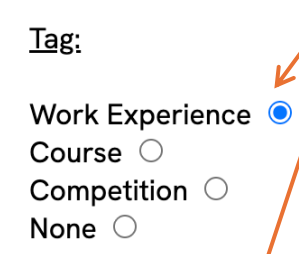
Calendar System for past, present and future events





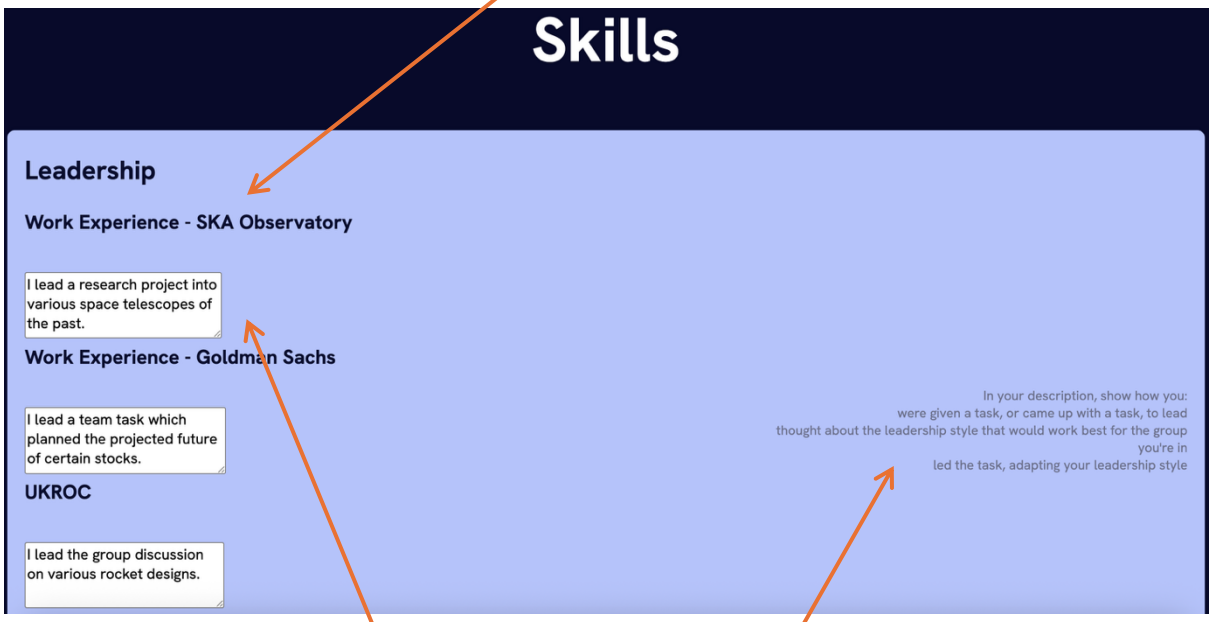
The calendar system currently only contains months for 2026 which allows for limited past, present and future event logging.

Colour coding of events



Events with a specific tag are colour coded according to the tag specified in the input form and the colour specified in the legend section of the calendar page.

Skills table that links to calendar



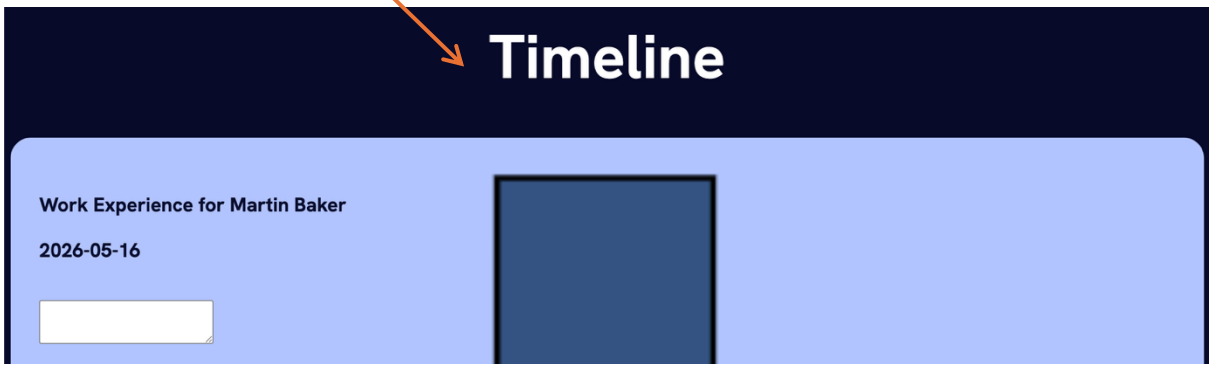
Skills description feature

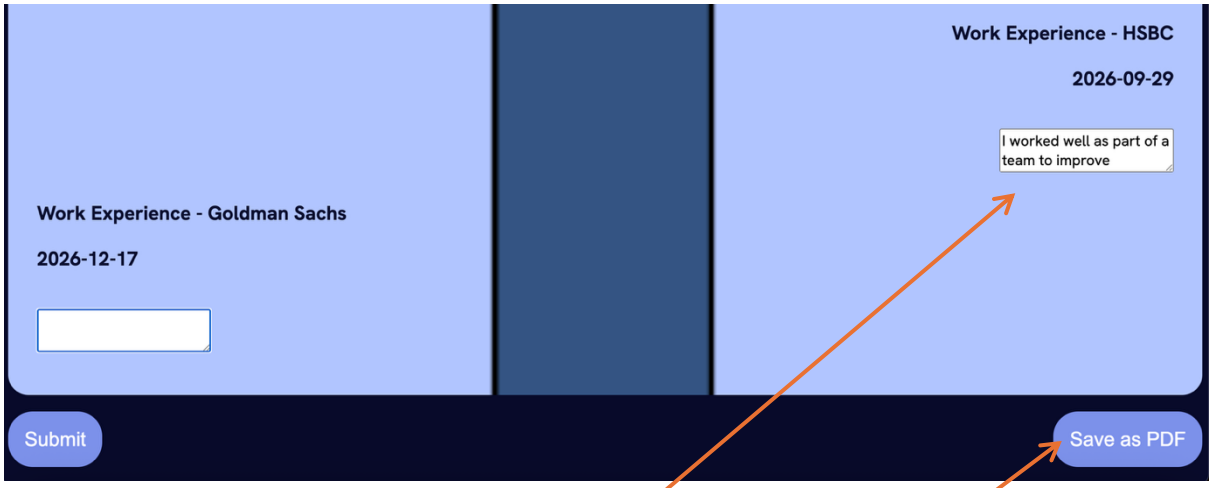
Example description of skills and extracurriculars

The events stored in the calendar are output on this page under the skills they have been tagged with. Each event has a text box underneath allowing for skills descriptions to be input in those text boxes. On the opposite side to the event names and skills descriptions is the example description.

Timeline page

Each event stored in the database is retrieved and output chronologically along the page.



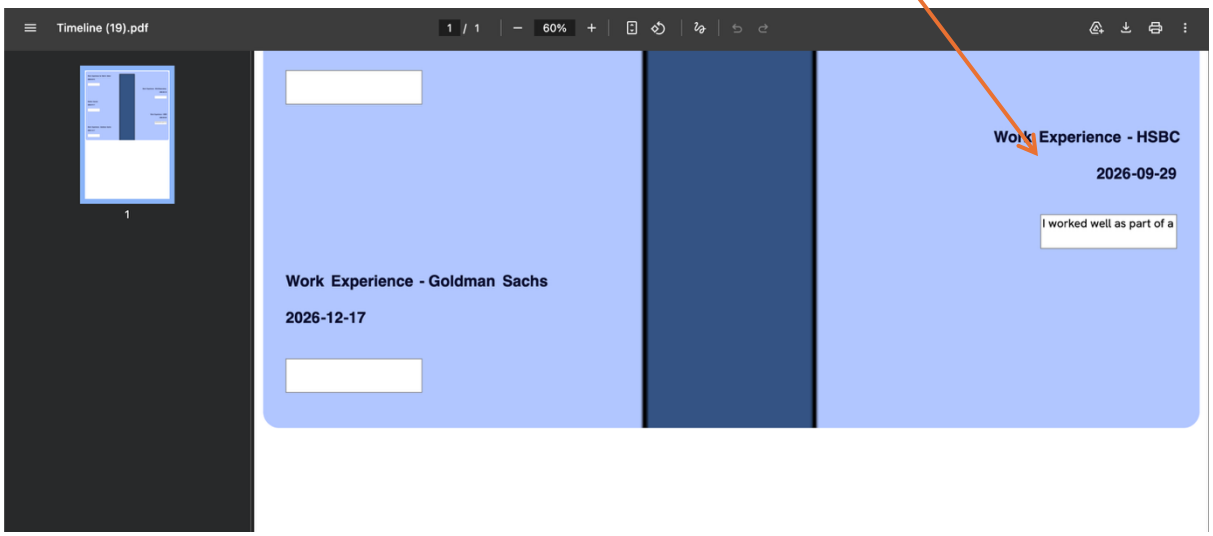


Reflection for each event on the timeline

A reflections textbox is output for every event in the timeline. The responses can be updated/saved by clicking on the submit button which stores the field values in the database.

'Save as PDF' feature on the timeline page

Once the button is pressed, a pdf file named 'Timeline.pdf' is downloaded which can then be opened and it shows a copy of the entire timeline page.



Navigation bar

The navigation bar is present at the top of every page. It allows the user to go to a specified page by clicking on it and the button changes to a different colour when it is hovered over. It also allows the user to logout which redirects them to the login page.

How could unmet criteria be addressed in future iterations?

The main unmet features include:

1. Calendar System for past and future events
2. Further developing the colour coding feature:
    A) User-defined tags
    B) User-defined colours for tags
    C) More specific tag options (e.g. virtual vs in person work experience)
3. Adding a notifications system to the calendar
4. Adding more customisation options (switchable themes, light/dark mode)
5. Adding the skills generator feature to the input form

These criteria couldn't be addressed with the given time constraints due to the complexity of the tasks. I would need to do more research and develop my skills in web-development languages to create these features. However, I can plan how these features can be addressed in future iterations.

1. Adding Past and Future sections to the Calendar

The highest priority issue with a partially met feature in this program is that the calendar page only allows for events to be input for 2026. Currently this is because I have one variable known as 'calendarData' (a nested dictionary) which is storing the data for the 2026 calendar.

However, this issue is an easy fix. I recognised this issue during development and therefore have passed a parameter known as year to the replace_month function and set it to 2026 within the function's parameter declaration. This pre-set value can be removed, and a year selector could be added to the calendar page to solve this issue.

The user would first select the year, which then redirects to the month selector that I have currently implemented. This month selector will have the previously selected year passed as a parameter to the replace_month functions in the buttons. Furthermore, another layer of nesting would need to be added to the 'calendarData' dictionary when I add information for past and future years. For example, the dictionary storing the year data for 2026 could be associated with the year (2026) as its key value, forming a key-value pair in a dictionary. Therefore, when the replace_month function accesses date information from 'calendarData', it would access it using the year as the key and then access the month and day information from the nested dictionary. For example, 'calendarData[month]' would become 'calendarData[year][month]'. This would easily solve the issue of past and future years not being accessible in the calendar.

2. Developing Colour Coding

The colour coding feature was the second partially met criteria of the program. Most of the main issues with the program highlighted by the stakeholders after their tests, were regarding the colour coding feature.

A) User-defined tags

User-defined tags were the most requested feature by stakeholders after testing the program. The stakeholders were currently limited to 'Work Experience', 'Course', and 'Competition' which some stakeholders thought were good, but many said that they felt limited.

One way of implementing user defined tags would be to change the 'Legend' section of the calendar page. The h1 elements currently used for the event names could be replaced with input fields, and a form tag could be added within the event div. Tag names could be updated from the input fields or to add a new tag, a button will need to be added which creates a new colour picker and input field.

The form tag could then GET or POST (user defined tags are not as sensitive as login credentials or event details so GET may work however I believe that POST should be used anyways) would link to a PHP file in which the contents of the input fields would be inserted/updated in the database. The Input Form tags will also need to be changed so that a radio button is created for each tag stored in the database and that the label of the button corresponds to the tag names input in the Legend.

Another important thing to note would be that a separate table will likely need to be created to store the input tag names. Therefore, the event_tag attribute from the EVENTS table will need to be replaced with a foreign key of the legend_id from a newly created LEGENDS table which will contain the attributes legend_id (primary key) and tag_name. The attribute legend_id will be dependent on the position of the tag in the

Legend and when a new input field for a tag is created, a new legend_id is auto incremented.

B) User-defined colours for tags

User-defined colours were another feature highly requested by the users. It allows another means of customisation which personalises the program to the user. This feature would require a similar solution as with the user-defined tags.

Currently in the program, the colours are shown as circular div elements. These div elements would need to be replaced by an input field with the 'type' attribute set to 'color' which I found in my research (linked in bibliography). This would allow the user to select a colour from a popup colour picker interface in which they can input the RGB values, the Hex code and the HSL values whilst also being able to manually select a colour using the slider. The colour of the selector will automatically change to whatever colour was selected; therefore, I will not need to style the button individually.

In my original implementation of the legend, the colours of the colour coding are pre-set CSS classes. To implement user defined colour coding however, I will have to change the class names to 'tag_(legend_id)' where the second half of the class name would be dependent on the legend_id which is the tag's position in the legend.

Furthermore, when a colour is selected and the submit button is entered, the value of its colour will be input to the database in a new field called 'colour_id' which will store the hexadecimal value of the colour associated with the legend_id.

Upon login, the legend div will call a JS/PHP function which will output the colour input fields with the value attribute of colour field set to the value stored in the database. Furthermore, it will set the days tagged with 'tag_(legend_id)' with the corresponding colour value, allowing for user defined colour coding.

C) More specific tag options

This feature is very easily implemented and is solved by implementing user-defined tags. User defined tags would allow the user to create tags as specific as they choose which solves this issue.

However, an alternative implementation that could be used would be to change the current tag options and add a few more tag options. Following the stakeholder feedback: 'Work Experience' could be renamed to 'Virtual Work Experience' and therefore another tag could be added for 'In-Person Work Experience'; 'Course' could be split into 'School Coursework', 'Super-curricular In-Person Course' and 'Super-curricular Virtual Course'; and 'Competition' could be split into 'Olympiad' and 'Sporting Tournament' for example.

This would require creating more coloured div and h1 elements and adding them in the Legend section which would be simple to do as my code allows for modularity.

Furthermore, more CSS classes will need to be created with unique colours to allow for colour coding of the events. Finally, these options will need to be added to the input form for selection.

3. Notifications system

A notifications system is one of the desirable features which was omitted due to time constraints. This can be solved by calling a notifications function to the calendar page. This function can be defined in another file called 'notifications.php'.

Upon login, using the built-in PHP date-time functions, the current date can be taken and converted to the format supported by MySQL. The date will then be stored as a local variable 'current_date'.

An if statement will check if fetching the event_name from the database where the 'event_date' = 'current_date' returns a value (for the logged in user_id). If so, the function will output a div with the class 'notification-box' with the text of '{event_name} is today!'.

If the statement returns null, the program will progress to the else statement. Here, the database is queried to return all the event_name and event_date values in the table with a date that is later than the current date (for the logged in user_id). The closest date and event name are stored as variables 'next_date' and 'next_event_name'. The 'next_date' and 'current_date' variables are then separated into year, month and date variables and the 'current_date' is subtracted from the 'next_date' and the result is stored as a variable 'difference_date'. Then a message is output that says '{event_name} is in {difference_date}!'.

The most challenging part is converting the dates into numerical values which can be subtracted and then storing them as variables however, it shouldn't be difficult to implement in a further iteration.

4. Customisation options

Further customisation options were a feature requested by stakeholders in their stakeholder testing. The stakeholders suggested pre-set themes that could be switched which can be simple to implement.

Another table known as THEMES will need to be added to the database with attributes 'theme_id' and 'theme_name'. Furthermore, a themes page will need to be created and added to the navigation bar. The themes page will have a radio selector and include a few example divs and text which would change colour based on the theme selected.

The radio buttons will be based on a few preset themes, and the selected theme button would be checked. Once a theme is selected and the submit button is pressed, the theme_id which is the foreign key in the USERS table will be updated to the theme_id corresponding to the selected theme.

Various stylesheets could be stored in a style folder which can be based on the original stylesheets renamed with the theme id at the end of the stylesheet name. To implement the stylesheets on each page, the 'href' in the link tag could contain a PHP tag at the end of the stylesheet name but before the '.css' which echoes the theme_id selected for the current logged in user_id.

5. Skills Generator

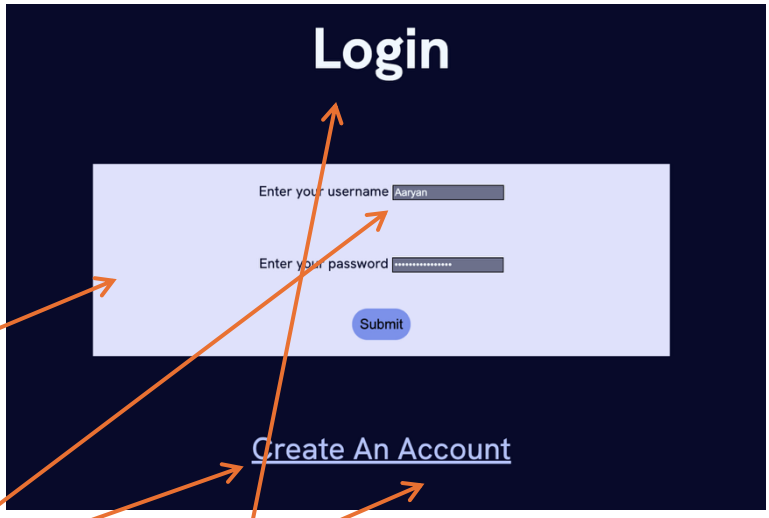
The skills generator was another desirable feature that was unmet due to time and skill constraints. This feature will be the most difficult feature to implement in a further iteration.

The skills generator could be added to the input form as an option underneath the event description textarea. A button could be pressed to use the skills generator which would take the input value of the event description textarea and send it to a generative AI model. This model could be based on an API from WordPress' AI chatbot feature (linked in bibliography), or it could use another model. The AI would then generate a word cloud image which would then be displayed underneath the textarea which could help the user choose which skills to select.

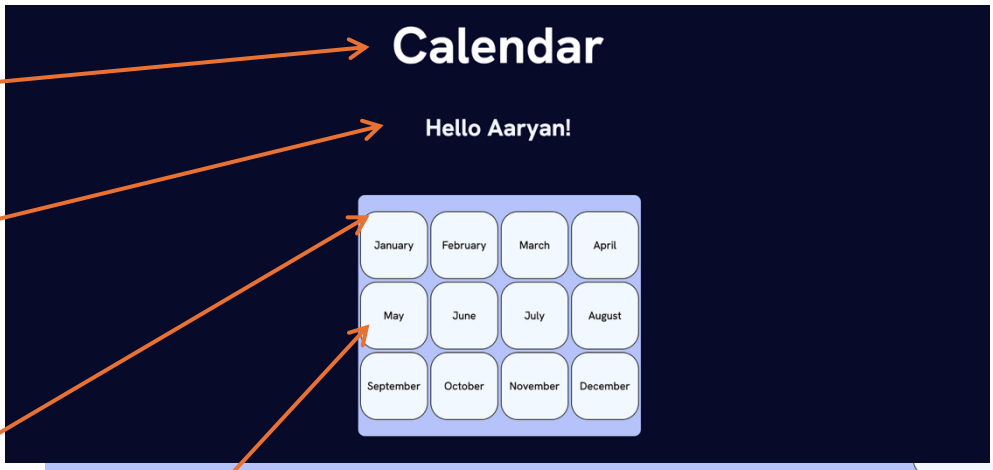
This feature will be the most difficult to implement, but upon further research, could be an incredibly useful tool for the users.

Usability Features

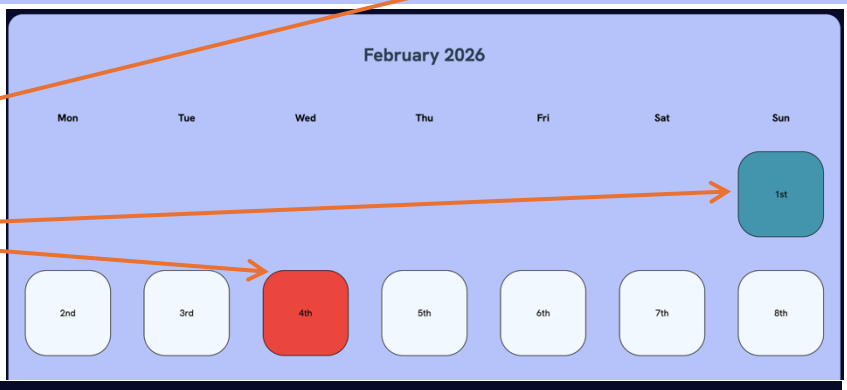
Evidence

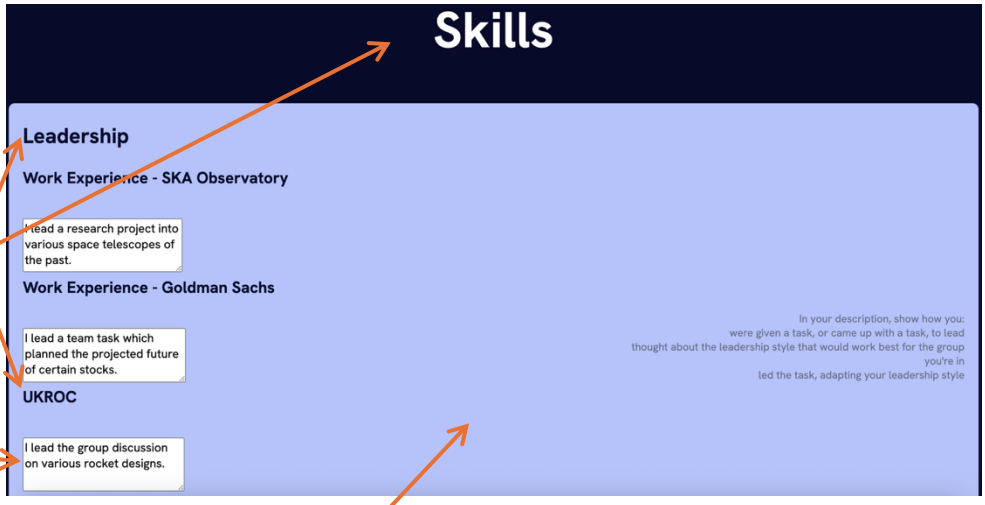


Important areas of the page are surrounded by a lighter colour div to contrast them and make them easier to notice and use. Furthermore, username and password input fields have a dark background colour with white text allowing for further contrast and readability of inputs. The title and option to create an account are in a very large font to clearly inform the user of the purpose of the webpage and the underlined hyperlink allows them to know where to click if they do not have an account.

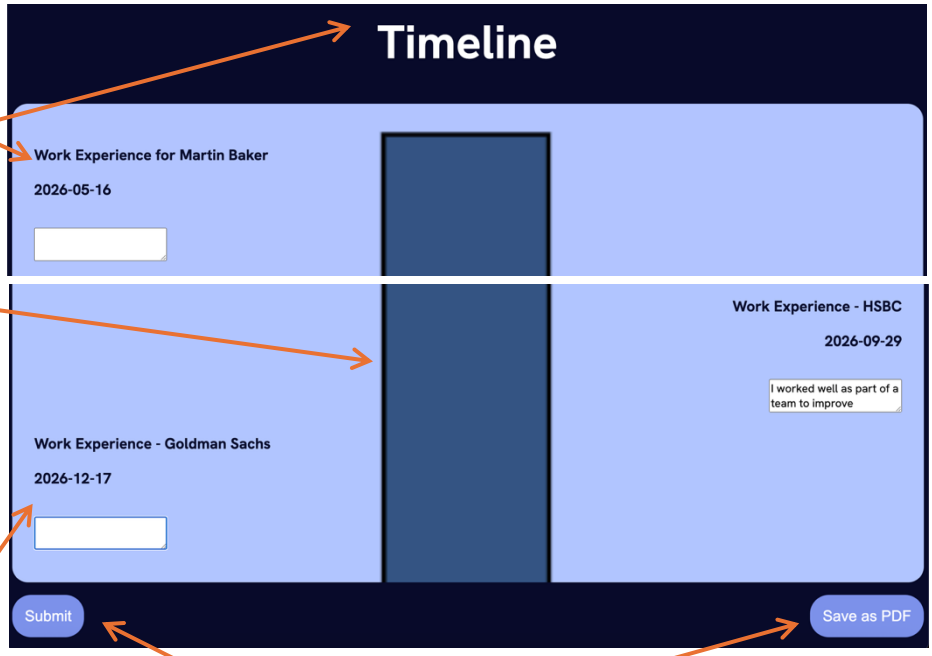


On the calendar page, the title is in a very large font to immediately inform the user of the page's purpose. The hello message is a personalisation touch to make the system feel more responsive for each user. The month selector has contrasting colours to the background (this establishes the common background div colour for the skills and timeline page to follow). The buttons on both the month selector and calendar are large and well labelled and change colours when they are hovered over, and the colour coding colours contrast well with the div colour. The legend has a different background colour to the rest of the calendar to show that it serves the purpose of acting as a reference, rather than being an interactive element.





The skills page has a large font size for the title of the page, the skill headings and the event names. Furthermore, the larger input text areas are coloured white compared to the darker coloured username and password input areas which can help visually differentiate the types of input fields from the user. The background div colour is also the same colour as the calendar page, to highlight that the area is interactable and to maintain consistency of design between pages.



The timeline page's title and subheadings are all large in font size which allows for good readability for the user. Furthermore, the background image of an arrow appears after 10 events have been input in the page which can motivate the user to apply for more events to further update their timeline page. Furthermore, the background colour of the div is the same as the skills and calendar page to maintain consistent design and to direct the user that this area is interactable. Furthermore, the buttons have been styled in a unique colour and font to highlight them compared to the other elements on the page.

Lastly, the navigation bar is consistently present on the top left of every webpage. It blends in with the background colours to not be obtrusive in design, however, when a page is hovered over, the button changes colour. This allows for responsiveness and allows the user to have confirmation that they have selected the right option.

How could unmet features be implemented in future iterations?

There are two main unmet usability features for this program:

1. The example skills descriptions are not formatted correctly
2. The timeline reflection/skills descriptions text boxes might be unclear for users

1. Improved formatting of example skills descriptions

According to stakeholder feedback, the example skills descriptions are a useful feature but do require further formatting polish. The stakeholders identified some issues with readability and therefore the first change would be to use a darker font for the descriptions. Making the font colour darker is easy to implement as it requires changing the 'color' attribute of the 'p' styling in 'skillsstylesheet.css' to a darker shade of the current colour, or another dark colour entirely. This could be done based on stakeholder feedback by sending out a request email before starting work on a further iteration of the project.

To further improve the formatting of the descriptions, instead of implementing them as a 'p' tag, I could implement them using a 'ul' tag which would format them as a bullet point list. This would improve the formatting of the descriptions and solve the readability issues. Furthermore, I could also format the bullet points to be formed in the centre of the tabular format aligned to the left of the second table data in each row. For example, the bullet points for an example description aligned on the right can be put in the middle of each row. This would reduce the amount of movement of the eyes required which may improve the program for better visual use.

2. User guidance for text boxes

Another piece of stakeholder feedback regarding usability was that the clients stated that some more indication of the fact that text boxes are resizable should be given. One user in particular stated that including a tutorial on how to resize text boxes would be a helpful feature; therefore, this could be implemented using a pop-up window which displays a gif on how to resize the text boxes which appears on the timeline and skills pages. Alternatively, I could also implement a written tutorial which could indicate to the user that "clicking on the bottom right corner of the text boxes and dragging the corner will allow you to resize the textboxes when required. This would help eliminate any confusion when writing into resizable text boxes. A further change I can make would be to change the default size of the text boxes to a larger size. This would make it so that the user does not even need to resize the text boxes to type large amounts of text and would also be more intuitive that a large amount of text should be input there.

Furthermore, many stakeholders also stated that more instructions on what to put on the timeline page could be helpful. I could implement this in two ways. One way would be to add a permanent p tag underneath the timeline title which will tell the user to "Enter your event reflections on this page". This would be a simple permanent fix.

Alternatively, a pop-up window could be implemented which appears on the first time the user accesses the timeline page to inform them of what to do. This would make the reminder feel less obtrusive and wouldn't affect the current design of the website.

Limitations

The main limitation of the program is that it is currently locally hosted. This means that the program isn't currently accessible on the WWW and that the database present on the user's device is not linked to other users.

Another limitation of the program is that it requires an internet connection to function. As shown in the final testing in the development and testing phase, the connection to the SQL database does not start and the fonts aren't correctly imported.

Furthermore, there is no multifactor authentication in the program. Multi-factor authentication would be a necessity when scaling the program to be globally hosted to prevent data breaches and invalid login attempts.

A final, minor, limitation of the program would be improving the current UI/UX. For example, currently the timeline page only shows the full arrow after 10 events have been added to the calendar. After 10 events have been added, the background image begins shifting and enlarging in the background which ruins the current design.

How could these limitations be addressed?

Global hosting is a necessity so users from all around the WWW will be able to access my program. The database and the program are currently locally hosted using XAMPP which means that there isn't a central database from which the users are retrieving data. Rather, each user has their own blank copy of the database and its tables upon running the database.php file. Using services such as Google Cloud or Wix web hosting (linked in bibliography) I could implement my website on a global scale which would remove the need for the user to download a localhost such as XAMPP to run the program. The database would then be hosted globally which would completely remove the need for database.php to be included with the program.

The website will require an internet connection to access as it requires a stable SQL connection, and the program needs to import fonts from google fonts to have the correct UI. However, a further iteration of the project could implement the program as a mobile app. The program could have a locally stored database on the user's device which would allow for offline querying of the database. Furthermore, the font required could be downloaded from Google fonts. This would further remove the need for an internet connection as the font will not need to be imported while hosting the website.

When scaling the program to be globally hosted, the main aspect that will require further focus will be security. Globally hosting the program increases the risks of malicious users attempting to access the website and therefore, further security features would need to be implemented. After doing further research on cybersecurity (linked in bibliography), I would like further iterations of the program to implement the principles of the CIA triad model (Confidentiality, Integrity, and Availability) which is a core model for protecting user data. To implement these, future iterations of the program could implement multi-factor authentication. This feature could be implemented by adding an email input to the signup field and sending an email with a security code to the user when they are signing in. This is a common method used in many modern programs to implement a further layer of security apart from common credential checking which would fall under the confidentiality principle. This would be a useful feature to implement which would address the current limitations of security.

Currently, the UI/UX mostly matches the drafted design in the design stage which the stakeholders approved. However, I intend to implement a better, more user-friendly UI. This could be clean and minimal, and I would base the design based on further stakeholder feedback. This could include more use of pop-ups or a more dynamic design (using animations etc). Improving the UI would be useful in improving the program for the current users and would also be helpful when hosting the website globally for users. An improved UI/UX would have a great impact on the SEO of my program. Implementing a good UI can increase the website's SEO rankings, traffic and user engagement.

Potential improvements/changes that could be made?

There are a few features that could be added to the program in further iterations:

- Reveal password feature
- 'Forgot my password' feature
- Further dividing days in the calendar

Reveal password feature

A 'reveal password' feature was a highly requested feature by stakeholders and would be a useful usability feature to implement. It would allow the user to be able to check the password that they used to create the account to minimise the chance of incorrect credentials being entered or the user forgetting the credentials.

To implement this, a button could be added underneath the input field and before the submit button. When clicked, it will first check the type attribute of the password input field. If the type is currently password, then it will change the type attribute of the password field (and confirmation password field on the signup page) to 'input' while

preserving the entered value. If the type attribute was input, it will revert the type back to password. This would allow the user to toggle between visible and hidden allowing the user privacy while maintaining readability.

'Forgot my password' feature

A necessary addition to the program would be to make a 'forgot my password' option available to the users. Currently, the user will only be able to use the program if they remember their credentials or have them stored in autofill (I have added the autofill attribute to the input fields for username and password on the login page to allow for this).

This may be an issue if the user forgets their credentials or if there are any issues with autofill. Therefore, to implement this, I will need to add an email input to the signup page. Adding an email input would allow me to implement a password reset in two ways. One simple way would be to make the user enter their username and email which would then allow them to reset their password. Another, more secure way of implementing this would be to send a password reset email to the user which they can use to reset their password.

Further dividing days in the calendar

This was a feature suggested by a user in the stakeholder feedback which would also be incredibly helpful to have. This feature, however, would require the program to be scaled even bigger than it currently is. This feature would be incredibly helpful to the user and would allow for more in-depth planning and would extend the use of the program so that it could also be used as a revision timetable.

Implementing a hours-based calendar system per day would require reworking the input form to allow for events to be input per hour. To allow for this, I will also need to add a 'event_time' attribute to the database to allow for multiple events to be stored per day and to be differentiated by time. Furthermore, the UI of the input form and calendar would need to be changed. The input form would need to have a time input added to the form alongside a 'add new event' button which would allow the user to add a second event for a different time.

Maintenance

The biggest issue with the program currently is that the calendar is limited to 2026. Currently, the user can only enter past, present and future data for 2026 and the calendar has been formatted according to the 2026 calendar. This is because the

calendarData variable currently stores a dictionary with nested dictionaries for the starting positions of each month in the week and the number of days in the month.

Further maintenance could add another layer of nesting (as suggested in the partially met criteria review) and allow for data for multiple years to be stored. Furthermore, a year selector could be implemented on the calendar page which then outputs the month selector with the specified year passed to the replace_month function.

Further iterations of the program can be improved by collecting more stakeholder feedback and implementing some of the currently unmet features. Further stakeholder suggestions or changes could be implemented by future developers as the code is very modular in nature and functions such as the error checking functions can be easily changed or replaced with updated conditions in the future.

The data in the database is retrieved and stored as session variables when required, and therefore, any new features could easily be implemented without worrying about those functions accidentally altering the user data in the database or affecting other functions.

My program is stored in one main folder which contains the subfolder ‘includes’ responsible for most of the backend processing of the website. This means that the frontend and backend code is logically separated for future developers and that creating new pages or new backend files can be done without affecting the other pre-existing files.

The code is also mostly annotated which would allow future developers to easily understand the logic for the function and allow them to easily pinpoint subroutines that need to be changed/improved in future iterations.

Future versions of the program could implement some of the potential improvements and unmet criteria highlighted previously as it would scale the project to be used as a multi-purpose 6th form tool for revision and applications.

Final Code

Main Folder

calendar.php

```php
<?php
require_once 'includes/config_session.inc.php';  //Allows for session variable regeneration (added security)
require_once 'includes/dbh.inc.php';  //Allows for database access
require_once 'includes/login_view.inc.php';  //Allows for the output_username() function to be accessed


if (!isset($_SESSION['user_id'])) {  //Checks if the user hasn't logged in legitamately (user id hasn't been assigned)
    header("Location: login.php");  //Otherwise, returns to login page
    exit();
}

$userId = $_SESSION['user_id'];

/*Retrieves relevant event information (date and tag) for the logged in user*/
$stmt = $pdo->prepare("SELECT EVENTS.date_of_event, EVENTS.event_tag FROM EVENTS INNER JOIN LINK_TABLE
                       ON EVENTS.event_id = LINK_TABLE.event_id WHERE LINK_TABLE.user_id = :user_id;");
$stmt->bindParam(":user_id", $userId);  //Prepared statement parameters are passed (user_id)
$stmt->execute();
$events = $stmt->fetchAll(PDO::FETCH_ASSOC);  //Fetch all to retrieve every entry that matches the SQL query

?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Calendar</title>
    <!--Allows for styling to be accessed-->
    <link rel="stylesheet" href="calendarstylesheet.css">
    <!--Allows for the replaceMonth() function to be accessed from calendargeneration.js-->
    <script src="calendargeneration.js"></script>
</head>
<body>

    <!--This is the navigation bar which allows the user to traverse between pages and logout from the website-->
    <nav>
    <ul>
    <li><a href="calendar.php">Calendar</a></li>
    <li><a href="skills.php">Skills</a></li>
    <li><a href="timeline.php">Timeline</a></li>
    <li><a href="includes/logout.inc.php">Logout</a></li>
    </ul>
    </nav>

    <h1 id="title">Calendar</h1>

    <?php
        output_username();  //Outputs a 'Hello (username) !' for the user who is currently signed in

        //Checks to see if the calendar page was redirected from the inputform and if the saving was successful or not
        if(isset($_GET['save']) && $_GET['save']==='success'){
            echo '<p id="form-success"><u>Event successfully stored/updated.</u></p>';
        }
    ?>

    <br><br>

        <!--Month selector for calendar (implementing next month/previous month functionality
        was challenging within the time constraints and with my current skills so I chose to implement a selector)-->
    <div id="calendar-grid">
        <br>
        <table>
            <tr class="calendar-grid-row">
                <td><button onclick="replace_month('January')" class="calendar-grid-month">January</button></td>
                <td><button onclick="replace_month('February')" class="calendar-grid-month">February</button></td>
                <td><button onclick="replace_month('March')" class="calendar-grid-month">March</button></td>
                <td><button onclick="replace_month('April')" class="calendar-grid-month">April</button></td>
            </tr>
```

```
            <tr class="calendar-grid-row">
                <td><button onclick="replace_month('May')" class="calendar-grid-month">May</button></td>
                <td><button onclick="replace_month('June')" class="calendar-grid-month">June</button></td>
                <td><button onclick="replace_month('July')" class="calendar-grid-month">July</button></td>
                <td><button onclick="replace_month('August')" class="calendar-grid-month">August</button></td>
            </tr>

            <tr class="calendar-grid-row">
                <td><button onclick="replace_month('September')" class="calendar-grid-month">September</button></td>
                <td><button onclick="replace_month('October')" class="calendar-grid-month">October</button></td>
                <td><button onclick="replace_month('November')" class="calendar-grid-month">November</button></td>
                <td><button onclick="replace_month('December')" class="calendar-grid-month">December</button></td>
            </tr>
        </table>
        <br>
    </div>

    <br><br>

    <!--This is where the selected calendar month HTML will be inserted-->
    <div id="calendar-container">
        <h2 id="selection-message">Select a month</h2>
    </div>

    <br><br><br><br><br><br>

    <!--Legend which acts as reference for colour coding-->
    <h2 id="subtitle">Legend</h2>

    <!--Two column layout to show a circle with the colour-coding colour and the tag name-->
    <div class="row" id="legend">
        <div class="column">
            <div class="circle-work-experience"></div>
            <div class="circle-course"></div>
            <div class="circle-competition"></div>
        </div>

        <div class="legend-tags">
            <h1>Work experience</h1>
            <h1 class="legend-tags-below">Course</h1>
            <h1 class="legend-tags-below">Competition</h1>
        </div>
    </div>

    <!--Creating a const in JS which is accessible by the replaceMonth() functions which stored the date and tag of
    tagged events for colour coding-->
    <script>
        const eventTags = {
            <?php
            $total = count($events);
            $i = 0;
            foreach ($events as $event) {  //Loops through associative array of event tags and dates fetched from database
                $i++;
                $date = $event['date_of_event'];  // Must be in format 'YYYY-MM-DD'
                $tag = $event['event_tag'];  // The tag string stored in DB
                echo "'$date':'$tag'";  // Adds them as key-value pairs into eventTags
                if ($i < $total) echo ",";  // Adds commas between pairs for correct syntax
            }
            ?>
        };
    </script>

</body>
</html>
```

calendargeneration.js

```
//Calendar Data for 2026 stored in nested dictionaries
//If startDay: 1 = Monday, 7 = Sunday etc for days of the week
//totalDays: number of days in month
const calendarData = {
    "January":   { startDay: 4, totalDays: 31 },
    "February":  { startDay: 7, totalDays: 28 },
    "March":     { startDay: 7, totalDays: 31 },
    "April":     { startDay: 3, totalDays: 30 },
    "May":       { startDay: 5, totalDays: 31 },
    "June":      { startDay: 1, totalDays: 30 },
    "July":      { startDay: 3, totalDays: 31 },
    "August":    { startDay: 6, totalDays: 31 },
    "September": { startDay: 2, totalDays: 30 },
    "October":   { startDay: 4, totalDays: 31 },
    "November":  { startDay: 7, totalDays: 30 },
    "December":  { startDay: 2, totalDays: 31 }
};


//Returns suffix for a day for the nodes on the calendar page
function suffix(n) {
    if (n >= 11 && n <= 13) return "th";  //Special case of dates
    if (n % 10 === 1) return "st";  //Checks if final digit is 1, if it is, return st
    if (n % 10 === 2) return "nd";  //Checks if final digit is 2, if it is, return nd
    if (n % 10 === 3) return "rd";  //Checks if final digit is 3, if it is, return rd
    return "th";
}
```

```
//Create a day cell (table data element) with a form that redirects to 'inputform.php'
function create_day_cell(day, month, year = 2026) {  //The year value has been set to 2026 (*This can be changed to allow for more years*)
    const td = document.createElement("td");  //Create the td element to store the button
    td.className = "calendar-cell";  //Adding a class for styling purposes

    //Creates a form element for the buttons (one form element per hidden input otherwise multiple values are sent in GET)
    const form = document.createElement("form");
    form.method = "GET";  //GET method as date is not sensitive data
    form.action = "inputform.php";  //Redirects to 'inputform.php'

    //Format date as YYYY-MM-DD to match database format
    const date = new Date(month + " " + day + ", " + year);
    let monthNum = date.getMonth() + 1;
    let dayNum = date.getDate();
    if (monthNum < 10){
        monthNum = "0" + monthNum;
    }
    if (dayNum < 10){
        dayNum = "0" + dayNum;
    }
    const fullDate = year + "-" + monthNum + "-" + dayNum;

    const hidden = document.createElement("input");  //Creates an input element
    hidden.type = "hidden";  //Sets the type of input to hidden (to send the date to the input form)
    hidden.name = "date";  //Sets the name of the input field for the GET superglobal to access the data
    hidden.value = fullDate;  //The value of the input field is the date formatted correctly for the database

    const submit = document.createElement("input");  //Creates an input element
    submit.type = "submit";  //Sets the type of the input to submit (these are the buttons on the calendar)
    submit.value = day + suffix(day);  //This is the string value displayed on the button (the day + suffix)
    submit.className = "calendar-day";  //Adds a class for styling purposes

    //Color-code cell based on eventTags
    if (eventTags[fullDate]) {  //If the date has a colour coding value associated with it in eventTags
    submit.classList.add(eventTags[fullDate]);  //Add the tag from eventTags to the class list of the button
}

    form.appendChild(hidden);  //Appends the hidden input to the form
    form.appendChild(submit);  //Appends the submit button to the form
    td.appendChild(form);  //Appends the form to the td element
```

```
    return td;  //Returns the td element
}

//Replace month function
function replace_month(month, year = 2026) {  //The year value has been set to 2026 (*This can be changed to allow for more years*)
    const container = document.getElementById("calendar-container");  //Retrieves the div to hold the calendar data
    container.innerHTML = ""; //Clears previous content

    //Month heading
    const heading = document.createElement("h2");  //Creates a h2 element
    heading.textContent = `${month} ${year}`;  //The text will be the month and the year
    heading.className = "calendar-header";  //Class added for styling purposes

    //Table setup
    const table = document.createElement("table");  //Table is created to hold days in a grid format
    table.className = "calendar-table";  //Class for styling purposes

    //Weekday header
    const headerRow = document.createElement("tr");  //A tr element is created to hold the day indicators
    const daysOfWeek = ["Mon","Tue","Wed","Thu","Fri","Sat","Sun"];  //Array storing abbreviated days of the week
    for (let d of daysOfWeek) {  //For every day in the array
        const th = document.createElement("th");  //Create a th element
        th.className = "calendar-cell calendar-weekday";  //Add a class for styling purposes
        th.textContent = d;  //Make the text content the day abbreviation
        headerRow.appendChild(th);  //Append the th to the tr
    }
    table.appendChild(headerRow);  //Append the tr to the table

    //(*calendarData[month] can be replaced with calendarData[year][month] to allow for more years*)
    const { startDay, totalDays } = calendarData[month];
    let day = 1;
    let tr = document.createElement("tr");

    //Add empty cells before first day of the month
    for (let i = 1; i < startDay; i++) {
        const td = document.createElement("td");
        td.className = "calendar-cell";
        tr.appendChild(td);
    }

    //Add the days to the calendar
    while (day <= totalDays) {
        tr.appendChild(create_day_cell(day, month, year));
        day++;

        if (tr.children.length === 7) {
            table.appendChild(tr);
            tr = document.createElement("tr");
        }
    }

    //Add remaining empty cells in last row
    while (tr.children.length > 0 && tr.children.length < 7) {  //While there are empty spaces in the last row
        const td = document.createElement("td");  //Create a td element
        td.className = "calendar-cell";  //Add a class for styling
        tr.appendChild(td);  //Append it to the row
    }
    //Add last row to calendar if it has values
    if (tr.children.length > 0){  //If the last row has at least one value
        table.appendChild(tr);  //Add it to the table
    }

    container.appendChild(heading);  //Add the heading to the div
    container.appendChild(table);  //Add the table to the div
}
```

calendarstylesheet.css

```css
@import url('https://fonts.googleapis.com/css2?family=Hanken+Grotesk:ital,wght@0,100..900;1,100..900&display=swap');
/*Importing correct font (based on Canva design) from Google fonts*/

body{
    text-align: center;
    font-family: 'Hanken Grotesk', sans-serif;
    background-color: #070a2a;
}

#title{
    color: white;
    font-size: 75px;
}

#form-success{
  color: aliceblue;
  font-size: 20px;
}

#subtitle{
    color: white;
    font-size: 50px;
    text-align: left;
}

#calendar-grid{
    background-color: #b5c4fa;
    margin-left: 35.75%;
    margin-right: 35.5%;
    border-radius: 10px;
    padding: 10px, 10px, 10px, 10px;
}

.calendar-grid-row{
    padding-bottom: 100px;
}
```

```css
.calendar-grid-month{
    font-family: 'Hanken Grotesk', sans-serif;
    width: 100px;                    /*Full cell width*/
    height: 100px;                   /*Full cell height*/
    background-color: aliceblue;   /*Blueish White background*/
    border: 1px black solid;
    border-radius: 25%;
    cursor: pointer;                 /*Pointer on hover*/
    font-size: medium;
    transition: background-color 0.2s;

}

.calendar-grid-month:hover{
    background-color: #2c3e50;
    color: white;
}

.calendar-grid-month:active{
    background-color: #2c3e50;
}

.calendar-table{
    width: 100%;
    height: 1100px;            /*Full width of the container*/
    border-collapse: collapse; /*Remove gaps between cells*/
    margin-top: 10px;          /*Small spacing below the header*/
}

.calendar-row{
    text-align: center;        /*Center all text in rows*/
}

.calendar-header{
    padding-top: 50px;
    padding-bottom: 50px;
    text-align: center;        /*Center month heading*/
    font-size: xx-large;       /*Larger font for the month*/
    margin-bottom: 5px;        /*Space below heading*/
    color: #2c3e50;            /*Dark blue color*/
}

.calendar-weekday{
    background-color: #b5c4fa;
    color: black;
    padding: 8px 0;
    font-weight: bold;
    font-size: large;
}

.calendar-cell{
    border: none;
    height: 40px;
    width: 14.28%;
    vertical-align: middle;
    padding: 2px;
}

.calendar-cell:empty{
    background-color: #b5c4fa; /*Very light background for empty cells*/
}


.calendar-day{
    font-family: 'Hanken Grotesk', sans-serif;
    width: 150px;                /*Full cell width*/
    height: 150px;               /*Full cell height*/
    background-color: aliceblue;
    border: 1px black solid;
    border-radius: 25%;
    cursor: pointer;             /*Pointer on hover*/
    font-size: medium;
    transition: background-color 0.2s; /*Smooth hover effect*/

}
```

```css
.work-experience{
    width: 150px;                  /*Full cell width*/
    height: 150px;                 /*Full cell height*/
    background-color: #ea473e;
    border: 1px black solid;
    border-radius: 25%;
    cursor: pointer;               /*Pointer on hover*/
    font-size: medium;
    transition: background-color 0.2s; /*Smooth hover effect*/
}

.course{
    width: 150px;                  /*Full cell width*/
    height: 150px;                 /*Full cell height*/
    background-color: #4395ae;
    border: 1px black solid;
    border-radius: 25%;
    cursor: pointer;               /*Pointer on hover*/
    font-size: medium;
    transition: background-color 0.2s; /*Smooth hover effect*/
}

.competition{
    width: 150px;                  /*Full cell width*/
    height: 150px;                 /*Full cell height*/
    background-color: #55bc6d;
    border: 1px black solid;
    border-radius: 25%;
    cursor: pointer;               /*Pointer on hover*/
    font-size: medium;
    transition: background-color 0.2s; /*Smooth hover effect*/
}

.calendar-day:hover {
    background-color: #2c3e50;
    color: white;                   /*White text on hover*/
}
```

```css
#calendar-container{
    text-align: center;
    background-color: #b5c4fa;
    border-radius: 25px;
}

#selection-message{
    font-size: xx-large;
}

.login-info{
    color: aliceblue;
    font-size: 35px;
}

#legend{
    text-align: left;
    background-color: white;
    padding-top: 25px;
    padding-left: 10px;
    padding-bottom: 25px;
    border-radius: 20px;
}

.legend-tags{
    text-align: left;
}

.legend-tags-below{
    padding-top: 2px;
}

.circle-work-experience {
    height: 50px;
    width: 50px;
    background-color: #ea473e;
    border-radius: 50%;
    margin-top: 15px;
}
```

```css
.circle-course {
    height: 50px;
    width: 50px;
    background-color: #4395ae;
    border-radius: 50%;
    margin-top: 15px;
}

.circle-competition {
  height: 50px;
  width: 50px;
  background-color: #55bc6d;
  border-radius: 50%;
    margin-top: 15px;
}
```

```css
.column {
  float: left;
  width: 70px;
}


/*Clear floats after the columns*/
.row:after {
  content: "";
  display: table;
  clear: both;
}
```

```css
nav{
position: left;
}

nav ul{
list-style: none;
background-color: #070a2a;
overflow: hidden;
margin: 0;
padding:0;
}

nav ul li{
float: left;
}

nav ul li a{
display: block;
color: aliceblue;
text-align: center;
padding: 14px 20px;
text-decoration: none;
border-color: black;
}

nav ul li a:hover {
background-color:  #b5c4fa;
}
```

database.php

```php
<?php
//Declare variables to be passed
$servername = "localhost";
$username = "root";
$password = "";

//Create connection
$conn = new mysqli($servername, $username, $password);
// Check connection
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}

// Create database
$sql = "CREATE DATABASE NEA";
if ($conn->query($sql) === TRUE) {
  echo "Database created successfully<br>";
} else {
  echo "Error creating database: " . $conn->error ."<br>";
}

//Close connection
$conn->close();
?>

<?php
//Declare variables to be passed
$db_server = "localhost";
$db_user = "root";
$db_pass = "";
$db_name = "NEA";
$conn = "";

//Connect to NEA database
try{
$conn = mysqli_connect($db_server, $db_user, $db_pass, $db_name);
}
catch(mysqli_sql_exception){
    echo "Could not connect" . $conn->connect_error ."<br>";
}
```

```php
//Create User table
$sql = "CREATE TABLE USERS (
user_id INT(6) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(30) NOT NULL,
hashed_password VARCHAR(255) NOT NULL
)";

//Check if table created successfully
if ($conn->query($sql) === TRUE){
    echo "Table USERS created successfully<br>";
} else{
    echo "Error creating table: ". $conn->error ."<br>";
}


//Create Events table
$sql = "CREATE TABLE EVENTS (
event_id INT(6) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
date_of_event DATE NOT NULL,
name_of_event VARCHAR(255) NOT NULL,
event_description TEXT NOT NULL,
event_reflection TEXT,
event_tag VARCHAR(255)
)";

//Check if table created successfully
if ($conn->query($sql) === TRUE) {
  echo "Table EVENTS created successfully<br>";
} else {
  echo "Error creating table: " . $conn->error ."<br>";
}
```

```php
//Create Skills table
$sql = "CREATE TABLE SKILLS (
skills_id INT(6) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
leadership BOOLEAN,
creativity BOOLEAN,
teamwork BOOLEAN,
initiative BOOLEAN,
problem_solving BOOLEAN,
organisation BOOLEAN,
communication BOOLEAN,
public_speaking BOOLEAN,
perseverance BOOLEAN,
time_management BOOLEAN,
planning_and_preparation BOOLEAN,
analytical_skills BOOLEAN,
practical_skills BOOLEAN,
academic_skills BOOLEAN,
computing_skills BOOLEAN
)";

//Check if table created successfully
if ($conn->query($sql) === TRUE) {
  echo "Table SKILLS created successfully<br>";
} else {
  echo "Error creating table: " . $conn->error ."<br>";
}
```

```php
//Create Skills Description table
$sql = "CREATE TABLE SKILLS_DESCRIPTION (
description_id INT(6) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
leadership_description TEXT,
creativity_description TEXT,
teamwork_description TEXT,
initiative_description TEXT,
problem_solving_description TEXT,
organisation_description TEXT,
communication_description TEXT,
public_speaking_description TEXT,
perseverance_description TEXT,
time_management_description TEXT,
planning_and_preparation_description TEXT,
analytical_skills_description TEXT,
practical_skills_description TEXT,
academic_skills_description TEXT,
computing_skills_description TEXT
)";

//Check if table created successfully
if ($conn->query($sql) === TRUE) {
  echo "Table SKILLS_DESCRIPTION created successfully<br>";
} else {
  echo "Error creating table: " . $conn->error ."<br>";
}
```

```php
//Create Link table
$sql = "CREATE TABLE LINK_TABLE (
link_id INT(6) NOT NULL AUTO_INCREMENT PRIMARY KEY,
user_id INT(6) UNSIGNED,
event_id INT(6) UNSIGNED,
skills_id INT(6) UNSIGNED,
description_id INT(6) UNSIGNED,
FOREIGN KEY (user_id) REFERENCES USERS(user_id),
FOREIGN KEY (event_id) REFERENCES EVENTS(event_id),
FOREIGN KEY (skills_id) REFERENCES SKILLS(skills_id),
FOREIGN KEY (description_id) REFERENCES SKILLS_DESCRIPTION(description_id)
)";

//Check if table created successfully
if ($conn->query($sql) === TRUE) {
  echo "Table LINK_TABLE created successfully<br>";
} else {
  echo "Error creating table: " . $conn->error ."<br>";
}

// Close connection
$conn->close();
```

inputform.php

```php
<?php
require_once 'includes/config_session.inc.php';  //Allows for session variable regeneration (added security)
//Allows for data from the database to be displayed in the form if there is an event stored for the date
require_once 'includes/inputformhandler.inc.php';
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Input</title>
    <!--Allows for styling to be accessed-->
    <link rel="stylesheet" href="inputformstylesheet.css">
</head>
<body>
    <!--This is the navigation bar which allows the user to traverse between pages and logout from the website-->
    <nav>
    <ul>
    <li><a href="calendar.php">Calendar</a></li>
    <li><a href="skills.php">Skills</a></li>
    <li><a href="timeline.php">Timeline</a></li>
    <li><a href="includes/logout.inc.php">Logout</a></li>
    </ul>
    </nav>
    <br>

    <!--This is the input form title. It displays a cleaned value (htmlspecialchars prevents malicious JS code
    from being displayed on the page) of the date in a YYYY-MM-DD format which is how it is stored in the database-->
    <b><p id="title"><?php echo htmlspecialchars($date); ?></p></b>

    <?php
    /*This piece of code checks to see if an event has been input without an event name*/
        if (isset($_GET['error']) && $_GET['error'] === 'missing_event_name') {  //If an event was input without a name
            echo '<p id="form-error"><u>Event name is required.</u></p>';  //Return an error
        }

    ?>

<!--This declares the form element and specifies where (inputform_input.inc.php) and using which method the data will be transferred
(this data is sent using post as it is sensitive data)-->
<form action="includes/inputform_input.inc.php?date=<?php echo htmlspecialchars($date); ?>" method="post">
    <div id="div1">

    <u><label for="eventname">Name of event:</label></u><br>

    <!--This section of the input form is for the event name. If the an event has been stored in the database for this date, the input field
    will show a cleaned value (htmlspecialchars prevents malicious JS code from being displayed on the page) of the event name-->

    <input type="text" name="eventname" id="eventname" value="<?php echo htmlspecialchars($eventName); ?>"><br><br><br>

    <u><p>Event Description:</p></u>

    <!--This section of the input form is for the event description. If the event has a description in the database, the textarea
    will show a cleaned value (htmlspecialchars prevents malicious JS code from being displayed on the page) of the event description-->

    <textarea name="event_description" id="event_description"><?php echo htmlspecialchars($eventDescription); ?></textarea><br><br><br>

    <u><label>Skills Shown:</label></u><br><br>

    <!--This section of the input form is for the skills the event has been tagged with. If the event has been tagged in the database
    with a certain skill, the checkbox button will already be checked when viewing the page-->

    <label for="leadership">Leadership</label>
    <input type="checkbox" name="leadership" id="leadership" value="1" <?php if ($leadership == 1) echo 'checked'; ?>><br><br>

    <label for="creativity">Creativity</label>
    <input type="checkbox" name="creativity" id="creativity" value="1" <?php if ($creativity == 1) echo 'checked'; ?>><br><br>

    <label for="teamwork">Teamwork</label>
    <input type="checkbox" name="teamwork" id="teamwork" value="1" <?php if ($teamwork == 1) echo 'checked'; ?>><br><br>

    <label for="initiative">Initiative</label>
    <input type="checkbox" name="initiative" id="initiative" value="1" <?php if ($initiative == 1) echo 'checked'; ?>><br><br>

    <label for="problem_solving">Problem Solving</label>
    <input type="checkbox" name="problem_solving" id="problem_solving" value="1" <?php if ($problem_solving == 1) echo 'checked'; ?>><br><br>
```

```
<label for="organisation">Organisation</label>
<input type="checkbox" name="organisation" id="organisation" value="1" <?php if ($organisation == 1) echo 'checked'; ?>><br><br>

<label for="communication">Communication</label>
<input type="checkbox" name="communication" id="communication" value="1" <?php if ($communication == 1) echo 'checked'; ?>><br><br>

<label for="public_speaking">Public Speaking</label>
<input type="checkbox" name="public_speaking" id="public_speaking" value="1" <?php if ($public_speaking == 1) echo 'checked'; ?>><br><br>

<label for="perseverance">Perseverance</label>
<input type="checkbox" name="perseverance" id="perseverance" value="1" <?php if ($perseverance == 1) echo 'checked'; ?>><br><br>

<label for="time_management">Time Management</label>
<input type="checkbox" name="time_management" id="time_management" value="1" <?php if ($time_management == 1) echo 'checked'; ?>><br><br>

<label for="planning_and_preparation">Planning and Preparation</label>
<input type="checkbox" name="planning_and_preparation" id="planning_and_preparation" value="1" <?php if ($planning_and_preparation == 1) echo 'checked'; ?>><br><br>

<label for="analytical_skills">Analytical Skills</label>
<input type="checkbox" name="analytical_skills" id="analytical_skills" value="1" <?php if ($analytical_skills == 1) echo 'checked'; ?>><br><br>

<label for="practical_skills">Practical Skills</label>
<input type="checkbox" name="practical_skills" id="practical_skills" value="1" <?php if ($practical_skills == 1) echo 'checked'; ?>><br><br>

<label for="academic_skills">Academic Skills</label>
<input type="checkbox" name="academic_skills" id="academic_skills" value="1" <?php if ($academic_skills == 1) echo 'checked'; ?>><br><br>

<label for="computing_skills">Computing Skills</label>
<input type="checkbox" name="computing_skills" id="computing_skills" value="1" <?php if ($computing_skills == 1) echo 'checked'; ?>><br><br>

<u><label>Tag:</label></u><br><br>

<!--This section of the input form is for the tags the event has been tagged with. If the event has been tagged in the database with a certain tag,
the radio button will already be checked when viewing the page-->

<label for="work-experience">Work Experience</label>
<input type="radio" name="tag" id="work-experience" value="work-experience" <?php if ($eventTag == 'work-experience') echo 'checked'; ?>><br>


    <label for="course">Course</label>
    <input type="radio" name="tag" id="course" value="course" <?php if ($eventTag == 'course') echo 'checked'; ?>><br>

    <label for="competition">Competition</label>
    <input type="radio" name="tag" id="competition" value="competition" <?php if ($eventTag == 'competition') echo 'checked'; ?>><br>

    <label for="none">None</label>
    <input type="radio" name="tag" id="none" value="none" <?php if ($eventTag == NULL) echo 'checked'; ?>><br><br>
</div>

<input type="submit" id="submit" value="Submit">
</form>

</body>
</html>
```

inputformstylesheet.css

```css
@import url('https://fonts.googleapis.com/css2?family=Hanken+Grotesk:ital,wght@0,100..900;1,100..900&display=swap');
/*Importing correct font (based on Canva design) from Google fonts*/

body{
    color: black;
    background-color: #070a2a;
    font-family: 'Hanken Grotesk', sans-serif;
    text-align: center;
}
#div1{
    font-family: 'Hanken Grotesk', sans-serif;
    text-align: left;
    background-color: white;
    height: 1050px;
    padding-left: 50px;
    padding-right: 50px;
    padding-bottom: 20px;
    padding-top: 75px;
    border: solid 1px #060a2e;
    border-collapse: separate;
    border-spacing: 0 10px;
    border-radius: 10px;
}

#title{
    color: white;
    font-size: xxx-large;
}

#eventname{
    font-family: 'Hanken Grotesk', sans-serif;
    background-color : #6c718c;
    color: aliceblue;
    border: 1px solid black;
    box-shadow: none;
}

#event_description{
    font-family: 'Hanken Grotesk', sans-serif;
}

#submit{
    background-color: #7c90ea;
    border-radius: 30px;
    font-size: x-large;
    border-color: #7c90ea;
    border-style: hidden;
    padding: 20px;
}

nav{
position: left;
}

nav ul{
list-style: none;
background-color: #070a2a;
overflow: hidden;
margin: 0;
padding:0;
}

nav ul li{
float: left;
}

nav ul li a{
display: block;
color: aliceblue;
text-align: center;
padding: 14px 20px;
text-decoration: none;
border-color: black;
}

nav ul li a:hover {
background-color:  #b5c4fa;
}

#form-error{
  color: red;
  font-size: 20px;
}
```

login.php

```php
<?php
require_once 'includes/config_session.inc.php';  //Allows for session variable regeneration (added security)
require_once 'includes/login_view.inc.php';  //Allows for check_login_errors() to be accessed
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Registration</title>
    <!--Allows for styling to be accessed-->
    <link rel="stylesheet" href="regstylesheet.css">
</head>

<body>
    <b><h1 id="Reg">Login</h1></b>

    <!--This declares the form element and specifies where (login.inc.php) and using which method the data will be transferred
    (this data is sent using post as it is sensitive data)-->
    <form action="includes/login.inc.php" method="post">
    <br><br>
        <div>
            <br>
            <label for="username">Enter your username   </label><input id="username" type="text" name="username" autocomplete="on">
            <br><br><br><br>
            <label for="password">Enter your password   </label><input id="password" type="password" name="password" autocomplete="on">
            <br><br><br>
            <input type="submit" id="Submit">
        </div>
    </form>

    <?php
        check_login_errors();  //Calls the function which checks for any errors in login such as incorrect login information or blank fields
    ?>

    <br><br><br><br>
    <!--A redirect link which takes you to the signup page-->
    <a href="signup.php">Create An Account</a>
</body>
</html>
```

regstylesheet.css

```css
@import url('https://fonts.googleapis.com/css2?family=Hanken+Grotesk:ital,wght@0,100..900;1,100..900&display=swap');
/*Importing correct font (based on Canva design) from Google fonts*/

body{
  background-color: #070a2a;
  font-family: 'Hanken Grotesk', sans-serif;
  text-align: center
}

form{
  color: #0b0e2e;
}

div{
  display: inline-block;
  margin: 10px auto;
  background-color: #dee2fb;
  height: 250px;
  width: 750px;
  font-size: large;
}

#Reg{
  color: aliceblue;
  font-size: 75px;
}

#Submit{
  background-color: #7c90ea;
  border-radius: 20px;
  font-size: large;
  border-color: #7c90ea;
  border-style: hidden;
  padding: 10px;
}

input[type="text"]{
  background-color : #6c718c;
  color: aliceblue;
  border: 1px solid black;
  box-shadow: none;
}
```

```css
input[type="password"]{
  background-color : #6c718c;
  color: aliceblue;
  border: 1px solid black;
  box-shadow: none;
}

.form-error{
  color: aliceblue;
  font-size: 35px;
}

.form-success{
  color: red;
  font-size: 50px;
}

.login-info{
  color: aliceblue;
  font-size: 20px;
}

a{
  position: static;
  font-size: 40px;
}

a:link {
  color: #b5c4fa;
  background-color: transparent;
  text-decoration: underline;
}

a:visited {
  color: #b5c4fa;
  background-color: transparent;
  text-decoration: none;
}

a:hover {
  color: #7c90ea;
  background-color: transparent;
  text-decoration: underline;
}

a:active {
  color: white;
  background-color: transparent;
  text-decoration: underline;
}
```

signup.php

```php
<?php
require_once 'includes/config_session.inc.php';  //Allows for session variable regeneration (added security)
require_once 'includes/signup_view.inc.php';  //Allows for signup_inputs() and check_signup_errors() to be accessed
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Registration</title>
    <!--Allows for styling to be accessed-->
    <link rel="stylesheet" href="regstylesheet.css">
</head>

<body>
    <!--This declares the form element and specifies where (signup.inc.php) and using which method the data will be transferred
    (this data is sent using post as it is sensitive data)-->
    <form action="includes/signup.inc.php" method="post">
        <br><br><b><label id="Reg">Sign Up</label></b><br><br><br><br><br>

        <!--Style is individually declared as there are 3 inputs for the signup page compared to the 2 for the login page-->
        <div style="height: 300px;">
            <br>

            <?php
            /*Calls the function that if there is an error with the passwords but the username is not taken,
            the username will stay in the username field (usability feature) otherwise all of the fields will be empty
            so the username doesn't need to retype username if it was valid*/
                signup_inputs();
            ?>

            <input type="submit" id="Submit">
        </div>
    </form>

    <?php
    check_signup_errors(); //Calls the function which checks for any errors in signup such as the username being taken or blank fields

    if(isset($_GET['signup']) && $_GET['signup']==='success'){ //Checks to see if the signup session variable is set and if it was a success
        echo '<p class="form-success"><u>Account created successfully.</u></p>'; //If it was, output a success message
    }

    ?>

    <br><br><br><br><br>
    <!--A redirect link which takes you to the login page-->
    <a href="login.php">Log in</a>
</body>
</html>
```

skills.php

```php
<?php
/*Remove the comment marks and this message to allow for error reporting for debugging

ini_set('display_errors', 1);
error_reporting(E_ALL);*/

require_once 'includes/config_session.inc.php';  //Allows for session variable regeneration (added security)

if (!isset($_SESSION['user_id'])) {  //Prevents unauthorised access from users who have not logged in legitamately (user id is not set)
    header('Location: login.php');
    die();
}

require_once 'includes/dbh.inc.php';  //Allows for database access
require_once 'includes/skills.inc.php';  //Allows for output_skills() to be accessed to create skills textboxes
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Skills</title>

    <script src="skillstextbox.js"></script>

    <link rel="stylesheet" href="skillsstylesheet.css">
</head>

<body>
    <nav>
    <ul>
    <li><a href="calendar.php">Calendar</a></li>
    <li><a href="skills.php">Skills</a></li>
    <li><a href="timeline.php">Timeline</a></li>
    <li><a href="includes/logout.inc.php">Logout</a></li>
    </ul>
    </nav>
    <br>

   <h1 id="skills">Skills</h1>

   <?php
       if(isset($_GET['update']) && $_GET['update']==='success'){
           echo '<p id="form-success"><u>Skills Descriptions successfully stored/updated.</u></p>';
       }
   ?>

   <form action="includes/skills_input.inc.php" method="post">
       <table>
           <tr>
               <td><h2 class="left-title-header">Leadership</h2>
               <?php output_skills($pdo, 'leadership', 'left-title', $_SESSION['user_id']); ?>

               <td><p id="description" class="rightp">In your description, show how you:
                   <br>were given a task, or came up with a task, to lead
                   <br>thought about the leadership style that would work best for the group you're in
                   <br>led the task, adapting your leadership style</p></td>
           </tr>



           <tr>
               <td><p id="description" class="leftp">In your description, show how you:
                   <br>thought about a change that you wanted to make
                   <br>came up with an idea to make the change
                   <br>brought the idea to life and made the change</p></td>
               <td><h2 class="right-title-header">Creativity</h2>

               <?php output_skills($pdo, 'creativity', 'right-title', $_SESSION['user_id']); ?>

           </tr>



           <tr>
               <td><h2 class="left-title-header">Teamwork</h2>
               <?php output_skills($pdo, 'teamwork', 'left-title', $_SESSION['user_id']); ?>
```

```
        <td><p id="description" class="rightp">In your description, show how you:
            <br>worked as part of a group
            <br>completed the tasks you were responsible for
            <br>reflected on how your group work was going
            <br>suggested ideas for improvement</p></td>
    </tr>



    <tr>
        <td><p id="description" class="leftp">In your description, show how you:
            <br>independently came up with an idea for a new long-term project
            <br>created detailed a plan for the project by yourself</p></td>

        <td><h2 class="right-title-header">Initiative</h2>
        <?php output_skills($pdo, 'initiative', 'right-title', $_SESSION['user_id']); ?>
    </tr>



    <tr>
        <td><h2 class="left-title-header">Problem Solving</h2>
        <?php output_skills($pdo, 'problem_solving', 'left-title', $_SESSION['user_id']); ?>

        <td><p id="description" class="rightp">In your description, show how you:
            <br>had a complex problem
            <br>came up with a solution
            <br>evaluated your solution (decided whether it solved the problem effectively or not)</p></td>
    </tr>



    <tr>
        <td><p id="description" class="leftp">In your description, show how you:
            <br>came across a problem
            <br>broke it down into steps
            <br>prepared a plan to tackle each of the steps</p></td>
```

```
        <td><h2 class="right-title-header">Organisation</h2>
        <?php output_skills($pdo, 'organisation', 'right-title', $_SESSION['user_id']); ?>
    </tr>



    <tr>
        <td><h2 class="left-title-header">Communication</h2>
        <?php output_skills($pdo, 'communication', 'left-title', $_SESSION['user_id']); ?>

        <td><p id="description" class="rightp">In your description, show how you:
            <br>explained something in a way that made it easy to understand
            <br>thought about the views of other people whilst you spoke
            <br>used language and tone of voice carefully to make others feel listened to</p></td>
    </tr>



    <tr>
        <td><p id="description" class="leftp">In your description, show how you:
            <br>presented confidently and kept a large audience interested
            <br>stayed calm before and during your presentation
            <br>used eye contact and gestures to keep your audience engaged</p></td>

        <td><h2 class="right-title-header">Public Speaking</h2>
        <?php output_skills($pdo, 'public_speaking', 'right-title', $_SESSION['user_id']); ?>
    </tr>



    <tr>
        <td><h2 class="left-title-header">Perseverance</h2>
        <?php output_skills($pdo, 'perseverance', 'left-title', $_SESSION['user_id']); ?>
```

```
        <td><p id="description" class="rightp">In your description, show how you:
            <br>had a goal
            <br>experienced more than one setback
            <br>achieved your goal, despite those setbacks</p></td>
    </tr>



    <tr>
        <td><p id="description" class="leftp">In your description, show how you:
            <br>were given a strict deadline for a complicated project
            <br>divided the available time effectively
            <br>completed the task to an excellent standard within the time frame</p></td>

        <td><h2 class="right-title-header">Time Management</h2>
        <?php output_skills($pdo, 'time_management', 'right-title', $_SESSION['user_id']); ?>
    </tr>



    <tr>
        <td><h2 class="left-title-header">Planning and Preparation</h2>
        <?php output_skills($pdo, 'planning_and_preparation', 'left-title', $_SESSION['user_id']); ?>

        <td><p id="description" class="rightp">In your description, show how you:
            <br>set an end goal
            <br>created a plan where you could track your progress, e.g. using data like numbers, or milestones
            <br>made a change to the plan when you needed to</p></td>
    </tr>

<tr>
    <td><p id="description" class="leftp">In your description, show how you:
        <br>decided if a piece of writing was successful or not by looking at both what was written (i.e. the language)
        and how it was written (i.e. the techniques the writer used)
        <br>looked at more than one piece of text at the same time
        <br>found similarities and differences between them</p></td>

    <td><h2 class="right-title-header">Analytical Skills</h2>
    <?php output_skills($pdo, 'analytical_skills', 'right-title', $_SESSION['user_id']); ?>
</tr>



<tr>
    <td><h2 class="left-title-header">Practical skills</h2>
    <?php output_skills($pdo, 'practical_skills', 'left-title', $_SESSION['user_id']); ?>

    <td><p id="description" class="rightp">In your description, show how you:
        <br>came up with a new idea
        <br>planned the requirements for a physical prototype
        <br>created the physical prototype
        <br>tried it out to see if it would work</p></td>
</tr>



<tr>
    <td><p id="description" class="leftp">In your description, show how you:
        <br>made sure you fully understood the concept
        <br>thought about the most important aspects of it
        <br>considered the order in which to explain things
        <br>thought about the audience you were writing for</p></td>

    <td><h2 class="right-title-header">Academic Skills</h2>
    <?php output_skills($pdo, 'academic_skills', 'right-title', $_SESSION['user_id']); ?>
</tr>

            <tr>
                <td><h2 class="left-title-header">Computer Skills</h2>
                <?php output_skills($pdo, 'computing_skills', 'left-title', $_SESSION['user_id']); ?>

                <td><p id="description" class="rightp">In your description, show how you:
                    <br>were given a coding task which you tackled using intermediate/high level programming techniques
                    <br>became familiar with emerging computer technologies
                    <br>started learning a new programming language</p></td>
            </tr>
        </table>

        <input type="submit" id="submit">
    </form>
</body>
</html>
```

skillsstylesheet.css

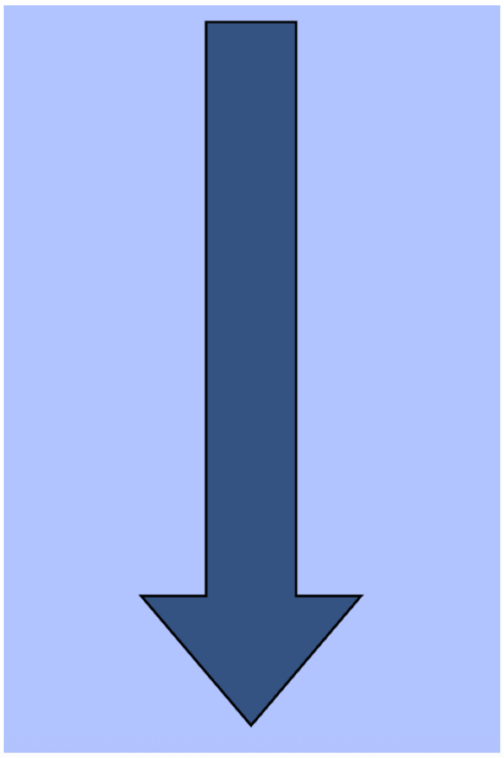
```css
@import url('https://fonts.googleapis.com/css2?family=Hanken+Grotesk:ital,wght@0,100..900;1,100..900&display=swap');
/*Importing correct font (based on Canva design) from Google fonts*/

body{
    color: #0b0e2e;
    background-color: #070a2a;
    font-family: 'Hanken Grotesk', sans-serif;
    text-align: center;
}

p{
    color: #6c718c;
}

table {
    border-collapse: separate;
    border-spacing: 0 10px;
    margin: -10px, 20px, 0px, 20px;
    width: 100%;
}

td {
    border: solid 1px #060a2e;
    border-style: solid none;
    padding: 1px;
    background-color: #b5c4fa;

}
td:first-child {
    border-left-style: solid;
    border-top-left-radius: 10px;
    border-bottom-left-radius: 10px;
}
td:last-child {
    border-right-style: solid;
    border-bottom-right-radius: 10px;
    border-top-right-radius: 10px;
}
#skills{
    color: white;
    font-size: 75px;
}
```

```css
#submit{
    background-color: #7c90ea;
    border-radius: 30px;
    font-size: x-large;
    border-color: #7c90ea;
    border-style: hidden;
    padding: 20px;
}

#form-success{
    color: aliceblue;
    font-size: 20px;
}

.left-title-header{
    font-size: xx-large;
    text-align: left;
    padding-left: 20px;

}
.right-title-header{
    font-size: xx-large;
    text-align: right;
    padding-right: 20px;

}
.left-title{
    font-size: x-large;
    text-align: left;
    padding-left: 20px;
}

.right-title{
    font-size: x-large;
    text-align: right;
    padding-right: 20px;
}

.leftp{
    text-align: left;
    padding-left: 20px;
}
```

```css
.rightp{
    text-align: right;
    padding-right: 20px;
}

.text-area-left{
    float: left;
    margin-left: 20px;
    font-family: 'Hanken Grotesk', sans-serif;
    font-size: large;
}

.text-area-right{
    float: right;
    margin-right: 20px;
    font-family: 'Hanken Grotesk', sans-serif;
    font-size: large;
}

nav{
position: left;
}

nav ul{
list-style: none;
background-color: #070a2a;
overflow: hidden;
margin: 0;
padding:0;
}

nav ul li{
float: left;
}
```

```css
nav ul li a{
display: block;
color: aliceblue;
text-align: center;
padding: 14px 20px;
text-decoration: none;
border-color: black;
}

nav ul li a:hover {
background-color:  #b5c4fa;
}
```

timeline_arrow_new.jpg



timeline.php

```php
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);

require_once 'includes/config_session.inc.php';
require_once 'includes/dbh.inc.php';
require_once 'includes/timeline.inc.php';

if (!isset($_SESSION['user_id'])) { //Prevents unauthorised access from users who have not logged in legitamately
    header('Location: login.php');
    die();
}
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Timeline</title>
    <link rel="stylesheet" href="timelinestylesheet.css">
    <!--<script src="timelinetextbox.js"></script>-->
    <script src="https://code.jquery.com/jquery-1.12.4.js"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/1.3.3/jspdf.min.js"></script>
    <script src="https://html2canvas.hertzen.com/dist/html2canvas.js"></script>

    <script>
        /*Function that converts HTML content into a downloadable PDF*/
        function CreatePDFfromHTML() {
            var HTML_Width = $(".html-content").width();  //Get the width of the HTML element
            var HTML_Height = $(".html-content").height();  //Get the height of the HTML element
            var top_left_margin = 15;  //Margin size for the PDF (gap between the edges of the div to the page of the pdf)
            var PDF_Width = HTML_Width + (top_left_margin * 2);  //Calculate the total PDF width (including margins)

            //Calculate the PDF height (1.5 ratio helps maintain a page-like aspect ratio)
            var PDF_Height = (PDF_Width * 1.5) + (top_left_margin * 2);
            var canvas_image_width = HTML_Width;  //Width of the image that will be placed into the PDF
            var canvas_image_height = HTML_Height;  //Height of the image that will be placed into the PDF
```

```
        //Calculate how many additional PDF pages are needed
        var totalPDFPages = Math.ceil(HTML_Height / PDF_Height) - 1;  //Math.ceil rounds up to ensure all content fits

        html2canvas($(".html-content")[0]).then(function (canvas) {  //html2canvas converts the HTML element into a canvas image
            var imgData = canvas.toDataURL("image/jpeg", 1.0);  //Convert the canvas into a JPEG image
            /*Create a new PDF document using jsPDF
            'p' = portrait orientation
            'pt' = unit of measurement (points)
            [PDF_Width, PDF_Height] = custom page size*/
            var pdf = new jsPDF('p', 'pt', [PDF_Width, PDF_Height]);

            //Add the image of the HTML content to the first page
            pdf.addImage(imgData, 'JPG', top_left_margin, top_left_margin, canvas_image_width, canvas_image_height);

            //Loop to create extra pages if the content is taller than one page
            for (var i = 1; i <= totalPDFPages; i++) {
                pdf.addPage(PDF_Width, PDF_Height);  //Add a new page to the PDF
                //Add the same image but shift it upward so the next section appears
                pdf.addImage(imgData, 'JPG', top_left_margin, -(PDF_Height*i)+(top_left_margin*4),canvas_image_width,canvas_image_height);
            }
            //Save the generated PDF file
            pdf.save("Timeline.pdf");
            //$(".html-content").hide();
        });
    }
    </script>
</head>
<body>

    <nav>
    <ul>
    <li><a href="calendar.php">Calendar</a></li>
    <li><a href="skills.php">Skills</a></li>
    <li><a href="timeline.php">Timeline</a></li>
    <li><a href="includes/logout.inc.php">Logout</a></li>
    </ul>
    </nav>
    <br>

    <h1 id="title">Timeline</h1>

    <?php
        if(isset($_GET['update']) && $_GET['update']==='success'){
            echo '<p id="form-success"><u>Event Reflections successfully stored/updated.</u></p>';
        }
    ?>

    <form action="includes/timeline_input.inc.php" method="post">
        <div id="timeline" class="html-content">
            <br><br>
            <?php
                output_events($pdo, $_SESSION['user_id']);
            ?>
            <br><br>
        </div>

        <div class="button-container">
            <input type="submit" id="submit" value="Submit">
            <button id="download" type="button" onclick="CreatePDFfromHTML()">Save as PDF</button>
        </div>
    </form>


</body>
</html>
```

timelinestylesheet.css

```css
@import url('https://fonts.googleapis.com/css2?family=Hanken+Grotesk:ital,wght@0,100..900;1,100..900&display=swap');
/*Importing correct font (based on Canva design) from Google fonts*/

body{
    color: #0b0e2e;
    background-color: #070a2a;
    font-family: 'Hanken Grotesk', sans-serif;
    text-align: center;
}

#title{
    color: white;
    font-size: 75px;
}

#timeline{
    background-image: url('timeline_arrow_new.jpg');
    background-repeat: no-repeat;
    /*background-attachment: fixed;*/
    background-size: cover;
    border-radius: 25px;
    text-align: left;
    padding-left: 15px;
    padding-right: 15px;
}

.left-title{
    font-size: x-large;
    text-align: left;
    padding-left: 20px;
}

.right-title{
    font-size: x-large;
    text-align: right;
    padding-right: 20px;
}

.left-text-area{
    font-family: 'Hanken Grotesk', sans-serif;
    font-size: large;
    float: left;
    margin-left: 20px;
}

.right-text-area{
    font-family: 'Hanken Grotesk', sans-serif;
    font-size: large;
    float: right;
    margin-right: 20px;
}

#form-success{
    color: aliceblue;
    font-size: 20px;
}

nav{
    position: left;
    font-family: 'Hanken Grotesk', sans-serif;

}

nav ul{
    list-style: none;
    background-color: #070a2a;
    overflow: hidden;
    margin: 0;
    padding:0;
}

nav ul li{
    float: left;
}

nav ul li a{
    display: block;
    color: aliceblue;
    text-align: center;
    padding: 14px 20px;
    text-decoration: none;
    border-color: black;
}

nav ul li a:hover {
    background-color:  #b5c4fa;
}

.button-container {
    display: flex;
    justify-content: space-between;
    margin: 20px 0;
    width: 100%;
}

.button-container input,
.button-container button {
    background-color: #7c90ea;
    border-radius: 30px;
    font-size: x-large;
    border-color: #7c90ea;
    border-style: hidden;
    padding: 20px;
}

.button-container input {
    background-color: #7c90ea;
    color: white;
}

.button-container button {
    color: white;
}
```

‘includes’ subfolder

config_session.inc.php

```php
<?php

ini_set('session.use_only_cookies', 1);
/*Makes sure that the website only uses a session id that has been created by a server in our website
Also makes the session id more complex when created -> difficult for people to guess the session id*/
ini_set('session.use_strict_mode', 1);

session_set_cookie_params([  //Function that allows for changing of cookie attributes
    'lifetime' => 1800,  //Lifetime (seconds)
    'domain' => 'localhost',  //Domain (XAMPP)
    'path' => '/',  //Sub-paths (/ means works on any paths)
    'secure' => true,  //Only use this cookie in HTTPS
    'httponly' => true  //Avoids the user changing anything about the cookie
]);

session_start();

if(isset($_SESSION["user_id"])){  //Checks if user_id was stored as a session variable
    if (!isset($_SESSION['last_regeneration'])) {  //If it was, checks if a session variable wasn't created

        regenerate_session_id_loggedin();  //If not, regenerate the logged in session id

    } else {  //If a session variable was created
        $interval = 60 * 30;  //Time until session id regeneration

        if(time() - $_SESSION['last_regeneration'] >= $interval){  //Check if more than 30 mins have passed

            regenerate_session_id_loggedin();  //Regenerate a logged in session id
        }
    }
}else{
    if (!isset($_SESSION['last_regeneration'])) {  //If it wasn't, checks if a session variable wasn't created

        regenerate_session_id();  //If not, regenerate session id

    } else {
        $interval = 60 * 30;  //Time until session id regeneration

        if(time() - $_SESSION['last_regeneration'] >= $interval){  //Check if more than 30 mins have passed

            regenerate_session_id();  //Regenerate session id

        }
    }
}



function regenerate_session_id(){
    session_regenerate_id(true); //Regenerate session id

    $_SESSION['last_regeneration'] = time(); //Reset last_regeneration variable
}

function regenerate_session_id_loggedin(){
    session_regenerate_id(true); //Regenerate session id

    $userId = $_SESSION["user_id"];  //Takes the user_id session variable
    $newSessionId = session_create_id();  //Creates a new session id
    $sessionId = $newSessionId . "_" . $userId;  //Concatenate user_id to session variable (creating a more complex compound session id)
    session_id($sessionId);  //Set this as the session id

    $_SESSION['last_regeneration'] = time(); //Reset last_regeneration variable
}
```

dbh.inc.php

```php
<?php

//Host type (XAMPP is a localhost)
$host = 'localhost';
$dbname = 'NEA';
//Default credentials using XAMPP
$dbusername = 'root';
$dbpassword = '';

try {  //Tries connecting to the database using the host type, database name and database credentials
    $pdo = new PDO("mysql:host=$host;dbname=$dbname", $dbusername, $dbpassword);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {  //If an error occured
    die("Connection failed: " . $e->getMessage());  //End the program and display a 'Connection failed: ' message with the error message
}
```

inputform_input.inc.php

```php
<?php
//Show errors temporarily for debugging
/*ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
error_reporting(E_ALL);*/

require_once 'dbh.inc.php';  //Allows for database linkage
require_once 'config_session.inc.php';  //Allows for session variable regeneration (added security)

//Check user is logged in
if (!isset($_SESSION['user_id'])) {
    header("Location: ../login.php");
    exit();
}
$userId = $_SESSION['user_id'];  //User id is stored

//Get date from GET and convert to MySQL format
if (isset($_GET['date'])) {  //If a date was set (page accessed legitamately)
    $timestamp = strtotime($_GET['date']);  //It is converted into a timestamp data structure
    if ($timestamp !== false) {  //If the date was correctly formatted
        $date = date('Y-m-d', $timestamp); //Date is stored in the MySQL format
    } else {  //Otherwise
        die("Invalid date format");  //End the program and output an error
    }
} else {  //Otherwise
    header("Location: ../calendar.php");  //Redirect back to the calendar page
    exit();
}

//Get form data safely
//If the event name is set, use it, otherwise use NULL
$eventName = isset($_POST['eventname']) ? $_POST['eventname'] : NULL;
//If the event description is set, use it, otherwise use NULL
$eventDescription = isset($_POST['event_description']) ? $_POST['event_description'] : NULL;
//If the tag is set, use it, otherwise use NULL
$tag = isset($_POST['tag']) ? $_POST['tag'] : NULL;

//Validate required field
if (empty($eventName)) {
    //Redirect back to the form with an error message
    header("Location: ../inputform.php?error=missing_event_name&date=" . urlencode($_GET['date']));  //URL encodes the date
```

```
    exit();
}

//Skills (checkboxes values will be 1 if checked, 0 if not)
$skills = [
    'leadership' => isset($_POST['leadership']) ? 1 : NULL,
    'creativity' => isset($_POST['creativity']) ? 1 : NULL,
    'teamwork' => isset($_POST['teamwork']) ? 1 : NULL,
    'initiative' => isset($_POST['initiative']) ? 1 : NULL,
    'problem_solving' => isset($_POST['problem_solving']) ? 1 : NULL,
    'organisation' => isset($_POST['organisation']) ? 1 : NULL,
    'communication' => isset($_POST['communication']) ? 1 : NULL,
    'public_speaking' => isset($_POST['public_speaking']) ? 1 : NULL,
    'perseverance' => isset($_POST['perseverance']) ? 1 : NULL,
    'time_management' => isset($_POST['time_management']) ? 1 : NULL,
    'planning_and_preparation' => isset($_POST['planning_and_preparation']) ? 1 : NULL,
    'analytical_skills' => isset($_POST['analytical_skills']) ? 1 : NULL,
    'practical_skills' => isset($_POST['practical_skills']) ? 1 : NULL,
    'academic_skills' => isset($_POST['academic_skills']) ? 1 : NULL,
    'computing_skills' => isset($_POST['computing_skills']) ? 1 : NULL
];

//Check if event already exists for this user & date
$sql = "SELECT EVENTS.event_id, LINK_TABLE.skills_id
        FROM EVENTS
        INNER JOIN LINK_TABLE ON EVENTS.event_id = LINK_TABLE.event_id
        WHERE EVENTS.date_of_event = ?
        AND LINK_TABLE.user_id = ?
        LIMIT 1
";

$stmt = $pdo->prepare($sql);  //Query is prepared
$stmt->execute([$date, $userId]);  //Date and user id parameters are bound and the statement is executed
$result = $stmt->fetch(PDO::FETCH_ASSOC);  //All of the data is fetched and stored as result

if ($result) {  //If a result was fetched
    //Update existing event data
    $updateEvent = $pdo->prepare("UPDATE EVENTS SET name_of_event = ?, event_description = ?, event_tag = ? WHERE event_id = ?
    ");
    $updateEvent->execute([$eventName, $eventDescription, $tag, $result['event_id']]);  //Parameters are bound and the statement is executed
```

```
    //Update existing skills data
    $updateSkills = $pdo->prepare("UPDATE SKILLS SET
            leadership = ?, creativity = ?, teamwork = ?, initiative = ?, problem_solving = ?,
            organisation = ?, communication = ?, public_speaking = ?, perseverance = ?,
            time_management = ?, planning_and_preparation = ?, analytical_skills = ?,
            practical_skills = ?, academic_skills = ?, computing_skills = ?
        WHERE skills_id = ?
    ");
    $updateSkills->execute([  //Parameters are bound and the statement is executed
        $skills['leadership'], $skills['creativity'], $skills['teamwork'], $skills['initiative'], $skills['problem_solving'],
        $skills['organisation'], $skills['communication'], $skills['public_speaking'], $skills['perseverance'],
        $skills['time_management'], $skills['planning_and_preparation'], $skills['analytical_skills'],
        $skills['practical_skills'], $skills['academic_skills'], $skills['computing_skills'],
        $result['skills_id']
    ]);
} else {  //If no result was fetched
    //Insert event into database
    $insertEvent = $pdo->prepare("INSERT INTO EVENTS (date_of_event, name_of_event, event_description, event_tag) VALUES (?, ?, ?, ?)");
    $insertEvent->execute([$date, $eventName, $eventDescription, $tag]);  //Parameters are bound and the statement is executed
    $eventId = $pdo->lastInsertId();  //The event id of the last insertion is retrieved

    //Insert empty SKILLS_DESCRIPTION row
    $insertDesc = $pdo->prepare("INSERT INTO SKILLS_DESCRIPTION () VALUES ()"); //Empty row (values can be added later)
    $insertDesc->execute();  //Statement is executed
    $descId = $pdo->lastInsertId();  //The description id of the last insertion is retrieved

    //Insert skills values into database
    $insertSkills = $pdo->prepare("INSERT INTO SKILLS (leadership, creativity, teamwork, initiative, problem_solving, organisation,
    communication, public_speaking, perseverance, time_management, planning_and_preparation, analytical_skills, practical_skills,
    academic_skills, computing_skills) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)");
    $insertSkills->execute(array_values($skills));  //Parameters are bound and the statement is executed
    $skillsId = $pdo->lastInsertId();  //The skills id of the last insertion is retrieved

    //Link table is updated
    $insertLink = $pdo->prepare("INSERT INTO LINK_TABLE (user_id, event_id, skills_id, description_id) VALUES (?, ?, ?, ?)");
    $insertLink->execute([$userId, $eventId, $skillsId, $descId]);  //Parameters are bound and the statement is executed

}

//Redirect after success
header("Location: ../calendar.php?save=success");
exit();
```

inputformhandler.inc.php

```php
<?php
require_once 'dbh.inc.php';  //Allows for database linkage
require_once 'config_session.inc.php';  //Allows for session variable regeneration (added security)

//Check user is logged in
if (!isset($_SESSION['user_id'])) {  //If the user id is not set
    header("Location: ../login.php");  //Redirect back to the login page
    exit();
}

$userId = $_SESSION['user_id'];  //Otherwise, retrieve user id

//Get date value from the url
if (isset($_GET['date'])) {  //If the date is set
    $date = $_GET['date'];  //Retrieve it as a variable date
} else {
    $date = '';  //Otherwise use an empty string
}

//Default values of input fields
$eventName = '';
$eventDescription = '';
$eventTag = '';

$leadership = 0;
$creativity = 0;
$teamwork = 0;
$initiative = 0;
$problem_solving = 0;
$organisation = 0;
$communication = 0;
$public_speaking = 0;
$perseverance = 0;
$time_management = 0;
$planning_and_preparation = 0;
$analytical_skills = 0;
$practical_skills = 0;
$academic_skills = 0;
$computing_skills = 0;
```

```php
/*Load event if it exists*/
if ($date != '') {  //If the date has been set

    //Retrieve the values from the database (limit 1 prevents accidental data collision)
    $sql = "SELECT EVENTS.name_of_event, EVENTS.event_description, EVENTS.event_tag, SKILLS.*
            FROM EVENTS
            INNER JOIN LINK_TABLE ON EVENTS.event_id = LINK_TABLE.event_id
            INNER JOIN SKILLS ON LINK_TABLE.skills_id = SKILLS.skills_id
            WHERE EVENTS.date_of_event = ?
            AND LINK_TABLE.user_id = ?
            LIMIT 1
    ";

    $stmt = $pdo->prepare($sql);  //The statement is prepared
    $stmt->execute([$date, $userId]);  //The parameters are passed and statement is executed
    $result = $stmt->fetch(PDO::FETCH_ASSOC);  //The retrieved values are stored as result

    if ($result) {  //If there was a result in the database
        //Set the parameters to the database values
        $eventName = $result['name_of_event'];
        $eventDescription = $result['event_description'];
        $eventTag = $result['event_tag'];

        $leadership = $result['leadership'];
        $creativity = $result['creativity'];
        $teamwork = $result['teamwork'];
        $initiative = $result['initiative'];
        $problem_solving = $result['problem_solving'];
        $organisation = $result['organisation'];
        $communication = $result['communication'];
        $public_speaking = $result['public_speaking'];
        $perseverance = $result['perseverance'];
        $time_management = $result['time_management'];
        $planning_and_preparation = $result['planning_and_preparation'];
        $analytical_skills = $result['analytical_skills'];
        $practical_skills = $result['practical_skills'];
        $academic_skills = $result['academic_skills'];
        $computing_skills = $result['computing_skills'];
    }
}
```

login_contr.inc.php

```php
<?php

declare(strict_types=1);  //Best practice to prevent errors in data types

function is_input_empty(string $username, string $password){
    if(empty($username) || empty($password)){  //Checks if either of the input fields are empty
        return true;
    } else {
        return false;
    }
}

function is_username_wrong(bool|array $result){  //To prevent data type errors if the result is false
    if(!$result){  //If there isn't a result
        return true;
    }else {
        return false;
    }
}

function is_password_wrong(string $password, string $hashed_password){
//Built in php function to check if an input variable is the same as another hashed variable
    if(!password_verify($password, $hashed_password)){  //If they are not the same
        return true;
    }else {
        return false;
    }
}
```

login_model.inc.php

```php
<?php

declare(strict_types=1);  //Best practice to prevent errors in data types

function get_user(object $pdo, string $username){
    //Returns the user_id, username and hashed_password where the input username matches the stored username
    $query = "SELECT * FROM USERS WHERE username = :username;";
    //Using prepared statements is best practice to avoid errors and to enforce security when accessing the database
    $stmt = $pdo->prepare($query);  //Query is prepared
    $stmt->bindParam(":username", $username);  //Username parameter is bound
    $stmt->execute();  //Statement is executed

    $result = $stmt->fetch(PDO::FETCH_ASSOC);  //The username is fetched from the database in an associative array
    return $result;  //The array containing the data is returned
}
```

login_view.inc.php

```php
<?php

declare(strict_types=1);  //Best practice to prevent errors in data types

function output_username(){
    if(isset($_SESSION["user_id"])){  //Checks if the user id is set
        echo "<p class='login-info'><b>Hello " . $_SESSION['user_username'] . "!</b></p>";  //If it is, output a hello message
    }else{
        echo "<p class='login-info'>You are not logged in</p>";  //If it isn't, say you are not logged in
    }
}

function check_login_errors(){
    if(isset($_SESSION["error_login"])){  //If errors are stored as a session variable
        $errors = $_SESSION["error_login"];  //Create a local variable to hold the errors

        echo "<br>";  //Start a new line

        foreach($errors as $error){  //For every error in the array
            echo '<p class="form-error"><b><u>' . $error . '</u></b></p>';  //Output the error
        }

        unset($_SESSION["error_login"]);  //Unset the session variable

    }
    else if(isset($_GET['login']) && $_GET['login'] === "success"){  //If the user logged in successfully
        echo '<br>';
        echo '<p class="form-success"><b><u>Login success</u></b></p>';  //Output a success message
    }
}
```

login.inc.php

```php
<?php

if($_SERVER["REQUEST_METHOD"] === "POST"){  //Check if page was accessed legitamately
    $username = $_POST["username"];  //Input username
    $password = $_POST["password"];  //Input password

    try {
        require_once 'dbh.inc.php';  //Allows for database linkage
        require_once 'login_model.inc.php';  //Allows for the results of fetching from the database to be accessed by this file
        require_once 'login_contr.inc.php';  //Allows for error-checking functions to be accessed by this file


        $errors = [];  //Storing any errors found in an array

        if (is_input_empty($username, $password)){  //Checks to see if any inputs are empty
         //If they are, create a key-pair value to store the "empty_input" error and the error message of "Fill in all fields"
            $errors["empty_input"] = "Fill in all fields";
        }

        $result = get_user($pdo, $username);  //Store the reasults of querying the database for user credentials in the result variable

        if(is_username_wrong($result)){  //Checks to see if the username was not found in the database
        //If it wasn't, create a key-pair value to store the "login_incorrect" error and the error message of "Incorrect login info"
            $errors["login_incorrect"] = "Incorrect login info";
        }

        //Checks to see if the username was found and if the input password doesn't match the stored password
        if(!is_username_wrong($result) && is_password_wrong($password, $result["hashed_password"])){
        //If it wasn't, create a key-pair value to store the "login_incorrect" error and the error message of "Incorrect login info"
            $errors["login_incorrect"] = "Incorrect login info";
        }

        require_once 'config_session.inc.php';  //Allows for session variables to be created/regenerated

        if($errors){  //If errors are present
            $_SESSION["error_login"] = $errors;  //Store them as a session variable

            header("Location: ../login.php");  //Redirect back to the login page
            die();  //End the program
        }
```

```
        $newSessionId = session_create_id();  //Generates a new session id
        $sessionId = $newSessionId . "_" . $result["user_id"];  //Concatenates the session id with the user id (increased security)
        session_id($sessionId);

        $_SESSION["user_id"] = $result["user_id"];  //The user id is stored as a session variable
        $_SESSION["user_username"] = htmlspecialchars($result["username"]);  //The username is stored as a session variable

        $_SESSION['last_regeneration'] = time();  //The last regeneration time is set to the current time

        header("Location: ../calendar.php?login=success");  //Redirect to the calendar page with 'success'
        $pdo = null;  //Unset database variables (best practice)
        $stmt = null;  //Unset database variables (best practice)
        die();  //
    } catch (PDOException $e) {  //If it fails, get the error
    die("Query failed: " . $e->getMessage());  //End the program and output the error
}
}
else{
    header("Location: ../login.php");  //If accessed incorrectly, redirect to the login page
    die();  //End the program
}
```

logout.inc.php

```
<?php

session_start();  //Session is started
session_unset();  //Session variables are unset
session_destroy();  //The session is ended
header("Location: ../login.php");  //The program is redirected to the login page
die();  //The code ends
```

signup_contr.inc.php

```
<?php

declare(strict_types=1);  //Best practice to prevent errors in data types

function is_input_empty(string $username, string $password, string $cpassword){
    if (empty($username) || empty($password) || empty($cpassword)){  //Checks if any input fields are empty
        return true;
    } else {
        return false;
    }
}

function is_password_short(string $password){
    if (strlen($password) > 12 && strlen($password) < 255){  //Checks if password is within the length requirement
        return false;
    } else {
        return true;
    }
}

function do_passwords_match(string $password, string $cpassword){
    if ($password === $cpassword){  //Checks if both passwords match
        return true;
    } else {
        return false;
    }
}

/*The $pdo object is passed to all functions that interact with the database as it allows for database connection*/
function is_username_taken(object $pdo, string $username){
    if (get_username($pdo, $username)){  //Checks if the get_username() function from signup_model.inc.php returned an output
        return true;
    } else {
        return false;
    }
}

function create_user(object $pdo, string $username, string $password){
    set_user($pdo, $username, $password);  //Calls the set_user() function from signup_model.inc.php and inputs the user data into the database
}
```

signup_model.inc.php

```php
<?php

declare(strict_types=1);  //Best practice to prevent errors in data types

function get_username(object $pdo, string $username){
    //Using prepared statements is best practice to avoid errors and to enforce security when accessing the database
    $query = "SELECT username FROM USERS WHERE username = :username;";
    $stmt = $pdo->prepare($query);  //Query is prepared
    $stmt->bindParam(":username", $username);  //Username parameter is bound
    $stmt->execute();  //Statement is executed

    $result = $stmt->fetch(PDO::FETCH_ASSOC);  //The username is fetched from the database in an associative array
    return $result;  //The array containing the username is returned
}

function set_user(object $pdo, string $username, string $password){
    //Using prepared statements is best practice to avoid errors and to enforce security when accessing the database
    $query = "INSERT INTO USERS (username, hashed_password) VALUES (:username, :hashed_password);";
    $stmt = $pdo->prepare($query);  //Query is prepared

    $options = [
        'cost' => 12  //Time taken to load to prevent hackers from brute-forcing the signup system
    ];

    $hashed_password = password_hash($password, PASSWORD_BCRYPT, $options);  //Hashing the password with the built-in hashing function in PHP

    $stmt->bindParam(":username", $username);  //Username parameter is bound
    $stmt->bindParam(":hashed_password", $hashed_password);  //Hashed password parameter is bound
    $stmt->execute();  //Statement is executed
}
```

signup_view.inc.php

```php
<?php

declare(strict_types=1);  //Best practice to prevent errors in data types

function signup_inputs(){
    //If a username was input that wasn't already taken
    if (isset($_SESSION["signup_data"]["username"]) && !isset($_SESSION["error_signup"]["username_taken"])){
        echo '<label>Enter your username   </label>';  //Output the label
        //Output the username input field with the username in it (usability)
        echo '<input type="text" name="username" id="username" value="' . $_SESSION["signup_data"]["username"] . '"><br><br><br>';

    } else{  //Otherwise
        echo '<label>Enter your username   </label>';  //Output the label
        echo '<input type="text" name="username" id="username"><br><br><br>';  //Output the username empty input field

    }

    echo '<label>Enter your password   </label>';  //Output the label
    echo '<input type="password" name="password" id="password"><br><br><br>';  //Output the password input field
    echo '<label>Confirm your password   </label>';  //Output the label
    echo '<input type="password" name="cpassword" id="cpassword"><br><br><br>';  //Output the confirm password input field
}

function check_signup_errors(){
    if(isset($_SESSION["error_signup"])){  //If errors are stored as a session variable
        $errors = $_SESSION["error_signup"];  //Create a local variable to hold the errors

        echo "<br>";  //Start a new line

        foreach($errors as $error){  //For every error in the array
            echo '<p class="form-error"><b><u>' . $error . '</u></b></p>';  //Output the error
        }

        unset($_SESSION["error_signup"]);  //Unset the session variable
    }
}
```

signup.inc.php

```php
<?php

if($_SERVER["REQUEST_METHOD"] === "POST"){  //Check if page was accessed legitamately

    $username = $_POST["username"];  //Input username
    $password = $_POST["password"];  //Input password
    $cpassword = $_POST["cpassword"];  //Input confirmation password

    try{

    require_once 'dbh.inc.php';  //Allows for database linkage
    require_once 'signup_model.inc.php';  //Allows for the results of fetching from the database to be accessed by this file
    require_once 'signup_contr.inc.php';  //Allows for error-checking functions to be accessed by this file


    $errors = [];  //Storing any errors found in an array

    if (is_input_empty($username, $password, $cpassword)){  //Checks to see if any inputs are empty
    //If they are, create a key-pair value to store the "empty_input" error and the error message of "Fill in all fields"
        $errors["empty_input"] = "Fill in all fields";
    }
    if (is_password_short($password)){  //Checks to see if the password is too short
    //If it is, create a key-pair value to store the "short_password" error and the error message of "Password is too short"
        $errors["short_password"] = "Password is too short";
    }
    if (!do_passwords_match($password, $cpassword)){  //Checks to see if password and cpassword don't match
    //If they are, create a key-pair value to store the "invalid_password" error and the error message of "Passwords do not match"
        $errors["invalid_password"] = "Passwords do not match";
    }
    if (is_username_taken($pdo, $username)){  //Checks to see if the username is already taken
    //If they are, create a key-pair value to store the "username_taken" error and the error message of "Username already taken"
        $errors["username_taken"] = "Username already taken";
    }

    require_once 'config_session.inc.php';  //Allows for session variables to be created/regenerated

    if($errors){  //If errors are present
        $_SESSION["error_signup"] = $errors;  //Store them as a session variable

        $signupData = [  //Create an associative array called signupData to store the username
            "username" => $username
        ];

        $_SESSION["signup_data"] = $signupData;  //Store it as a session variable


        header("Location: ../signup.php");  //Redirect back to the signup page
        die();  //End the program
    }

    create_user($pdo, $username, $password);  //Calls the create user function from signup_contr.inc.php

    header("Location: ../signup.php?signup=success");  //Redirect to the signup page with 'success'

    $pdo = null;  //Unset database variables (best practice)
    $stmt = null;  //Unset database variables (best practice)

    die();  //End the program
    } catch (PDOException $e) {  //If it fails, get the error
    die("Query failed: " . $e->getMessage());  //End the program and output the error
}

}else {
    header("Location: ../signup.php");  //If accessed incorrectly, redirect to the signup page
    die();  //End the program
}
```

skills_input.inc.php

```php
<?php
declare(strict_types=1);  //Best practice to prevent errors in data types

require_once 'dbh.inc.php';  //Allows for database linkage
require_once 'config_session.inc.php';  //Allows for session variables to be created/regenerated

//Ensure user is logged in
if (!isset($_SESSION['user_id'])) {
    header('Location: ../login.php');
    exit();
}

$userId = $_SESSION['user_id'];  //Retrieve the user id from the session variable

if ($_SERVER['REQUEST_METHOD'] === 'POST') {  //If the page was correctly accessed

    //Loop through submitted textareas
    foreach ($_POST as $textareaName => $descriptionText) {

    if (substr($textareaName, 0, 6) === 'event_') {  //If the textareaname begins with event_

        $withoutPrefix = substr($textareaName, 6);  //Remove event_ from the name
        $firstUnderscore = strpos($withoutPrefix, '_');  //Stores the index position of the first underscore in the text

        if ($firstUnderscore === false) {  //If the underscore wasn't found
            continue;  //The input is skipped as the format is invalid (validation)
        }

        $eventId = (int)substr($withoutPrefix, 0, $firstUnderscore);  //Event id is retrieved from the name without prefix
        $skill = substr($withoutPrefix, $firstUnderscore + 1);  //The skill name is retrieved

        //If the descriptionText is already set, use it, otherwise use "" (ternary operator)
        $descriptionClean = isset($descriptionText) ? $descriptionText : "";

        $descColumn = "SKILLS_DESCRIPTION.{$skill}_description";  //The column is specified based on skill

        //Updates the skills description for the logged in user for the specific event id
        $query = "UPDATE EVENTS
                  INNER JOIN LINK_TABLE ON EVENTS.event_id = LINK_TABLE.event_id
                  INNER JOIN SKILLS_DESCRIPTION ON SKILLS_DESCRIPTION.description_id = LINK_TABLE.description_id
                  SET $descColumn = :skill_description

                  WHERE EVENTS.event_id = :event_id
                  AND LINK_TABLE.user_id = :user_id";

        $stmt = $pdo->prepare($query);  //Query is prepared
        $stmt->bindParam(':skill_description', $descriptionClean, PDO::PARAM_STR);  //Skills description parameter is bound
        $stmt->bindParam(':event_id', $eventId, PDO::PARAM_INT);  //Event id parameter is bound
        $stmt->bindParam(':user_id', $userId, PDO::PARAM_INT);  //User id parameter is bound
        $stmt->execute();  //Statement is executed
    }
}


    //Redirect back to skills page after update
    header('Location: ../skills.php?update=success');
    exit();
} else {  //Otherwise
    header('Location: ../skills.php');  //Redirect back to the skills page
    exit();
}
```

skills.inc.php

```php
<?php

declare(strict_types=1);  //Best practice to prevent errors in data types

require_once 'dbh.inc.php';  //Allows for database linkage

    function get_data(object $pdo, string $skill, int $userId): array {  //Function returns an array therefore it has been declared
    //Allows for data for the specified skill to be returned
    $skillColumn = "SKILLS.$skill";
    $descColumn = "SKILLS_DESCRIPTION.{$skill}_description";

    //Returns the event_id, name_of_event, skill value and skill description for the logged in user in ascending order
    $query = "SELECT EVENTS.event_id, EVENTS.name_of_event, $skillColumn AS skill, $descColumn AS skill_description
              FROM EVENTS
              INNER JOIN LINK_TABLE ON EVENTS.event_id = LINK_TABLE.event_id
              INNER JOIN SKILLS ON LINK_TABLE.skills_id = SKILLS.skills_id
              LEFT JOIN SKILLS_DESCRIPTION ON LINK_TABLE.description_id = SKILLS_DESCRIPTION.description_id
              WHERE LINK_TABLE.user_id = :userid
              ORDER BY EVENTS.date_of_event ASC;";

    $stmt = $pdo->prepare($query);  //Query is prepared
    $stmt->bindParam(':userid', $userId, PDO::PARAM_INT);  //User id parameter is bound
    $stmt->execute();  //Statement is executed

    return $stmt->fetchAll(PDO::FETCH_ASSOC);  //All of the data is fetched and returned
}


function create_text_areas(array $results, string $skill, string $class){
    foreach ($results as $result) {  //For each returned event data from the database

        if ($result['skill'] == 1 && $class == 'left-title') {  //If the skill has been set and if the title should be aligned to the left

            if ($result['skill_description'] === NULL){  //If there is no skill description set
                $output = '';  //Output a blank string
            } else {
                $output = htmlspecialchars($result['skill_description'], ENT_QUOTES, 'UTF-8');  //If there is, output a cleaned version of it
            }


            //The name of event is output with the specified class
            echo '<h3 class="' . $class . '">' . htmlspecialchars($result['name_of_event'], ENT_QUOTES, 'UTF-8') . '</h3><br>';
            //The text area is output with the left align class, a name containing the event id and skill name, and the value of the output variable
            echo '<textarea class="text-area-left" name="event_' . (int)$result['event_id'] ."_". $skill . '">'. $output . '</textarea><br><br><br>';

        }
        elseif ($result['skill'] == 1 && $class == 'right-title') {  //If the skill has been set and if the title should be aligned to the right

            if ($result['skill_description'] === NULL){  //If there is no skill description set
                $output = '';  //Output a blank string
            } else {
                $output = htmlspecialchars($result['skill_description'], ENT_QUOTES, 'UTF-8');  //If there is, output a cleaned version of it
            }

            //The name of event is output with the specified class
            echo '<h3 class="' . $class . '">' . htmlspecialchars($result['name_of_event'], ENT_QUOTES, 'UTF-8') . '</h3><br>';
            //The text area is output with the right align class, a name containing the event id and skill name, and the value of the output variable
            echo '<textarea class="text-area-right" name="event_' . (int)$result['event_id'] ."_". $skill . '">'. $output . '</textarea><br><br><br>';
        }
    }
}

//This function combines both functions into one that can be called in the skills page
function output_skills(object $pdo, string $skill, string $class, int $userId){
    $results = get_data($pdo, $skill, $userId);  //The return value of the get_data function is stored as results

    create_text_areas($results, $skill, $class);  //The data is passed to the create_text_area function
}
```

timeline_input.inc.php

```php
<?php
declare(strict_types=1);  //Best practice to prevent errors in data types

require_once 'dbh.inc.php';  //Allows for database linkage
require_once 'config_session.inc.php';  //Allows for session variables to be created/regenerated

//Ensure user is logged in
if (!isset($_SESSION['user_id'])) {
    header('Location: ../login.php');
    exit();
}

$userId = $_SESSION['user_id'];  //Retrieve the user id from the session variable

if ($_SERVER['REQUEST_METHOD'] === 'POST') {  //If the page was correctly accessed

    //Loop through submitted textareas
    foreach ($_POST as $textareaName => $reflectionText) {

        if (substr($textareaName, 0, 6) === 'event_') {  //If the textareaname begins with event_

            $eventId = (int) str_replace('event_', '', $textareaName);  //Event id is retrieved from the name without prefix
            $reflectionClean = htmlspecialchars($reflectionText, ENT_QUOTES, 'UTF-8');  //Cleaned reflection is retrieved

            //Updates the event reflection for the logged in user for the specific event id
            $query = "UPDATE EVENTS
                    INNER JOIN LINK_TABLE
                    ON EVENTS.event_id = LINK_TABLE.event_id
                    SET EVENTS.event_reflection = :reflection
                    WHERE EVENTS.event_id = :event_id
                    AND LINK_TABLE.user_id = :user_id";

            $stmt = $pdo->prepare($query);  //Query is prepared
            $stmt->bindParam(':reflection', $reflectionClean, PDO::PARAM_STR);  //Event reflection parameter is bound
            $stmt->bindParam(':event_id', $eventId, PDO::PARAM_INT);  //Event id parameter is bound
            $stmt->bindParam(':user_id', $userId, PDO::PARAM_INT);  //User id parameter is bound
            $stmt->execute();  //Statement is executed
        }
    }

    //Redirect back to timeline after update
    header('Location: ../timeline.php?update=success');
    exit();
} else {  //Otherwise
    header('Location: ../timeline.php');  //Redirect back to the timeline page
    exit();
}
```

timeline.inc.php

```php
<?php

declare(strict_types=1);  //Best practice to prevent errors in data types

require_once 'dbh.inc.php';  //Allows for database linkage

function get_data(object $pdo, int $userId): array {  //Function returns an array therefore it has been declared

    //Returns the event_id, date_of_event, name_of_event and event_reflection for the logged in user in ascending order
    $query = "SELECT EVENTS.event_id, EVENTS.date_of_event, EVENTS.name_of_event, EVENTS.event_reflection FROM EVENTS
    INNER JOIN LINK_TABLE ON EVENTS.event_id = LINK_TABLE.event_id
    INNER JOIN USERS ON LINK_TABLE.user_id = USERS.user_id
    WHERE USERS.user_id = :user_id
    ORDER BY EVENTS.date_of_event ASC";

    $stmt = $pdo->prepare($query);  //Query is prepared
    $stmt->bindParam(':user_id', $userId, PDO::PARAM_INT);  //User id parameter is bound
    $stmt->execute();  //Statement is executed

    return $stmt->fetchAll(PDO::FETCH_ASSOC);  //All of the data is fetched and returned
}



function create_text_areas(array $results){
    $i = 0;
    foreach ($results as $result) {  //For each returned event data from the database
        if ($i % 2 === 0) {  //If i is even
            $titleClass = 'left-title';  //The title is aligned to the left
            $textClass = 'left-text-area';  //The text area is aligned to the left
        } else {
            $titleClass = 'right-title';  //The title is aligned to the right
            $textClass = 'right-text-area';  //The text area is aligned to the right
        }

        // Output event name and date safely
        //The name of event is output with the specified class
        echo '<h3 class="' . $titleClass . '">' . htmlspecialchars($result['name_of_event']) . '</h3>';
        //The date of event is output with the specified class
        echo '<h3 class="' . $titleClass . '">' . htmlspecialchars($result['date_of_event']) . '</h3><br>';
```

```php
        // Textarea with name = event_EVENTID, contains sanitized reflection
        //If there is a reflection, output the cleaned value, otherwise output ''
        $eventReflection = htmlspecialchars(isset($result['event_reflection']) ? $result['event_reflection'] : '');
        //The text area is output with the correct class, a name containing the event id, and the value of the eventReflection variable
        echo '<textarea class="' . $textClass . '" name="event_' . (int)$result['event_id'] . '">' . $eventReflection . '</textarea><br><br><br>';

        $i++; //i is incremented
    }
}

//This function combines both functions into one that can be called in the timeline page
function output_events(object $pdo, int $userId) {
    $results = get_data($pdo, $userId);  //The return value of the get_data function is stored as results

    create_text_areas($results);  //The data is passed to the create_text_areas function
}
```

Bibliography

Skills Log. *Student Features*. [online] Available at: https://skillslog.com.au/student-features/ [Accessed 29th Jul. 2025].

Skills-Tracker. *Students*. [online] Available at: https://www.skills-tracker.com/students/ [Accessed 29th Jul. 2025].

Unifrog. *Add an example of a skill*. [online] Available at: https://www.unifrog.org/student/skills [Accessed 30th Jul. 2025].

Google Calendar. *Calendar*. [online] Available at: https://calendar.google.com/calendar/ [Accessed 30th Jul. 2025].

Wikipedia. *XAMPP*. [online] Available at: https://en.wikipedia.org/wiki/XAMPP [Accessed 10th Aug. 2025].

Apache Friends. *Download*. [online] Available at: https://www.apachefriends.org/download.html [Accessed 10th Aug. 2025].

Dani Krossing. *1 | Introduction to PHP Programming for Beginners | 2023 | Learn PHP Full Course for Beginners*. [online] www.youtube.com Available at: https://www.youtube.com/watch?v=m52ljs78S24&list=PL0eyrZgxdwhwwQQZA79OzYwl5ewA7HQih [Accessed 2nd Oct. 2025].

Dani Krossing. *28 | Let's Create A Signup System in PHP! | 2023 | Learn PHP Full Course for Beginners*. [online] www.youtube.com Available at: https://www.youtube.com/watch?v=Ojk70Ag8Ofs&list=PL0eyrZgxdwhwwQQZA79OzYwl5ewA7HQih&index=28 [Accessed 5th Oct. 2025].

Dani Krossing. *29 | Let's Create A Login System in PHP! | 2023 | Learn PHP Full Course for Beginners*. [online] www.youtube.com Available at: https://www.youtube.com/watch?v=XDz9SMYyTQo&list=PL0eyrZgxdwhwwQQZA79OzYwl5ewA7HQih&index=29 [Accessed 7th Oct. 2025].

W3Schools. *HTML <input type="hidden">*. [online] Available at: https://www.w3schools.com/tags/att_input_type_hidden.asp [Accessed 8th Oct. 2025].

Stack Overflow. *Download a div in a HTML page as pdf using javascript*. [online] Available at: https://stackoverflow.com/questions/17293135/download-a-div-in-a-html-page-as-pdf-using-javascript [Accessed 12th Oct. 2025].

Auth0. *NIST Password Guidelines and Best Practices for 2020*. [online] Available at: https://auth0.com/blog/dont-pass-on-the-new-nist-password-guidelines/ [Accessed 19th Oct. 2025].

Security.org. *How Secure Is My Password?* [online] Available at: https://www.security.org/how-secure-is-my-password/ [Accessed 19th Oct. 2025].

Microsoft. *SQL Server security best practices*. [online] Available at: https://learn.microsoft.com/en-us/sql/relational-databases/security/sql-server-security-best-practices?view=sql-server-ver17 [Accessed 20th Oct. 2025].

Bro Code. *PHP how to connect to MySQL database*. [online] www.youtube.com Available at: https://www.youtube.com/watch?v=-1DTYAQ25bY [Accessed 11th Nov. 2025].

GeeksForGeeks. *How to Fix Access Denied for User 'root'@'localhost' in MySQL*. [online] Available at https://www.geeksforgeeks.org/mysql/how-to-fix-mysql-error-access-denied-for-user/ [Accessed 12th Nov. 2025].

Stack Overflow. *phpMyAdmin access denied for user 'root'@'localhost' (using password: NO)*. [online] Available at: https://stackoverflow.com/questions/45111527/phpmyadmin-access-denied-for-user-rootlocalhost-using-password-no [Accessed 12th Nov. 2025].

W3Schools. *SQL FOREIGN KEY Constraint*. [online] Available at: https://www.w3schools.com/sql/sql_foreignkey.asp [Accessed 14th Nov. 2025].

Google Fonts. *Hanken Grotesk*. [online] Available at: https://fonts.google.com/specimen/Hanken+Grotesk [Accessed 18th Nov. 2025].

Stack Overflow. *MySQL Database won't start in XAMPP Manager-osx*. [online] Available at: https://stackoverflow.com/questions/21267064/mysql-database-wont-start-in-xampp-manager-osx [Accessed 28th Nov. 2025].

MySQL. *13.2.1 Date and Time Data Type Syntax*. [online] Available at: https://dev.mysql.com/doc/refman/8.4/en/date-and-time-type-syntax.html#:~:text=MySQL%20displays%20DATE%20values%20in,using%20either%20strings%20or%20numbers.&text=A%20date%20and%20time%20combination,23%3A59%3A59.499999'%20. [Accessed 10th Dec. 2025].

Stack Overflow. *Weird padding at bottom of page*. [online] Available at: https://stackoverflow.com/questions/43764029/weird-padding-at-bottom-of-page [Accessed 7th Jan. 2026].

Stack Overflow. *How to add a border radius on a table row?* [online] Available at: https://stackoverflow.com/questions/4094126/how-to-add-a-border-radius-on-a-table-row [Accessed 13th Jan. 2026].

FreakyJolly.com. *Generate Multipage PDF using Single Canvas of HTML Document using jsPDF*. [online] Available at: https://www.freakyjolly.com/multipage-canvas-pdf-using-jspdf/ [Accessed 27th Jan. 2026].

MDN. *<input type="color">*. [online] Available at: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input/color [Accessed 26th Feb. 2026].

WordPress. *Plug into possibility - Chatbot*. [online] Available at: https://wordpress.com/plugins/browse/chatbot [Accessed 27th Feb. 2026].

Google Cloud. *Google Cloud web hosting*. [online] Available at: https://cloud.google.com/solutions/web-hosting [Accessed 5th Mar. 2026].

Wix. *How to host a website in 5 steps*. [online] Available at: https://www.wix.com/blog/how-to-host-a-website [Accessed 5th Mar. 2026].

GeeksForGeeks. *Understanding CIA Triad*. [online] Available at: https://www.geeksforgeeks.org/computer-networks/the-cia-triad-in-cryptography/ [Accessed 9th Mar. 2026].